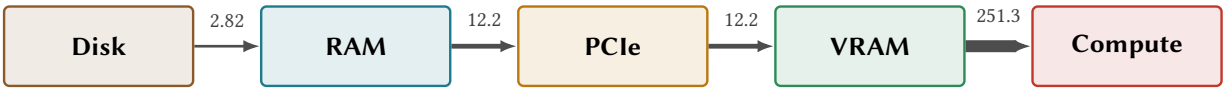
troncons

Running a model far larger than memory: compress to the extreme, decompress at the exact instant of need

Research thesis — Part I and II: the measured foundations



Throughputs measured on Stone's machine, GB/s (disk: 16 MiB random reads; PCIe: host-to-device DMA; VRAM: shader reads). Arrow width grows with throughput.

Stone

Toronto, 15 October 2026

Working version. Chapters on the architecture, the implementation and the results are to come; every figure is regenerated from the measurement data committed in the `troncons` repository.

Abstract

The aim of *troncons* is to run, on one ordinary laptop (16 GB of RAM, an 8 GB RTX 4060, a 1 TB NVMe disk), a language model several times larger than all of its memory, cleanly: weights stored compressed to the extreme on disk, decompressed only for the computation that needs them, at the exact moment it needs them, then released. Read literally, "without RAM" is impossible; the reading retained here is that *the memory in use no longer depends on the size of the model*, only on a bounded working window.

This first part lays the measured foundations. Every stage between the disk and the compute units was measured on the target machine with tools written for the purpose, each with a control case that was forced to fail: the disk delivers 2.82 GB/s (47 % of its advertised rate, because the volume is encrypted by BitLocker behind Intel RST VMD), the host-to-device link 12.2 GB/s, the video memory 251.3 GB/s. From these we prove the roof of the problem, $t \geq \max_i O_i/W_i$, and its corollaries: a crossing factor $r^\star = 4.33$ beyond which compression is wasted if the processor decompresses, and no crossing at all if the graphics card decompresses (architecture B).

Four levers are then studied and measured. *Extreme quantization*: on the Gaussian source, a shift-register trellis reaches a mean squared error of 0.068 at 2 bits, closing 90 % of the gap between the optimal scalar quantizer (0.1175) and the Shannon bound (0.0625); we wrote and measured the first portable trellis decoder (Vulkan, 124 GB/s equivalent), and proved a bit-allocation rule by memory stage (with the measured throughputs, a VRAM-resident matrix deserves 2.18 more bits than one read through the link, 3.24 more than one read from disk). *On-demand loading*: what decides everything for a mixture of experts is the cache hit rate; we derive the hit rate implied by any published measurement. *Speculative decoding*: we prove $\mathscr{E}(\alpha, k) = (1 - \alpha^{k+1})/(1 - \alpha)$ tokens per read of the weights, and that for fine-grained experts read from disk speculation pays only if $\mathscr{E}(\alpha, k) > U(k + 1)/k_e$, which no independent routing satisfies. *The target models*: 49 open models inventoried from their headers; gpt-oss-120b (63.4 GB, native MXFP4) is the first "ultra heavy" target, now downloaded and verified.

Lossless coding completes them. The 4-bit codes of the machine's models carry 3.86 bits out of 4, which an exact calculation recovers to the hundredth from any bell-shaped source under a small-group min-max, refuting for these formats the published "0.6 to 1 bit". The gain of MXFP4 lies in its scales: with those coded by the median of their row, and decoded exactly, the experts of gpt-oss-120b reach a rate of 1.1026 where zstd takes 1.04. Our interleaved rANS decoder, proved correct, runs at 70 GB/s under Vulkan and 82 under CUDA; fused into the matrix–vector product it is 3.7 times slower, so a chunk is decoded once, on arrival in video memory, and gives its whole rate back to the disk.

One equation then contains the four levers: the time of a token of a mixture of experts served by the GPU, stage by stage, under a one-constant locality model. Its proven elasticities say that on this machine the disk holds the maximum (157 ms per token against 58 for the link and 36 for the GPU, gpt-oss-120b): every bit removed and every byte of throughput gained is worth almost one for one, and faster GPU kernels return under 2 %. Three candidate architectures are computed on it. A, exact, serves the 63.4 GB model with 1.9 GB of peak RAM. B rests on a new mechanism, the exact refinement of a native 4-bit code into a base and a complement, which costs nothing in entropy (proven) and whose base loses 0.27 bit less than truncation at equal error (measured); counting the complements struck out an earlier ×1.42 gain. The arbiter decided to build A first, in a format that

prepares B. First measurements on gpt-oss-20b show a token-to-token expert reuse $\rho = 0.577$, under which speculation would pay from an acceptance rate of 0.42.

The thesis exists in two complete versions, English and French; every figure is regenerated from the committed measurement data.

Résumé

Le but de *troncons* est de faire tourner, sur un portable ordinaire (16 Go de RAM, une RTX 4060 de 8 Go, un disque NVMe de 1 To), un modèle de langue plusieurs fois plus gros que toute sa mémoire, proprement : des poids stockés compressés à l'extrême sur le disque, décompressés seulement pour le calcul qui en a besoin, à l'instant exact où il en a besoin, puis libérés. Au pied de la lettre, « sans RAM » est impossible ; la lecture retenue est que *la mémoire occupée ne dépend plus de la taille du modèle*, seulement d'une fenêtre de travail bornée.

Cette première partie pose les fondations mesurées. Chaque étage entre le disque et les unités de calcul a été mesuré sur la machine cible, avec des outils écrits pour cela, chacun avec un témoin qu'on a forcé à échouer : le disque livre 2.82 GB/s (47 % de son annonce, parce que le volume est chiffré par BitLocker derrière Intel RST VMD), le lien hôte-carte 12.2 GB/s, la mémoire vidéo 251.3 GB/s. On en démontre le toit du problème, $t \geq \max_i O_i/W_i$, et ses corollaires : un facteur de croisement $r^\star = 4{,}33$ au-delà duquel la compression est perdue si le processeur décompresse, et aucun croisement si la carte graphique décompresse (architecture B).

Quatre leviers sont ensuite étudiés et mesurés. *La quantification extrême* : sur la source gaussienne, un treillis à décalage atteint une erreur quadratique de 0,068 à 2 bits et ferme 90 % de l'écart entre le scalaire optimal (0,1175) et la borne de Shannon (0,0625) ; nous avons écrit et mesuré le premier décodeur de treillis portable (Vulkan, 124 GB/s équivalents), et démontré une règle de partage des bits selon l'étage de mémoire (avec les débits mesurés, une matrice résidente en VRAM mérite 2,18 bits de plus qu'une matrice lue par le lien, 3,24 de plus qu'une matrice lue du disque). *Le chargement à la demande* : pour un mélange d'experts, le taux de succès du cache décide de tout ; on dérive le succès qu'implique toute mesure publiée. *Le décodage spéculatif* : on démontre $\mathscr{E}(\alpha, k) = (1 - \alpha^{k+1})/(1 - \alpha)$ jetons par lecture des poids, et que pour des experts fins lus du disque la spéculation ne paie que si $\mathscr{E}(\alpha, k) > U(k+1)/k_e$, ce qu'aucun routage indépendant ne satisfait. *Les modèles visés* : 49 modèles ouverts inventoriés dans leurs en-têtes ; gpt-oss-120b (63,4 Go, MXFP4 natif) est la première cible « ultra lourde », téléchargée et vérifiée.

Le codage sans perte les complète. Les codes de 4 bits des modèles de la machine portent 3,86 bits sur 4, ce qu'un calcul exact retrouve au centième pour toute source en cloche sous un min-max par petits groupes, ce qui réfute pour ces formats les « 0,6 à 1 bit » publiés. Le gain du MXFP4 est dans ses échelles : avec leurs échelles codées par la médiane de leur ligne, et décodés exactement, les experts de gpt-oss-120b atteignent un taux de 1,1026 là où zstd prend 1,04. Notre décodeur rANS entrelacé, démontré exact, tourne à 70 GB/s sous Vulkan et 82 sous CUDA ; fondu au produit matrice-vecteur, il est 3,7 fois plus lent : un tronçon se décode donc une fois, à son arrivée en mémoire vidéo, et rend tout son taux au disque.

Une seule équation contient ensuite les quatre leviers : le temps d'un jeton d'un mélange d'experts servi par la carte, étage par étage, sous un modèle de localité à une constante. Ses élasticités, démontrées, disent que sur cette machine le disque tient le maximum (157 ms par jeton contre 58 pour le lien et 36 pour la carte, gpt-oss-120b) : chaque bit retiré et chaque octet de débit gagné valent presque un pour un, et des noyaux plus rapides rapportent moins de 2 %. Trois architectures candidates y sont calculées. A, exacte, sert le modèle de 63,4 Go avec 1,9 Go de RAM de pointe. B repose sur un mécanisme neuf, le raffinement exact d'un code 4 bits natif en une base et un complément, qui ne

coûte rien en entropie (démontré) et dont la base perd 0,27 bit de moins que la troncature à erreur égale (mesuré) ; compter les compléments a fait barrer un gain de ×1,42 annoncé plus tôt. L'arbitre a décidé de construire A d'abord, dans un format qui prépare B. Les premières mesures sur gpt-oss-20b donnent une réutilisation des experts d'un jeton au suivant $\rho = 0{,}577$, sous laquelle la spéculation paierait dès un taux d'acceptation de 0,42.

Le mémoire existe en deux versions complètes, anglaise et française ; chaque figure se régénère depuis les données de mesure commitées.

Contents

Abstract iii

Résumé v

List of Figures xiii

List of Tables xv

Listings xvii

Notation xix

I The problem 1

1 Introduction and motivation 3
- 1.1 The mission . . . 3
- 1.2 Where we start . . . 4
- 1.3 What “without RAM or anything” means . . . 4
- 1.4 The research question . . . 4
- 1.5 What this first part contributes . . . 5
- 1.6 The method . . . 5
- 1.7 Outline . . . 6

2 State of the art 7
- 2.1 The memory wall, at batch size one . . . 7
- 2.2 The working window: an idea from 1968 . . . 8
- 2.3 Running a model larger than memory . . . 9
- 2.4 Extreme quantization . . . 9
- 2.5 Lossless compression . . . 9
- 2.6 Speculation . . . 10
- 2.7 What each family changes in the roof . . . 10
- 2.8 Positioning . . . 10

3 The machine and its physics 13
- 3.1 The machine . . . 13
- 3.2 The measurement method . . . 14
- 3.3 The disk . . . 14
  - 3.3.1 The instrument . . . 14
  - 3.3.2 Results . . . 15
  - 3.3.3 Why so far from the advertised rate . . . 16
  - 3.3.4 The cost of BitLocker, measured by a second path . . . 16

3.3.5 IoRing, BypassIO, DirectStorage . . . 16
3.4 The RAM . . . 17
3.5 From host to device . . . 18
3.5.1 The instrument, and the trap the control uncovered . . . 18
3.5.2 Results . . . 18
3.6 The card . . . 19
3.6.1 Thermals, and the discovery of the battery . . . 20
3.7 The baseline: llama.cpp . . . 21
3.8 The roof . . . 22
3.8.1 The model . . . 22
3.8.2 The roof theorem . . . 22
3.8.3 The two corollaries that decide the architecture . . . 23
3.8.4 Ceilings by stage . . . 24
3.8.5 The decoder budget, and why B . . . 24
3.9 What is missing in the world . . . 25

4 Problem formulation **29**
4.1 The token as a chain of stages . . . 29
4.2 The terms for a mixture of experts served by the GPU . . . 30
4.3 Locality, in one constant . . . 30
4.4 The numbers of the machine . . . 31
4.5 The problem, stated . . . 32
4.6 What holds the maximum: the elasticities . . . 33
4.7 A worked example . . . 34
4.8 The working window, in numbers . . . 35
4.9 The term that was missing from speculation . . . 35
4.10 The witnesses of the equation . . . 36

II The levers, measured **39**

5 Extreme quantization **41**
5.1 The lever . . . 41
5.2 The formats, bit by bit . . . 42
5.3 The Gaussian source and its bound . . . 42
5.4 The optimal scalar quantizer . . . 44
5.5 The theoretical bench, redone here . . . 45
5.6 The shift-register trellis . . . 46
5.7 The cost of decoding on the card . . . 48
5.7.1 In CUDA . . . 48
5.7.2 In Vulkan and DirectX 12: the first portable trellis measured . . . 48
5.7.3 A multiplication-free trellis is not faster . . . 50
5.8 The llama.cpp ladder, measured on Qwen3.5 . . . 50
5.9 At equal bytes read: today's best trade-off . . . 52
5.10 Splitting bits by stage . . . 53
5.11 The techniques, in one table . . . 55
5.12 What is missing in the world . . . 55

6 Lossless compression at the speed of compute **57**
6.1 The judge: when lossless pays . . . 57
6.2 What a quantized code contains . . . 58
6.2.1 The estimator, and its bias . . . 58
6.2.2 The codes: almost uniform, and order 0 says it all . . . 59
6.2.3 The metadata, and what a byte coder sees . . . 60
6.3 Why so little: the entropy of a min-max . . . 60
6.3.1 A published contradiction, and a corrected sentence . . . 62
6.4 MXFP4: the gain is in the scales . . . 63
6.5 The interleaved rANS coder . . . 65
6.5.1 The format . . . 65
6.5.2 Correctness . . . 66
6.5.3 The heart of v4 . . . 69
6.6 Four decoders, and what each one taught . . . 69
6.7 The key question: decode inside the product, or just before? . . . 71
6.8 Beyond lossless: a quantization tailored to the coder . . . 74
6.9 The state of the art, technique by technique . . . 75
6.10 What is missing in the world . . . 75

7 On-demand loading **79**
7.1 The law of loading . . . 79
7.2 The mixture of experts in three quantities . . . 79
7.3 Granularity . . . 80
7.4 Dense models . . . 81
7.5 Mixtures of experts . . . 81
7.6 Expert locality from one token to the next . . . 82
7.7 Double-buffered pipelines . . . 83
7.8 The judge applied to the machine . . . 84
7.9 What is missing in the world . . . 85

8 Sparsity and speculative decoding **89**
8.1 The judge, and the lever with the right slope . . . 89
8.2 Speculative decoding . . . 90
8.2.1 The verification rule . . . 90
8.2.2 The amortization theorem . . . 91
8.2.3 Trees . . . 91
8.2.4 The variants . . . 92
8.3 Mixtures of experts and speculation . . . 92
8.4 Activation sparsity . . . 94
8.4.1 The measurement: unions, window, hot neurons . . . 94
8.4.2 A mask shared by the verified block . . . 96
8.5 The computation for Stone's machine . . . 96
8.5.1 Acceptance rates, measured . . . 96
8.5.2 The resident substitute, transposed . . . 97
8.5.3 Dense models read from disk . . . 97
8.6 What is missing in the world . . . 97

9 The target models **99**
9.1 What “ultra heavy” means here . . . 99
9.2 The inventory, read from the headers . . . 99
9.3 Bytes read per token . . . 100
9.4 The physical judge . . . 102
9.5 What changed in 2026 and serves troncons . . . 102
9.6 The judging protocol . . . 103
9.7 The downloads . . . 104
9.8 What is missing in the world . . . 104

III Towards the system **105**

10 Architecture: the candidates, the decision, what comes next **107**
10.1 Three angles . . . 107
10.2 A: everything through the GPU, bytes coded end to end, lossless . . . 107
10.3 B: precision by stage . . . 109
10.3.1 The mechanism . . . 109
10.3.2 The measurement on the real weights . . . 111
10.3.3 The number that had to be struck out . . . 111
10.3.4 Where B really gains: B3 and the λ knob . . . 111
10.4 C: reading less often . . . 112
10.5 The debate and the decision . . . 113
10.6 The first measurements of the target model . . . 115
10.7 What comes next . . . 116

11 Discussion and limits **119**
11.1 What the first part establishes . . . 119
11.2 What is solid, what is fragile . . . 119
11.3 The ideas killed by the numbers . . . 120
11.4 The struck-out numbers . . . 120
11.5 The limits of the method . . . 120
11.6 Reading “without RAM or anything” . . . 121

12 Future work **123**
12.1 The deciding measurements, first . . . 123
12.2 Milestone J1: building A . . . 123
12.3 The research questions . . . 124
12.4 The gesture that falls to Stone . . . 124
12.5 Possible downloads . . . 124
12.6 The thesis itself . . . 125

A Measurement protocols **127**
A.1 The labels . . . 127
A.2 The witness, and its break . . . 127
A.3 The threshold declared beforehand . . . 127
A.4 ABBA series and noise . . . 128
A.5 The second path . . . 128

A.6 Power and the shared machine . . . 128
A.7 The quality judge . . . 128
A.8 The struck-out sentence . . . 128

B Tools **129**
B.1 The thesis toolchain . . . 129
B.2 The tools of the studies . . . 129

C Logs and reproduction of the figures **135**
C.1 Rebuilding the thesis . . . 135
C.2 The logs . . . 135
C.3 The data of each graph . . . 136

Bibliography **139**

List of Figures

2.1 Timeline of the works, by family . . . 7

3.1 Disk throughput versus block size and queue depth . . . 15
3.2 XTS-AES decryption versus thread count . . . 17
3.3 Dependent-access latency versus buffer size . . . 18
3.4 PCIe link generation during copies . . . 19
3.5 Matrix–vector products written here, on the RTX 4060 . . . 20
3.6 Five minutes of GEMV, mains versus battery . . . 21
3.7 The data path, from disk to compute units . . . 26
3.8 Computed ceiling versus measured throughput, by stage . . . 26
3.9 Architectures A and B . . . 27
3.10 Timing diagram: serial versus overlapped . . . 27
3.11 The roof of Qwen3.5-9B entirely on disk, versus r . . . 28
3.12 The roof of the three target MoE models, real placement . . . 28

4.1 The three locality scenarios . . . 31
4.2 The ceiling against the share read from disk . . . 34
4.3 The working window: ceiling against RAM . . . 35

5.1 Floating-point formats, bit by bit . . . 42
5.2 The MXFP4 block . . . 42
5.3 The Q4_K super-block . . . 43
5.4 The IQ2_XXS super-block . . . 43
5.5 Distortion versus rate on the Gaussian source . . . 46
5.6 The shift-register trellis and its computed decoder . . . 47
5.7 Decoding the trellis in Vulkan and DirectX 12 . . . 49
5.8 Perplexity versus bytes read per token, Qwen3.5 4B and 9B . . . 51
5.9 Decoding speed versus bytes read per token . . . 52
5.10 At equal bytes: Llama 2 in trellis codes . . . 53
5.11 At equal bytes: Qwen3 on MMLU . . . 54

6.1 The law of 4-bit codes, Q4_K and MXFP4 . . . 59
6.2 Entropy of the codes of a min-max by group size . . . 61
6.3 Entropy of the codes by quantization range . . . 62
6.4 Coding the scales by the row median . . . 63
6.5 Real lossless rate of every expert tensor . . . 65
6.6 Entropy of the E8M0 scales, tensor by tensor . . . 66
6.7 The tile of the interleaved rANS coder . . . 67
6.8 The decoding-table entry, bit by bit . . . 67
6.9 The shared stream: vote and bit count . . . 68
6.10 Throughput of the rANS decoders on the RTX 4060 Laptop . . . 70

6.11 The MXFP4 product, raw, fused with the decoder, or in two passes . . . 73
6.12 Where to decode: on arrival of the chunk . . . 73
6.13 At equal error, what an ECSQ needs . . . 75

7.1 A mixture-of-experts layer and its cache hierarchy . . . 80
7.2 Cache policy hit rates on synthetic routing . . . 84
7.3 Required cache hit rate, model by model . . . 87
7.4 The ceiling of gpt-oss-120b versus cache hit rate . . . 88
7.5 The four responses to a cache miss . . . 88

8.1 The time of a pass of n tokens . . . 90
8.2 Tokens produced per read of the weights, $\mathscr{E}(\alpha, k)$ . . . 92
8.3 Chain and tree verification . . . 93
8.4 The union of the masks of n consecutive tokens . . . 95

9.1 Total and active parameters of 2026 mixtures of experts . . . 100

10.1 Ceilings of the candidate architectures . . . 108
10.2 The byte path of architecture A . . . 109
10.3 Timeline of a token in A . . . 109
10.4 Base and complement of an E2M1 code . . . 110
10.5 B3: throughput against exactness . . . 112
10.6 Measurement F1: bits per weight of each format variant . . . 114
10.7 A chunk of the .trc format . . . 115
10.8 llama.cpp baseline on gpt-oss-20b . . . 116
10.9 Measured locality on gpt-oss-20b against the assumptions . . . 117

List of Tables

2.1 Three ways to occupy only what is used . . . 8
2.2 What each family changes in the roof inequality . . . 11
2.3 What existing systems cover of the troncons problem . . . 11

3.1 Stone's machine, survey of 2026-10-15 . . . 13
3.2 Disk throughput, excerpt of the ABBA sweep . . . 15
3.3 Host ↔ device transfers, 64 MiB . . . 19
3.4 llama.cpp on Qwen3.5-9B Q4_K_M, first pass . . . 21
3.5 Tokens/s ceilings by stage . . . 24

4.1 The quantities of the token equation . . . 32
4.2 What each quantity is worth . . . 33
4.3 The witnesses of the token equation . . . 36

5.1 Ceilings versus bits per weight . . . 41
5.2 Bits per weight, format by format . . . 44
5.3 Squared error at 2 bits on the Gaussian source . . . 45
5.4 GEMV in CUDA, four formats . . . 48
5.5 The llama.cpp ladder on Qwen3.5-4B and 9B . . . 50
5.6 Extreme quantization techniques . . . 55

6.1 Entropy of the K-quant blocks of Qwen3.5-9B . . . 59
6.2 What a byte coder sees on a K block . . . 60
6.3 Entropy of the E8M0 scales by context . . . 63
6.4 Real lossless rate of MXFP4, exact decoding . . . 64
6.5 The v4 decoder under CUDA and Vulkan, ABBA . . . 71
6.6 The key question, measured . . . 72
6.7 Lossless weight coders, with respect to the mission . . . 76

7.1 Locality read from published measurements . . . 80
7.2 Predicting and prefetching experts . . . 82
7.3 Expert cache policies . . . 82
7.4 Measured expert locality . . . 83
7.5 The judge on the machine, disk at 2.4 then 2.82 GB/s . . . 85

8.1 Values of $\mathscr{E}(\alpha, k)$ . . . 93
8.2 The variants of speculative decoding . . . 94
8.3 Speculation and MoE models read from disk . . . 94
8.4 Activation sparsity methods . . . 95
8.5 The quality cost of a shared mask . . . 96
8.6 Measured greedy acceptance rates . . . 97
8.7 Speculation and dense models read from disk . . . 97

9.1 The target mixtures of experts, read from their headers . . . 101
9.2 Bytes read per token: headers against formula . . . 101
9.3 The candidate files and their ceilings . . . 102

10.1 Ceilings of A . . . 110
10.2 Base layers of an E2M1 code . . . 111
10.3 The milestones decided . . . 113

11.1 The ideas killed by the numbers . . . 120
11.2 The struck-out numbers . . . 121

12.1 The open research questions . . . 124
12.2 Downloads that could help . . . 125

Listings

5.1 The vectorized Viterbi of the shift-register trellis 47
5.2 The mul1 decoder in WGSL 48

6.1 Inner loop of the rANS v4 decoder (WGSL) 69

Notation

Units are decimal: 1 GB = 10^9 bytes, 1 GB/s = 10^9 bytes per second; binary multiples are written KiB, MiB, GiB. Every number of this thesis carries its provenance: [meas.] measured here on Stone's machine with a committed tool and log, [calc.] computed from named numbers, [lit.] literature (publisher's number, not reproduced).

Symbol	Meaning
Throughput and the roof	
B	bytes of weights read per decoded token (batch of one)
W_i	effective throughput of stage i (disk, RAM, PCIe, VRAM), bytes/s
O_i	bytes that must cross stage i for one token
f_V, f_R, f_D	fractions of the weights resident in VRAM, RAM, on disk ($f_V + f_R + f_D = 1$)
r	compression factor of stored weights (bytes served / bytes stored)
$r^\star$	crossing factor $W_{\text{PCIe}}/W_{\text{disk}}$
t	time per token, s
Quantization	
N	number of (active) parameters read per token
b	bits per weight, metadata included
R	rate, bits per sample (per weight)
$D(R)$	distortion–rate function (mean squared error at rate R)
σ^2	variance of the source
$\Delta(q)$	distance in bits of quantizer q to the Shannon bound
L, k	state width and bits per weight of a shift-register trellis
c_i, n_i	sensitivity and size (weights) of matrix i
Lossless coding	
$H(\cdot), h(\cdot)$	Shannon entropy (bits) of a discrete variable; differential entropy of a density
W_{dec}, W_g	output throughput of the decoder on the card; throughput at which the matrix–vector product consumes decoded weights
M, L	rANS probability precision (2^{12}) and lower bound of the state (2^{16}, Chapter 6 only)
f_s, c_s	quantised frequency and cumulative of symbol s
$e, \tilde{e}, \delta$	E8M0 scale of an MXFP4 block, median of its row, offset $e - \tilde{e}$
Mixture of experts and cache	
E, k_e	routed experts per layer, experts chosen per token
B_t, B_e	always-read bytes and routed-expert bytes per token ($B = B_t + B_e$)

Symbol	Meaning
h	hit rate of the expert cache (share of routed bytes served from memory)
Speculative decoding	
α	acceptance rate of one drafted position
k	number of drafted tokens per verification pass
$\mathscr{E}(\alpha, k)$	expected tokens produced per full read of the active weights
$U(n)$	expected size of the union of experts of n tokens
$n^\star$	batch size where compute catches up with reading

Part I

The problem

Chapter 1

Introduction and motivation

1.1 The mission

This thesis answers a request written by Stone on 15 October 2026. Here it is in his exact words, in French, mistakes included, because they are what is authoritative and everything else is only a reading of them:

> *j'ai aussi une mission je veux de la recherche scientique et de l'implémentation prend les meilleurs avancées au monde en stockage optimisation et domaine connexes (enfin les plus utiles) et fait en sorte de tout les assemblées dans le but de produire et d'implémenter une vrai solution pour prendre un modele ultra lourd quelcoque d'une ai le faire tourner proprement sans ram ni rien en fait un moyen de compresser à l'extreme puis décompresser en un temps record au besoin à l'instant exacte du noeud bref tu sais ce qui manque et tu sais quoi faire je ne veux pas que des recherches déja existante je veux que tu fasse une vrai recherche de spécialiste puis tu implémente la solution fonctionnelle avec une team d'ingénieur si ce qu'on cherche n,existe pas alors tant mieux creer le ceci est ta mission principale on vas continuer la suite relais plus tard on doit faire tomber se levier d'abord*

In English, faithfully: *"I also have a mission: I want scientific research and implementation. Take the world's best advances in storage, optimization and related fields (well, the most useful ones) and assemble them all in order to produce and implement a real solution to take any ultra-heavy AI model and run it cleanly without RAM or anything; in fact, a way to compress to the extreme, then decompress in record time when needed, at the exact instant of the node. In short, you know what is missing and you know what to do. I do not want only existing research; I want you to do real specialist research, then implement the working solution with a team of engineers. If what we are looking for does not exist, so much the better: create it. This is your main mission; we will continue with relais later, we must bring down this lever first."*

It has an older source, in two sentences of 2 and 4 October 2026, rendered here in English: *"does it exist, otherwise we could invent a kind of instant call, in other words a new way to compress data so that it appears right when we need it and disappears afterwards"*; and *"I am thinking a bit of chunks like in Minecraft, a kind of ultra-advanced engineering that can make the load of an AI disappear when it is no longer useful and reload another one instantly, I mean running an AI only when it is needed and not everything at once, an AI taking up space only when active"*.

The image of Minecraft's "chunks" is exact and gives the project its name. In that game, the world is cut into sections of 16×16 columns; only those around the player are loaded in memory, the others sleep on disk, and the engine loads and unloads continuously as the player moves. A language model is a world of weights that computation traverses layer after layer, token after token: *troncons* (French for "sections") wants to load, around the point where computation is, the section of weights it needs, and nothing else.

1.2 Where we start

The machine. An ordinary laptop: an Intel i7-13620H processor, **16 GB of RAM on a single module**, an NVIDIA RTX 4060 Laptop card with **8 GB**, a 1 TB NVMe disk with 404 GB free, encrypted by BitLocker. After the system, the display and the context, about 6.5 GB of VRAM and 11 GB of RAM remain for weights, 17.5 GB in total. Chapter 3 measures each stage.

The models. The open models that matter in 2025–2026 are mixtures of experts from 20 billion to 2.8 trillion parameters (Chapter 9). The first target, gpt-oss-120b, weighs 63.4 GB in its original format: **3.6 times** all the usable memory of the machine. Others weigh five to twenty times more.

What exists. The engines that run a model larger than memory (Chapter 2) know how to move weights, but they reason with data-sheet throughputs, on workstations with 128 or 192 GB of RAM, with formats decoded by CUDA, and almost all under Linux. None is made for a 16 GB Windows laptop whose encrypted disk delivers less than half its advertised throughput.

1.3 What "without RAM or anything" means

Taken literally, the request is impossible: every computation reads its operands from a memory, and a matrix product requires the matrix to be, at least in pieces, in the registers of the unit that computes. The reading retained, to be confirmed by Stone and never substituted for him, is the following.

Definition 1.1 (Bounded-window execution)**.** An execution of a model with P bytes of weights is *bounded-window* if its peak memory (RAM and VRAM) is bounded by a constant M_{window} that does not depend on P, but only on the size of the largest section of weights a computation needs at once and on the prefetch depth.

Under this reading, "without RAM" means that **the memory in use no longer depends on the size of the model**. The difficulty then changes nature: it is no longer to make the model fit, but to move, at every token, the bytes it needs through the slowest of the stages that separate it from the compute units. The judge is physics, quantified by theorem 3.2:

$$\text{tokens/s} \;\leq\; \min_i \frac{W_i}{O_i},$$

where W_i is the measured throughput of stage i and O_i the number of bytes a token must move through it. The four levers of this thesis each act on one term of this inequality: compressing to the extreme reduces O_i (Chapter 5), and lossless coding reduces it further by a rate r upstream of the decoder (Chapter 6); decompressing on the card changes the stage the compressed bytes cross (Section 3.8); a cache that hits keeps bytes in a faster stage (Chapter 7); speculative decoding returns several tokens per read (Chapter 8).

1.4 The research question

> *On a consumer machine whose memory is several times smaller than the model, which assembly of weight representation, placement, loading and compute maximizes tokens per second at measured quality, and what does the world lack to reach it?*

"I do not want only existing research": the state of the art is read thoroughly, but the deliverable is a contribution, an assembly or a mechanism that does not yet exist, proven by measurement, and a program that works. This first part lays the foundations; the architecture, the implementation and the results will come in the chapters announced in Chapter 10.

1.5 What this first part contributes

1. **A measured, and proven, roof.** Every stage of the machine is measured with a control that can fail (Chapter 3): the disk delivers 2.82 GB/s, 47 % of its advertised rate, because the volume is encrypted and goes through Intel RST VMD; the host-to-device link 12.2 GB/s; the VRAM 251.3 GB/s. We prove the throughput bound of a chain of stages, its steady state through a flow-shop lemma with identical jobs, and two corollaries that settle the architecture: beyond a compression factor $r^\star = 4.33$, compressing is wasted if the processor decompresses, and never if the card decompresses.
2. **The battery regime.** On battery, the machine is another machine: card power limit lowered from 80 to 52.6 W, card memory at 6 GHz instead of 8, and everything that goes through the processor divided by 4 to 6. No published inference benchmark records the power source.
3. **The first portable trellis decoder.** A shift-register trellis, the code family closest to the Shannon bound at 2 bits, decoded in a Vulkan shader at 124 GB/s equivalent, as fast as 4-bit scalar while reading half the bytes (Chapter 5).
4. **A rule for splitting bits by stage**, proven through the Lagrangian, then corrected with the measured throughputs: a VRAM-resident matrix deserves 2.18 more bits than a matrix read over the link, 3.24 more than one read from disk.
5. **The equal-bytes measurement of the llama.cpp ladder on Qwen3.5**: with the best portable 2-bit format, shrinking beats compressing up to about 3.2 bits per weight.
6. **The MXFP4 scales, coded by the median of their row**: a field-wise lossless coding, decoded exactly, that yields a rate of 1.1026 over the 60.9 GB of experts of gpt-oss-120b where zstd takes 1.04; an interleaved rANS decoder, proved exact, at 70 GB/s under Vulkan and 82 under CUDA; the measurement that decoding inside the product is 3.7 times slower, hence the rule of decoding on arrival of the chunk; and the exact calculation that explains the 3.86 bits of 4-bit codes and refutes, for small-group formats, the published "0.6 to 1 bit" entropies (Chapter 6).
7. **The cache hit rate implied by a published measurement**, which reads locality in a measurement that does not publish it (Chapter 7).
8. **The amortization theorem of speculation** and the exact condition under which it pays for a mixture of experts read from disk, which independent routing never satisfies; and the measurement that activation sparsity and speculation do not compose on a SwiGLU model (Chapter 8).
9. **An inventory of 49 models read from their headers** and a judging protocol in which one measurement has already corrected an assumption (Chapter 9).
10. **The token equation and its elasticities**, proven: on this machine the disk holds the maximum, and any lever that does not touch the bytes read from disk is ornament; the condition for speculation corrected by expert reuse, $\alpha > 1 - \rho$ (Chapter 4).
11. **The exact refinement of a native 4-bit code**, which costs nothing in entropy and whose base beats truncation by 0.27 bit; three candidate architectures, the correction of a wrongly announced gain, and the decision to build the exact architecture first (Chapter 10).

1.6 The method

The required standard fits in a few rules, applied to every number (Chapter A):

- every measurement has its **control**, a case whose answer is known in advance, and every new control must be able to fail: its condition is forced to be false, we check that the instrument fails, then restore it;
- a **threshold** is never set by eye: it derives from a named quantity and is declared before the measurement;
- a cost never rests on a single pass: **ABBA series**, with the gap of a program to itself as reference noise;
- every number we depend on is redone through an independent **second path**;
- when a measurement contradicts a published number, even ours, **the sentence is crossed out** and we say what changed: never a silent retraction; this thesis keeps several of them, ~~like this one~~, in place;
- every number carries its provenance: [meas.] measured here, [calc.] computed, [lit.] literature not reproduced;
- whatever produces a published number is committed, tools and logs, and every figure is regenerated from its data by a committed script (Chapter C).

Each chapter opens with what was acquired at the start, what it establishes and what remains, and closes with what could be wrong.

1.7 Outline

The first part states the problem: the state of the art (Chapter 2), the machine and its physics (Chapter 3), the formulation of the problem in equations (Chapter 4). The second measures the levers one by one: extreme quantization (Chapter 5), lossless compression at the speed of compute (Chapter 6), on-demand loading (Chapter 7), sparsity and speculative decoding (Chapter 8), the target models (Chapter 9). The third prepares the system: what comes next (Chapter 10), discussion and limits (Chapter 11), future work (Chapter 12). The appendices give the measurement protocols, the tools and the logs.

Chapter 2

State of the art

Where we start, what this chapter establishes, what remains

This chapter gathers, as of 15 October 2026, what the world knows how to do to run a model larger than memory. Each family is read at its primary source (paper, official repository at a named commit, release notes) and judged by the same rule: the bytes a token must read, divided by the throughput of the stage that serves them. Details, measurements and complete numbers are in the chapters of the second part; here we say what exists, what each family gains, what it costs, and what is missing for Stone's machine.

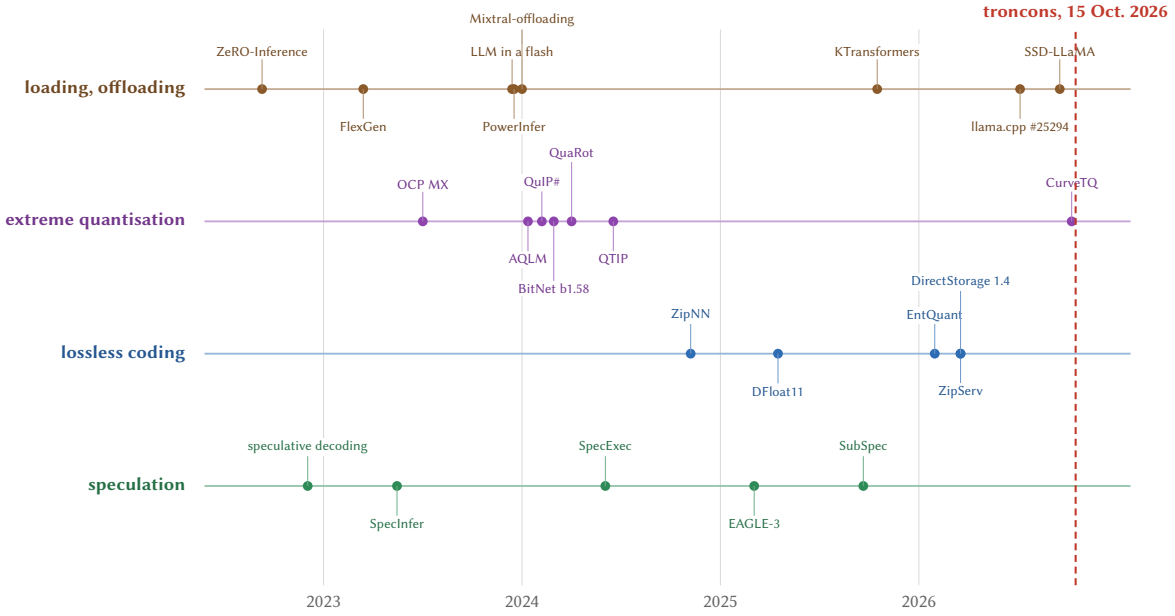


Figure 2.1: The works cited in this chapter, by family and by date of first public version (date of the bibliography entry). Four families have matured in parallel since 2023; none targets a 16 GB Windows laptop reading its weights from an encrypted disk. The theoretical roots are much older.

Data: `memoire/sources/*.bib` (dates), `memoire/figures/schemas/etat.tex`.

2.1 The memory wall, at batch size one

A decoded token, at batch size one, multiplies each weight matrix by a single vector: each weight is read once and serves two operations. The arithmetic intensity, about two operations per weight

read, is far below what the card can compute per byte (about 50 operations per byte of VRAM on the RTX 4060, Section 3.6): decoding is memory-bound. It is the classical reading of the roofline model of Williams, Waterman and Patterson [1], which bounds performance by the minimum of compute and of intensity × throughput. This thesis extends the idea to a hierarchy of stages (disk, RAM, link, VRAM) crossed at every token (theorem 3.2): when the model does not fit in VRAM, the bounding stage is no longer the VRAM but the link or the disk, 20 to 90 times slower.

2.2 The working window: an idea from 1968

Stone's picture, an AI that "occupies space only when active", has had a name in computer science for more than half a century. Denning [2] defines the *working set* of a program at time t as the set of pages it referenced in the window $(t-\tau, t]$,

$$W(t,\tau) = \{\,\text{pages referenced between } t-\tau \text{ and } t\,\},$$

and holds that the memory to give a program is that of its working set, the rest being free to sleep on disk. Two years earlier Belady [3] had shown which replacement policy is optimal when the future is known: evict the page whose next use is farthest away. These two results bound everything Chapter 7 measures: the bounded-window execution of Definition 1.1 is the working-set property applied to weights, with a window whose size does not depend on the model; and Belady's optimum is the ceiling of any expert cache.

Video-game engines turned this idea into a rendering technique. Sparse virtual textures [4] cut a huge texture into tiles, send the card only the tiles the current frame needs, and translate the virtual address into a physical one in the shader, like a page table; DirectStorage [5] moves the reading and decompression of these data onto the card. This is exactly the Minecraft "chunk" Stone has in mind, and Table 2.1 lays out the correspondence term by term.

Table 2.1: Three ways to occupy memory only for what is used: virtual memory, the virtual textures of game engines, and troncons. The last row is the difference that matters.

	virtual memory	virtual textures	troncons
unit	the page	the texture tile	the chunk: one expert matrix, 4.4 MB in gpt-oss-120b
who asks	the processor, on access	the shader, while rendering the frame	the router of each layer, at every token
what predicts	temporal locality	the camera, which moves little from frame to frame	expert reuse between neighbouring tokens (ρ, Chapter 4)
the cache	the RAM	a physical texture in VRAM	the VRAM, then pinned RAM
compression	none	decompression on the card	rANS decoded on the card (Chapter 6)
the miss	the program waits for the page	a coarser tile is shown meanwhile	the token waits for the expert: a wrong computation is not allowed

The last row is the fundamental difference. A game engine may display a blurry tile until the right one arrives: the eye forgives. A language model cannot compute with an approximate expert

without changing its output. The closest equivalent of the coarse tile is a progressive code whose base layer serves at once and whose complement then corrects it bit for bit: the exact refinement of Proposition 10.1, and the chapter that carries it (Chapter 10) quantifies what it costs.

2.3 Running a model larger than memory

Weight offloading. The first generation loads layers on demand: AirLLM [6] reads a layer, computes it, frees it, and solves peak memory while doing nothing for throughput; DeepSpeed ZeRO-Inference [7] and FlexGen [8] amortize the read over large batches, at the price of latencies of thousands of seconds. llama.cpp [9] maps the file into memory and splits layers between the card and the processor; it is the project's baseline.

Activation sparsity. Deja Vu [10], PowerInfer [11, 12] and LLM in a flash [13] read only the active neurons of a token, predicted by small networks; they obtain factors of 2 to 25 on ReLU models, most of whose neurons are zero. Recent models use SwiGLU, where sparsity is weak (Chapter 8).

Mixtures of experts. An MoE activates only a small share of its parameters per token (3 to 10 % for 2026 models, Figure 9.1), which makes it the natural target: only the chosen experts need to be read. Mixtral-offloading [14] keeps an LRU cache of experts in VRAM and prefetches through the next layer's router; ProMoE [15], HOBBIT [16], ExpertFlow [17], SeqMoE [18] and Edge0 [19] predict better; Fiddler [20] and KTransformers [21] compute experts where they are. Since 2026, engines read experts from disk with an explicit cache: llama.cpp pull request #25294 [22], moe-stream [23] and SSD-LLaMA [24], the latter compressing experts in the read loop. Chapter 7 judges them.

2.4 Extreme quantization

Quantization reduces b, the bits per weight, and multiplies the ceiling by $16/b$. Four generations overlap (Chapter 5):

- **group-wise scalar quantizers**: llama.cpp K-quants, GPTQ, EfficientQAT [25], HIGGS [26]; simple to decode, they lose a lot below 3 bits;
- **vector codes**: llama.cpp i-quants (grids drawn from the E8 lattice), QuIP# [27], AQLM [28] and PV-tuning [29], VPTQ [30];
- **trellis codes**: QTIP [31] and its simplified version EXL3, and since autumn 2026 CurveTQ [32] (rotation-free), XOR-Trellis [33], Tetra [34] (Leech lattice); at 2 bits on the Gaussian source, they close 90 % of the gap between the optimal scalar quantizer and the Shannon bound;
- **models born at low rate**: BitNet [35], ParetoQ [36], and the MXFP4 and NVFP4 formats [37, 38] in which gpt-oss, Kimi K2.6 and K3, DeepSeek V4 publish their weights.

Rotations (QuaRot [39], SpinQuant [40]) make the source Gaussian and serve as a building block for the codes of the next two generations. The finding that weighs for this thesis: **the best codes decode only in CUDA**, and the best 2-bit format decodable under Vulkan (IQ2_XXS) is worth, on the Gaussian source, exactly the optimal scalar quantizer.

2.5 Lossless compression

After quantization, redundancy remains: codes are not uniform and block scales are highly predictable. Recent work exploits it through entropy coding. DFloat11 [41] Huffman-codes the exponent of BF16

weights and shrinks the model to about 70 % of its size losslessly; ZipNN [42], Huff-LLM [43], ECF8 [44], ZipServ [45] and the work approaching the Shannon bound [46] follow the same path on other formats; EntroLLM [47], EntroPack [48], EntQuant [49], WaterSIC [50] and vram-compression [51] combine quantization and entropy coding, the last with a CUDA kernel that fuses rANS decoding into the matrix–vector product.

On the decoder side, asymmetric numeral systems (ANS) [52] and their interleaving [53] have produced massively parallel GPU decoders [54, 55], libraries (nvCOMP [56], DietGPU [57]) and, under Windows, GDeflate and Zstandard [58] decompression on the GPU through DirectStorage [5]. The project's entropy study (Chapter 6) measures on the machine's models a lossless ceiling of 5.3 % for K-quants and a real rate of 1.1026 for the MXFP4 experts of gpt-oss-120b (9.3 % fewer bytes, decoded exactly), mostly in the scales, which nobody codes; and an rANS decoder at 70 GB/s of codes produced under Vulkan, 82 under CUDA. It also shows, through an exact calculation, why the "0.6 to 1 bit" entropies published for int4 do not apply to small-group formats. The overall lesson is certain: **after a good quantization, lossless compression is worth a few per cent**; the compression lever is the representation, and lossless coding plays on the metadata.

2.6 Speculation

Speculative decoding [59, 60] has a cheap draft propose several tokens and verifies them in one pass of the target, losslessly. Its variants change the draft (small model, Medusa heads [61], EAGLE layer [62, 63], native MTP module [64, 65], DFlash block diffusion [66], n-grams [67]) or the shape (trees of SpecInfer [68], Sequoia [69], SpecExec [70]). Two works target the offloaded regime: SpecExec, which verifies trees of thousands of nodes when weights cross the link, and SubSpec [71], which keeps a low-precision copy of the target in VRAM as the draft. For MoE models, Cascade [72] shows that speculation can slow things down, and MoE-Spec [73], AcceptMoE [74], EcoSpec [75] bound the expert set of the verification.

2.7 What each family changes in the roof

All these families read in the same inequality, tokens/s $\leq \min_i W_i/O_i$ (theorem 3.2), multiplied by $\mathscr{E}$, the number of tokens produced per read of the weights. Table 2.2 says which term each one moves, and under which condition; this common reading is what allows Chapter 4 to combine them rather than juxtapose them.

2.8 Positioning

Table 2.3 shows the shape of the gap. Every building block exists somewhere: reading experts from disk (Linux), decompression on the GPU (for textures, or from VRAM), codes close to the bound (in CUDA), speculation in the offloaded regime (with 128 GB of RAM). **Nobody has assembled them on a 16 GB Windows laptop without the CUDA toolkit**, and the chapters that follow show, with measured numbers, that this assembly is not a juxtaposition: the levers multiply (quantization manufactures the speculation draft, compression acts twice on an expert cache) or cancel out (speculation and fine-grained experts, sparsity and verification).

Table 2.2: What each family of techniques changes in $\mathscr{E} \cdot \min_i W_i/O_i$, and the condition under which the gain is real. b: bits per weight; r: lossless rate; h: cache hit rate; α: acceptance rate.

family	term moved	condition, and what measures it here
quantization	every O_i, by a factor $16/b$	quality holds (perplexity, KL divergence, Chapter 5) and the kernel keeps up with VRAM
lossless coding	the O_i of the stages upstream of the decoder, by a factor r	the decoder exceeds r times the throughput of the relieved stage (corollary 6.2)
decompression on the card	the stage that carries compressed bytes	beyond $r^\star$, only the card can decompress without becoming the bottleneck (Section 3.8)
cache and prefetch	moves hB_e to a faster stage	routing locality exists (h measured, Chapter 7)
placement	f_V, f_R, f_D	free VRAM and RAM, measured after the system
activation sparsity	O_i, by a density factor	the model has sparse activations (ReLU); weak under SwiGLU (Chapter 8)
speculation	multiplies by $\mathscr{E}(\alpha, k)$	the verification pass reads barely more than a one-token pass; for an MoE, $\alpha > 1 - \rho$ (Proposition 4.8)

Table 2.3: What existing systems cover of the troncons problem, according to their publication and code [lit.]. "Disk": weights read from disk at every token; "16 GB": published or measured with less RAM than the model; "GPU decodes": decompression on the card of the bytes coming over the link.

System	Windows	no CUDA	disk	16 GB	GPU decodes	trellis
llama.cpp (`mmap`, `-ncmoe`)	yes	yes (Vulkan)	via mmap	yes	no	no
llama.cpp #25294	no	no	yes	no (128 GB)	no	no
moe-stream	yes	CPU only	yes	yes	no	no
SSD-LLaMA	no	no	yes	16–32 GB	after transfer	no
KTransformers	no	no	no	no	no	no
PowerInfer, LLM in a flash	partial	no	yes (flash)	yes	no	no
SpecExec, SubSpec	no	no	no (RAM)	no	no	no
EXL3, QTIP	no	no	no	—	yes (in VRAM)	yes
DirectStorage 1.4	yes	yes	yes	—	yes (textures)	no
troncons (target)	**yes**	**yes**	**yes**	**yes**	**yes**	**yes**

Chapter 3

The machine and its physics

Where we start, what this chapter establishes, what remains

At the start, we only had ceilings computed from data sheets (NVMe link 7.877 GB/s, PCIe 15.754 GB/s, VRAM 256 GB/s, RAM 41.6 GB/s) and a single number measured by another study, under load (PCIe 6.4 GB/s). **This chapter establishes** the real throughput of every stage on Stone's machine, with control cases and ABBA series, in two regimes (mains and battery), then proves the roof that follows: a lower bound on the time per token, its steady state, and two corollaries that decide the architecture. **What remains** is the RAM and llama.cpp measured on a quiet machine on mains power, the share of BitLocker in the disk ceiling, and a 4-bit Vulkan kernel at VRAM speed. Source: `fondations/01-machine.md` and `fondations/journaux/01-machine/`.

This chapter is the judge of the whole thesis. An idea about compression, caching or speculation is worth only what it gains on the slowest stage a token crosses; we therefore need to know, to within 1 % if possible, what each stage really delivers. We first describe the machine, then the measurement method, then each stage from the slowest to the fastest, and we end with the roof theorem.

3.1 The machine

Table 3.1: Stone's machine, survey of 2026-10-15 (WMI, nvidia-smi, FSCTL_MANAGE_BYPASS_IO).

Component	Survey
Processor	Intel i7-13620H: 6 performance cores, 4 efficient cores, 16 threads; L2 9.5 MiB, L3 24 MiB
RAM	**a single** 16 GiB Micron DDR5 module rated 5 600, configured at 5 200 MT/s (slot Controller0-ChannelA-DIMM0); ceiling $5\,200 \times 8 = 41.6$ GB/s [calc.]
Graphics card	NVIDIA RTX 4060 Laptop (AD107), 8 188 MiB of GDDR6 on 128 bits at 16 Gbit/s (memory clock read at 8 001 MHz), driver 591.91; PCIe 4.0 ×8 link at most
Disk	Samsung NVMe MZVMA1T0HCLD (1 TB), firmware MXE7001Q, **behind the Intel RST VMD controller** (driver `iaStor`); volume C: **encrypted by BitLocker**
System	Windows 11 Pro 10.0.26300; virtualization-based security (VBS) and memory integrity (HVCI) enabled
Tools	CUDA 13.1.2 unpacked without installation; wgpu 30.0.1 (Vulkan, DirectX 12); Rust 1.98.1; llama.cpp b11515

Two traits of this machine weigh on everything else. The first is the **single RAM module**: one 64-bit channel, half the bandwidth of a two-module laptop, and a RAM only 5.3 times faster than the disk link. The second is the **disk path**: every byte read crosses the BitLocker encryption filter and

Intel's VMD controller, two default settings of a consumer Windows laptop, which we will see cost more than half of the advertised throughput.

3.2 The measurement method

Every number in this chapter follows the same rules (Chapter A):

- **a control case**, i.e. a case whose answer is known in advance, forced to be wrong once to check that the instrument fails (`--fausser`);
- **a threshold declared before** the measurement, derived from a named quantity (`00-seuils-declares.md`);
- **ABBA series**: the list of configurations is measured in one direction then in the other; each cell is seen twice, in mirror order, slow drift cancels, and the gap $|A - B|$ relative to the mean serves as the reference noise;
- **an independent second path** for every number we depend on;
- **the power source** recorded at every pass (`Win32_Battery`, `BatteryStatus` = 2 on mains), since the discovery of Section 3.6.1.

A permanent bias, stated in advance. Other computations of the project were running on the machine during all these measurements (llama.cpp quantizations using up to 8.7 cores, perplexity runs on the card using up to 7.7 GiB of VRAM, free RAM between 1.0 and 4.4 GiB). Every pass logs that load. The "best" numbers of this chapter are therefore **lower bounds** of what the machine gives at rest; the medians say what it gives when shared.

3.3 The disk

3.3.1 The instrument

The `disque.exe` bench (Rust, `windows-sys`) reads a 32 GiB test file, twice the RAM, opened with `FILE_FLAG_NO_BUFFERING | FILE_FLAG_OVERLAPPED`, into buffers aligned by `VirtualAlloc`, with q reads in flight (the queue depth, QD) through a completion port (IOCP) or through IoRing. The file carries a verifiable pattern: every 4 KiB page starts with its number XORed with a constant, the rest is an xorshift64* stream seeded with that number. Every read checks the header of every page received, and one full page in 64. The sweep crosses blocks from 4 KiB to 16 MiB with depths 1 to 64, random and sequential, in ABBA order: 188 passes, then 40 passes of IOCP against IoRing.

The controls. Forced to be wrong (the expected value shifted by one page), the instrument counts 978 688 wrong pages out of 978 688; without the fault, 0 wrong pages on all passes. No unbuffered read exceeded the link (7.877 GB/s). The second path is the cumulative system counter (`Win32_PerfRawData_PerfDisk_PhysicalDisk`), read every 0.5 s: re-read through the Windows cache, 1 GiB is delivered at 1.65 GB/s while the disk reads only 3.4 % of those bytes; the cache is therefore visible when it plays, and absent from the unbuffered passes, where the disk sees at least 100 % of the bytes.

A crossed-out rule. The declared rule "the buffered engine re-reading 1 GiB must exceed 7.877 GB/s" was wrong: ~~the re-read cache exceeds the link~~. The trial delivered the cache at 3.1 GB/s with 2.98 s of CPU for a 3 s window: copying from the Windows cache is bounded by one CPU thread, not by the RAM. It was crossed out before the sweep and replaced by the stronger counter-based control.

3.3.2 Results

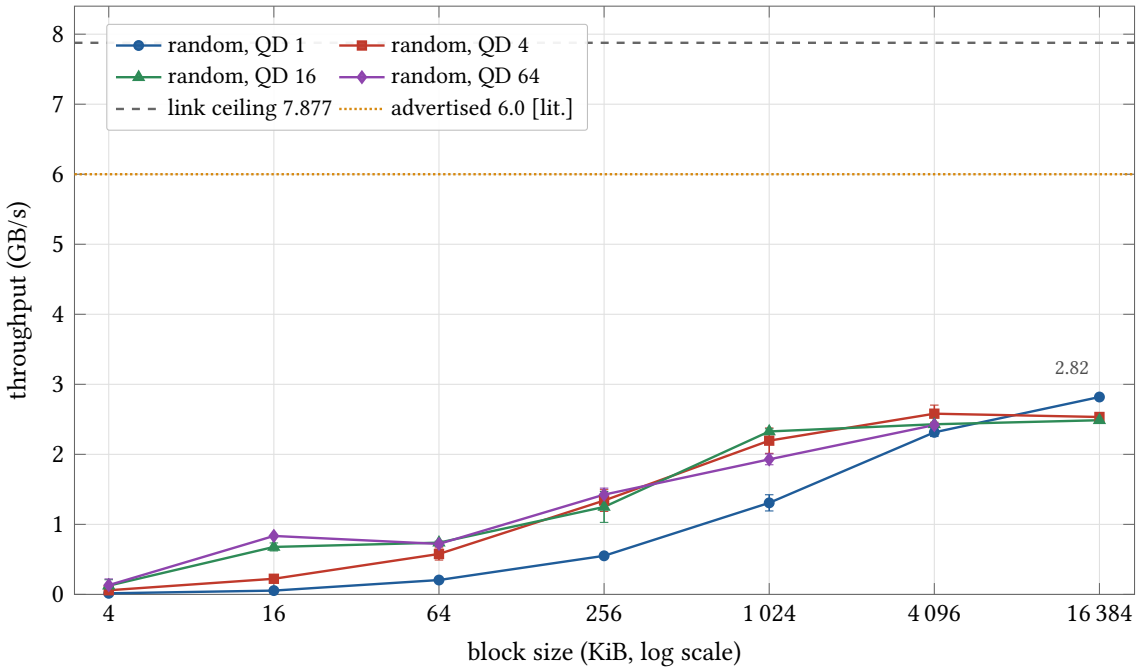


Figure 3.1: Disk throughput for unbuffered random reads, versus block size and queue depth [meas.]. Each point is the mean of passes A and B; the error bar is the half-gap $|A - B|/2$. The large-block plateau stays between 2.4 and 2.9 GB/s whatever the depth, at 36 % of the link and 47 % of the advertised rate.

Data: `fondations/journaux/01-machine/disque-balayage.csv` (94 cells), via `memoire/outils/donnees_figures.py`.

Table 3.2: Disk throughput in GB/s, mean of passes A and B; in brackets, the noise $|A - B|/\text{mean}$ [meas.].

block	rand. QD 1	rand. QD 4	rand. QD 16	rand. QD 64	seq. QD 1	seq. QD 16
4 KiB	0.015 [53 %]	0.059 [44 %]	0.125 [136 %]	0.132 [134 %]	0.046 [11 %]	0.188 [3 %]
64 KiB	0.205 [11 %]	0.578 [30 %]	0.739 [6 %]	0.721 [10 %]	0.390 [16 %]	2.309 [0 %]
1 MiB	1.307 [18 %]	2.195 [17 %]	2.327 [3 %]	1.928 [8 %]	1.599 [8 %]	1.900 [40 %]
4 MiB	2.314 [5 %]	2.581 [10 %]	2.430 [0 %]	2.418 [2 %]	1.041 [13 %]	1.631 [92 %]
16 MiB	**2.819 [1 %]**	2.533 [0 %]	2.487 [2 %]	—	2.340 [43 %]	2.206 [17 %]

What we draw from it (Figure 3.1 and Table 3.2):

- **The large-block plateau is at 2.4–2.9 GB/s**, whatever the depth. The best reproducible point is the 16 MiB random read at QD 1: **2.82 GB/s, noise 1 %**. Two cells reached 3.84 GB/s in pass A without reproducing in pass B; they are not retained as a ceiling.
- Small blocks are bounded by the processor: 4 KiB random tops out near 54 000 I/O per second at best, the median latency at QD 1 is 200 to 230 µs, and the bench spends about 4 s of CPU per GB read in 4 KiB pieces (16 µs per I/O), against 0.07 to 0.11 s/GB beyond 1 MiB.
- **IOCP and IoRing are indistinguishable**: over five cases in double ABBA, the gap (0 to 5 %) stays below the declared rule (at least twice the noise and 3 %). IoRing costs slightly less CPU on large blocks, with no throughput gain.

3.3.3 Why so far from the advertised rate

Samsung advertises 6 000 MB/s sequential reads and 900 000 IOPS for the PM9C1a [lit.] [76, 77]; identifying the MZVMA1T0HCLD part with that model is likely (MZVMA prefix), not proven. What we know, measured:

1. **BitLocker is active and its `fvevol` filter refuses BypassIO** (status `0xC00004C9`). Microsoft's documentation describes exactly this case [78]: every byte read goes through the software decryption of the volume.
2. **The storage driver is `iaStor` (Intel RST VMD)**, not `stornvme`; the same page reserves the full BypassIO path for StorNVMe.
3. The processor was loaded by other computations of the project; small, CPU-bound blocks suffer the most.

The shares of BitLocker, of VMD and of the disk itself cannot be separated without reading the disk unencrypted, which requires Stone to touch a security setting. Delegated answer of 2026-10-15, reversible: we keep BitLocker, the protection of his data if the laptop is stolen, and we design for 2.82 GB/s.

3.3.4 The cost of BitLocker, measured by a second path

We can, however, measure what the decryption that `fvevol` applies to every byte costs on this processor. The `aes_xts.py` tool calls the Windows CNG API (`bcrypt.dll`, algorithm `XTS-AES` [79]) on 4 KiB units, like a BitLocker sector, from 1 to 8 threads.

The mechanism. The XTS mode [80] encrypts the 16-byte block j of unit number i as

$$C_j = E_{K_1}(P_j \oplus T_j) \oplus T_j, \qquad T_j = E_{K_2}(i) \otimes \alpha^j \in \mathrm{GF}(2^{128}), \tag{3.1}$$

where E_K is AES under key K and α the primitive element of the field. Each block costs one AES (10 rounds for a 128-bit key, 14 for 256), executed by the AES-NI instructions, plus a multiplication by α (a shift and a conditional XOR). Blocks are independent: decryption parallelizes perfectly across units. Controls: vector 1 of annex B of IEEE 1619-2007 and the encrypt-decrypt round trip on 1 MiB; `--fausser` makes both fail.

On one thread, decryption runs at 3.2–4.2 GB/s, **the same order as the disk plateau**; on eight threads, 8.7 GB/s (Figure 3.2). If `fvevol` decrypts one request on a single thread, a QD 1 read is bounded by one core. But the plateau does not rise with depth while the processor can decrypt 8.7 GB/s: either `fvevol` serializes, or the bound is elsewhere (VMD, the disk, its thermal management). We cannot decide without reading without BitLocker; it is stated as a quantified hypothesis. What BitLocker costs the project in CPU time is, however, measured: 1/4.2 to 1/3.2, i.e. **0.24 to 0.31 core-seconds per GB read**. At 2.82 GB/s, that is 0.7 to 0.9 core busy at all times, which our own CPU-side decompression could not use.

3.3.5 IoRing, BypassIO, DirectStorage

IoRing is present in the kernel at maximum version 400, newer than the `windows-sys` headers; usable, with no measurable gain here [81]. BypassIO is refused on C:. DirectStorage does not ship with Windows: its SDK is a 2.8 MB NuGet package (version 1.3.0) or 4.0 MB (preview 1.4.0-preview2, which adds Zstandard decompressed on the GPU) [82, 83]. It is not measured here, and would run without BypassIO anyway.

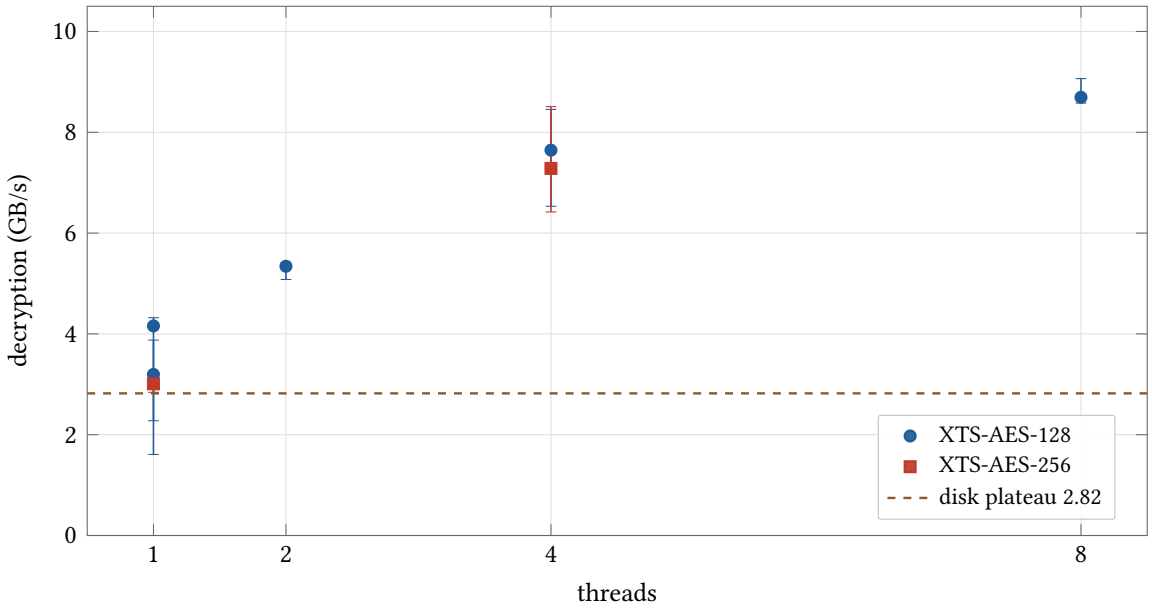


Figure 3.2: XTS-AES decryption through the Windows CNG API, 4 KiB units, on battery [meas.]. Point: median; bar: minimum and maximum of the series. Two single-thread passes (A then B) are plotted. On one thread, decryption runs at the pace of the disk plateau.

Data: `fondations/journaux/01-machine/aes-xts-batterie.log`.

3.4 The RAM

The `memoire.exe` bench measures reads (sum of 64-bit words, four accumulators), writes and copies, from 1 to 16 threads, on 512 MiB buffers (21 times the L3), and latency by pointer chasing on a Sattolo cycle, one hop per 64-byte line. Control: the sum of the pattern is known, $n(n-1)/2$.

What the first measurement taught the instrument. The first measurement (machine heavily loaded) gave 15.6 GB/s for a single-thread read, then collapsed to 2–4 GB/s for every thread count. With 1 to 2 GiB of free RAM, Windows trims pages from the bench's working set and stores them in its compression store: a "RAM" read then pays a page fault and a decompression. The bench now raises its working-set floor, **locks its buffers** (`VirtualLock`) and counts page faults. New control, declared before: fewer than 1 % of the pages read faulted. Forced to be wrong (a fresh buffer every pass), it fails: 197 033 faults for 196 608 pages read. Locked: 15 to 267 faults, control held.

The locked measurement, made on battery, has an ABBA throughput noise above 100 %: nothing can be taken from it as a ceiling. Only two facts (Figure 3.3): the latency of a dependent access is 0.75 to 0.96 µs beyond the L3, with no page fault; and 16 threads read more slowly than one. A DDR5 DRAM answers in about a hundred nanoseconds; a TLB miss costs a few hundred more when translation is *nested*, which is the case here since VBS and HVCI are running (each miss walks two page tables). Added to a processor at 1.5 GHz on battery, this is the most likely explanation: **a hypothesis, not proven** (it would take large pages, which require an administrator privilege). The best CPU-to-RAM throughput remains 15.6 GB/s (one thread, first pass, control held), 37 % of a module's ceiling; the measurement on a quiet machine on mains power is pending.

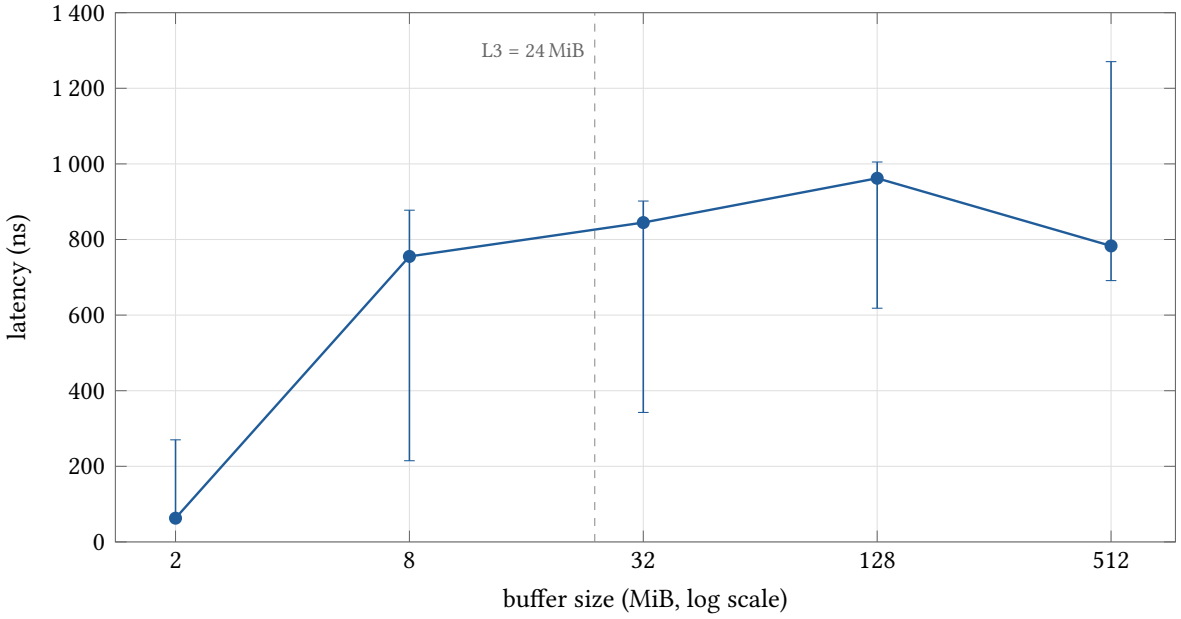


Figure 3.3: Latency of a dependent access (pointer chasing), locked buffers, no page fault, on battery [meas.]. Point: median; bar: minimum and maximum. Beyond the L3, an access costs 0.75 to 0.96 µs, seven to ten times a DRAM latency.

Data: `fondations/journaux/01-machine/memoire-batterie-verrou.log`.

3.5 From host to device

3.5.1 The instrument, and the trap the control uncovered

The `carte.exe transfert` bench (wgpu 30.0.1 [84], Vulkan and DirectX 12) measures in ABBA order, with GPU timestamps: true DMA from a buffer placed in host RAM to VRAM; the processor writing into a mapped buffer; the naive `queue.write_buffer` write; device-to-host and device-to-device copies. Control: byte-for-byte round trip.

A discovery by the ceiling control. The first trial gave "host to device" at 80–220 GB/s: the control "never above the link" flagged it as false. The reason: **under Vulkan, the driver places write-mapped buffers in CPU-visible VRAM (resizable BAR)**; the measured copy was a VRAM-to-VRAM copy, and the real crossing of the link had happened while the processor wrote into the mapping. Under DirectX 12, the same buffer lives in host RAM (copy at 12.2 GB/s). An exception confirms it: when the VRAM was almost full, the buffer fell back to host RAM and the copy gave 7.9 GB/s, below the ceiling.

3.5.2 Results

DMA from host RAM reaches **12.2 GB/s, 77 % of the** PCIe 4.0 ×8 link (Table 3.3). One sentence was crossed out during the study: ~~at 256 MiB, DMA drops to 4.4–6.3 GB/s, a size effect~~. The size sweep gave 3.2–4.3 GB/s at *every* size, 64 MiB included, then three series at 64 MiB came back to 12.1–12.2 GB/s a quarter of an hour later: the variable is time (the load of other computations on the card), not size. Independent second path: the loading study measured 6.4 GB/s through the CUDA runtime in pinned memory, under load; it falls in the range of the loaded series, and 6.4 was not a CUDA ceiling but a state of the machine.

Table 3.3: Transfers between host and device, 64 MiB [meas.].

Operation	quiet machine	under load	remark
host → device, DMA from host RAM	**12.10 / 12.24 / 12.20**	3.2 to 6.3	77 % of the link
CPU → BAR (Vulkan)	9.1	0.45 to 4.5	wall clock, mapping included
`write_buffer` (Vulkan)	7.5 to 8.2	0.1 to 4.3	one write per submission
device → host	6.2 to 9.4	3.3 to 6.8	

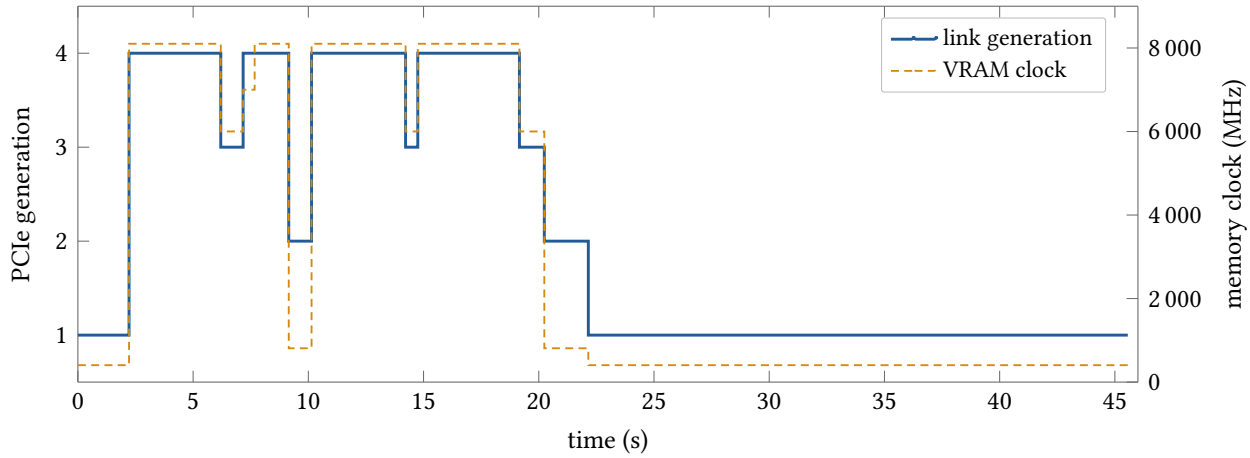


Figure 3.4: PCIe link generation and VRAM clock read by nvidia-smi every 0.1 s during three series of copies [meas.]. The link sleeps in generation 1 and only climbs to generation 4 under load; 216 readings in generation 1, 118 in generation 4.

Data: `fondations/journaux/01-machine/carte-lien-pcie.csv`.

The link changes generation. Read during copies (Figure 3.4), the link is in generation 1 at rest, generation 4 under load, generation 3 on battery. The low level of about 3.8 GB/s of the slow series resembles the ceiling of a generation 2 ×8 link (4 GB/s); not proven. **Every transfer measurement must record the link generation during the measurement, not before.** The loading study saw it too (Chapter 7): a pipeline that only copies leaves the link in generation 3; the card must compute while we copy.

3.6 The card

VRAM. A read shader (sum of `vec4<u32>` over 512 MiB, sum checked against the CPU) reaches **251.3 GB/s, 98.2 % of the** 256 GB/s ceiling. Below 32 MiB, copies exceed 256 GB/s: that is the L2 cache, expected, and it is the positive control of the cache.

Compute. In independent chains, timed by the GPU (Vulkan): f32 FMA up to 12.6 Tflop/s, f16 13.4, `dot4I8Packed` 12.3 Top/s, i.e. 77 % of the computed ceiling (24 SMs × 128 lanes × 2 × 2.67 GHz = 16.4 Tflop/s). A 4-bit decode (shift, mask, convert, FMA) from registers runs at **2.61 Top/s, i.e. 1.31 trillion weights per second**: 2.6 times what the VRAM delivers at 4 bits. The controls compare against a CPU reference running the same sequence (gap 0, 0 and $4 \cdot 10^{-7}$); forced to be wrong, they fail.

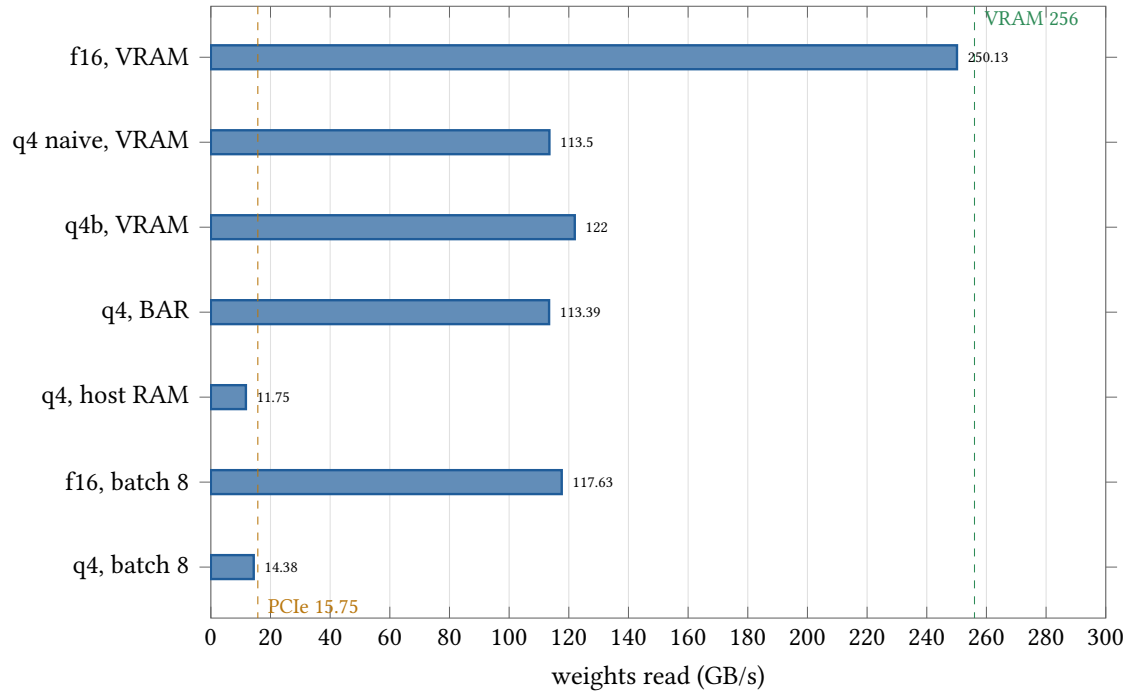


Figure 3.5: Matrix–vector products written here (Vulkan, RTX 4060), best GPU-timed median, control held [meas.]. f16 reaches the VRAM; 4-bit tops out at 122 GB/s with these kernels; with weights left in host RAM (zero copy), the card computes at the link's throughput.

Data: `fondations/journaux/01-machine/carte.jsonl` and `carte-complement.jsonl`.

The matrix–vector product. It is the heart of a decoded token: each weight matrix multiplies a single activation vector, each weight is used once, and the kernel is memory-bound. Figure 3.5 yields four facts. f16 reaches the VRAM (250 GB/s): the machine can read its 251 GB/s in a real computation. 4-bit tops out at 122 GB/s while decoding from registers costs 2.6 times less than what the VRAM delivers: the shortfall is in the kernel (access to the scales and the vector, memory parallelism), not in the machine. **With zero copy**, the card computes a GEMV whose weights stay in host RAM at 11.75 GB/s, the DMA throughput: for a model larger than the VRAM, weights can be consumed straight from pinned RAM, with no staging buffer. Finally, batches of 8 vectors collapse with these kernels, which re-read the vector for every block: the kernel must be rewritten before any use in speculative decoding.

3.6.1 Thermals, and the discovery of the battery

The rule declared before the thermal pass says "throttling if the last 30 seconds fall below 95 % of the first 30". On mains, it holds (ratio 1.001, noise 6.3 %). This test is not a maximal load: the loop submits 20 GEMVs then waits, the card is about 70 % busy; it is a decoding load, the project's.

The second pass, at 07:25, revealed that **the machine was on battery**, and that **on battery the machine is another machine**: the driver lowers the card's power limit from 80 W to 52.6 W, the card memory stays at 6 001 MHz (25 % less VRAM throughput, 192 GB/s at most [calc.]), the link drops to generation 3, the processor runs near 1.5 GHz. On short measurements, host-to-device DMA stays at 11.8 GB/s, but CPU writes into the BAR fall from 9.1 to 1.7 GB/s and `write_buffer` from 7.5 to 1.25 GB/s: **everything that goes through the processor collapses**. For the project, a weight loader that relies on the processor loses a factor of 4 to 6 on battery; a loader that only does DMA and

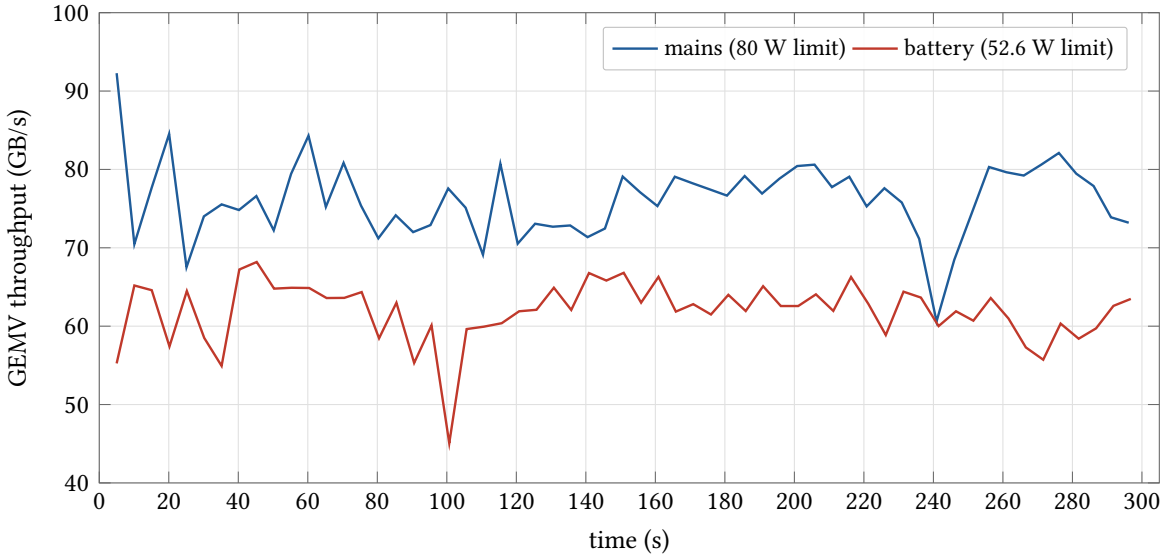


Figure 3.6: Five minutes of q4b GEMV in a loop, throughput per 5 s slice [meas.]. On mains: end/start ratio 1.001, no throttle reason, 2 685 MHz, 75 °C, 72 W. On battery: the card is throttled from the first second (enforced limit 52.6 W, memory at 6 001 MHz).

Data: `fondations/journaux/01-machine/thermique-secteur-charge.csv`, `thermique-batterie.csv`.

decodes on the card loses little. It is a measured argument for architecture B (Section 3.8).

3.7 The baseline: llama.cpp

The `pilote_llama.py` driver runs `llama-bench` [85] (b11515, CUDA 12.4 backend) on Qwen3.5-9B Q4_K_M (5 680 522 464 bytes, 8 953 803 264 parameters), generation only, one configuration per process, in ABBA order, and records during the life of the process the peak memory, the page faults and the VRAM.

Table 3.4: llama.cpp b11515 on Qwen3.5-9B Q4_K_M, first pass (3 configurations out of 12, shared machine) [meas.].

`-ngl`	loading	tok/s	s.d.	working set	added VRAM	faults
99 (all in VRAM)	mmap	16.5	1.2	5 279 MiB	5 118 MiB	1 376 607
99	direct read	**34.1**	5.4	**964 MiB**	5 083 MiB	270 240
24 (of 32 layers)	mmap	**2.8**	2.1	5 296 MiB	7 114 MiB	1 563 456

Cautious reading (Table 3.4): 34.1 tokens/s all in VRAM is 69 % of the roof of 49.3 (Section 3.8), i.e. $34.1 \times 5.10 = 174$ GB/s of weights read [calc.], better than our 122 GB/s. The gap between mmap and direct read (16.5 against 34.1) does not come from loading (the weights are in VRAM in both cases): another process took VRAM during the measurement (the `-ngl 24` pass adds more VRAM than the all-in-VRAM pass). We keep only the order of magnitude; the complete ABBA awaits a quiet machine on mains power. What is certain: with eight of 32 layers left on the processor, generation drops by a factor of 12.

3.8 The roof

3.8.1 The model

A decoded token, at batch size one, reads B bytes of weights: every weight it uses, once each. The weights reside on three stages: a fraction f_V in VRAM, f_R in host RAM, f_D on disk ($f_V+f_R+f_D=1$), the disk part stored compressed by a factor r (bytes served over bytes stored). Each stage i has a measured effective throughput W_i: $W_{\text{VRAM}} = 251.3$, $W_{\text{PCIe}} = 12.2$, $W_{\text{disk}} = 2.82$ GB/s (`debits-mesures.json`, each value with its log). Figure 3.7 summarizes this path, Figure 3.8 the efficiency of each stage.

We distinguish two architectures (Figure 3.9), according to where the bytes coming from the disk are decompressed:

Architecture A (the processor decompresses, PCIe carries the served bytes): $O_{\text{VRAM}} = f_V B$, $O_{\text{PCIe}} = (f_R + f_D)B$, $O_{\text{disk}} = f_D B/r$, and the CPU decoder must output $f_D B$ bytes.
Architecture B (the card decompresses, PCIe carries the compressed bytes): $O_{\text{VRAM}} = f_V B$, $O_{\text{PCIe}} = (f_R + f_D/r)B$, $O_{\text{disk}} = f_D B/r$, and the GPU decoder must output $f_D B$ bytes, preferably in registers, without writing them back to VRAM.

3.8.2 The roof theorem

We model a stage by its throughput constraint alone.

Assumption 3.1 (Throughput-bounded stage)**.** Over any time interval of length τ, stage i moves at most $W_i\tau$ bytes. A token is finished when every stage has moved its O_i bytes.

Theorem 3.2 (Throughput bound of a chain of stages)**.** *Under assumption 3.1, whatever the schedule, the time t of one token satisfies*

$$t \;\geq\; \max_i \frac{O_i}{W_i}, \qquad \textit{hence} \qquad \textit{tokens/s} \;\leq\; \min_i \frac{W_i}{O_i}. \tag{3.2}$$

If the stages work one after another without overlap, $t = \sum_i O_i/W_i$.

Proof. Let $[0, t]$ be the interval during which the token is processed. By assumption 3.1, stage i moves at most $W_i t$ bytes in that interval; it must move O_i, hence $W_i t \geq O_i$, i.e. $t \geq O_i/W_i$, for every i. The bound holds for the maximum. Without overlap, the activity intervals of the stages are disjoint and each lasts at least O_i/W_i; if they work at their throughput, each lasts exactly O_i/W_i, and the durations add up. ∎

For a dense model read entirely through one stage, $B = Nb/8$ and the theorem gives the form used throughout the thesis:

$$\text{tokens/s} \;\leq\; \frac{8\,W}{N\,b}. \tag{3.3}$$

Its slope in b is $\partial \ln(\text{tokens/s})/\partial \ln b = -1$: halving the number of bits per weight doubles the ceiling, at any stage. No other parameter of the chain has this slope without a counterpart.

Is the bound attainable? Yes, up to a filling penalty that can be quantified. Split the token into L equal pieces (the layers) that each stage processes in order; piece ℓ can only enter stage $i+1$ once it has left stage i (double buffering between stages). This is a permutation flow shop with identical jobs.

Lemma 3.3 (Flow shop with identical jobs)**.** *Let there be m stages in series and L identical jobs, the job requiring p_i seconds at stage i, each stage processing the jobs in order, one at a time. The total duration is*

$$T_L \;=\; \sum_{i=1}^{m} p_i \;+\; (L-1)\,\max_i p_i. \tag{3.4}$$

Proof. Let $C_{i,\ell}$ be the instant job ℓ leaves stage i. The constraints give the recurrence $C_{i,\ell} = \max(C_{i-1,\ell}, C_{i,\ell-1}) + p_i$, with $C_{0,\ell} = C_{i,0} = 0$. Unrolled, $C_{m,L}$ is the maximum, over all paths from cell $(1,1)$ to cell (m,L) that advance by one stage or one job at each step, of the sum of the p_i of the visited cells. Such a path visits $m + L - 1$ cells and passes at least once through every stage; its sum is $\sum_i p_i$ plus the $L - 1$ extra cells, each at most $\max_i p_i$. The path that goes down all the stages while spending its $L - 1$ extra cells on the slowest stage attains this bound. ■

Corollary 3.4 (Steady state). *With L equal pieces, $p_i = O_i/(LW_i)$ and the time per token is*

$$t_L = \max_i \frac{O_i}{W_i} + \frac{1}{L}\Big(\sum_i \frac{O_i}{W_i} - \max_i \frac{O_i}{W_i}\Big). \tag{3.5}$$

It tends to the bound of theorem 3.2 as L grows, and the filling penalty is at most $1/L$ of the gap between the sum and the maximum. If the weights of the next token can be read during the computation of the current one (dense model, large enough prefetch window), filling is paid only once and the time per token tends to $\max_i O_i/W_i$.

Proof. Substitute p_i in Equation (3.4) and divide by the number of tokens (one). ■

For gpt-oss-120b, $L = 36$ layers: even without overlap between tokens, the time per token is within 3 % of the sum–maximum gap above the bound. Figure 3.10 shows the two regimes.

3.8.3 The two corollaries that decide the architecture

Corollary 3.5 (The crossing factor). *In architecture A, with all non-VRAM weights on disk ($f_R = 0$), the disk stops being the limiting stage as soon as*

$$r \geq r^\star = \frac{W_{\text{PCIe}}}{W_{\text{disk}}} = \frac{12.2}{2.82} = 4.33 \quad \textit{[calc.]}. \tag{3.6}$$

Beyond it, compressing further is useless in A. With a share f_R in RAM, the threshold becomes $r^\star f_D/(f_R + f_D)$, smaller.

Proof. In A, $O_{\text{disk}}/W_{\text{disk}} = f_D B/(rW_{\text{disk}})$ and $O_{\text{PCIe}}/W_{\text{PCIe}} = (f_R + f_D)B/W_{\text{PCIe}}$. The disk is no longer limiting when the first term does not exceed the second, i.e. $r \geq W_{\text{PCIe}} f_D/(W_{\text{disk}}(f_R + f_D))$, which equals $r^\star$ for $f_R = 0$. Beyond, the maximum of theorem 3.2 is the PCIe term, which does not depend on r. ■

Corollary 3.6 (In architecture B, no crossing). *In architecture B with $f_R = 0$, the disk remains the limiting stage for every r, and the ceiling of the disk stream grows as $r\,W_{\text{disk}}/(f_D B)$ until the GPU decoder or the VRAM bounds it.*

Proof. In B, disk and PCIe both carry $f_D B/r$; the ratio of their times is $W_{\text{PCIe}}/W_{\text{disk}} = 4.33 > 1$, independent of r: the disk is always the slower of the two. Its time $f_D B/(rW_{\text{disk}})$ decreases as $1/r$, which gives the stated ceiling, until another term (the decoder, the VRAM) exceeds it. ■

The upper bound of the GPU decoder is read from the measurement of Section 3.6: 1.31 trillion weights per second decoded from registers, i.e. $1.31 \cdot 10^{12} \times 5.08/8 = 832$ GB/s served [calc.]. It is not an entropy-decoding bound (Chapter 10 returns to it), but it says that, on this card, compute is not the limiting stage.

Table 3.5: Tokens-per-second ceilings if all weights read go through the named stage [calc.] on measurements. B: bytes read per token (for MoE models, always-read plus experts, uniform routing).

Model	B (GB)	VRAM	PCIe	disk, $r = 1$	disk, $r = r^\star$
Qwen3.5-9B Q4_K_M	5.10	**49.3**	2.39	**0.55**	2.39
Qwen3.5-4B Q4_K_M	2.73	92.1	4.47	1.03	4.47
gpt-oss-120b MXFP4	3.60	69.8	3.39	0.78	3.39
Qwen3.8-Flash-Next UD-Q2_K_XL	4.44	56.6	2.75	0.64	2.75
Kimi-K2.6 UD-Q2_K_XL	15.42	16.3	0.79	0.18	0.79

Data: `fondations/journaux/01-machine/toit-modeles.csv`.

3.8.4 Ceilings by stage

The real placement (Figure 3.12) assumes, like the model study (Chapter 9), 6.5 GB of VRAM and 11 GB of RAM available for weights, always-read tensors first in VRAM, experts spread in the same order, uniform routing and no dynamic cache: it is the pessimistic bound. For gpt-oss-120b, it gives **1.98 tokens/s** overlapped (1.53 in series): 1.84 GB per token served by the VRAM, 0.34 by the RAM, 1.42 by the disk.

Second path, and what it corrects. The tool of the model study (`modeles_physique.py`) gives 5.1 to 5.6 tokens/s for gpt-oss-120b. Redone with its own assumptions, the roof tool finds the same order (1.42 GB per token at 7.877 GB/s, i.e. 5.5 tokens/s): the two models agree. The gap comes from the disk throughput, the link ceiling on one side, the measurement on the other: **the real ceiling of gpt-oss-120b on this machine, without cache and without compression, is about 2 tokens/s, not 5.5.** The judge of the loading study, rerun with the measured disk, finds 1.99 tokens/s by a third path (Chapter 7).

3.8.5 The decoder budget, and why B

At a compressed input rate equal to the link (12.2 GB/s, architecture A at the crossing), the card has $2.61 \cdot 10^{12} / 12.2 \cdot 10^{9} \approx$ **214** operations per compressed byte [calc.]; at the disk rate (architecture B), **926**. An entropy decoder of the rANS family performs on the order of ten operations per symbol: compute is not the bound; the question is the parallelism and memory access of the decoder.

Three measured reasons to prefer architecture B: in A, any factor beyond 4.33 is lost (corollary 3.5); in A, the processor already pays 0.24 to 0.31 core-seconds per GB for BitLocker before any decompression (Section 3.3.4); on battery, everything that goes through the processor loses a factor of 4 to 6 while DMA stays at 11.8 GB/s (Section 3.6.1). And the card can consume weights without copying them into VRAM (zero copy, 11.75 GB/s).

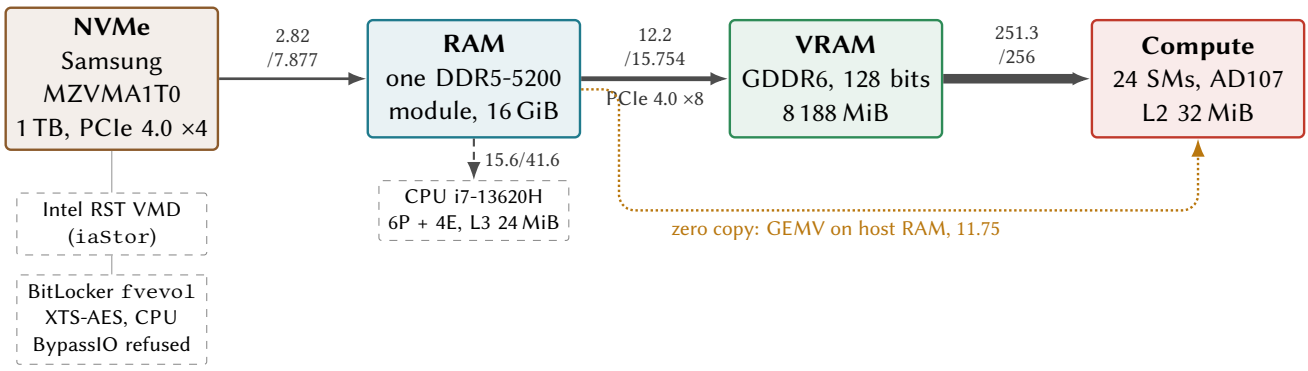
What $r^\star$ means in bits. Q4_K_M serves 5.08 bits per weight on average; stored at $r^\star = 4.33$, it would fit in $5.08/4.33 =$ **1.17** bits per weight on disk. ~~i.e. about 1.05 bits per stored weight for 4.5 bits served~~ (crossed out in the original study: Q4_K_M is not at 4.5 bits, it mixes Q6_K tensors). No lossless code gets there on 4-bit codes, whose measured entropy leaves a few per cent: $r^\star$ is reachable only by changing the representation (Chapter 5). On a large MoE, compression is not the only lever anyway: the hit rate of the expert cache multiplies the effect of r (Chapters 7 and 9).

3.9 What is missing in the world

1. **A measured, not advertised, roof for out-of-memory inference on a laptop.** Work on inference from storage reasons with data-sheet or workstation throughputs. Here, the disk delivers 47 % of its advertised rate because of two default settings; a ceiling computed at the link is wrong by a factor of 2.8.
2. **The battery regime.** No inference benchmark we read records the power source; here it changes the card's power limit, its memory clock, the link generation, and divides by 4 to 6 everything that goes through the processor.
3. **The CPU cost of BitLocker for a weight stream**, 0.24 to 0.31 core-seconds per GB, and the open question of whether `fvevol` parallelizes.
4. **GPU-side decompression fed by DMA from pinned RAM or by zero copy, under Vulkan.** DirectStorage decompresses on the GPU, but into Direct3D 12 resources and for textures; inference engines decode their formats on the GPU, but from weights already in VRAM. The path "disk → pinned RAM → PCIe (compressed bytes) → decoding on the card" exists, to our knowledge, nowhere under Vulkan.
5. **A 4-bit Vulkan kernel at VRAM speed**: our kernels top out at 48 %, llama.cpp under CUDA reaches 69 %; the machine offers 98 %.

What could be wrong

- **The machine was never quiet.** The "best" numbers are lower bounds of the rest state; the disk ceiling is the most exposed (up to 0.6 GB/s of foreign reads during pass A).
- **The roof assumes independent throughputs.** The RAM is shared between DMA to the card, disk reads and the processor, on a single module. Perfect overlap may be unattainable; the serial column brackets from below.
- $B = 5.10$ **GB per token** ignores the KV cache and activations: right for a short context, too low for a long one.
- **Identifying the disk with the PM9C1a** is likely, not proven.
- **The RAM latency attributed to nested translation** is a hypothesis.
- **The llama.cpp measurements are contaminated** by another load on the card; the mmap / direct-read gap is not a fact.
- **The thermal test is not a maximal load.**
- **The decoder bound (832 GB/s served)** measures a decode of 4-bit fields; a real entropy decoder has other bottlenecks.
- **BitLocker's XTS key size** (128 or 256 bits) is unknown; the gap is about 10 %.



Bold: best reproducible throughput measured here (GB/s). After the slash: computed ceiling of the stage.

Figure 3.7: The data path on Stone's machine, from disk to compute units. Each arrow carries the best reproducible throughput measured here and the computed ceiling of the stage; thickness grows with throughput. The dotted path is "zero copy": the card computes directly on weights in host RAM, across the link.

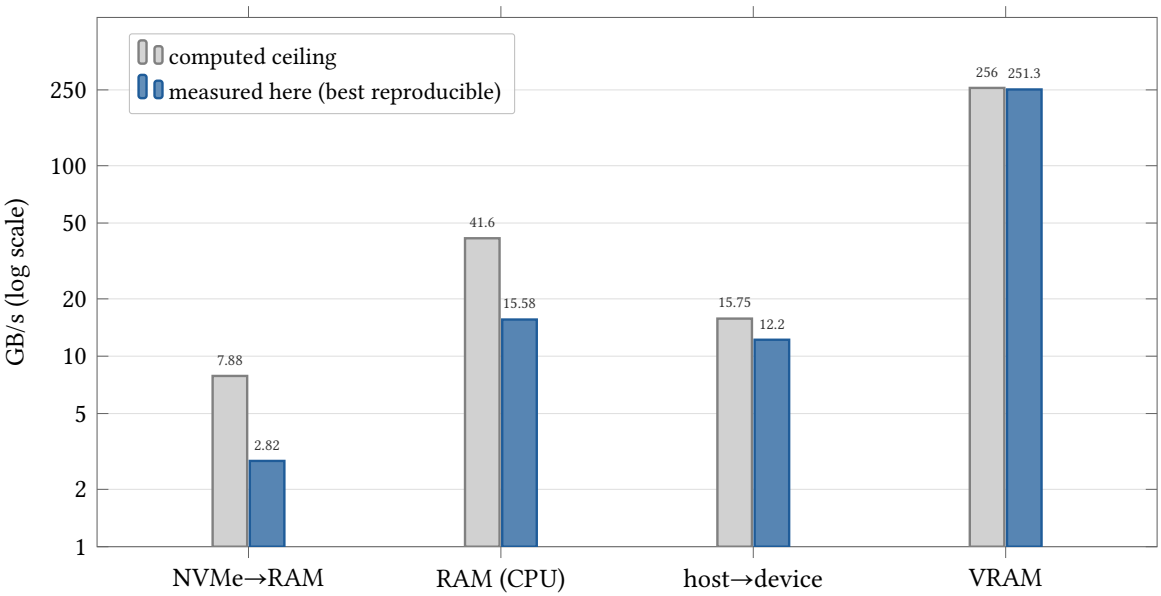
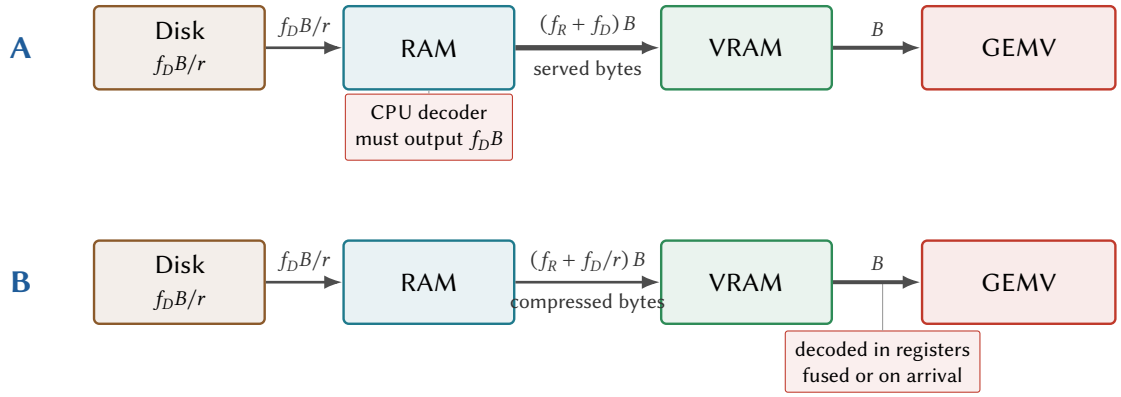


Figure 3.8: Computed ceiling versus measured throughput, by stage (log scale) [meas.] [calc.]. The disk delivers 36 % of its link, the RAM 37 % of one module, DMA 77 % of the PCIe link, the VRAM 98 %.

Data: `fondations/journaux/01-machine/debits-mesures.json`, `memoire.log`.



In A, any factor r beyond $r^\star = W_{\text{PCIe}}/W_{\text{disk}} = 4.33$ is lost: the link carries the served bytes. In B, disk and link both carry $f_D B/r$; the roof grows with r until the GPU decoder or the VRAM bounds it (corollary 3.6).

Figure 3.9: The two architectures. In A, the processor decompresses and the link carries the served bytes; in B, the card decompresses and the link carries the compressed bytes. The quantities on the arrows are bytes per token.

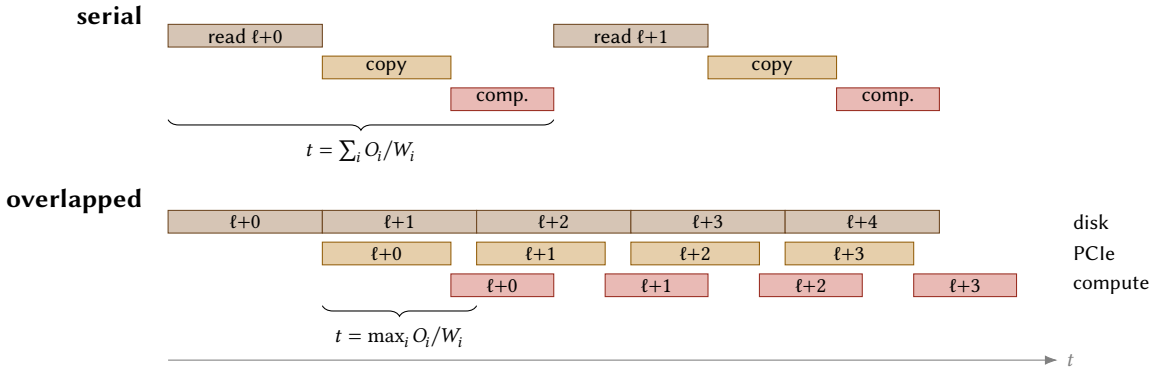


Figure 3.10: Timing diagram of the stages for a sequence of layers. In series, the time is the sum of the stages; with double buffering, each stage works on a different layer and the slowest one sets the pace (lemma 3.3 and corollary 3.4).

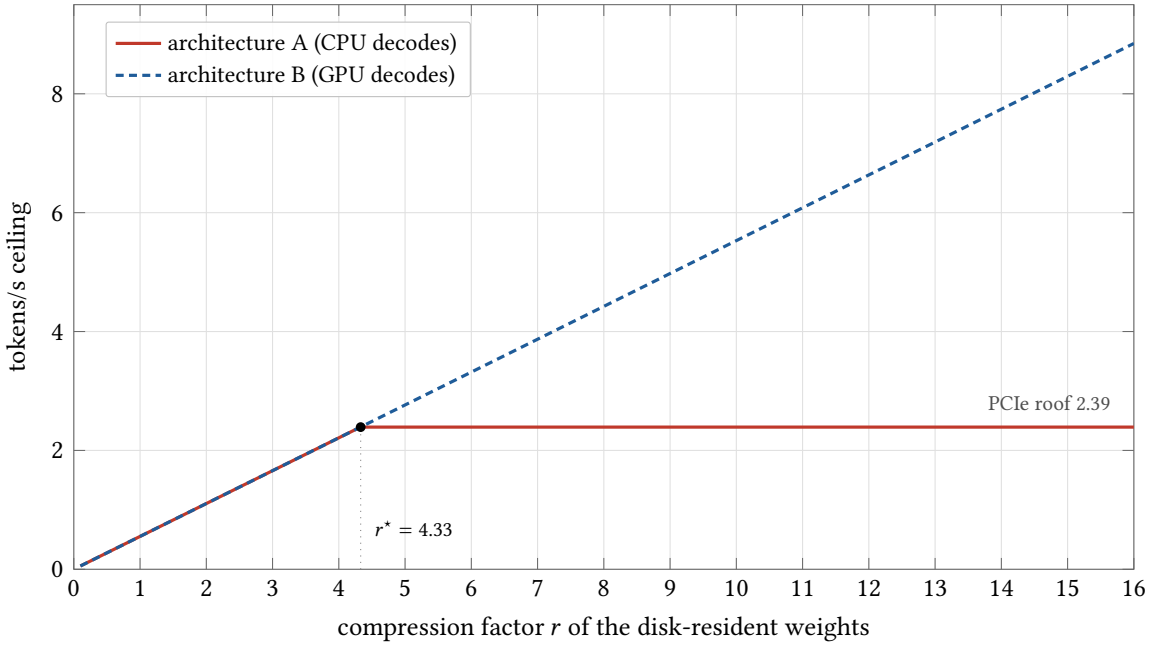


Figure 3.11: Tokens-per-second ceiling of Qwen3.5-9B Q4_K_M ($B = 5.10$ GB per token) entirely on disk, versus the compression factor r [calc.]. In architecture A, any factor beyond $r^\star = 4.33$ is lost; in architecture B, the ceiling keeps growing.

Data: `fondations/journaux/01-machine/toit-9b-compression.csv` (tool `outils/banc/toit.py`).

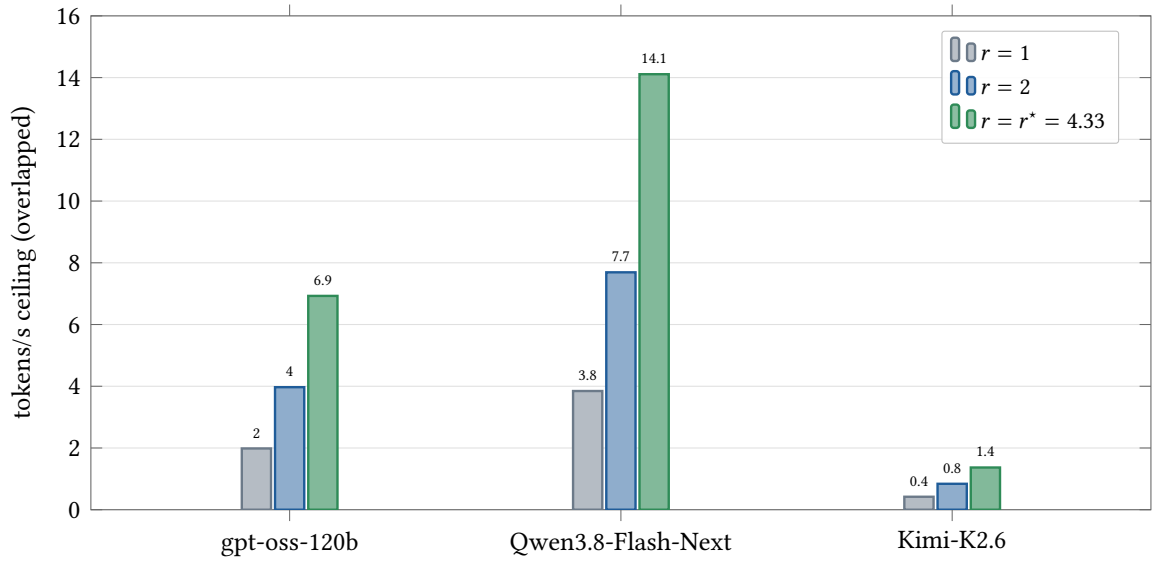


Figure 3.12: Ceiling (overlapped stages) of the three target MoE models on the machine, architecture A, uniform routing, no cache, for three compression factors of the disk part [calc.]. The no-overlap floor is in the source file.

Data: `fondations/journaux/01-machine/toit-placements.csv`.

Chapter 4

Problem formulation

Where we start, what this chapter establishes, what remains

Where we start: one theorem and four levers, each measured on its own: the throughput bound of a chain of stages (theorem 3.2), then quantization, loading, speculation and the target models (Chapters 5 and 7 to 9). **This chapter establishes** a single equation that contains them all, the time of one token of a mixture of experts served by the GPU, term by term; a one-constant locality model with its three scenarios fitted on the literature; the optimization problem troncons must solve, with its memory and quality constraints; the elasticity of the ceiling to each quantity, proven, which says which ones matter and which ones are ornament on this machine; a worked example; the term that was missing from the condition for speculation; and the witnesses that make the equation fall when it is falsified. **What remains** is to replace the locality model by the measured curve of gpt-oss-120b, and the estimated baseline by the measured one (Chapter 10). Source: `fondations/07-ce-qui-manque.md` (§ 1 and appendix A), tool `outils/07-synthese/plafonds.py`, log `fondations/journaux/07-synthese/plafonds.json`.

4.1 The token as a chain of stages

A decoding token, at batch size one, applies every active weight once. Its bytes cross the chain of Figure 3.7: disk, RAM, PCIe link, VRAM, compute units. Theorem 3.2 bounds its time by the largest stage time, and corollary 3.4 says that this maximum is reached up to a fill penalty. Three terms are still missing to describe a real engine; the studies met them one at a time.

Definition 4.1 (Token equation)**.** The time of one produced token is

$$t \;=\; \frac{1}{\mathscr{E}}\Big[\max\big(T_{\mathrm{D}},\, T_{\mathrm{P}},\, T_{\mathrm{G}}\big) + t_{\mathrm{lat}}\Big] \;+\; t_0, \tag{4.1}$$

where T_{D}, T_{P}, T_{G} are the busy times of the disk, the link and the GPU (Section 4.2), and:

- t_0 is the fixed cost of a token (kernel launches, synchronizations, recurrent layers);
- $t_{\mathrm{lat}} \geq 0$ is the read latency that prefetching did not hide; it is zero if the disk queue never runs dry;
- $\mathscr{E} \geq 1$ is the mean number of tokens returned by one read of the weights (theorem 8.3); $\mathscr{E} = 1$ without speculation.

Where each term comes from. $t_0 \approx 3.5$ ms [calc.] from a measurement of the quantization study: Qwen3.5-4B Q4_K_M fully in VRAM returns 68.9 to 69.9 tokens/s depending on the ABBA series, i.e. 14.3 to 14.5 ms per token, whereas its 2.73 GB read at 251 GB/s only need 10.9 (`fondations/journaux/02-quantification/vitesse-4B-3.json`, field `reference_jetons_s`). The term t_{lat} is the gap between the ceiling (the maximum) and the floor (the sum) of theorem 3.2; it can dominate: in Edge0, the time blocked on reads ranges from 101.9 to 575 ms per step while the disk works at most

12 % of the step [lit.] (Chapter 7). The factor $\mathscr{E}$ is that of Chapter 8; for a mixture of experts, the read of a verification pass grows with the number of verified tokens, which Section 4.9 handles.

4.2 The terms for a mixture of experts served by the GPU

We take the quantities of Chapter 7: B_t always-read bytes (attention, routers, norms, head), resident in VRAM; B_e routed-expert bytes read per token; E_{tot} bytes of all experts. A share h_v of the expert bytes is served by the VRAM, h_r by the RAM, $h_s = 1 - h_v - h_r$ by the disk. The bytes that cross the disk and the link are compressed by factors r_d and r_p (bytes served over bytes carried). The GPU decodes at W_{dec} (bytes served produced per second) and computes the matrix-vector product at W_g on decoded weights. Finally $\delta = \max(0, B_t - V)$ is the share of the always-read bytes that does not fit in the usable VRAM V.

Proposition 4.2 (Stage times of a MoE whose GPU decodes everything)**.** *If the GPU computes every layer and decodes on arrival everything that does not come from its own memory, the stage times of Equation (4.1) are*

$$T_{\text{D}} = \frac{h_s\, B_e}{r_d\, W_{\text{D}}}, \tag{4.2}$$

$$T_{\text{P}} = \frac{1}{W_{\text{P}}}\Big(\frac{h_r\, B_e}{r_p} + \frac{h_s\, B_e}{r_d} + \delta\Big), \tag{4.3}$$

$$T_{\text{G}} = \frac{(1 - h_v)\, B_e}{W_{\text{dec}}} + \frac{B_t + B_e}{W_g}. \tag{4.4}$$

Proof. The disk only reads the expert bytes absent from VRAM and RAM, that is $h_s B_e$ bytes served, carried as $h_s B_e / r_d$ bytes; hence Equation (4.2) by assumption 3.1. The link carries everything that enters the GPU: the experts coming from RAM (compressed by r_p), those coming from disk (compressed by r_d, since they are only decoded on arrival) and the overflow of always-read bytes. The GPU decodes everything that was not already decoded in its memory, $(1-h_v)B_e$ bytes served, then reads each active weight once, $B_t + B_e$ bytes, for the product. Decoder and product occupy the same multiprocessors: their times add instead of overlapping. ■

Remark 4.3 (Three meanings of the word "architecture"). In Section 3.8, architecture A is the one where the CPU decompresses, architecture B the one where the GPU decompresses; corollary 3.6 settled for B there. Proposition 4.2 is written for that architecture B. The three *candidates* A, B and C of Chapter 10 are all built on it and differ along another axis: the precision served at each stage. We keep the names used by the studies so as not to break their cross-references.

The disk and the link overlap with the GPU: their copy engines are free during compute, and the measured hidden share (Equation (7.1)) is 0.98 to 1.01 [meas.] (`fondations/journaux/04-chargement/`). This is what licenses the maximum in Equation (4.1).

4.3 Locality, in one constant

The shares h_v and h_r depend on the capacity of each stage and on how successive tokens reuse their experts. Lacking a measurement on the target model, we posit a model with a single constant.

Assumption 4.4 (Power-law hit rate)**.** If a stage holds a share c of the expert bytes, it serves a share $h(c) = c^{\beta}$ of the requested bytes, with $0 < \beta \leq 1$. For two nested stages (VRAM of capacity c_v, RAM of capacity c_r), $h_v = h(c_v)$ and $h_v + h_r = h(c_v + c_r)$.

$\beta = 1$ is uniform routing, with no reuse at all: the worst case, the one of the model study. The two other values are fitted on the only usable published point, a cache on one layer of Qwen3-30B-A3B (128 experts, 8 active): at 13.4 % of the budget, an LRU serves 65.9 % of the requests and Belady's oracle 79.1 % [lit.] (Chapter 7). One constant puts one curve through one point:

$$\beta_{\text{LRU}} = \frac{\ln 0.659}{\ln 0.134} = \frac{-0.4170}{-2.0099} = 0.2075, \tag{4.5}$$

$$\beta_{\text{Belady}} = \frac{\ln 0.791}{\ln 0.134} = \frac{-0.2345}{-2.0099} = 0.1167 \quad \text{[calc.]}. \tag{4.6}$$

A second point, as a check: moe-stream implies at least 53 % hits at 31 % of the budget (lemma 7.1); the LRU scenario gives $0.31^{0.2075} = 0.784 \geq 0.53$ there: consistent. Figure 4.1 shows the three curves.

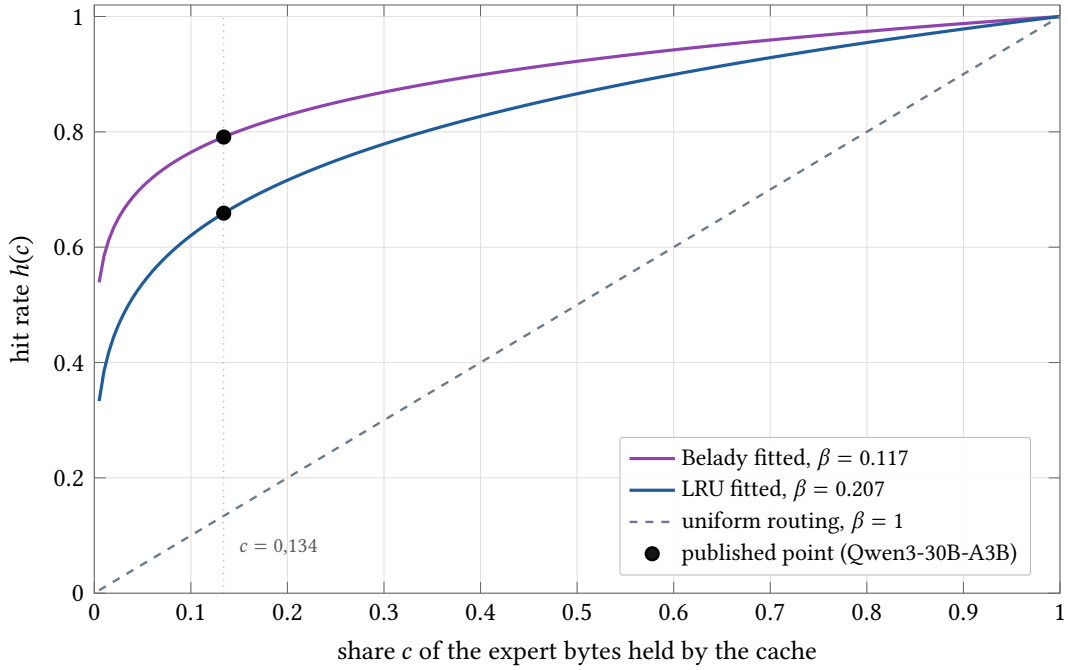


Figure 4.1: The three locality scenarios $h(c) = c^{\beta}$ [calc.]. Each β puts its curve through the corresponding published point (LRU, Belady; Qwen3-30B-A3B, one layer); uniform routing ($\beta = 1$) is the worst case. These are assumptions, not measurements of the target model: the first measurement on gpt-oss is in Chapter 10.

Data: `fondations/journaux/07-synthese/plafonds.json` (field `betas`), extracted by `memoire/outils/donnees_figures.py`.

This model is a stated assumption, not a law. It assumes that the shape of the Qwen3-30B curve (8 active out of 128) holds for gpt-oss-120b (4 active out of 128), and it extrapolates a single point. Every ceiling of the candidate architectures depends on it by a factor of 5; this is why the three scenarios are always given together.

4.4 The numbers of the machine

Two caveats hold for everything that follows. **The machine was shared**: every measurement ran alongside other agents, and the "best" throughputs are lower bounds of the idle machine (Chapter 3). **On battery** it is another machine: the GPU drops to 52.6 W, the link to Gen 3, everything that goes through the CPU loses a factor of 4 to 6; DMA holds at 11.8 GB/s, the decoder falls to 26.8 GB/s (Section 3.6.1). The numbers of this chapter are those on mains power.

Table 4.1: The quantities of the token equation on Stone's machine, on mains power. Throughputs are those of the measurement chapters; each row names its log.

Quantity	Value		Log
W_{D}, disk, 4 MiB reads at QD 4 (size of one expert matrix)	2.58 GB/s	[meas.]	`01-machine/disque-analyse.md`
W_{D}, disk, 16 MiB reads at QD 1	2.82 GB/s	[meas.]	`01-machine/debits-mesures.json`
disk, 4 KiB random (page faults)	0.22 GB/s	[meas.]	`01-machine/disque-balayage.csv`
W_{P}, host → device DMA from pinned RAM	12.2 GB/s	[meas.]	`01-machine/carte-complement.jsonl`
VRAM read by a shader	251.3 GB/s	[meas.]	`01-machine/carte.jsonl`
W_g, raw MXFP4 matrix-vector product, Vulkan	209.4 GB/s	[meas.]	`03-entropie/`
W_{dec}, rANS v4 decoder, Vulkan	70 GB/s	[meas.]	`03-entropie/`
decoder plus reassembly of the three class streams	41.6 GB/s	[meas.]	`03-entropie/`
r, lossless coding of gpt-oss-120b (three classes)	1.1026	[meas.]	`03-entropie/`
V, M: VRAM and RAM usable for weights	6.5 and 11 GB	assum.	study 06
t_0, fixed cost of a token	3.5 ms	[calc.]	`02-quantification/vitesse-4B-3.json`
B_t, B_e, E_{tot} of gpt-oss-120b	1.686; 1.909; 61.07 GB	[meas.]	`07-synthese/inventaire-gptoss.json`

Data: Paths relative to `fondations/journaux/`.

4.5 The problem, stated

What Stone asks now reads as a constrained optimization problem. Let θ be the set of decisions of an engine: the weight format (bits per weight, coding, hence B_t, B_e, r_d, r_p), the placement and cache policy (hence c_v, c_r and, through assumption 4.4, h_v, h_r, h_s), the scheduling of reads (hence t_{lat}), speculation (hence $\mathscr{E}$), and the miss policy (serve exact, wait, serve a less precise version, re-route).

Definition 4.5 (The troncons problem)**.** For a given model and Stone's machine, find

$$\theta^\star \in \arg\max_{\theta} \ \frac{1}{t(\theta)} \quad \text{subject to} \quad \begin{cases} \mathrm{VRAM}_{\max}(\theta) \le 8\,188\ \mathrm{MiB}, \\ \mathrm{RAM}_{\max}(\theta) \le M_{\mathrm{window}} \quad \text{(independent of the model size, Definition 1.1)}, \\ \mathrm{KL}\big(p_\theta \,\|\, p_{\mathrm{exact}}\big) \le \kappa, \quad \text{same top choice} \ge \pi_0, \end{cases} \tag{4.7}$$

where $t(\theta)$ is given by Equation (4.1), p_θ is the next-token distribution returned by the engine, p_{exact} the one returned by the original weights, and (κ, π_0) a quality budget declared before the measurement.

The chosen budget is derived from a measured quantity, not set by eye: $\kappa = 0.0055$ and $\pi_0 = 95.96\,\%$, the measured gap between two correct engines reading the same weights (Section 9.6). The reason: a loss smaller than the gap between two correct implementations cannot be told apart from a change of engine. An exact solution (KL = 0 by construction) meets the third constraint whatever κ.

What makes the problem hard. The constraints couple what the studies measured separately. Lossy compression shrinks B_e but spends the budget κ. Lossless compression raises r_d but loads the decoder (T_G). A large RAM cache raises h_r but breaks the window if M_{window} is small. Speculation raises $\mathscr{E}$ but makes the engine read the union of the experts of several tokens. The next section says which of these couplings matters on this machine.

4.6 What holds the maximum: the elasticities

Proposition 4.6 (Elasticities of the ceiling). *Let $d = 1/t$ be the ceiling given by Equation (4.1) with $\mathscr{E} = 1, t_{\text{lat}} = 0$, and assume the disk strictly holds the maximum: $T_D > \max(T_P, T_G)$. Let $\eta = T_D/(T_D + t_0)$. Then*

$$\frac{\partial \ln d}{\partial \ln W_D} = \frac{\partial \ln d}{\partial \ln r_d} = +\eta, \qquad \frac{\partial \ln d}{\partial \ln h_s} = \frac{\partial \ln d}{\partial \ln B_e} = -\eta, \qquad \frac{\partial \ln d}{\partial \ln x} = 0 \quad \text{for } x \in \{W_P, W_{\text{dec}}, W_g, r_p\}, \tag{4.8}$$

and $\partial \ln d / \partial \ln t_0 = -(1 - \eta)$.

Proof. Under the strict assumption, the maximum equals T_D in a neighbourhood of the point (the three functions are continuous), so $t = T_D + t_0$ is differentiable there and depends neither on W_P, nor on W_{dec}, nor on W_g, nor on r_p: their elasticities are zero. For $x \in \{W_D, r_d, h_s, B_e\}$, Equation (4.2) gives $\partial \ln T_D / \partial \ln x = \mp 1$, and

$$\frac{\partial \ln d}{\partial \ln x} = -\frac{\partial \ln t}{\partial \ln x} = -\frac{T_D}{T_D + t_0} \frac{\partial \ln T_D}{\partial \ln x} = \pm \eta.$$

B_e also appears in T_P and T_G, which are not the maximum: its derivative only goes through T_D. Finally $\partial \ln t / \partial \ln t_0 = t_0/(T_D + t_0) = 1 - \eta$. ∎

For gpt-oss-120b in the fitted-LRU scenario (Section 4.7), $T_D = 157$ ms, $T_P = 58$ ms, $T_G = 36$ ms: the disk holds the maximum with a margin of 2.7 over the link, and $\eta = 157/160.5 = 0.978$ [calc.]. Table 4.2 derives what each gain is worth.

Table 4.2: Elasticity of the ceiling of gpt-oss-120b, candidate architecture A, fitted-LRU scenario [calc.] (Proposition 4.6, $\eta = 0.978$).

Quantity	Elasticity	10 % gained is worth	Remark
W_D (disk throughput)	+0.98	+9.8 %	BitLocker and VMD take 53 % of it (Chapter 3)
r_d (compression of bytes read)	+0.98	+9.8 %	lossless: 1.10 reached; lossy: quantization
B_e (bits per active weight)	−0.98	+10.8 %	the slope of quantization (Chapter 5)
h_s (share read from disk)	−0.98	+10.8 %	one hit point gained is worth +4.4 % at $h_s = 0.234$ (160.5 to 153.8 ms)
W_P, W_{dec}, W_g	0	0	as long as they do not hold the maximum
t_0	−0.02	+0.2 %	3.5 ms out of 160

Corollary 4.7 (GPU kernels are ornament in the disk regime). *As long as the disk holds the maximum, making the GPU kernels faster (decoder, matrix-vector product, fixed cost) returns less than 2 % on gpt-oss-120b.*

Proof. By Proposition 4.6, only t_0 (elasticity -0.022) and the zero-elasticity quantities depend on the kernels. Removing t_0 entirely would take t from 160.5 to 157 ms, i.e. +2.2 % at most; any partial gain returns less. (The 2 % figure is rounded from $1 - \eta = 0.022$.) ■

This is an idea killed by the number, for this regime: work on the speed of the VRAM kernels only becomes a lever again once the disk stops limiting, for instance on an unencrypted volume where the link takes over (Chapter 10). Figure 4.2 shows both regimes on one curve.

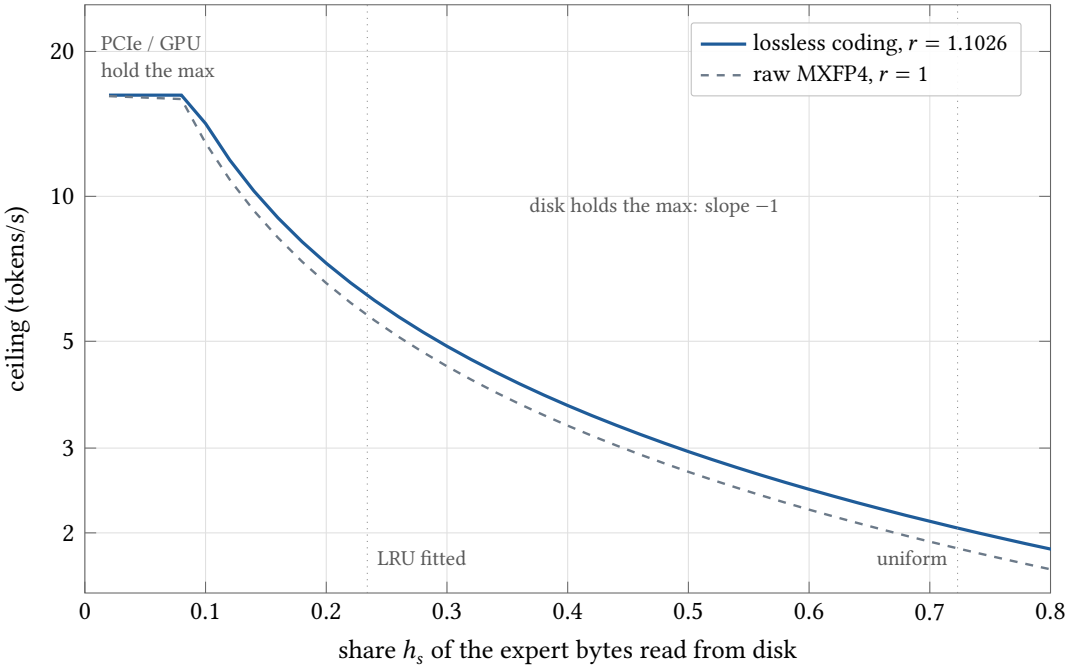


Figure 4.2: Ceiling of gpt-oss-120b against the share h_s of the expert bytes read from disk, candidate architecture A [calc.]. On the right, the disk holds the maximum and the curve has slope -1 on a log scale (Proposition 4.6); on the left, below $h_s \approx 0.1$, the link and the GPU take the maximum back and the curve flattens. Lossless coding shifts the whole disk regime by a factor $r = 1.1026$.

Data: `fondations/journaux/07-synthese/csv/plafond-gptoss120b-contre-hs.csv` (tool `outils/07-synthese/plafonds.py`).

4.7 A worked example

gpt-oss-120b MXFP4, three-class lossless coding ($r_d = r_p = 1.1026$), candidate architecture A, fitted-LRU scenario, $V = 6.5$ GB, 11 GB warm cache in RAM (`fondations/07-ce-qui-manque.md`, appendix A.3) [calc.].

1. VRAM capacity for experts: $c_v = (6.5 - 1.686)/61.07 = 0.0788$, hence $h_v = 0.0788^{0.2075} = 0.590$.
2. RAM capacity: the warm cache keeps coded experts, so 11 GB hold $11 \times 1.1026 = 12.13$ GB served: $c_r = 12.13/61.07 = 0.1986$. Then $h_v + h_r = 0.2774^{0.2075} = 0.766$, hence $h_r = 0.176$ and $h_s = 0.234$.
3. Disk: $T_D = 0.234 \times 1.909/1.1026/2.58 = 0.157$ s.
4. Link: $T_P = (0.176 + 0.234) \times 1.909/1.1026/12.2 = 0.058$ s (the always-read bytes fit in VRAM: $\delta = 0$).
5. GPU: $T_G = 0.410 \times 1.909/41.6 + (1.686 + 1.909)/209.4 = 0.0188 + 0.0172 = 0.036$ s (three-class-stream decoder, 41.6 GB/s).
6. Ceiling: $t = 0.157 + 0.0035 = 0.161$ s, i.e. **6.24 tokens/s**; serial floor: $t = 0.157 + 0.058 + 0.036 + 0.0035 = 0.254$ s, i.e. **3.93 tokens/s**.

The same six lines, redone by the tool, give 6.2414 and 3.9322 (`fondations/journaux/07-synthese/csv/architectures-gptoss120b.csv`, row `lru_cale, A_sans_perte`).

4.8 The working window, in numbers

Definition 1.1 asks for a peak memory that does not depend on the model size. In Equation (4.1), RAM only appears through c_r, the capacity of the warm cache: a setting, not a consequence of the model. The rest of the peak RAM comes from the arrival buffers (the misses of one token, double-buffered) and from the engine's structures.

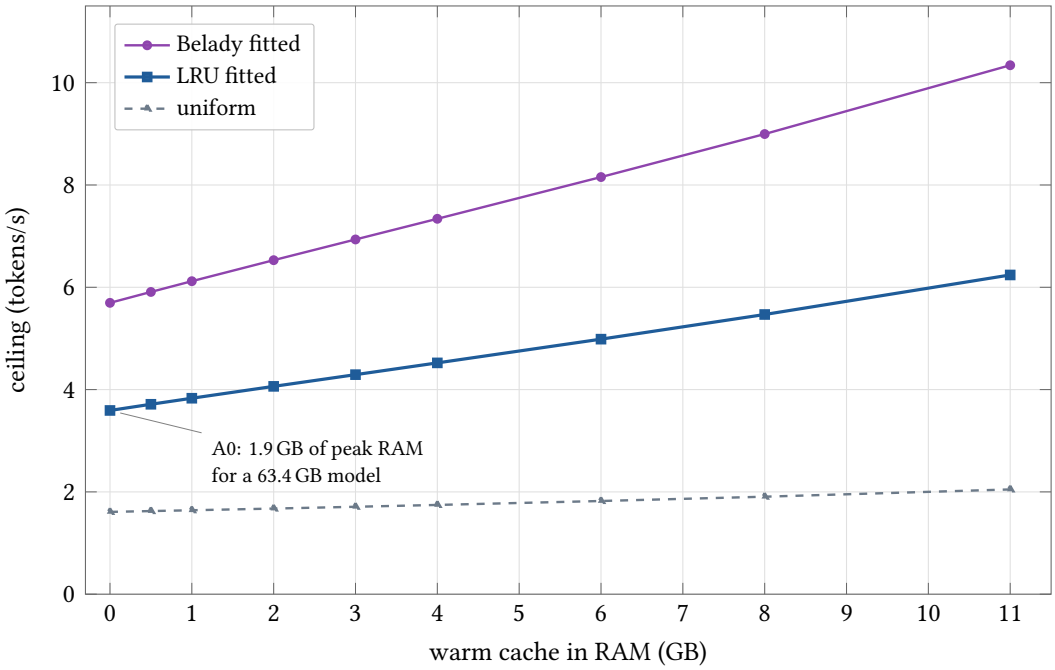


Figure 4.3: Ceiling of gpt-oss-120b (63.4 GB) against the warm cache in RAM, candidate architecture A [calc.]. The first column is A0: with no RAM cache at all, 1.9 GB of peak RAM and a ceiling of 3.6 tokens/s in the fitted-LRU scenario. Each GB of cache adds about 0.24 token/s in this scenario; peak RAM goes from 1.9 GB (no cache) to 12.3 GB (11 GB of cache).

Data: `fondations/journaux/07-synthese/csv/plafond-gptoss120b-contre-ram.csv`.

This is the answer, in numbers, to "without RAM or anything" (Figure 4.3): RAM does not vanish, it stops depending on the model. The advocate of A has since tightened the window: with buffers of a few layers instead of two whole tokens, A0 goes down to 0.64 GB of peak RAM (1.26 GB if the embedding table stays in RAM) [calc.] (`decisions/partisan-A.md`, § 0, item 7). From this follows the direct witness of Definition 1.1: gpt-oss-20b and gpt-oss-120b have the same expert size (13.25 MB), the same number of active experts and the same embedding table, so their peak RAM in A0 must agree within 0.5 MB, while their files are 12.1 and 63.4 GB.

4.9 The term that was missing from speculation

Corollary 8.8 concludes that speculation never pays on fine-grained *independent* experts. But two neighbouring tokens reuse part of their experts; a single number is enough to account for it.

Proposition 4.8 (Speculation under expert reuse). *Let $\rho \in [0,1]$ be the mean share of the experts of a token that were already among those of the previous token, in the same layer. A verification pass of one drafted token ($k = 1$) reduces the expert bytes read per produced token if and only if*

$$\alpha \;>\; 1 - \rho. \tag{4.9}$$

Proof. Without speculation, each token reads k_e experts per layer. The verification pass covers two positions; the second token brings $k_e(1-\rho)$ new experts on average, so the pass reads $U(2) = k_e(2-\rho)$ experts per layer. It returns $\mathscr{E}(\alpha, 1) = 1 + \alpha$ tokens (theorem 8.3). Bytes per produced token decrease if and only if $k_e(2-\rho)/(1+\alpha) < k_e$, i.e. $\alpha > 1-\rho$. ■

Under independence, $\rho = k_e/E$ and we recover corollary 8.8: $\alpha > 1 - 4/128 = 0.969$ for gpt-oss-120b. With the published reuse rates, the condition drops to $\alpha > 0.556$ for Qwen3-30B-A3B ($\rho = 0.444$) and $\alpha > 0.75$ for Qwen3.6-35B-A3B ($\rho \approx 0.25$) [calc.] on [lit.] (`fondations/journaux/07-synthese/plafonds.json`, field `moe_speculation_condition_k1`). If ρ were 0.44 on the target model, a draft at $\alpha = 0.8$ would return 1.8 tokens for 1.56 times the experts of one token: +16 % throughput, where Chapter 8 concluded to a loss. **The conclusion "optimal $k = 0$" therefore depends on a number nobody publishes for gpt-oss-120b**; the routing trace that gives $h(c)$ also gives ρ.

4.10 The witnesses of the equation

An equation that cannot be made to fall proves nothing. The tool `plafonds.py` redoes three numbers published by other studies, with the threshold declared before the run: 1 % deviation on each value.

Table 4.3: The witnesses of the token equation [calc.]. The break replaces the measured disk throughput by the link ceiling: the witness that depends on it falls, the other two hold.

Witness	Expected	Obtained	Holds	Broken
T1: gpt-oss-120b, placement of Section 3.8, $r = 1$, overlapped	1.98	1.994	yes	**falls** (5.569)
T2: Qwen3.5-9B entirely on disk, $r = 16$, the GPU decodes	8.85	8.847	yes	holds
T3: the placement of T1 at link throughput (7.877 GB/s), number of study 06	5.6	5.569	yes	holds

Data: `fondations/journaux/07-synthese/plafonds.json` and `plafonds-casse.json`.

A gap corrected along the way. The entropy study put the effect of the lossless rate (1.1026) on the cache capacity at +14.4 %, assuming the whole cache coded, VRAM included. Yet its own measurement says that the product fused with the decoder is 3.7 times slower than the raw product: the GPU must keep its experts decoded, and only RAM grows. The effect is then +13.1 % [calc.] (field `gain_taux_sur_cache_uniforme` of `plafonds.json`).

What could be wrong

- **The model $h(c) = c^{\beta}$ extrapolates one point.** It comes from an LRU on one layer of Qwen3-30B-A3B, on one domain; the curve of gpt-oss-120b may be flatter or steeper. The three scenarios bracket a belief; they do not bound reality.

- **The ceiling assumes perfect overlap** and a disk queue that never runs dry ($t_{\text{lat}} = 0$). The serial floor is closer to what published engines achieve, which land between 18 and 45 % of their bound (Chapter 7).
- $t_0 = 3.5$ **ms comes from another model**, under CUDA; a Vulkan engine that synchronizes after each router (36 times per token) may have a larger fixed cost, and the elasticity of t_0 would grow with it.
- **11 GB of pinned RAM**: the Windows display driver model may limit the host memory visible to the GPU; if the warm cache has to live in pageable memory, every RAM hit goes through a CPU copy. Not verified.
- **The condition** $\alpha > 1 - \rho$ assumes a constant α and a ρ independent of position; it says nothing about $k > 1$, where the union of n tokens decreases more slowly than a one-parameter model.
- **The quality budget** κ comes from a hybrid model (Qwen3.5-4B); the gap between two correct engines on gpt-oss may differ and must be remeasured on it.

Part II

The levers, measured

Chapter 5

Extreme quantization

Where we start, what this chapter establishes, what remains

At the start, the literature announced "nearly lossless" 2-bit quantizations on older families (Llama 2), with CUDA kernels, and no measurement existed on the machine's models or under Vulkan. **This chapter establishes** four things: the Shannon bound and each coder's distance to it, redone by a program written here; the first portable trellis decoder (Vulkan and DirectX 12), measured on the RTX 4060; the real ladder of llama.cpp quantizations on Qwen3.5 4B and 9B, at equal bytes read; and a rule for splitting bits by memory stage, proven, then corrected here with the throughputs measured in Chapter 3. **What remains** is a rotation-free trellis coder decoded under Vulkan, a nested code tied to placement, and a BF16 baseline instead of Q4_K_M. Source: `fondations/02-quantification.md`, `02-quantification-derivations.md`, `fondations/journaux/02-quantification/`.

5.1 The lever

For a dense model of N parameters stored at b bits per weight, metadata included, a decoded token reads every weight once: $B = Nb/8$ bytes, plus the KV cache and activations, negligible for a short context. Theorem 3.2 then gives Equation (3.3), tokens/s $\leq 8W/(Nb)$: **halving b doubles the ceiling, at any stage**. For a mixture of experts, N becomes the number of active parameters per token. Table 5.1 quantifies this lever with the throughputs measured on the machine.

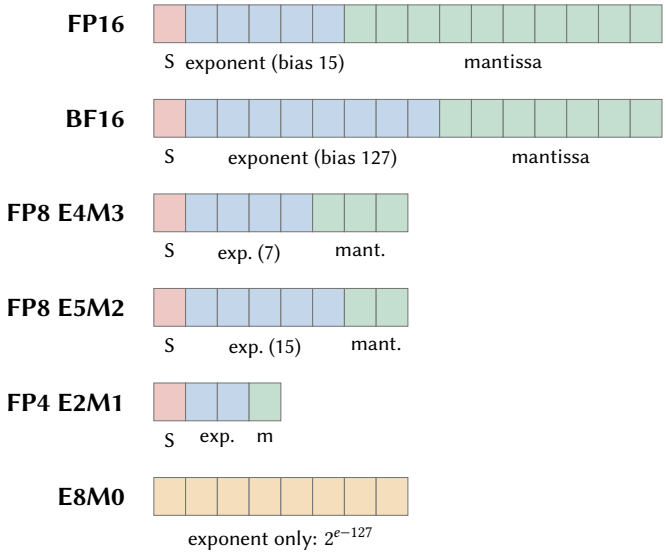
Table 5.1: Tokens-per-second ceiling, weights read entirely from the named stage, no cache, tokens/s $= 8W/(Nb)$ [calc.] on the measured throughputs (VRAM 251.3; PCIe 12.2; disk 2.82 GB/s).

Model (parameters read), stage	$b = 16$	$b = 4.5$ (Q4_K)	$b = 2.06$ (IQ2_XXS)	$b = 1.56$ (IQ1_S)
9 G dense, VRAM	14.0	49.9	108.9	143.8
9 G dense, PCIe	0.68	2.42	5.29	6.98
9 G dense, disk	0.16	0.56	1.22	1.61
70 G dense, disk	0.020	0.072	0.156	0.206
MoE with 3 G active, PCIe	2.03	7.23	15.8	20.8
MoE with 3 G active, disk	0.47	1.67	3.65	4.81

Reading: outside VRAM, only mixtures of experts become interactive, and quantization multiplies the ceiling by $16/b$. No other lever of this thesis has that slope without a throughput counterpart; that is its reason for being. The counterpart is elsewhere: quality.

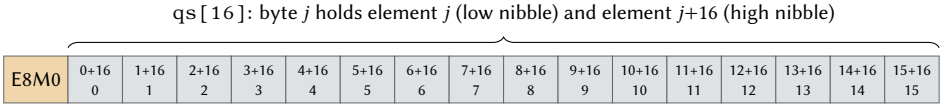
5.2 The formats, bit by bit

A quantization format is a bit layout and a reconstruction rule. The diagrams that follow are read from the IEEE 754 standard [86], the OCP MX specification [37, 87] and the llama.cpp code [9] (`ggml/src/ggml-common.h` and `ggml-quants.c`, commit `843e7dc`); the `static_asserts` of the code give each block's size in bytes and serve as the control for each row of Table 5.2.



E2M1 values: $\pm\{0, \frac{1}{2}, 1, \frac{3}{2}, 2, 3, 4, 6\}$. llama.cpp stores them doubled ($\{0, 1, 2, 3, 4, 6, 8, 12\}$) and halves the E8M0 scale.

Figure 5.1: Floating-point formats, bit by bit: FP16 and BF16 (16 bits), FP8 as E4M3 and E5M2 [88], FP4 as E2M1, and the E8M0 scale of the MX formats (exponent only). BF16 keeps the FP32 exponent and sacrifices the mantissa [89]; E8M0 encodes only a power of two.



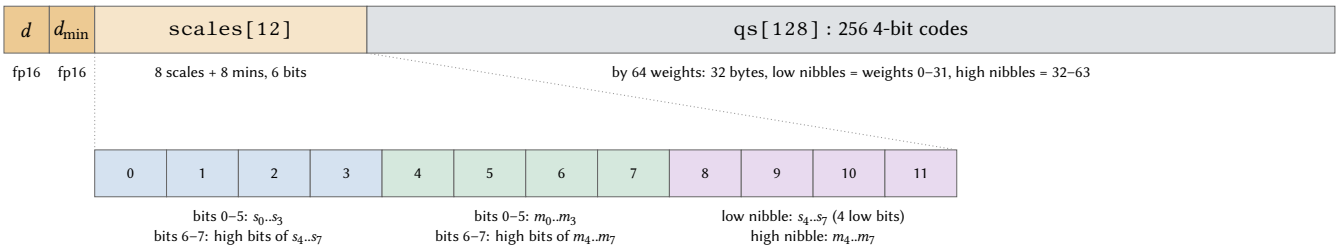
$w_j = 2^{e-127} \cdot \mathrm{E2M1}(c_j)$; $8 + 32 \times 4 = 136$ bits for 32 weights = **4,25** bits/weight.

Figure 5.2: The MXFP4 block: 32 weights share an E8M0 exponent; each weight is a 4-bit E2M1 code. It is the native format of the gpt-oss experts, the first target of this thesis.

The bit read per token is not the bit of the type. An "IQ2_XXS" file mixes types: we measure it tensor by tensor (`quant-gguf-bits.py`, control: the end of the last tensor equals the end of the file). The "IQ2_XXS" of Qwen3.5-9B reads 2.757 GB per token, i.e. 2.76 bits per weight: the output stays in Q5_K (699 MB, 25 % of the bytes read), 32 matrices in Q4_K (472 MB), and the matrices truly in IQ2_XXS amount to only 1 516 MB [meas.]. **At 2 bits, the output head becomes the largest item** of a small model with a large vocabulary: 29 % of the bytes read per token in the 4B "IQ2_XXS", whose embedding table, tied to the output (248 320 × 2 560), is reread in full at every token.

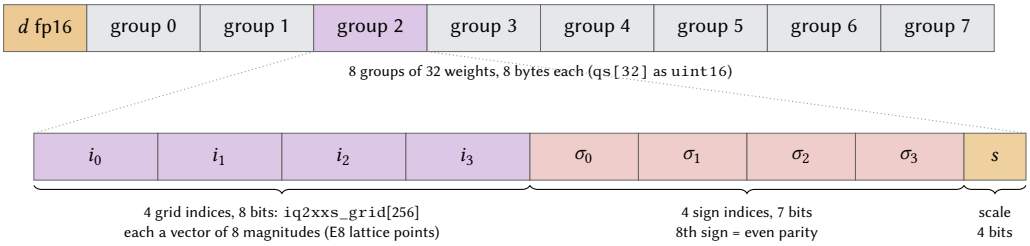
5.3 The Gaussian source and its bound

Why the Gaussian. The methods closest to the bound today (QuIP# [27], QTIP [31], HIGGS [26], QuaRot [39]) multiply each weight matrix, left and right, by a Hadamard transform with random



$w = d\, s_j\, c - d_{\min}\, m_j$ for a weight of sub-block j (32 weights) with code $c \in \{0..15\}$; $(16 + 16 + 96 + 1024)/256 = \mathbf{4{,}5}$ bits/weight.

Figure 5.3: The llama.cpp Q4_K super-block: 256 weights in 144 bytes. Two fp16 scales (d, $d_{\min}$) multiply eight 6-bit scales and eight 6-bit minimums, packed into 12 bytes according to the lower diagram (function `get_scale_min_k4`).



$w = d\,(\tfrac{1}{2} + s)\,\tfrac{1}{4}\; g_{i_l}[k]\; (-1)^{\sigma_l[k]}$; $(16 + 8 \times 64)/256 = \mathbf{2{,}0625}$ bits/weight.

Figure 5.4: The llama.cpp IQ2_XXS super-block: 256 weights in 66 bytes. A group of 32 weights fits in 64 bits: four indices into a grid of 256 eight-dimensional vectors drawn from the E8 lattice, four 7-bit sign indices (the 8th sign is fixed by parity), a 4-bit sub-scale.

signs, $W' = H_g\,\mathrm{diag}(s) W\,\mathrm{diag}(s') H_d$: after this rotation, the weights of a row are nearly independent Gaussians and outliers disappear. Quantization becomes a Gaussian source coding problem. HIGGS moreover links this error to quality: for small errors, the perplexity increase is approximately a layer-by-layer weighted sum of the relative squared errors ("linearity theorem") [lit.]. Each gain in squared error thus translates roughly linearly into a quality gain.

Theorem 5.1 (Shannon, memoryless Gaussian source [90, 91])**.** *For an i.i.d. source $X \sim \mathcal{N}(0, \sigma^2)$ and squared-error distortion, the rate–distortion function is $R(D) = \frac{1}{2}\log_2(\sigma^2/D)$ for $0 < D \le \sigma^2$; equivalently, the smallest mean squared error achievable at a rate of R bits per sample is*

$$D(R) = \sigma^2\, 2^{-2R}. \tag{5.1}$$

Proof. The source coding theorem with a fidelity criterion [90] states that the smallest rate achievable by long codes at average distortion D is $R(D) = \min I(X;\hat{X})$, the minimum running over conditional laws $p(\hat{x} \mid x)$ with $\mathbb{E}(X - \hat{X})^2 \le D$. It remains to compute this minimum.

Lower bound. For any admissible law,

$$\begin{aligned} I(X;\hat{X}) &= h(X) - h(X \mid \hat{X}) = h(X) - h(X - \hat{X} \mid \hat{X}) \\ &\ge h(X) - h(X - \hat{X}) \ge \tfrac{1}{2}\log_2(2\pi e\sigma^2) - \tfrac{1}{2}\log_2\!\big(2\pi e\, \mathbb{E}(X - \hat{X})^2\big) \ge \tfrac{1}{2}\log_2 \frac{\sigma^2}{D}. \end{aligned}$$

The second equality holds because, given $\hat{X}$, X and $X - \hat{X}$ determine each other by translation; the first inequality because conditioning reduces entropy; the second because, for a given second moment, the normal law maximizes differential entropy.

Table 5.2: Bit accounting, format by format [calc.] (layouts read in `ggml-common.h`; control: the `static_asserts`).

Format	Block	block bits	bits/weight
Q2_K	256 weights: 2-bit codes (512), 16 bytes of 4+4-bit scales and mins (128), fp16 d and $d_{\min}$ (32)	672	**2.625**
Q4_K	256: 4-bit codes (1 024), 12 bytes of 6-bit scales (96), d, $d_{\min}$ (32)	1 152	**4.5**
Q6_K	256: 4 low bits (1 024) + 2 high bits (512), 16 int8 scales (128), d (16)	1 680	**6.5625**
IQ2_XXS	8 groups of 32: 4 8-bit indices, 4×7 sign bits, 4-bit scale; fp16 d	528	**2.0625**
IQ3_XXS	256: 8-bit index per 4 weights (512); per 32 weights, signs and scale (256); d (16)	784	**3.0625**
IQ1_S	256: 8 low index bits per 8 weights (256); per 32 weights, high bits, scale, shift (128); d (16)	400	**1.5625**
TQ1_0	256: 240 trits at 5 per byte, 16 trits at 4 per byte; d (16)	432	**1.6875**
MXFP4	32: E2M1 codes (128); E8M0 exponent (8)	136	**4.25**
NVFP4	64: E2M1 codes (256); 4 UE4M3 scales, one per 16 (32)	288	**4.5**
Trellis, free state	sequence of T weights, k bits per weight, L state bits	$Tk + L - k$	$k + \frac{L-k}{T}$
Tail-biting trellis	tile of 256 weights read circularly	$256k$	k

Achievability. Let $\hat{X} \sim \mathcal{N}(0, \sigma^2 - D)$ and $Z \sim \mathcal{N}(0, D)$ be independent, and $X = \hat{X} + Z$ (the "backward" test channel). Then $X \sim \mathcal{N}(0, \sigma^2)$, $\mathbb{E}(X - \hat{X})^2 = D$ and $I(X; \hat{X}) = h(X) - h(Z) = \frac{1}{2}\log_2(\sigma^2/D)$: all inequalities are equalities. Hence $R(D) = \frac{1}{2}\log_2(\sigma^2/D)$, and inverting, $D(R) = \sigma^2 2^{-2R}$. ■

At $R = 2$, $D = 0.0625\,\sigma^2$. The bound says what no coder can beat; it does not say how to reach it. To compare coders, we measure their distance to it in bits.

Definition 5.2 (Distance in bits to the bound)**.** A quantizer q reaching error D_q at rate R is

$$\Delta(q) \;=\; R - R(D_q) \;=\; R + \tfrac{1}{2}\log_2 \frac{D_q}{\sigma^2}$$

bits away from the bound: the rate an optimal code would save at equal quality.

Lemma 5.3 (An error ratio in bits)**.** *Two quantizers with the same slope $D \propto 2^{-2R}$ around the operating point, with errors $D_1 > D_2$ at the same rate, are $\frac{1}{2}\log_2(D_1/D_2)$ bit apart at equal quality.*

Proof. The first must gain δ bits to reach D_2: $D_1 2^{-2\delta} = D_2$, i.e. $\delta = \frac{1}{2}\log_2(D_1/D_2)$. ■

5.4 The optimal scalar quantizer

The Lloyd–Max quantizer [92, 93] is the best scalar quantizer with $M = 2^R$ levels $y_1 < \cdots < y_M$ and thresholds t_j. Its two necessary optimality conditions are:

1. nearest neighbour: $t_j = (y_j + y_{j+1})/2$;
2. centroid: $y_j = \mathbb{E}[X \mid t_{j-1} < X \le t_j] = \dfrac{\varphi(t_{j-1}) - \varphi(t_j)}{\Phi(t_j) - \Phi(t_{j-1})}$ for $X \sim \mathcal{N}(0,1)$, where φ and Φ are the normal density and distribution function.

They are iterated to the fixed point on the exact density (`quant-mse-gaussien.py`). The error then has a closed form,

$$D = 1 - \sum_j y_j^2 \left[\Phi(t_j) - \Phi(t_{j-1})\right], \tag{5.2}$$

because $\mathbb{E}[X^2] = 1$ and the error is orthogonal to the reconstruction at the centroid. We find 0.36338 at 1 bit (levels $\pm\sqrt{2/\pi} = \pm 0.79788$), **0.117482 at 2 bits** (levels ±0.45278 and ±1.51042), 0.034548 at 3 bits: Max's (1960) values [calc.].

Remark 5.4 (Why the bench was redone). The Panter–Dite formula [94] for the high-resolution optimal scalar quantizer, $D \approx \frac{\sqrt{3}\pi}{2}\sigma^2 2^{-2R} = 2.72\,\sigma^2 2^{-2R}$, would give 0.170 at $R = 2$ instead of 0.1175: at 2 bits we are not in high resolution, and only computed or measured errors count (Figure 5.5).

5.5 The theoretical bench, redone here

The control. Lloyd–Max computed analytically against Monte Carlo on 2^{20} samples, threshold declared before running: 4 standard errors. The control holds at 1, 2 and 3 bits. **It was at first too weak**: the first fault (a single level shifted by 0.05) did not make it fail (+0.0008 error at 1 bit, below the threshold), because nearest-neighbour assignment reallocates the samples and absorbs the shift. Strengthened fault (all levels shifted by 0.1): it fails at 1 and 2 bits. Both logs are kept.

Table 5.3: Mean squared error at about 2 bits on $\mathcal{N}(0,1)$, Monte Carlo standard error in parentheses; distance Δ to the bound (Definition 5.2).

Quantizer	bits/weight	MSE [meas.]	published [lit.]	Δ [calc.]
Lloyd–Max scalar	2	0.1175 (0.0002)	0.118	0.455
VQ dim. 8, random Gaussian codebook of 2^{16} words	2	0.0935 (0.0004)	—	0.291
VQ dim. 8, codebook computed IQn_KT-style	2	0.0928 (0.0004)	—	0.285
QuIP# E8P (E8 lattice)	2	not redone	0.089	—
llama.cpp IQ2_XXS	2.0625	**0.1070 (0.0005)**	—	0.450
Shift-register trellis $L = 16$, random table	2.0034	0.0680 (0.0007)	0.068	0.064
Trellis $L = 16$, "3INST" code (QTIP)	2.0034	0.0687 (0.0007)	0.069	0.070
Trellis $L = 16$, "mul1" code (EXL3)	2.0034	0.0682 (0.0007)	—	0.066
Shannon bound	2	0.0625 [calc.]	0.063	0

Three results (Table 5.3 and Figure 5.5):

- **The three trellis codes reproduce QTIP's table 1 within the standard error**, by a program written from the mathematical definition, without their code. The trellis closes $(0.1175-0.068)/(0.1175-0.0625) = 90\,\%$ of the gap between the scalar quantizer and the bound; the 8-dimensional VQ closes 44 % [calc.]. At 1 bit (24 sequences of 4 096 weights), the trellis reaches 0.2631 against 0.3634 for Lloyd–Max and 0.25 for the bound: it closes 88 % of the gap.
- **IQ2_XXS, measured on the same source, is exactly the optimal scalar quantizer at its rate**: 0.107 at 2.0625 bits, when Lloyd–Max brought to that rate gives $0.1175 \times 2^{-0.125} = 0.108$ [calc.]. The coder was written from the format (256-vector grid read in `ggml-common.h`, sign parity, 4-bit sub-scales), optimal for a given scale; its control, a grid $\{0.45278; 1.51042\}^8$ without parity, gives back Lloyd–Max (0.1173 ± 0.0009) and fails when broken. On a Gaussian source, i.e. after rotation, the E8 grid of IQ2_XXS brings nothing; it was chosen on real, unrotated weights, where it wins. The parity constraint costs as much as the bit it saves.

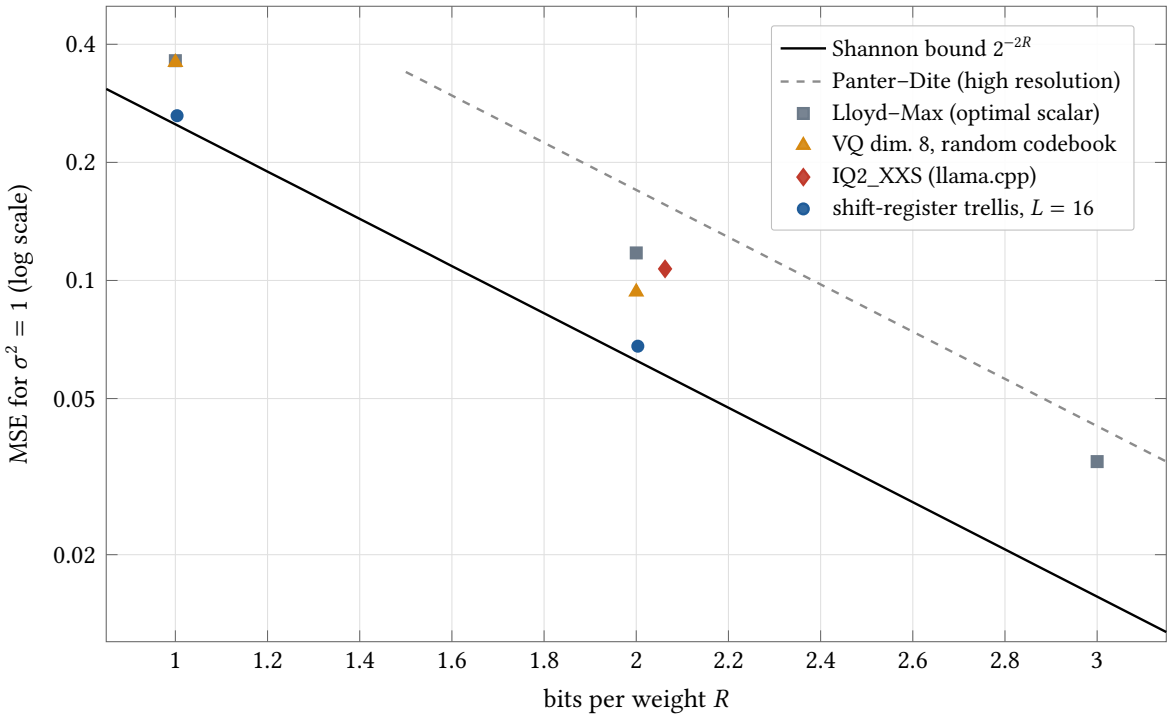


Figure 5.5: Distortion versus rate on the $\mathcal{N}(0,1)$ source [meas.], with the Shannon bound (theorem 5.1) and the Panter–Dite high-resolution formula. The shift-register trellis closes 90 % of the gap between the optimal scalar quantizer and the bound at 2 bits, the 8-dimensional VQ closes 44 %; IQ2_XXS equals the optimal scalar quantizer at its rate.

Data: `fondations/journaux/02-quantification/csv/fig-q1-*.csv` (logs `mse-gaussien-passe1-partielle.log`, `mse-treillis-2b.json`, `mse-iq2xxs.json`).

- **The "trellis" of ik_llama.cpp is not one**: read in its code, each group of 8 weights is decoded from an independent 16-bit seed, found by clustered nearest neighbour. Measured: the same error as an 8-dimensional VQ with a random codebook (0.0928 against 0.0935). A true shift-register trellis would have 27 % less error at the same rate.

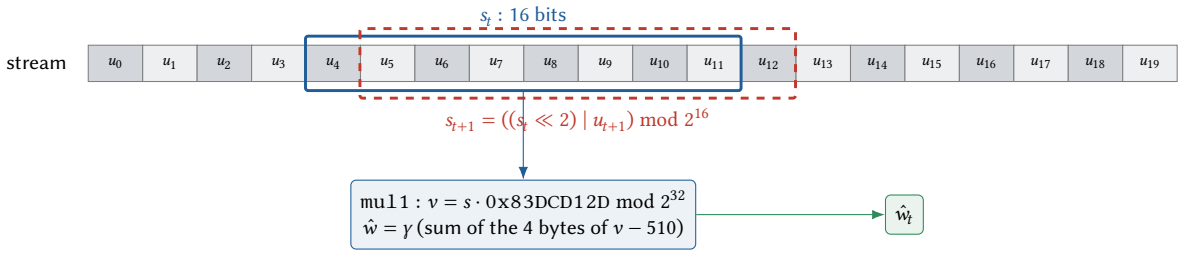
The distances of Table 5.3 use the real rate, priming bits included (2.0034 bits for the trellis); with the nominal 2-bit rate, the random-table trellis is 0.061 bit away, the figure of the original study.

5.6 The shift-register trellis

The code. The state at time t is an L-bit window s_t of the coded stream; each weight shifts in k new bits: $s_{t+1} = ((s_t \ll k) \mid u_t) \bmod 2^L$, with $u_t \in \{0, \dots, 2^k - 1\}$. The reconstruction is $\hat{w}_t = f(s_{t+1})$, where $f : \{0,1\}^L \to \mathbb{R}$ is a table (random Gaussian) or a computed function. It is trellis-coded quantization [95] with a "shift-register" trellis: since f depends only on L bits, the table of 2^{16} values can be computed instead of read, for example by EXL3's "mul1" code (Figure 5.6). The coder searches the minimum-distortion stream with the Viterbi algorithm [96].

Lemma 5.5 (The predecessors of a state)**.** *The predecessors of a state s' are exactly the 2^k states $s = (s' \gg k) \mid (h \ll (L-k))$, $h \in \{0, \dots, 2^k - 1\}$.*

Proof. The transition $s' = ((s \ll k) \mid u) \bmod 2^L$ fixes the $L-k$ low bits of s, which become the $L-k$ high bits of s', and leaves its k high bits free, lost in the shift. Conversely, any s of this form leads to s' with $u = s' \bmod 2^k$. ∎



Each weight shifts $k = 2$ new bits into the window; the reconstruction depends only on the 16-bit window, so the codebook of 2^{16} values is computed, not read. The encoder chooses the stream by Viterbi (minimum total squared error); the decoder needs one multiplication and one `dot4` per weight. Tail-biting: the tile of 256 weights is read circularly, so the cost is exactly k bits per weight.

Figure 5.6: The shift-register trellis ($L = 16$, $k = 2$) and its computed "mul1" decoder. The state is a 16-bit window of the coded stream; each weight shifts 2 bits into it; the reconstructed value is a computed function of the window.

Corollary 5.6 (Vectorized Viterbi). *The cumulative cost* $J_{t+1}(s') = \min_h J_t\big(s(h, s')\big) + (w_{t+1} - f(s'))^2$ *is computed by reshaping the vector* J_t *of size* 2^L *into a* $(2^k, 2^{L-k})$ *matrix and taking the minimum along the* h *axis:* $O(2^L)$ *operations per weight.*

Proof. By lemma 5.5, the predecessors of s' depend only on $s' \gg k$, and, in index order, they occupy column $s' \gg k$ of the $(2^k, 2^{L-k})$ matrix whose row is h. The minimum of each column serves the 2^k states s' sharing the same value of $s' \gg k$, hence the repetition. ■

Listing 5.1 is the implementation of this corollary in the bench, in numpy; it takes six lines.

Listing 5.1: The "mul1" code and the vectorized Viterbi of the shift-register trellis, as they run in the bench (corollary 5.6); comments in French, as in the repository.

```python
# Extrait de outils/quant-mse-gaussien.py (commit 3adf9c3), lignes 119-136, recopié tel quel.
def code_mul1(etats):
    """EXL3 « mul1 » : x <- s * 0x83DCD12D mod 2^32, somme des 4 octets, centrée sur 510."""
    x = (etats.astype(np.uint64) * 0x83DCD12D) & 0xFFFFFFFF
    somme = (x & 0xFF) + ((x >> 8) & 0xFF) + ((x >> 16) & 0xFF) + ((x >> 24) & 0xFF)
    return somme.astype(np.float64) - 510.0


def viterbi_treillis(suite, table, L, k):
    """Coût minimal (somme des erreurs quadratiques) d'une suite sur le treillis à
    décalage : état suivant = ((état << k) | b) mod 2^L ; valeur émise = table[état].
    État initial libre. Les prédécesseurs de s' sont (s' >> k) | (h << (L - k))."""
    n_etats = 1 << L
    cout = (suite[0] - table) ** 2
    for t in range(1, len(suite)):
        # min sur les 2^k bits sortants h, pour chaque valeur de (s' >> k)
        m = cout.reshape(1 << k, n_etats >> k).min(axis=0)
        cout = np.repeat(m, 1 << k) + (suite[t] - table) ** 2
    return float(cout.min())
```

The exact rate. With a free initial state, a sequence of T weights costs $(Tk + L - k)/T$ bits per weight: 2.0034 bits for $T = 4\,096$, $k = 2$, $L = 16$. With QTIP's and EXL3's "tail-biting", one imposes $s_T = s_0$ (the tile is read circularly) and the cost is exactly k bits. CurveTQ [32] measures that tail-biting on blocks of 64 costs, at the same useful rate, +4.10 perplexity against a stored initial state (0.22 bit per weight) [lit.].

5.7 The cost of decoding on the card

5.7.1 In CUDA

The `quant-gemv-treillis.cu` bench (CUDA 13.1.2, `-arch=sm_89`), written from scratch, performs a 16 384 × 16 384 matrix–vector product (268 million weights, well beyond the 32 MiB L2 cache) in four formats. The two 2-bit kernels share layout and traversal: their gap is the cost of decoding the trellis. Control: 64 rows recomputed on the CPU, relative error declared below 10^{-3}; it holds for all four formats and, broken (code constant altered on the CPU side only), fails on the trellis and only on it.

Table 5.4: 16 384 × 16 384 product in CUDA on the RTX 4060, ABBA pairs with a clean reference [meas.].

Format	weight bytes	time (ms)	equivalent GB/s	speed-up / FP16
FP16	512 MiB	2.17 – 2.25	238 – 247	1
scalar 4 bits	128 MiB	0.567 – 0.580	231 – 237	3.8
scalar 2 bits	64 MiB	0.317 – 0.319	211 – 212	6.9
mul1 trellis 2 bits	64 MiB	0.430 – 0.433	155 – 156	5.1

The trellis is compute-bound, not memory-bound (Table 5.4): about ten integer operations per weight cap it at 624 billion weights per second in this naive kernel; it costs 36 % more time than the 2-bit scalar and remains 1.33 times faster than 4-bit.

5.7.2 In Vulkan and DirectX 12: the first portable trellis measured

The `quant-vulkan-gemv` bench (Rust, wgpu 30.0.1, WGSL shaders written from scratch, translated by Naga to SPIR-V for Vulkan and to HLSL for DirectX 12) redoes the same measurement, same matrix, same tile, same control. The trellis is decoded in two ways: a hand-written byte sum, or `dot4U8Packed`, the WGSL equivalent of `dp4a`. Three passes were needed and are kept: the first indexed a local array with a loop variable (2-bit fell below 4-bit); the second reread the whole activation vector for every row (1 GiB of L2 traffic for 64 MiB of weights); the third does four rows per group of 32 lanes and reads the activations once for all four. A fourth pass adds a 16-bit window extraction without `select` (Listing 5.2).

Listing 5.2: The shift-register trellis decoder in the WGSL shader of the GEMV: extraction of four 16-bit windows straddling two words, then the "mul1" code through `dot4U8Packed`; comments in French, as in the repository.

```
// Extrait de outils/quant-vulkan-gemv/src/main.rs (commit 89b509c), lignes 50-61 :
// le décodeur du treillis à décalage, code « mul1 », dans le shader WGSL de la GEMV.
// Le gabarit SOMMEV du dépôt est remplacé ici par sa valeur dans la variante retenue
```

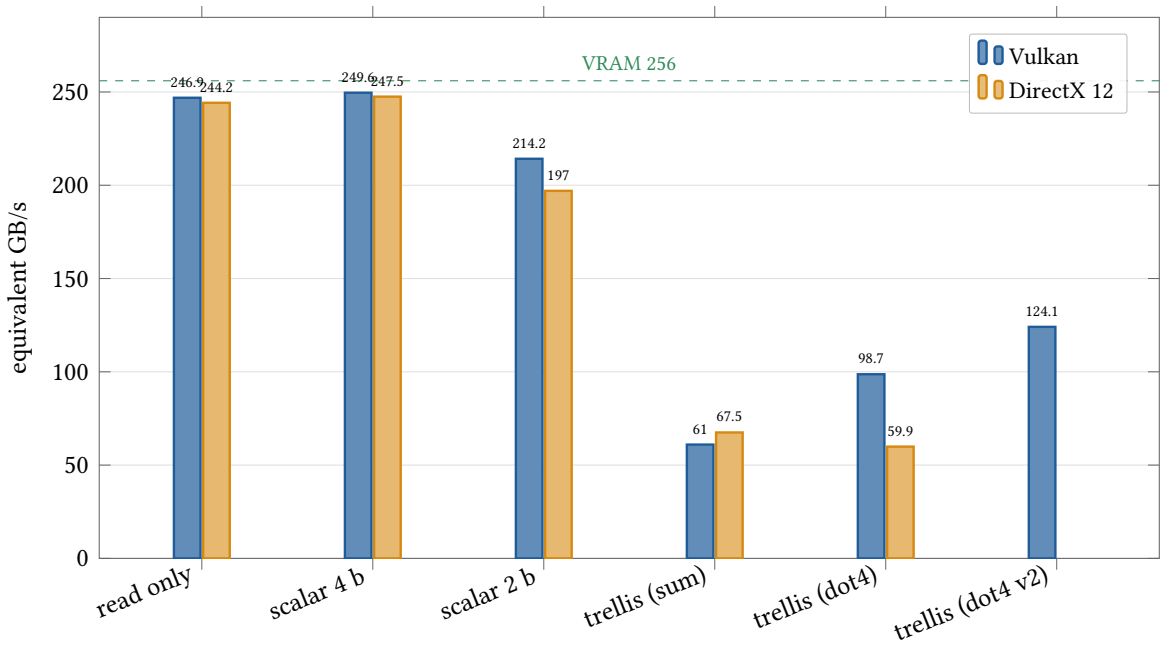


Figure 5.7: 16 384 × 16 384 matrix–vector product on the RTX 4060, equivalent GB/s of weights, medians of passes 3 and 4 [meas.]. With `dot4U8Packed` and select-free extraction (v2), the 2-bit trellis runs as fast as the 4-bit scalar while reading half the bytes.

Data: `fondations/journaux/02-quantification/csv/fig-q2-*.csv` (logs `vulkan-gemv-3.json`, `vulkan-gemv-4.json`, `dx12-gemv-3.json`).

```
// (dot4U8Packed, ligne 509 du même fichier) ; rien d'autre n'est changé.
fn fenetre4(w0: u32, w1: u32, off: vec4<u32>) -> vec4<u32> {
    let c = min(off, vec4<u32>(31u));
    let bas = ((vec4<u32>(w0) >> c) | (vec4<u32>(w1) << (vec4<u32>(32u) - c))) & vec4<u32>(0xffffu);
    let d = select(vec4<u32>(0u), off - vec4<u32>(32u), off >= vec4<u32>(32u));
    let haut = (vec4<u32>(w1) >> d) & vec4<u32>(0xffffu);
    return select(bas, haut, off >= vec4<u32>(32u));
}
fn mul1v(e: vec4<u32>) -> vec4<f32> {
    let v = e * vec4<u32>(0x83DCD12Du);
    let s = vec4<u32>(dot4U8Packed(v.x, 0x01010101u), dot4U8Packed(v.y, 0x01010101u),
                      dot4U8Packed(v.z, 0x01010101u), dot4U8Packed(v.w, 0x01010101u));
    return (vec4<f32>(s) - vec4<f32>(510.0)) * (1.0 / 147.7);
}
```

What these measurements say (Figure 5.7):

- **A shift-register trellis decodes in a portable shader**, inside the product, with no intermediate matrix: 124 GB/s equivalent in Vulkan, 68 in DirectX 12. To our knowledge, no such measurement is published.
- `dot4U8Packed` is worth a factor of 1.6 in Vulkan (native integer instruction) and nothing in DirectX 12 (where Naga seems to emulate it): **Vulkan is the API to target**.
- Decoding costs when the weights are in VRAM: the 2-bit trellis stays 1.7 times slower than the 2-bit scalar and as slow as the 4-bit scalar. **Outside VRAM, this cost disappears**: at 124 GB/s, the

decoder is ten times faster than the measured PCIe link (12.2 GB/s) and 44 times faster than the disk. In the troncons regime, decoding is free and only bits count.

5.7.3 A multiplication-free trellis is not faster

XOR-Trellis [33] announces a decoder at about 14 XOR gates per value, without publishing its masks. The question asked here: does a computed code made only of parities, written from scratch, keep the error of QTIP's and EXL3's codes, and does it decode faster? The code: 16-bit state s, index $j = \sum_{q=0}^{3} \text{parity}(s \,\&\, m_q)\, 2^q$, a linear function of s over GF(2), value taken from the palette of the 16 Gaussian quantiles. If the four masks have rank 4, each index is reached by 2^{12} states and the law of the emitted values is uniform over the palette.

Result [meas.]: **error 0.0712 ± 0.0009** at 2.0034 bits (masks `0xdfea`, `0x30b1`, `0xb566`, `0x753c`), against 0.0682 for mul1 on the same sequences, i.e. $\frac{1}{2}\log_2(0.0712/0.0682) = 0.031$ bit of loss (lemma 5.3). The control runs mul1 through the same machinery and finds *exactly* 0.068236 (same generator consumed in the same order). But in Vulkan, this decoder runs at 52 GB/s equivalent against 119 to 121 for mul1: four bit counts per value cost more than an integer multiplication and a `dot4`, which are native instructions of the card. The table-free variant (uniform palette) runs at the same speed: it is not the table that costs, it is the counts. **On an RTX 4060, mul1 remains the best code to decode**; the advantage of XOR-Trellis is for a dedicated circuit.

5.8 The llama.cpp ladder, measured on Qwen3.5

Protocol. llama.cpp b11515, the machine's Q4_K_M as baseline (bias declared at the end of the chapter); importance matrix on 128 chunks of 512 tokens of wikitext-2 "train"; quantization to each type with llama.cpp's default mix; real bits read in the file; perplexity and Kullback–Leibler divergence against the baseline logits on 40 chunks of 512 tokens of wikitext-2 "test". The importance matrix $I_j = \sum_t x_{t,j}^2$ accumulates the square of each input column on the calibration text; quantization then minimizes $\sum_j I_j (w_j - q_j)^2$, the diagonal approximation of the Hessian $X^\top X$ used by GPTQ and QuIP#.

Table 5.5: The llama.cpp ladder on Qwen3.5-4B and 9B; ±: llama-perplexity standard error over the 40 chunks [meas.].

Model	type	bits/weight	GB read/token	PPL	KLD	same 1st token
4B	Q4_K_M (base)	5.193	2.730	8.723 ± 0.229	0	100 %
4B	IQ4_XS	4.762	2.503	8.908 ± 0.236	0.030	91.9 %
4B	IQ3_XXS	3.602	1.894	9.777 ± 0.259	0.148	82.1 %
4B	Q2_K	3.623	1.905	10.957 ± 0.300	0.254	77.4 %
4B	IQ2_M	3.292	1.731	10.956 ± 0.293	0.269	76.4 %
4B	IQ2_XXS	2.905	1.527	15.493 ± 0.437	0.608	65.8 %
4B	IQ1_S	2.565	1.348	48.027 ± 1.467	1.780	44.7 %
9B	Q4_K_M (base)	5.066	5.097	7.174 ± 0.178	0	100 %
9B	IQ3_XXS	3.509	3.490	7.913 ± 0.198	0.113	85.0 %
9B	IQ2_M	3.213	3.159	8.612 ± 0.217	0.206	80.2 %
9B	IQ2_XXS	2.761	2.757	10.605 ± 0.271	0.425	72.3 %

Data: `fondations/journaux/02-quantification/csv/echelle.csv`, logs `echelle-4B/`, `echelle-9B/`.

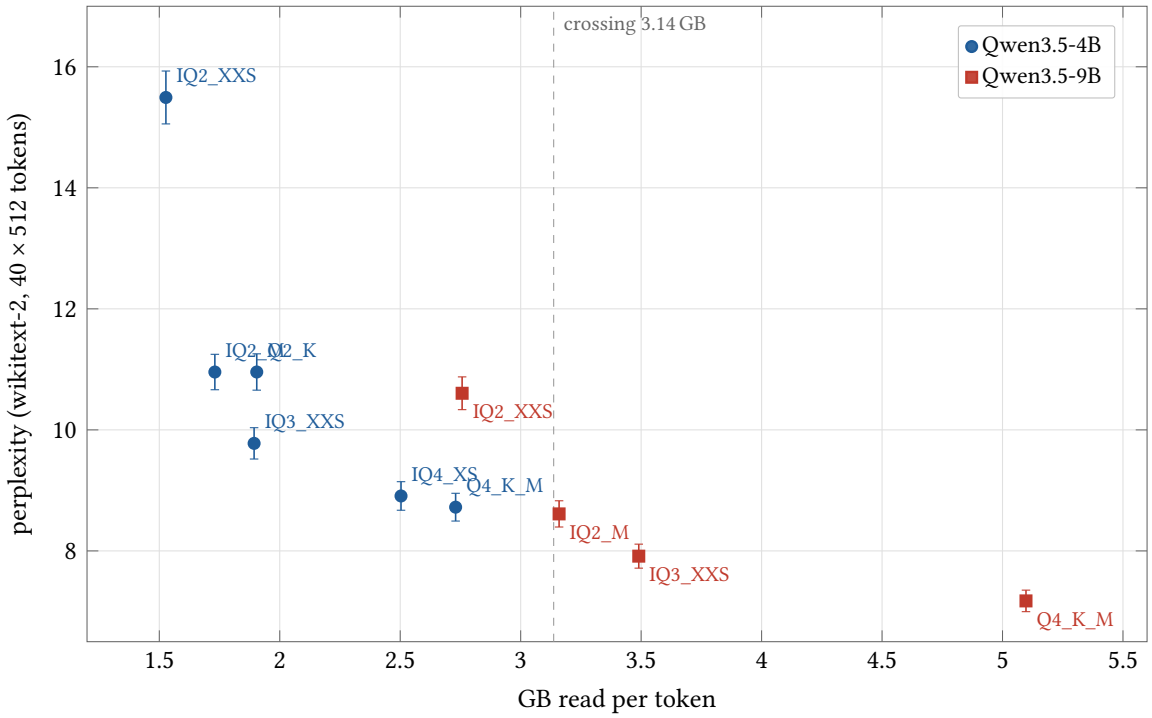


Figure 5.8: Perplexity versus bytes read per token, Qwen3.5-4B and 9B under llama.cpp [meas.]. Bars: ± one standard error. Below about 3.14 GB read per token, the 4B wins; above, the compressed 9B dominates the whole front of the 4B.

Data: `fondations/journaux/02-quantification/csv/fig-q3-4B.csv`, `fig-q3-9B.csv`.

What the ladder says (Table 5.5 and Figure 5.8):

- **At equal bytes, the i-quant beats the K-quant by 11 %**: IQ3_XXS 9.777 against Q2_K 10.957 for 1.89–1.91 GB read per token (llama.cpp's "Q2_K" is in fact at 3.6 bits per weight: its sensitive matrices go up to Q3_K and Q4_K).
- **IQ1_S is unusable on an ordinary 4B** (perplexity multiplied by 5.5); 1.56 bits only suits models trained for it.
- **The 4B at 4 bits beats the 9B at 2 bits by 22 % in perplexity, within 1 % of bytes** (2.730 against 2.757 GB; 8.723 against 10.605); the two-standard-error intervals, [8.27; 9.18] and [10.06; 11.15], are disjoint. With today's best portable 2-bit format, shrinking beats compressing for Qwen3.5.
- **The crossing happens near 3.2 bits per weight, not at 2.** By linear interpolation of the 9B front between IQ2_XXS and IQ2_M, it reaches 8.723 at $2.757 + \frac{10.605-8.723}{10.605-8.612} \times 0.402 = 3.137$ GB, i.e. $3.137 \times 8/7.94 = 3.16$ bits per weight read [calc.]. Beyond it, the compressed 9B is better than all of the measured 4B.

In VRAM, bytes saved only half turn into speed. The retained speed pass (quiet machine, `-lm none`, 128 tokens × 5, ABBA, reference noise 0.1 to 2.8 % against a 5 % threshold) gives Figure 5.9. From Q4_K_M to IQ2_XXS, bytes read are divided by 1.79 (4B) and 1.85 (9B), speed multiplied by only 1.34 and 1.49: the share of the 256 GB/s ceiling falls from 74–84 % to 55–68 %. Decoding the i-quants (grids in shared memory) and the fixed cost per token take the rest. Outside VRAM, the ratio reverses: at the link's throughput, i-quant decoding (at least 142 GB/s effective, measured) runs ten times faster than the link, and only bytes count.

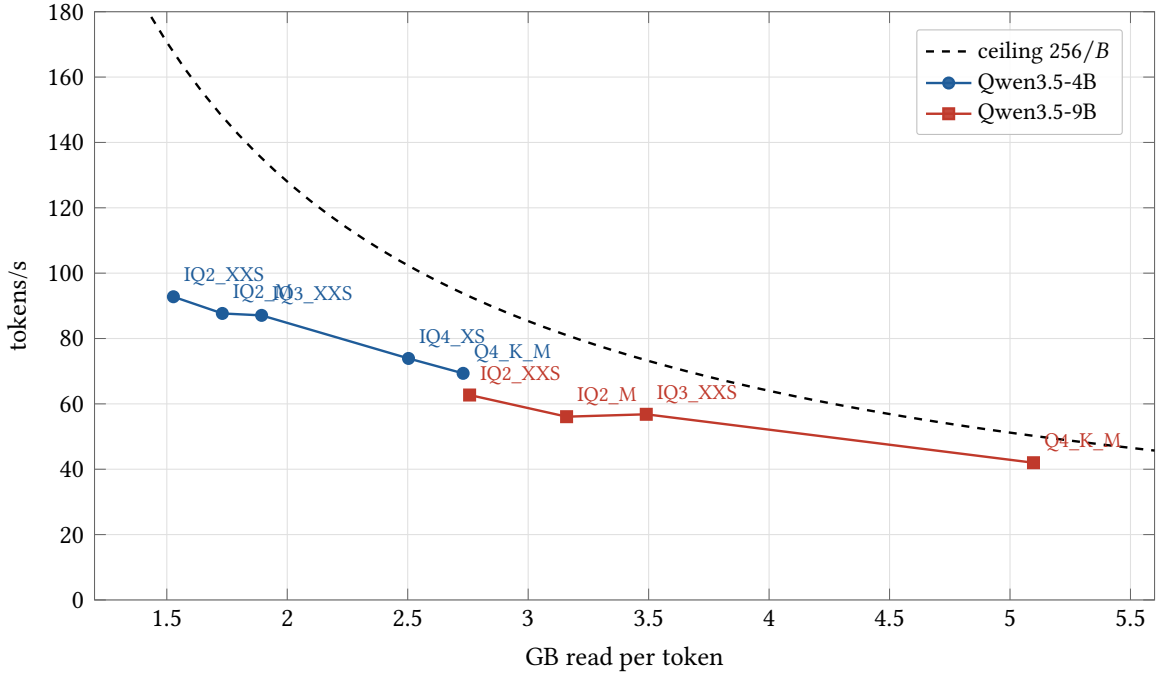


Figure 5.9: Decoding speed under llama.cpp (CUDA, batch 1, all in VRAM) versus bytes read per token [meas.]. The dotted curve is the ceiling of theorem 3.2 at 256 GB/s: the most compact formats drift away from it, because their decoding is paid for in VRAM.

Data: `fondations/journaux/02-quantification/csv/fig-q6-vitesse-*.csv` (logs `vitesse-4B-3.json`, `vitesse-9B-3.json`).

5.9 At equal bytes read: today's best trade-off

We compare at equal bytes read, since that is what each token pays. The only published set where all methods and all sizes come from the same program is QTIP's table on Llama 2 (Figure 5.10).

- **With a trellis and fine-tuning, a model twice as large at 2 bits equals or beats the half-size model at 4 bits**, on Llama 2 [lit.]. With scalar quantization (EfficientQAT [25]), it is the opposite: the crossing depends on the method as much as on size.
- **On a heavily trained family (Qwen3), with the codes of September 2026, the crossing happens from 8B** on MMLU [34] (Figure 5.11). The authors write that this comparison was chosen after the measurement.
- **With llama.cpp's i-quants, the crossing does not happen at 9B** (Section 5.8).

From what size do 2 bits become acceptable? Delegated answer of 2026-10-15, reversible: "acceptable" means the model of size $2N$ at $b/2$ bits is at least as good as the model of size N at $b \approx 4$ bits, at equal bytes read; it is the only criterion that says whether compressing beats shrinking. The relative loss of "IQ2_XXS" against the baseline of the same size is $\ell(4.21\,\text{G}) = 0.777$ and $\ell(8.95\,\text{G}) = 0.479$ [meas.]. A power law through these two points gives $\ell(N) = 0.479\,(N/8.95)^{\alpha}$ with $\alpha = \ln(0.479/0.777)/\ln(8.95/4.21) = -0.640$ [calc.]. It falls below the quality gap of a doubling of size, 22 % between 4B and 9B, near $N = 8.95 \cdot (0.22/0.479)^{1/\alpha} \approx 30$ billion parameters, and below 10 % near 100 billion. **With IQ2_XXS on Qwen3.5, the threshold lies beyond 30B; with a second-generation code, Qwen3 crosses it at 8B: the coder moves the threshold by a factor of 4 to 10.** Only two points carry this law: it is an order of magnitude.

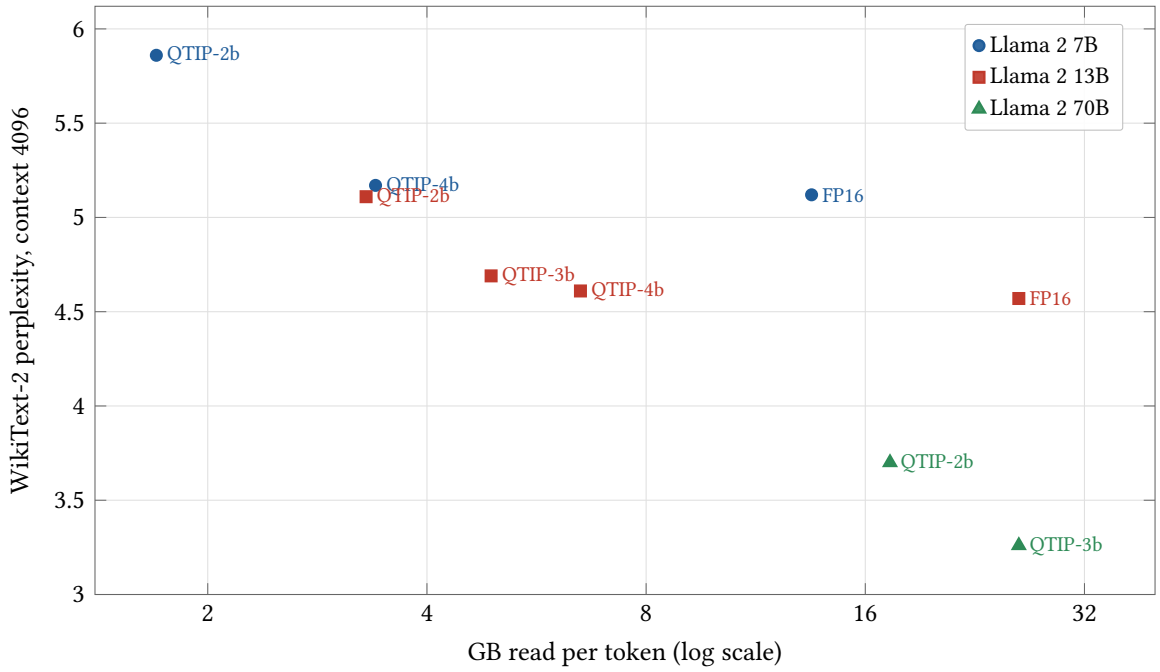


Figure 5.10: At equal bytes read, Llama 2 quantized with QTIP trellis codes [lit.] [calc.]. A 13B at 2 bits (3.3 GB, 5.11) equals a 7B at 4 bits (3.4 GB, 5.17); a 70B at 2 bits (17 GB, 3.70) beats a 13B in FP16 (26 GB, 4.57).

Data: `fondations/journaux/02-quantification/csv/fig-q4-llama2-*.csv` (QTIP, table 5).

5.10 Splitting bits by stage

The published tools (dynamic HIGGS, EXL3's recipes, llama.cpp's `--tensor-type`) minimize the error under a *storage* budget. In troncons, a byte read in VRAM and a byte brought across the link do not have the same price.

Assumption 5.7 (Linearity, Shannon slope, given placement)**.** (H1) The quality loss is $\sum_i c_i \varepsilon_i$, where ε_i is the relative error of matrix i and $c_i \geq 0$ its sensitivity (HIGGS linearity theorem). (H2) $\varepsilon_i = \kappa\, 2^{-2b_i}$ (theorem 5.1, up to a constant per coder). (H3) Matrix i, of n_i weights, is served by a stage of fixed throughput W_i. (H4) The transfer time per token $\sum_i n_i b_i/(8W_i)$ does not exceed a budget T.

Proposition 5.8 (Optimal bits per matrix)**.** *Under assumption 5.7, the problem* $\min_b \sum_i c_i \kappa 2^{-2b_i}$ *subject to* $\sum_i n_i b_i/W_i \leq 8T$ *has solution*

$$b_i^\star = \tfrac{1}{2} \log_2\!\left(\frac{2 \ln 2\, \kappa\, c_i\, W_i}{\lambda\, n_i}\right), \tag{5.3}$$

where $\lambda > 0$ *is set by the active constraint.*

Proof. The Lagrangian $\mathscr{L} = \sum_i c_i \kappa 2^{-2b_i} + \lambda(\sum_i n_i b_i/W_i - 8T)$ is convex in b: a sum of decreasing exponentials and a linear term. The first-order condition $\partial \mathscr{L}/\partial b_i = -2 \ln 2\, c_i \kappa\, 2^{-2b_i} + \lambda n_i/W_i = 0$ gives $2^{-2b_i} = \lambda n_i/(2 \ln 2\, c_i \kappa W_i)$, hence the formula. Convexity makes it the global minimum; the constraint is active at the optimum, since the loss strictly decreases in each b_i. ∎

Corollary 5.9 (The bits-per-stage rule)**.** *Two matrices with the same sensitivity density* c_i/n_i*, one served by a stage of throughput* W_1*, the other by a stage* W_2*, receive*

$$b_1^\star - b_2^\star = \tfrac{1}{2} \log_2 \frac{W_1}{W_2} \tag{5.4}$$

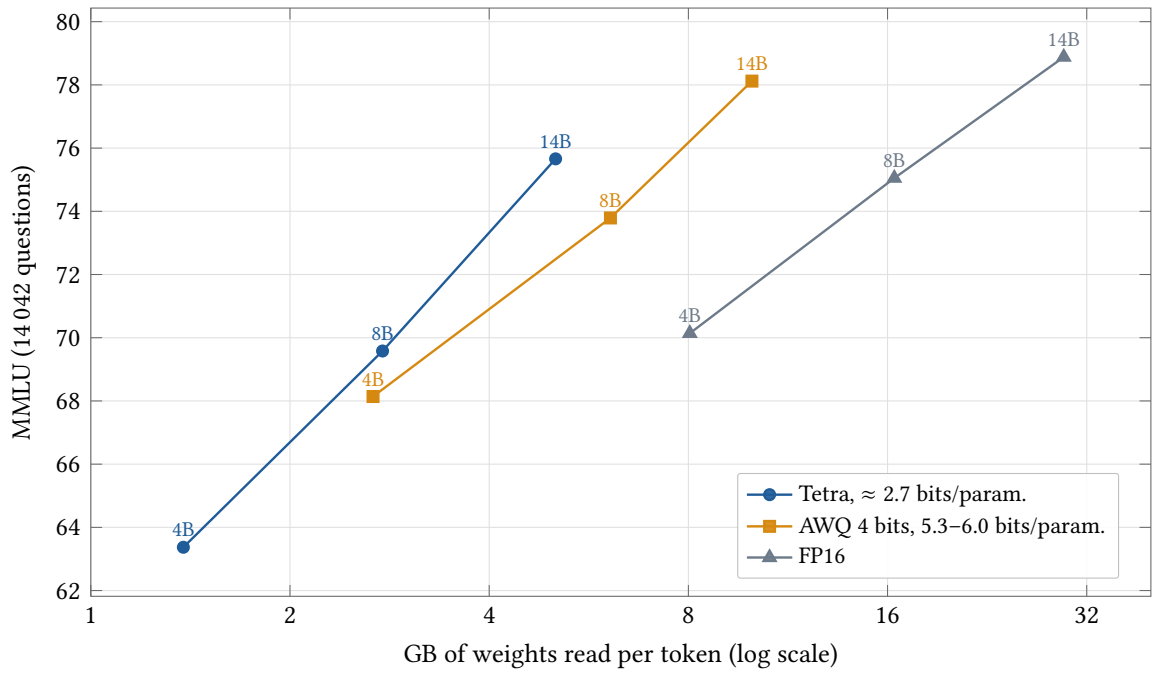


Figure 5.11: At equal bytes read, Qwen3 on MMLU: Tetra's Leech-lattice code at about 2.7 bits per parameter against AWQ 4-bit and FP16 [lit.] [calc.]. Qwen3-8B Tetra (2.76 GB) beats Qwen3-4B AWQ (2.67 GB), 69.6 against 68.1.

Data: `fondations/journaux/02-quantification/csv/fig-q4-qwen3-mmlu-*.csv` (Tetra, table 3).

bits of difference. With the throughputs measured in Chapter 3: ***2.18 bits*** *between VRAM and the PCIe link* ($\frac{1}{2}\log_2(251.3/12.2)$)*,* ***3.24 bits*** *between VRAM and disk* ($\frac{1}{2}\log_2(251.3/2.82)$)*, 1.06 bits between link and disk [calc.].*

Proof. Take the ratio of the two instances of Equation (5.3): λ, κ and c_i/n_i cancel. ■

The original study had quantified this rule with a link at 7 GB/s: ~~a VRAM-resident matrix receives $\frac{1}{2}\log_2(256/7) \approx 2.6$ more bits than a matrix read over PCIe~~. That throughput was a loaded state of the machine; at rest, the link delivers 12.2 GB/s (Section 3.5), and the gap drops to 2.18 bits. Weights read from disk, for their part, deserve 3.24 bits fewer than residents: residents at 4 to 5 bits, the disk stream around 1 to 2 bits.

Proposition 5.10 (Frequency decides placement, sensitivity decides bits)**.** *Add a usage probability p_i per token (experts of an MoE; $p_i = 1$ for a dense model): time becomes $\sum_i p_i n_i b_i / W_i$ and loss $\sum_i p_i c_i \kappa 2^{-2b_i}$. Then (a) $b_i^\star$ is given by Equation (5.3) and does not depend on p_i; (b) the gain of making matrix i resident is proportional to p_i and is, when $W_V \gg W_P$, about $\lambda\, p_i\, n_i\, (\frac{1}{2\ln 2} + b^\star_{i,P})/W_P$.*

Proof. (a) In the first-order condition, p_i multiplies both terms and cancels. (b) At the optimum, $c_i\kappa 2^{-2b^\star} = \lambda n_i/(2\ln 2\, W)$; the contribution of the matrix to the Lagrangian is thus $\Lambda_i(W) = p_i\big(c_i\kappa 2^{-2b^\star} + \lambda n_i b^\star/W\big) = \lambda\, p_i\, n_i\, (\frac{1}{2\ln 2} + b_i^\star(W))/W$, and the residency gain is $\Lambda_i(W_P) - \Lambda_i(W_V)$, dominated by the first term when $W_V \gg W_P$. ■

Hence the VRAM filling rule: **sort by usage frequency first, sensitivity second**. To our knowledge, no published tool poses this joint (bits, stage) problem; Q-Palette [97] poses it at fixed placement with a profiled latency cost.

The nested code. ApexQuant [98] requantizes the residual after a fresh random rotation, pass after pass; every prefix of the passes is a valid model, and the "price of elasticity" is a constant distance to the bound per pass: 0.45 bit for a scalar quantizer, 0.10 bit for a trellis [lit.]. In troncons, this gives a single file for both stages: a streamed matrix reads only its pass 1 (about 2 bits), a resident matrix reads passes 1 and 2 (about 4 bits), and when the residency window moves, nothing is requantized. Cost [calc.]: about 0.10 bit on the residents, i.e. 2.5 % of VRAM bytes; gain over two separate copies (2 + 4 bits): 1.9 bits per weight of disk, 32 % of storage. It is the rule of corollary 5.9 made dynamic; no published engine reads precision prefixes according to the memory stage.

5.11 The techniques, in one table

Table 5.6: Extreme quantization techniques, state at 2026-10-15. Losses: WikiText-2 perplexity, FP16 → quantized [lit.].

Technique	bits/weight	Published loss (examples)	Vulkan	Licence
K-quants (llama.cpp)	2.625–6.5625	Q2_K: +3.52 on Llama 3 8B	**yes**	MIT
i-quants + imatrix	1.5625–4.25	IQ2_XXS: 7B 5.12→7.07; 70B 3.12→4.08	**yes**	MIT
EfficientQAT [25]	≈ 2.25	L2-7B 5.47→6.86; L3-70B 2.85→6.08	no	MIT
HIGGS [26]	2–4.25	L3.1-8B 5.607→6.388 at 3.25 bits	no	Apache-2.0
QuIP# [27]	2, 3, 4	70B 3.12→3.91 at 2 bits	no	GPL-3.0
AQLM [28] + PV-tuning [29]	1.0–2.3	L2-7B 5.12→5.84; L3-70B 2.59→4.57	no	Apache-2.0
VPTQ [30]	1.375–4	L2-70B 2.07 bits: 3.93	no	MIT
QTIP [31]	2, 3, 4	2 bits: 7B 5.86; 70B 3.70	no	GPL-3.0
CurveTQ [32]	2.29 all-in	Qwen3-4B 9.72→11.50; Qwen3-32B 6.17→6.90	no (CUDA)	—
XOR-Trellis [33]	≈ 2	L3.1-8B 5.84→8.00 (QTIP 8.48)	not published	—
Tetra [34]	2.148 read	MMLU Qwen3-8B 69.6 (FP16 75.1)	no (CUDA)	Apache-2.0
GSQ [99]	2.13 / 3.13	projectable to Q2_K / Q3_K_M	**yes if projected**	Apache-2.0
ApexQuant [98]	$P \times b$, prefixes	every prefix valid	not published	undeclared
EXL3	1–8	repository curves	**no** (CUDA)	MIT
IQn_KT (ik_llama.cpp)	1.75–4.0	error ≈ random VQ 8 [meas.]	no	MIT
BitNet b1.58 [35]	1.58	native, on par with full models (2B4T)	via TQ1_0	MIT
MXFP4 / NVFP4 [38]	4.25 / 4.5	native NVFP4 "comparable to FP8"	storage yes	—

5.12 What is missing in the world

1. **No trellis decoder (QTIP, EXL3 class) in Vulkan or DirectX 12**, apart from this chapter's. At stake [calc.]: at 2 bits, trellis 0.068 against 0.093 for an 8-dimensional VQ and 0.107 for IQ2_XXS, i.e. 0.23 to 0.33 bit per weight at equal quality (lemma 5.3); without fine-tuning, the published perplexity gain is smaller (QTIP 3INST 6.82 against IQ2_XXS 7.07 on Llama 2 7B). CurveTQ shows that a rotation-free trellis, with curvature in the Viterbi metric and an initial state stored per block, beats QTIP: the decoder to write is thus simpler than the one measured here, and the code to port is mul1 with `dot4U8Packed`.
2. **Nobody allocates bits according to transfer cost** (Proposition 5.8); it is a direct contribution for troncons.
3. **No nested code is tied to placement.**
4. **No published equal-bytes frontier for heavily trained families below 3 bits with portable kernels**; Figure 5.8 gives a first point for Qwen3.5.

5. **Quantizing a large model on the machine**: QTIP and PV-tuning require days of datacenter GPUs; a trellis quantizer working matrix by matrix from disk, on the card, in Vulkan, does not exist.
6. **The best 2-bit format decodable in Vulkan is still a poorly filled K-quant**: GSQ raises a Qwen3-8B Q2_K from 50.03 to 56.28 on average without changing the format [lit.].

What could be wrong

- **The card and the processor were shared**: quantization times are upper bounds; quality measurements do not depend on it.
- **The baselines are Q4_K_M, not BF16**: requantization adds the Q4_K_M error, estimated at about 10 % of the loss at 2 bits, which makes the low-rate results pessimistic by as much. The KL divergence is measured against the Q4_K_M.
- **The importance matrix is computed on the same domain as the test** (optimistic bias for the i-quants).
- **20 480 evaluation tokens**; the absolute perplexity is not comparable to published numbers at context 4096.
- **Literature numbers come from different programs**; only comparisons inside one paper or inside our ladder are clean.
- **The error on a Gaussian is not model quality**; the step to perplexity relies on a linearity theorem valid for small errors, which 2 bits no longer are.
- **The bits-per-stage rule** assumes additive errors and the Shannon slope: it gives the direction and the order of magnitude, not the value.
- **The October 2026 front is not reproduced**; CurveTQ, Tetra, XOR-Trellis and ApexQuant are less than six weeks old.
- **The extrapolation to 30B** rests on two points.

Chapter 6

Lossless compression at the speed of compute

Where we start, what this chapter establishes, what remains

At the start, the literature promised two contradictory things: lossless coders that win 30 % on BF16 weights (DFloat11, ZipNN), and the statement that quantized GGUF files "do not compress at all". An ISCA 2026 paper further announced "0.6 to 1.0 bit of true entropy" for 4-bit weights, which would have meant a factor of 4 hidden in every quantized model. No entropy decoder for weights had a published throughput under Vulkan, nor on a laptop card. **This chapter establishes** what a quantized code on the machine really contains (3.86 bits out of 4, order 0 says it all); why, through an exact calculation that recovers the measurements to the hundredth; that the target format, MXFP4, hides its gain in its scales, and that a field-wise coding takes it exactly (rate 1.1026 on gpt-oss-120b, decoded bit for bit); an interleaved rANS coder whose correctness is proved, decoded on the card at 70 GB/s through Vulkan and 82 GB/s through CUDA; and the measured answer to the key question: decoding *inside* the product is 3.7 times slower, decoding *on arrival* costs only 3.6 % of the card's time and gives the whole rate back to the disk. **What remains** is the quantization tailored to the coder (0.41 to 0.46 bit per weight to take, measured on surrogates), writing the decoder's output directly at its final place, and reproducing DietGPU and Recoil on the machine. Source: `fondations/03-entropie.md`, `fondations/03-entropie-derivations.md` (version of commit `f75d2cc`), `fondations/journaux/03-entropie/`.

6.1 The judge: when lossless pays

Compressing losslessly by a rate r divides by r the bytes that cross the disk, the RAM and the link. It only wins if the decoder keeps up: its output throughput must exceed r times that of the stage it relieves, and its compute time must not eat the gain. In this chapter, W_{dec} is the output throughput of the decoder on the card (decoded bytes per second) and W_g the throughput at which the matrix–vector product consumes decoded weights, as in Chapter 4.

Proposition 6.1 (Time of a token, overlapped chain, decoding on the card)**.** *Let a token read B bytes of (uncompressed) weights through the disk and the link, stored at rate r. If the stages overlap and if the decoder and the product share the card, then*

$$t(r) = \max\Big(\frac{B}{r\,W_{\text{disk}}},\ \frac{B}{r\,W_{\text{PCIe}}},\ B\Big(\frac{1}{W_{\text{dec}}} + \frac{1}{W_g}\Big)\Big). \tag{6.1}$$

Proof. This is theorem 3.2 applied to three stages: the disk and the link see B/r bytes; the card runs the decoding (B/W_{dec} seconds) then the product (B/W_g) in series, which makes it a single stage

of throughput $(1/W_{\text{dec}} + 1/W_g)^{-1}$. Overlap makes the bound reachable up to the fill penalty of lemma 3.3. ■

Corollary 6.2 (The gain is exactly r, then saturates). *If $W_{\text{disk}} \le W_{\text{PCIe}}$, the gain $t(1)/t(r)$ equals exactly r as long as $r\,W_{\text{disk}} \le (1/W_{\text{dec}} + 1/W_g)^{-1}$, and saturates beyond at $W_{\text{disk}}\,(1/W_{\text{dec}} + 1/W_g)$.*

Proof. Under the condition, the first term of (6.1) dominates for every rate between 1 and r, so $t(1)/t(r) = (B/W_{\text{disk}})/(B/(rW_{\text{disk}})) = r$. Beyond, the third term dominates $t(r)$ and no longer depends on r. ■

The calculation, with the machine's throughputs. $W_{\text{disk}} = 2.82$ GB/s (Chapter 3), $W_{\text{dec}} = 70$ GB/s (v4 decoder under Vulkan, on mains, Section 6.6) and $W_g = 209$ GB/s (raw MXFP4 product, Section 6.7) give $(1/70 + 1/209)^{-1} = 52.4$ GB/s [calc.], against $r\,W_{\text{disk}} = 1.10 \times 2.82 = 3.1$ GB/s. The condition holds by a factor of 17: **every percent of compression becomes a percent of tokens per second**, and the card is only 6 % busy with decoding and product together. On battery, where the card is capped at 43 W ($W_{\text{dec}} = 26.8$, $W_g = 66.4$), the card's term is still 19.1 GB/s, six times $r\,W_{\text{disk}}$ [calc.].

The rate acts twice on a mixture of experts. A memory that holds a share h of the experts in the clear holds $h\,r$ of them if they are coded; the bytes read from disk per token go from $(1-h)B_e$ to $(1-hr)B_e/r$. The study thus put +14.4 % tokens per second for gpt-oss-120b at $r = 1.1026$ and $h = 26$ % (uniform routing). ~~+14.4 % tokens per second~~ Chapter 4 corrected this figure: the card must keep its experts *decoded* (Section 6.7), and only the RAM grows; the effect is +13.1 % [calc.].

6.2 What a quantized code contains

6.2.1 The estimator, and its bias

The entropy of a field (the 4-bit codes, the scales of a block) is estimated from the observed frequencies: $\hat{H} = -\sum_s \hat{p}_s \log_2 \hat{p}_s$, $\hat{p}_s = n_s/N$, over all blocks of a model. A plug-in estimator underestimates the entropy; one must know by how much before reading a difference of a few thousandths of a bit.

Lemma 6.3 (First-order bias). *For N independent draws from a law with K symbols of non-zero probability,*

$$\mathbb{E}[\hat{H}] = H - \frac{K-1}{2N \ln 2} + O(N^{-2}). \tag{6.2}$$

Proof. Let $\phi(p) = -p \ln p$, so $\phi''(p) = -1/p$. A second-order Taylor expansion around p_s gives $\mathbb{E}[\phi(\hat{p}_s)] = \phi(p_s) + \frac{1}{2}\phi''(p_s)\,\mathrm{Var}(\hat{p}_s) + O(N^{-2})$, the first-order term vanishing since $\mathbb{E}[\hat{p}_s] = p_s$. With $\mathrm{Var}(\hat{p}_s) = p_s(1-p_s)/N$, each symbol contributes $-(1-p_s)/(2N)$, and the sum over the K symbols is $-(K - \sum_s p_s)/(2N) = -(K-1)/(2N)$ nats. Divide by $\ln 2$ for bits. ■

For the Q4_K codes of Qwen3.5-9B, $K = 16$ and $N \approx 7.4 \cdot 10^9$: the bias is $15/(2 \times 7.4 \cdot 10^9 \times 0.693) = 1.5 \cdot 10^{-9}$ bit [calc.], negligible. For the fp16 field d of a small tensor ($K \approx 5\,000$ values seen, $N = 40\,000$ blocks), it reaches 0.09 bit [calc.]: gains on fp16 metadata are therefore optimistic, and the rate of a real coding is preferred to them.

6.2.2 The codes: almost uniform, and order 0 says it all

The study wrote its own GGUF reader and split each block into its fields (Figures 5.2 and 5.3); the dequantization redone from these fields equals that of llama.cpp's `gguf` library with zero difference, and the control (two nibbles swapped in our path only) breaks the equality [meas.] (`temoin-champs.log`). Table 6.1 gives what remains in each block type of Qwen3.5-9B.

Table 6.1: Entropy of the K-quant blocks of Qwen3.5-9B Q4_K_M, by type [meas.]. "Ideal": order-0 coding of every field (codes and metadata); rate = stored bits / ideal. zstd at level 3 and lz4 on whole tensors. The 4B gives 5.187 → 4.930 bits per weight (rate 1.0521).

type	share	stored	H_0 codes	H | prev.	ideal	rate	zstd 3	lz4
Q4_K	58.5 %	4.500	3.864 / 4	3.858	4.256	1.0574	1.0215	1.000
Q5_K	14.7 %	5.500	4.861 / 5	4.853	5.253	1.0470	1.0000	1.000
Q6_K	26.7 %	6.562	5.789 / 6	5.785	6.274	1.0460	1.0000	1.000
Q8_0	0.1 %	8.500	7.556 / 8	7.185	7.807	1.0888	1.0546	1.000
model	100 %	**5.062**			**4.808**	**1.0529**	**1.0125**	**1.000**

Data: `fondations/journaux/03-entropie/entropie-9B-resume.txt`, `entropie-4B-resume.txt`.

Three facts stand out. **The codes are almost uniform**: 3.864 bits out of 4 for Q4_K, 4.86 out of 5, 5.79 out of 6. **Context brings nothing**: the previous code removes 0.005 bit, the sub-block scale no more; everything is in order 0. **One table is enough**: one frequency table per tensor against one for the whole model, 3.86519 against 3.86521 bits per code [meas.] (`rans-4B-*.json`); the roles (`ffn_gate`, `attn_*`, ...) differ by less than 0.01 bit. Figure 6.1 shows the shape of the law: a bell shifted toward codes 7 and 8, with raised tails at both ends, the signature of a scale–minimum fit that clips to fill the 16 levels.

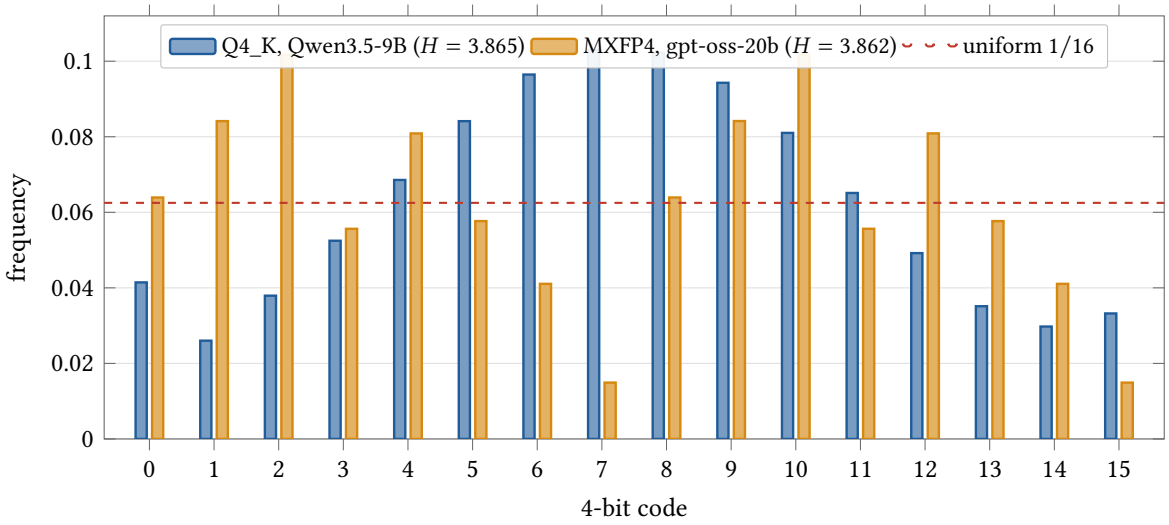


Figure 6.1: Frequency of each of the 16 codes, all Q4_K tensors of Qwen3.5-9B and all MXFP4 experts of gpt-oss-20b [meas.]. Both laws are close to uniform (3.865 and 3.862 bits out of 4); MXFP4's is that of an E2M1 sign–magnitude code (codes 0 to 7 positive, 8 to 15 negative).

Data: `fondations/journaux/03-entropie/csv/fig-histogramme-codes.csv` (from `entropie-9B.json` and `entropie-gptoss20b.json`).

6.2.3 The metadata, and what a byte coder sees

The metadata of a Q4_K block weigh 0.5 bit per weight: d and $d_{\min}$ in fp16 carry 9.46 to 9.53 and 10.11 to 10.35 bits out of 16, the 6-bit scales and minima carry 4.89 and 5.19 bits [meas.]. Their order-0 ideal, per weight, is

$$\frac{H(d)+H(d_{\min})}{256}+\frac{8\left(H(\text{scale}_6)+H(\text{min}_6)\right)}{256}=\frac{9.46+10.35}{256}+\frac{8\,(4.89+5.19)}{256}$$
$$=0.0774+0.315=0.392 \text{ bit} \quad \text{[calc.]},$$

a gain of 0.108 bit per weight, against $4-3.864=0.135$ for the codes. **Coding the codes alone leaves almost half of the gain on the table.**

Table 6.2: Rate of an order-0 coding, depending on what the coder sees [meas.]. The first row bounds DEFLATE, GDeflate and the literals of zstd; the second is ZipNN's "byte grouping" transposed to the fields of K blocks; the third codes whole symbols (5 and 6 bits for Q5_K and Q6_K).

order-0 coding	Q4_K	Q5_K	Q6_K	9B model
bytes, whole block, one table	1.0267	1.0031	1.0085	1.0183
bytes, one stream per field	1.0519	1.0179	1.0325	1.0416 [calc.]
whole symbols per field	1.0574	1.0470	1.0460	1.0529

Data: `fondations/journaux/03-entropie/flux-octets-9B.json`, `csv/flux-octets.csv`.

Table 6.2 reconciles the published facts. ZipNN writes that GGUF files "do not compress" [42]: true for an LZ and for a byte coder over the whole block (1.8 % at best). No longer true for a coder that knows the fields: for Q4_K, a byte coder per field takes almost everything (1.052 out of 1.057). For Q5_K and Q6_K, the 5th and 6th bits are stored apart (field qh), and a byte coder sees them as noise (7.999 bits per byte on the qh of Q5_K [meas.]): alphabets of 32 and 64 symbols are needed.

6.3 Why so little: the entropy of a min-max

The codes of a K-quant are almost uniform because the quantization is made for that: a min-max per group (or its optimized version, which clips) spreads the weights of a group over the whole grid. How much entropy is left then? The question has an exact answer, which this thesis computes along a path independent of the measurement (`memoire/outils/calculs_entropie.py`).

Definition 6.4 (Two 4-bit grids)**.** Let X be a source of variance σ^2, quantized over a range of extent R. The *min-max grid* (with zero point) has 16 levels $-R/2+k\,R/15$, $k=0,\dots,15$, and codes each weight by the nearest level, the two extreme levels taking the tails. The *symmetric grid* has 15 levels $k\,R/14$, $k=-7,\dots,7$ (the maximum of $|X|$ is $R/2$, level 0 at the centre).

Proposition 6.5 (Entropy of the codes at high resolution)**.** *If the density p of X is regular enough to be almost constant on each cell of step Δ, and if the range covers almost all the mass, then*

$$H(\text{codes})=h(X)-\log_2\Delta+o(1), \qquad h(X)=-\int p\log_2 p, \tag{6.3}$$

with $h=\frac{1}{2}\log_2(2\pi e\,\sigma^2)$ for a Gaussian and $h=\log_2(\sqrt{2}\,e\,\sigma)$ for a Laplacian.

Proof. Cell k has probability $p_k = \int_{\text{cell}} p \approx p(x_k)\,\Delta$. Then

$$-\sum_k p_k \log_2 p_k \;\approx\; -\sum_k p(x_k)\,\Delta \log_2 p(x_k) \;-\; \log_2 \Delta \sum_k p(x_k)\,\Delta;$$

the first sum is a Riemann sum of $-\int p \log_2 p$, the second tends to 1. For the Gaussian and for the Laplacian of parameter $b = \sigma/\sqrt{2}$,

$$-\mathbb{E}[\log_2 p(X)] = \tfrac{1}{2}\log_2(2\pi\sigma^2) + \frac{\mathbb{E}[X^2]}{2\sigma^2 \ln 2} = \tfrac{1}{2}\log_2(2\pi e\sigma^2), \qquad -\mathbb{E}[\log_2 p(X)] = \log_2(2b) + \frac{\mathbb{E}|X|}{b \ln 2} = \log_2(2eb).$$

■

The calculation for a min-max over groups of 32. For $\sigma = 1$, $h = \frac{1}{2}\log_2(2\pi e) = 2.047$ bits. The mean range of 32 Gaussian draws is about 4.1, so $\Delta \approx 4.1/15 = 0.273$ and $H \approx 2.047 + 1.87 = 3.92$ bits [calc.]. The exact calculation (fixed-seed Monte Carlo, 2^{18} weights per point, Figure 6.2) gives 3.865; the approximation overestimates by 0.05 bit because the two extreme weights of every group always land on codes 0 and 15. **The study's measurements on real weights** (min-max on three tensors, kurtosis 3.8 to 6.6) give 3.855 to 3.864 bits at $g = 32$ and 3.728 to 3.748 at $g = 64$, **against** 3.865 **and** 3.753 **computed for a Gaussian**: agreement to the hundredth. An entropy of 3.86 bits is not a mysterious property of neural networks: it is what any bell-shaped source gives under a min-max over small groups.

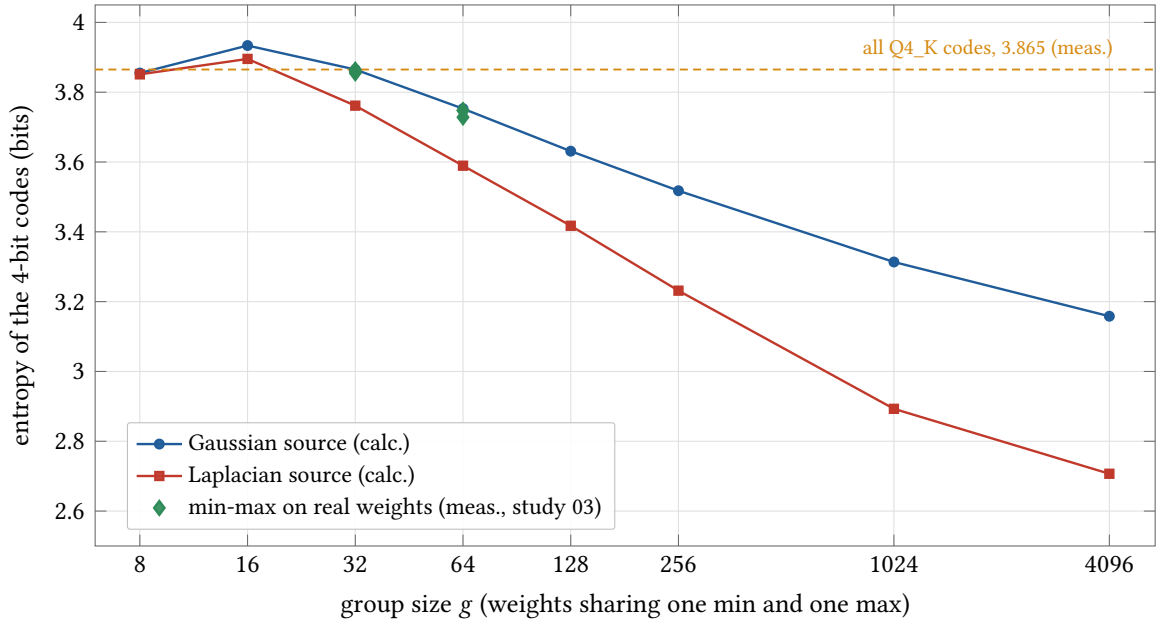


Figure 6.2: Entropy of the 4-bit codes of a min-max over groups of g weights [calc.], for a Gaussian and a Laplacian (kurtosis 6) source, and the min-max measured by the study on real weights [meas.]. Entropy decreases only slowly with group size: still 3.16 bits (Gaussian) and 2.71 (Laplacian) at $g = 4\,096$.

Data: `memoire/figures/donnees/entropie-groupes.csv` and `entropie-groupes-mesures.csv` (`calculs_entropie.py`, from `rd-proxy-4B.json` for the measurements).

6.3.1 A published contradiction, and a corrected sentence

An ISCA 2026 paper writes that int4 weights "retain only 0.6–1.0 bits of true entropy" [46] [lit.]; a discussion around MLX puts it at about 2.17 bits [100] [lit.], EntroLLM at 1.39 bit for a per-tensor uint4 [47] [lit.]. The study measures 3.864 to 3.865 bits on all Q4_K codes of two models and 3.862 on MXFP4.

For which range does one fall to 1 bit? Figure 6.3 answers exactly. **On the min-max grid, never**: zero falls on the boundary between codes 7 and 8, half of a symmetric source goes to each side, and the entropy decreases toward 1 bit without reaching it (still 1.01 bits at $R = 50\sigma$ for a Gaussian [calc.]). **On the symmetric grid**, 1 bit needs $R = 33.9\sigma$ (Gaussian) or 31.1σ (Laplacian), and 0.6 bit 45σ [calc.]: a maximum of $|X|$ at 17 to 22 standard deviations, which only a per-tensor or per-channel scale dominated by outliers produces, the case of EntroLLM, paid in perplexity (10.11 against 9.03 in fp16 on phi3 [lit.]).

The study wrote: "~~To fall to 1 bit, the range of a group would have to exceed about ten times the standard deviation of its weights.~~" The exact calculation contradicts it: at $R = 10\sigma$, the entropy is still 2.47 to 2.66 bits depending on grid and law; 1 bit needs 31 to 34σ on a symmetric grid and is never reached on a min-max grid [calc.] (`entropie-seuils.csv`). The study's conclusion comes out strengthened, not weakened: none of these published entropies applies to K-quants nor to MXFP4, and an announced gain of "1.8 to 6 times on top of quantization" is wrong for them by a factor of 30 to 100 on the gain (0.24 bit instead of 2 to 3). The published figure is not struck out: it was not reproduced on their weights.

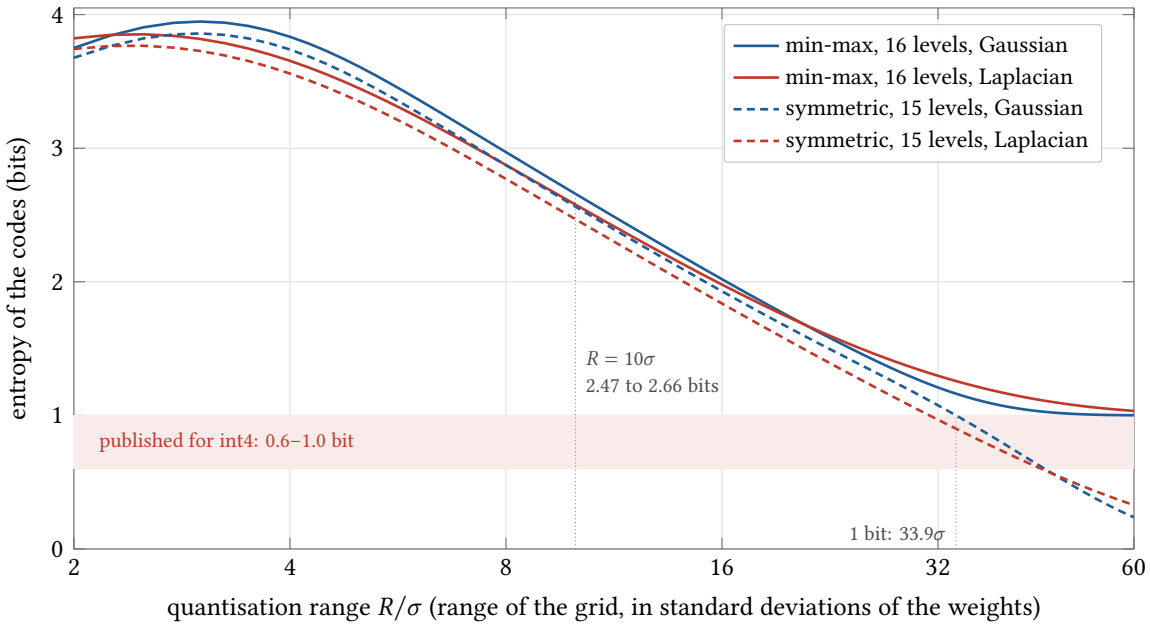


Figure 6.3: Exact entropy of the codes of a Gaussian or Laplacian source of unit variance, as a function of the extent R of the 4-bit grid [calc.]. Solid: 16-level min-max grid, bounded below by 1 bit. Dashed: 15-level symmetric grid, which drops below 1 bit at $R \approx 31$ to 34σ. The red band is the interval published for int4 [46].

Data: `memoire/figures/donnees/entropie-plage.csv`, `entropie-seuils.csv` (`calculs_entropie.py`).

6.4 MXFP4: the gain is in the scales

The experts of gpt-oss are in native MXFP4: blocks of 32 weights, 4-bit E2M1 codes, one one-byte E8M0 scale per block, 4.25 bits per weight (Figure 5.2). Its codes carry 3.862 bits, like those of K-quants. But **its scales carry most of the gain**, and this is where this chapter finds what nobody codes (Table 6.3).

Table 6.3: Entropy of the E8M0 scales of the 72 expert tensors of gpt-oss-20b, means weighted by block count [meas.]. The "row median" context is the best and introduces no dependency between blocks. On gpt-oss-120b (first 36 tensors): 1.232 at order 0, 1.004 given the left block, 0.910 given the median (plus 0.010 for the median).

quantity	bits	reading
$H(e)$, order 0	1.554 / 8	from 0.89 (`ffn_up`) to 2.28 (`ffn_down` of layer 0)
$H(e \mid \text{left block})$	1.152	
$H(e \mid \text{block above})$	1.538	neighbouring rows hardly resemble each other
$H(e \mid \text{left and above})$	1.146	above adds nothing to left
$H(e \mid \text{median of its row})$	**1.011**	plus 0.015 for the median; no dependency between blocks
$H(\text{code})$	3.862 / 4	almost uniform
$H(\text{code} \mid e)$	3.823	0.039 bit of code–scale mutual information

Data: `fondations/journaux/03-entropie/mxfp4-contexte-gptoss20b.json`, `mxfp4-contexte-gptoss120b.json`.

The estimator's control: the scales of the first tensor, randomly permuted, keep $H(e) = 2.283$ and lose the whole context gain ($H(e \mid \text{left}) = 2.283$) [meas.]. The estimator does not invent structure.

Why the row median. A matrix row (an output channel) has its own norm; the scale of a block follows it to within ±1, because an E8M0 scale is only an exponent, $\lceil \log_2 \max_{\text{block}} |w| \rceil$ up to an offset. Knowing the median $\tilde{e}$ of the row says almost everything, and the reconstruction reads no other block (Figure 6.4).

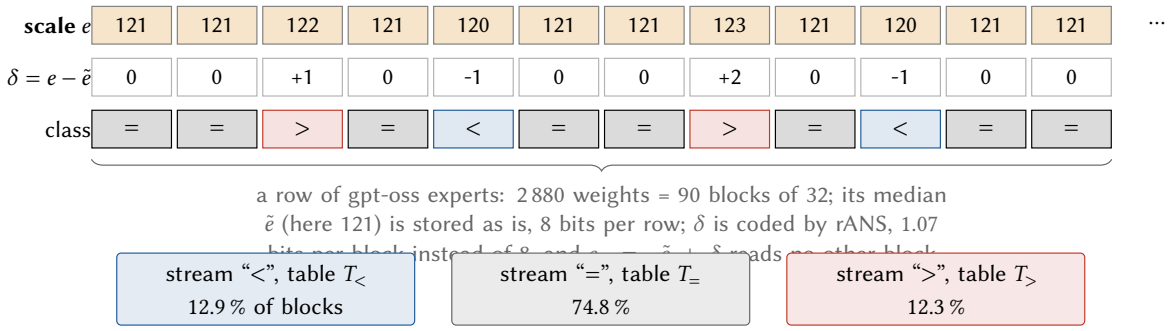


Figure 6.4: Coding the E8M0 scales of one row (example values). The row median is stored as is, the offset $\delta = e - \tilde{e}$ is coded by rANS. The block's class (below, at, above the median) selects the table of the code stream that carries it; the proportions are those measured on gpt-oss-20b [meas.].

Data: `fondations/journaux/03-entropie/proportions-classes-gptoss20b.txt`, `dispersion-classes-secteur.json`.

Proposition 6.6 (Local reconstruction of the scales). *With the coding $e = \tilde{e} + \delta$, where $\tilde{e}$ is the row median stored as is and δ a symbol of the rANS stream, every scale is rebuilt from its own symbol and*

one byte per row. Coding by difference with the left neighbour, $e_j = e_{j-1} + \delta'_j$, forces a prefix sum along the row and costs 0.126 *bit more per scale on gpt-oss-20b.*

Proof. The first claim is immediate: e depends only on $\tilde{e}$ and δ. For the second, $e_j = e_0 + \sum_{i \le j} \delta'_i$: every scale depends on all the symbols before it on the row, which is the definition of a prefix sum. The cost comes from Table 6.3: $H(e \mid \text{left}) - (H(e \mid \tilde{e}) + 0.015) = 1.152 - 1.026 = 0.126$ bit [calc.]. ■

The balance in bits per weight, step by step. Over the 72 tensors of gpt-oss-20b (10.15 GB), with a byte-alphabet rANS and one table per tensor [meas.] (`taux-mxfp4-gptoss20b.json`):

- codes: 3.866 bits per weight (for an entropy of 3.862);
- offsets δ: $H(\delta) = 1.071$ bit per block, i.e. $1.071/32 = 0.0335$ bit per weight;
- medians: 8 bits per row of 2 880 weights, i.e. $8/2\,880 = 0.0028$ bit per weight;
- two tables of 384 bytes per tensor, counted;
- scales, real total: 0.037 bit per weight, instead of 0.25;
- **total:** 3.903 **bits per weight for** 4.25 **stored, rate** $4.25/3.903 = 1.0888$.

The scales give back $0.25 - 0.037 = 0.213$ bit per weight, the codes $4 - 3.866 = 0.134$ [calc.]: **the scales are worth one and a half times the codes**. Each of the 72 tensors is decoded and compared byte for byte, codes and rebuilt scales: all exact; the control (a falsified median) makes the reconstruction inexact [meas.]. On gpt-oss-120b (108 tensors, 60.9 GB), the rate is 1.0887, within 0.0001 of the 20b: it does not depend on model size.

Three scale classes. The code–scale mutual information (0.039 bit) is cashed in without changing the decoder: blocks are split into three streams according to where their scale sits relative to the median (below, at, above), each stream having its own table; each workgroup on the card loads only the table of its stream, and the reassembly, guided by δ which the decoder already has, is checked byte for byte (Table 6.4).

Table 6.4: Real lossless rate of the MXFP4 experts, codes and scales, exact decoding of every tensor [meas.]. Three classes take 0.053 bit, more than the measured mutual information (0.039); five classes add only 0.0008 bit.

model	variant	codes (b/w)	total (b/w)	rate	exact
gpt-oss-20b, 72 tensors	one table per tensor	3.8662	3.9032	1.0888	yes
	three scale classes	**3.8135**	**3.8505**	**1.1037**	yes
	five classes ($\lvert\delta\rvert \le 2$)	3.8127	3.8498	1.1040	yes
gpt-oss-120b, 108 tensors	one table per tensor	3.8695	3.9039	1.0887	yes
	three scale classes	**3.8201**	**3.8545**	**1.1026**	yes

Data: `fondations/journaux/03-entropie/taux-mxfp4-classes-gptoss{20b,120b}.json`, `taux-mxfp4-classes5-gptoss20b.json`.

The physical reason for the class gain: a scale above its row median signals a block with one large weight; its other codes are then small, and their law tightens toward zero. The class relative to the row says more than the absolute value of the scale, and the byte alphabet adds the dependency between the two codes of one byte, which share their block. Figure 6.5 shows that the gain is uniform from one tensor to the next: from 1.0846 to 1.1041 with one table, from 1.0987 to 1.1177 with three classes, over the 72 tensors of the 20b (extreme values recomputed here from the log, equal to the study's [calc.]).



Figure 6.5: Real lossless rate of every expert tensor, one table per tensor (circles) and three scale classes (discs), by layer and role [meas.]. The down tensors, whose scales are the most disordered at order 0, gain the most (1.1051 on average with three classes on the 20b, against 1.1022 for `gate`): the row median absorbs that disorder (Figure 6.6).

Data: `memoire/figures/donnees/entropie-taux-{20b,120b}-{down,gate,up}.csv` (`donnees_figures.py`, from `taux-mxfp4-classes-gptoss{20b,120b}.json`).

All of gpt-oss-20b, field by field. Over the 11.5 GB of quantized tensors [meas.] (`entropie-gptoss20b-resume.txt`): the MXFP4 experts (9 682 MiB) have an order-0 ideal of 1.0869 (zstd 3: 1.0405; lz4: 1.000); the Q8_0 attention (645 MiB), 1.069 to 1.095; the output head, 1.0758; the embeddings (`token_embd`, 587 MiB), **1.4371**. The embeddings compress by 44 %, but a token reads only one row of them: this gain serves the disk and the RAM, not throughput. The attention, always read and resident in VRAM in the plan of Chapter 9, gains nothing in speed from lossless coding (Section 6.7). **What matters for throughput is the experts read from disk**: 1.1026 real on the 120b.

6.5 The interleaved rANS coder

Duda's asymmetric numeral systems (ANS) [52] code a symbol of probability q in $\log_2(1/q)$ bits like an arithmetic coder, but with a single integer state and integer operations; the range variant (rANS) decodes through a table. Giesen [53] showed that several independent coders can be interleaved in a single stream. The study wrote its own format and decoders, without taking a line from the repositories it read (`outils/03-entropie/rans/`).

6.5.1 The format

Probabilities are quantized on $M = 2^{12}$; the state x fits in 32 bits; its lower bound is $L = 2^{16}$ (in this chapter, L denotes that bound); renormalization proceeds by 16-bit words. A symbol s has a frequency $f_s \geq 1$ and a cumulative $c_s = \sum_{t<s} f_t$, with $\sum_s f_s = M$.

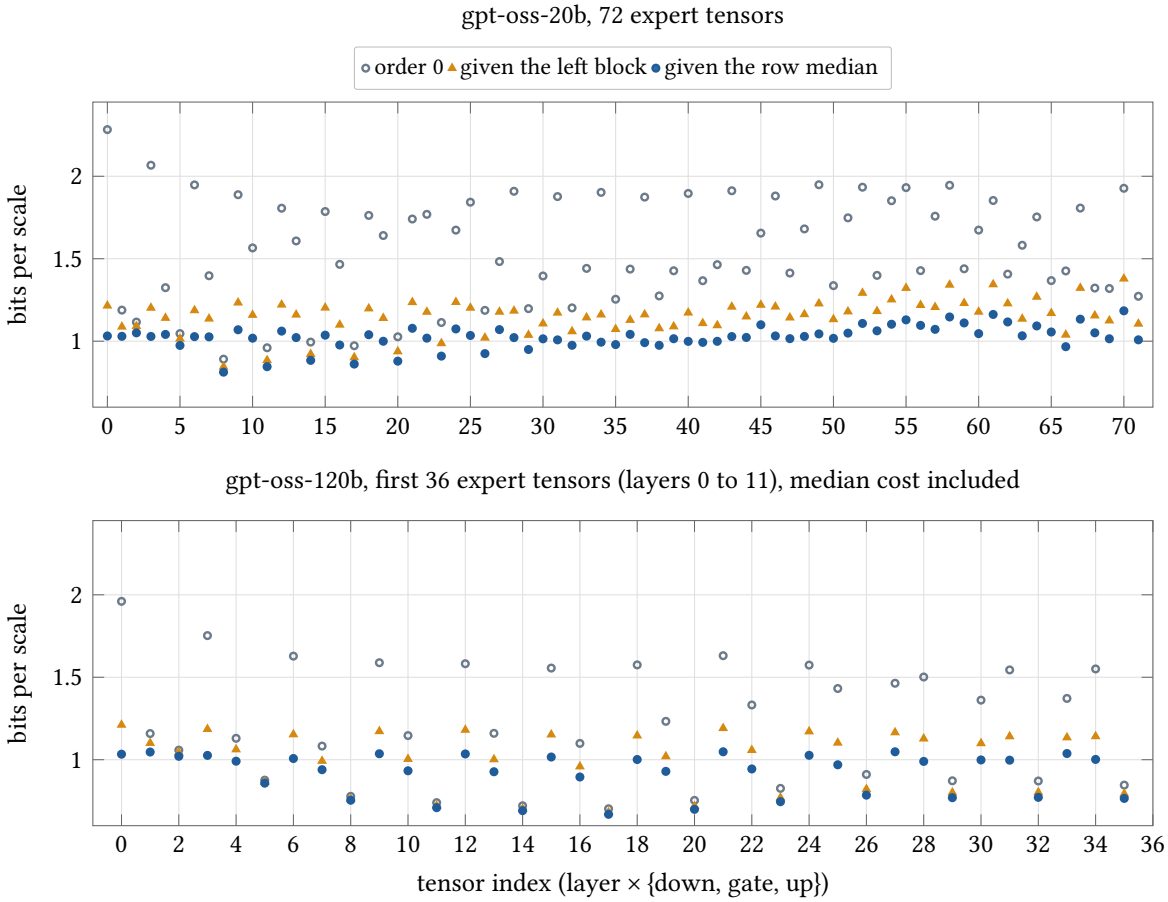


Figure 6.6: Entropy of the E8M0 scales of every expert tensor by context [meas.]. Given the row median, all scales fall near 1 bit, whatever their disorder at order 0; the `ffn_down` tensors (indices 0, 3, 6, ...) are the most disordered at order 0 and the most helped by context.

Data: `memoire/figures/donnees/entropie-echelles-{20b,120b}.csv` (from `mxfp4-contexte-gptoss{20b,120b}.json`).

Definition 6.7 (Coding and decoding steps)**.**

$$C(s,x) = \left\lfloor \frac{x}{f_s} \right\rfloor M + (x \bmod f_s) + c_s, \qquad D(x) = \big(s,\ f_s\lfloor x/M \rfloor + (x \bmod M) - c_s\big),$$

where, in D, s is the unique symbol with $c_s \le x \bmod M < c_s + f_s$ (the *slot* $x \bmod M$ designates the symbol).

Figures 6.7 and 6.8 give the bit-exact layout: a tile is a subgroup of 32 lanes that decodes N macro-steps; lane ℓ of macro-step m writes 32-bit word $32m + \ell$, so that **the 32 lanes write 128 contiguous bytes at every macro-step**. The decoding table has 4 096 slots of 32 bits; v4 (256-symbol alphabet, the whole byte of two nibbles) stores in each entry the symbol (8 bits), $f - 1$ (12 bits) and the bias = slot − cumulative (12 bits). The bias is $< f \le 4\,096$ and thus fits in 12 bits; so does $f - 1$.

6.5.2 Correctness

Lemma 6.8 (Inversion)**.** *For every symbol s and every $x \ge 0$, $D(C(s,x)) = (s,x)$.*

Proof. Let $q = \lfloor x/f_s \rfloor$ and $\rho = x \bmod f_s$, so $0 \le \rho < f_s$ and $x = qf_s + \rho$. Then $y = C(s,x) = qM + \rho + c_s$

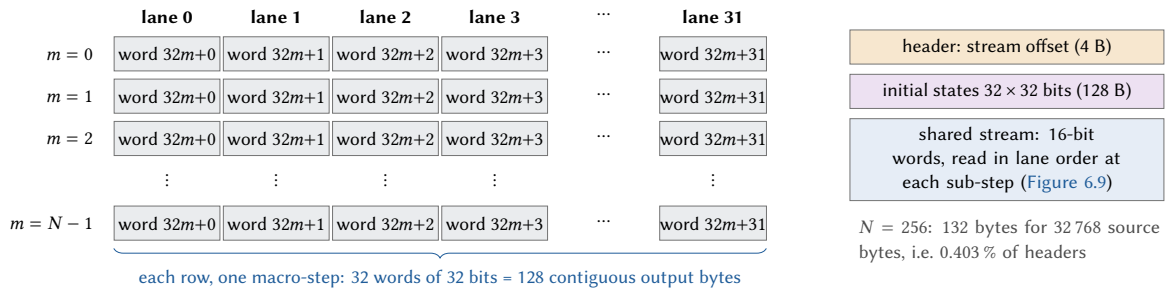


Figure 6.7: The coder's tile: 32 lanes × N macro-steps. Each macro-step writes 128 contiguous bytes (a perfectly coalesced write); the tile carries a stream offset, 32 initial states and its shared stream.

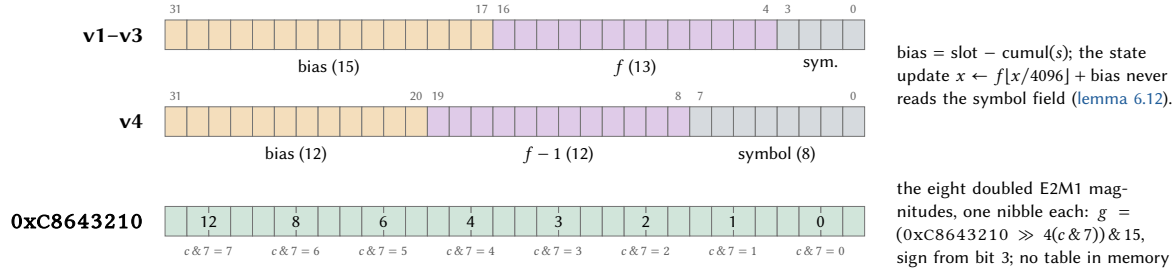


Figure 6.8: The 32-bit entry of the decoding table (4 096 slots, 16 KiB of shared memory), for the 16-symbol alphabet (v1 to v3) and the byte alphabet (v4); below, the 32-bit word that holds the eight doubled E2M1 magnitudes and replaces any MXFP4 dequantization table.

with $0 \le \rho + c_s < c_s + f_s \le M$. Hence $\lfloor y/M \rfloor = q$ and $y \bmod M = \rho + c_s \in [c_s, c_s + f_s)$: the slot does designate s. Finally $f_s\lfloor y/M \rfloor + (y \bmod M) - c_s = f_s q + \rho = x$. ■

Lemma 6.9 (Interval, and a single read per step)**.** *If the decoder keeps $x \in [L, 2^{32})$ and applies $x \leftarrow D(x)_2$, then, if $x < L$, $x \leftarrow 2^{16}x + w$ with w the next word of the stream, then a single word always suffices, and the state returns to $[L, 2^{32})$.*

Proof. After D, $x' = f_s\lfloor x/M \rfloor + (x \bmod M) - c_s \ge f_s\lfloor L/M \rfloor = 16 f_s \ge 16$. If $x' < L$, then $2^{16}x' + w \ge 2^{20} > L$ and $2^{16}x' + w \le 2^{16}(L-1) + 2^{16} - 1 < 2^{32}$. If $x' \ge L$, also $x' \le x < 2^{32}$, since $x' \le M\lfloor x/M \rfloor + (x \bmod M) = x$ as soon as $f_s \le M$. ■

The card's loop therefore has a single read branch, replaced by a `select`: no lane divergence.

Lemma 6.10 (The coder, symmetrically)**.** *The coder keeps $x \in [L, 2^{32})$, starts from $x = L$ and, to code s: if $x \ge f_s 2^{20}$, it emits $x \bmod 2^{16}$ and sets $x \leftarrow \lfloor x/2^{16} \rfloor$; then $x \leftarrow C(s, x)$. One emission always suffices, and $C(s, x) \in [L, 2^{32})$.*

Proof. Let us show that before C, $x \in [16 f_s, 2^{20} f_s)$. Without emission, $x < 2^{20} f_s$ by the condition, and $x \ge L = 16M \ge 16 f_s$. With emission, $x \ge 2^{20} f_s$ gives $\lfloor x/2^{16} \rfloor \ge 16 f_s$, and $x < 2^{32}$ gives $\lfloor x/2^{16} \rfloor < 2^{16} \le 2^{20} f_s$. Then, with $q = \lfloor x/f_s \rfloor \in [16, 2^{20} - 1]$: $C(s, x) = qM + \rho + c_s \ge 16M = L$ and $C(s, x) \le (2^{20} - 1)M + (M - 1) = 2^{32} - 1$. ■

The emission condition before C and the read condition after D are two faces of one transition: by lemma 6.8, the state the decoder reads just after D is the coder's state just before C, in $[16 f_s, 2^{20} f_s)$; it is below L exactly when the coder had emitted.

Lemma 6.11 (Stream shared by 32 lanes). *At sub-step k, let $b_\ell \in \{0,1\}$ indicate "lane ℓ renormalizes". If the decoder gives lane ℓ the position $p + \sum_{j<\ell} b_j$ then advances p by $\sum_j b_j$, and if the coder walks the symbols in exactly reverse order (macro-steps, sub-steps and lanes decreasing), emitting before coding, then reverses its stack, then every lane reads exactly the word its coder emitted.*

Proof. The coding of one lane is a stack: last emitted, first read. The lanes being interleaved, the global stack receives, for sub-step k walked backwards, the words of lanes $31, 30, \dots, 0$ that emit. Reversed, it presents for sub-step k, in forward order, the words of lanes $0, 1, \dots, 31$; and every lane that emitted when coding is exactly a lane that reads when decoding (lemma 6.8 lane by lane, and lemma 6.10). The position of lane ℓ is therefore the number of words read before it in this sub-step, $\sum_{j<\ell} b_j$. ■

On the card, $\sum_{j<\ell} b_j = \text{popcount}(\text{ballot}(b) \,\&\, (2^\ell - 1))$: one subgroup vote and one bit count, instead of a prefix sum (Figure 6.9). This principle of a single stream read in lane order is the one Microsoft set down in 2022 for GDeflate [5]; DietGPU also does it through `__ballot_sync` and `__popc` [57]. The study's v2 decoder found it independently.

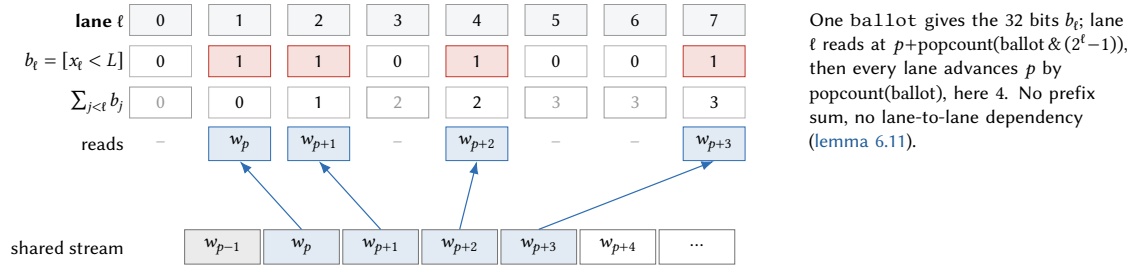


Figure 6.9: One sub-step of the shared stream, 8 lanes shown out of 32. The lanes whose state dropped below L read consecutive words, in increasing lane order.

Lemma 6.12 (What the state check guards, and what it does not). *Every lane starts from L when coding, so the exact decoder ends on L: a free check. But the state update depends only on f and the bias read at the slot, and on the stream words; it does not depend on the entry's symbol field. A corruption confined to the symbol field changes the output without changing any state: the check $x_{\text{end}} = L$ does not see it.*

Proof. In D, the update reads $x' = f\lfloor x/M \rfloor + \text{bias}$, with $\text{bias} = (x \bmod M) - c_s$: two fields of the entry, neither of which is the symbol. ■

This is exactly what the broken control showed: a bit flipped in the table in shared memory makes v1 to v3 fail twice (wrong output, and 129 278 to 129 280 lanes out of 129 280 off L), but v4 only on the output, **all its lanes ending on** L [meas.]: the corruption hit the symbol field. In production, a fingerprint of the table or of the output is needed on top of the state check; the plan of architecture A compares every decoded chunk with its fingerprint (Chapter 10).

Proposition 6.13 (Real code length). *For a memoryless source of law p coded with the quantized table $q_s = f_s/M$, the mean length per symbol tends to $H(p) + D(p \,\|\, q)$, plus tile headers and stream ends.*

Proof. Step C multiplies the state by M/f_s up to rounding; over n symbols the state has grown by a factor $\prod_i M/f_{s_i}$, and every 16-bit word emitted divides the state by 2^{16}, the state staying in $[L, 2^{32})$. The number of bits emitted is thus $\sum_i \log_2(M/f_{s_i}) + O(1)$ per lane, whose expectation per symbol is $\sum_s p_s \log_2(1/q_s) = H(p) + D(p \,\|\, q)$. Headers and final states add up. ■

The numbers hold together. Headers cost $4 + 32 \times 4 = 132$ bytes per tile of $32 \times 4 \times 256 = 32\,768$ source bytes, i.e. 0.403 % [calc.]. Over all Q4_K nibbles of the 4B (1 209 794 560 bytes) [meas.]: $H(p) = 3.8652$ bits per code, the 12-bit table gives 3.8652 (so $D(p \,\|\, q) < 10^{-4}$), the real coding 3.8774, a rate of 1.0316 on the nibbles and of 1.0280 on whole Q4_K, raw metadata. The gap to the entropy, 0.0122 bit per code, is $0.0122 \times 65\,536/8 = 100$ bytes per tile, less than the 132 bytes of header and states ($132 \times 8/65\,536 = 0.0161$ bit per code, a tile carrying 65 536 codes) [calc.]. The difference is not an error: the 32 stored states themselves carry part of the payload. The coder's final state, which serves as the decoder's initial state, is an integer of $[2^{16}, 2^{32})$, carrying up to 16 bits of information above L per lane; the count says they carry 32 bytes per tile, 8 bits per lane on average [calc.]. (The study attributed the gap to "the rounding of tile ends", which is the same cause, put differently.)

6.5.3 The heart of v4

Listing 6.1 shows the inner loop of the v4 decoder, as committed, with no change other than comments. A step costs one shared-memory read, one multiplication, one vote, two bit counts, two shuffles and a few masks: about 25 integer operations for one byte produced.

Listing 6.1: Inner loop of the rANS v4 decoder, extracted verbatim from `outils/03-entropie/rans/src/decodeur.wgsl` (function `decoder_v4`); only the end-of-line comments are added, in French as in the repository.

```
// Extrait de outils/03-entropie/rans/src/decodeur.wgsl (fonction decoder_v4), boucle intérieure,
// code inchangé ; seuls les commentaires de fin de ligne sont ajoutés ici.
for (var m = 0u; m < t.y; m++) {                // un macro-pas
    var mot = 0u;
    for (var k = 0u; k < 4u; k++) {             // 4 sous-pas, un octet chacun
        let e = lut[x & 4095u];                 // fente = x mod M
        x = (((e >> 8u) & 0xFFFu) + 1u) * (x >> 12u) + (e >> 20u);
        mot |= (e & 0xFFu) << (8u * k);         // symbole : hors de l'état
        let besoin = x < 65536u;                // x < L : lire un mot
        let vote = subgroupBallot(besoin).x;
        let pos = ptr + countOneBits(vote & masque_inf);
        let wi = (pos >> 1u) - base;
        let a = subgroupShuffle(cour, wi & 31u);  // fenêtre en registres
        let b = subgroupShuffle(suiv, wi & 31u);
        let w32 = select(a, b, wi >= 32u);
        let w = (w32 >> ((pos & 1u) * 16u)) & 0xFFFFu;
        x = select(x, (x << 16u) | w, besoin);  // une seule lecture
        ptr += countOneBits(vote);
        if ((ptr >> 1u) - base >= 32u) {        // fenêtre suivante
            base += 32u;
            cour = suiv;
            suiv = mots[base + 32u + voie];
        }
    }
    sortie[dest] = mot;                         // 128 octets contigus
    dest += 32u;
}
```

6.6 Four decoders, and what each one taught

v1 Subgroup prefix sum, then a read of the word from global memory in the middle of the dependency chain. Latency-bound: at every sub-step at least one lane out of 32 renormalizes (the probability that

none does is about $0.76^{32} \approx 1.5 \cdot 10^{-4}$), so every sub-step waits for a global read of several hundred cycles.

v2 A vote and a bit count replace the prefix sum; the stream is read through windows of 32 words of 32 bits loaded ahead, one per lane, distributed by shuffle. A sub-step consumes at most 32 words of 16 bits, so the next window has always been requested at least one sub-step before being read: latency is hidden. About twice v1.

v3 v2 with a compact 4 KiB table, to fit more workgroups per multiprocessor. **No gain** (42.1 against 42.1 GB/s on mains): occupancy was not the ceiling. A dead idea, killed by the number.

v4 The byte alphabet: half as many steps per byte produced, for the same chain per step. 1.67 times v2 on mains (70.2 against 42.1 GB/s), at the same rate (1.0316).

Two references bound these throughputs: the *copy* with the same read and write geometry, without decoding (the uncompressed path), and a grid-stride `vec4` copy, a measure of the memory. Every pass is checked: each decoder returns exactly the original bytes, compared word by word over the whole batch, and no lane ends off L [meas.].

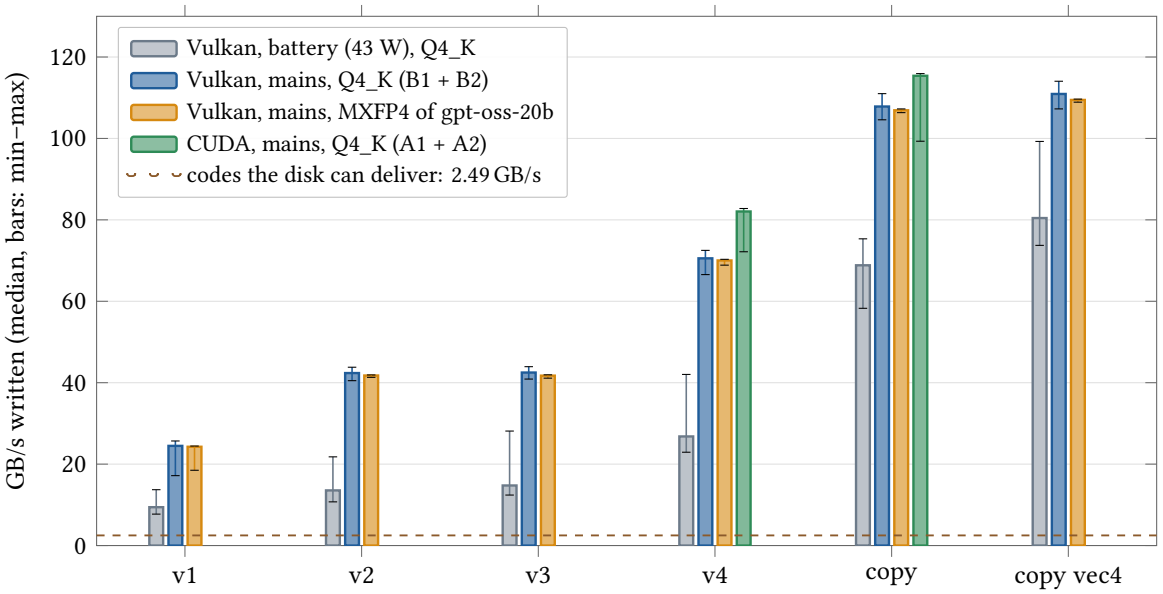


Figure 6.10: Written throughput of the four decoders and the two reference copies, median and range of every series [meas.]. On mains, v4 produces 70 to 72 GB/s through Vulkan, the same on MXFP4 codes as on Q4_K nibbles (throughput hardly depends on the symbol law), and 82 GB/s through CUDA. On battery (card capped at 43 W) it loses a factor of 2.6. The bottom line is what the disk can deliver.

Data:
`memoire/figures/donnees/entropie-debits-{batterie,secteur,mxfp4,cuda}.csv`
(`donnees_figures.py`, from `csv/debits-carte-series.csv` and `csv/debits-cuda-series.csv`).

The second path: the same decoder in CUDA. So that the throughput does not rest on a single tool, the algorithm was rewritten in CUDA (`__ballot_sync`, `__popc`, `__shfl_sync`), compiled by nvcc 13.1 for sm_89 and fed the *same stream* written to disk (256 MiB of Q4_K nibbles, 8 192 tiles). The CUDA output equals the source byte for byte; the broken control makes it fail [meas.]. Two decoders written in two languages and compiled by two toolchains return the same bytes from the same stream: the format is confirmed independently of the code that wrote it.

Table 6.5: The v4 decoder under CUDA (A) and Vulkan (B), passes in the order A B B A, all on mains, medians [meas.].

pass	v4 decoder (GB/s written)	copy, same geometry
A1, CUDA	80.8	112.8
B1, Vulkan	70.2	107.5
B2, Vulkan	71.6	110.4
A2, CUDA	82.1	115.7

Data: `fondations/journaux/03-entropie/cuda-4B-A{1,2}-secteur.json`, `rans-4B-B{1,2}-secteur.json`.

NVIDIA's compiler gets $(80.8 + 82.1)/(70.2 + 71.6) = 1.149$ times more from the same decoder [calc.]; the copy differs by only 4 %. The gap is in the code generated for the loop (naga then the Vulkan driver's compiler, against nvcc), not in the memory. It is one of the quantified reasons for choosing CUDA for milestone J1 (Chapter 10).

What the decoder costs in compute. At the trial, v4 reads about 63 GB/s of stream and writes 65 GB/s: its 128 GB/s of traffic stay below the 189 GB/s (read and written) of the copy, **it is not memory-bound**. The battery signature says it is bound by the multiprocessor clock: under the power cap the decoder loses 59 % and the copy 27 % [meas.]. Whether by arithmetic throughput or by each lane's dependency chain remains to be seen. The study wrote: "~~At 130 billion steps per second, that makes about 3.3 trillion operations per lane per second [...] The decoder is thus at about half of the arithmetic ceiling.~~" But v4 takes one step per *byte*: 65 GB/s written are 65 billion steps per second (130 billion *codes*), i.e., at about 25 operations per step, $1.6 \cdot 10^{12}$ operations per second [calc.], for a card that offers on the order of $24 \times 128 \times (2.0 \text{ to } 2.5)\,\text{GHz} = 6 \text{ to } 7.7 \cdot 10^{12}$ (order of magnitude, not measured). The decoder is thus at about a quarter of the arithmetic ceiling, not half: what bounds it is more probably the latency of one step's chain (table read, multiplication, vote, shuffle, in that order) than the units' throughput. The consequence for what follows is favourable: more than a factor of 2 may remain, through more independent states per lane (two streams interleaved in the same lane overlap their chains). Proposition 6.14 does not depend on it: it counts operations per byte, whether the step is bound by throughput or by latency.

The processor does not keep up. The scalar rANS decoder costs about 16 cycles per code (the shared-pointer chain is serial); it does 0.075 GB/s per thread on a micro-bench [meas.]. Even with AVX-512, at about 0.7 GB/s per core as Recoil announces [55] [lit.], six performance cores would do 4 GB/s, barely more than the disk, while the card offers twenty-five times more. **Decoding belongs to the card**; the processor reads the disk and fills the transfer buffers.

6.7 The key question: decode inside the product, or just before?

Proposition 6.14 (Resident regime: when fusing would pay)**.** *The weights reside in VRAM, of throughput W_{VRAM}. A product fused with the decoder must, per byte of weights, read $1/r$ byte of stream and execute $a_{\text{dec}} + a_{\text{mv}}$ operations; the raw path reads 1 byte and executes a_{mv} operations. With A the card's arithmetic throughput,*

$$\frac{t_{\text{fused}}}{t_{\text{raw}}} = \frac{\max\big(1/(r\,W_{\text{VRAM}}),\ (a_{\text{dec}} + a_{\text{mv}})/A\big)}{\max\big(1/W_{\text{VRAM}},\ a_{\text{mv}}/A\big)}, \tag{6.4}$$

and fusion wins only if $(a_{\text{dec}}+a_{\text{mv}})/A < 1/W_{\text{VRAM}}$*, that is, if the card offers at least* $a_{\text{dec}}+a_{\text{mv}}$ *operations per byte of VRAM.*

Proof. Each path is bound by the slower of its two resources (theorem 3.2 applied to memory and compute); the ratio follows. If $(a_{\text{dec}} + a_{\text{mv}})/A \geq 1/W_{\text{VRAM}}$, the numerator is at least $1/W_{\text{VRAM}}$ and the denominator, if the raw path is memory-bound, is exactly $1/W_{\text{VRAM}}$: the ratio is ≥ 1. ■

The calculation. A v4 step costs about 25 operations and produces one byte: $a_{\text{dec}} \approx 25$. The card offers $A/W_{\text{VRAM}} \approx 6$ to $7.7\cdot10^{12}/251\cdot10^{9} \approx 24$ to 31 operations per byte [calc.]. Decoding alone already eats almost the whole budget before the first multiplication: the fused kernel will be compute-bound, at a throughput close to that of the decoder alone, while the raw one is memory-bound. The prediction: a ratio of about $446/130 = 3.4$ (Q4_K weights the VRAM can deliver against codes produced by the decoder, in billions per second [calc.]).

The measurement. A bench written for this (`outils/03-entropie/rans/src/bin/banc_gemv.rs`) runs a batch-1 matrix–vector product on **real MXFP4 weights of gpt-oss-20b** (three expert tensors, 256 MiB of codes, 131 072 rows of 4 096 weights), along four paths that share exactly the same dequantization (the word `0xC8643210` of Figure 6.8): *raw* (raw codes and scales); *fused* (v4 decoder, each decoded word consumed in registers); *decode only*; *two passes* (decode then raw in the same submission).

Lemma 6.15 (An exact check of a floating-point product)**.** *If every term* $w_i x_i$ *of a sum of n terms is an integer of absolute value at most a, and if* $n\,a < 2^{24}$*, then the sum computed in f32 is exact, in any order.*

Proof. Every partial sum is an integer of absolute value at most $na < 2^{24}$. Every integer of absolute value below 2^{24} is exactly representable in f32 (24-bit significand); adding two such numbers whose sum stays below 2^{24} is therefore exact, and no rounding occurs at any step, whatever the order. ■

With all scales at 2^0 and integer x in $[-2, 2]$, every term is at most $12 \times 2 = 24$ (largest doubled E2M1 magnitude times $|x|$), and $4\,096 \times 24 = 98\,304 < 2^{17}$: the card must equal the processor reference *bit for bit.* Result [meas.]: **0 wrong rows out of 131 072** for raw, fused and two passes. The controls: a single corrupted table slot makes 38 329 rows wrong, all slots 130 564. A floating-point tolerance would have let the single slot through. On the real inputs, the error relative to the maximum of $|y|$ is $9.5 \cdot 10^{-8}$ (raw) and $1.3 \cdot 10^{-7}$ (fused) against an f64 reference.

Table 6.6: The key question measured, five palindromic series, card alone, on mains [meas.]; last column, the same measurement on battery.

path	median time	G weights/s	equiv. GB/s	ratio to raw	battery (GB/s)
raw	1.36 ms	394.1	**209.4**	1	66.4
fused	4.99 ms	107.5	**57.1**	**0.27**	19.5
decode only	3.72 ms	144.5	76.7	—	29.4
two passes	5.13 ms	104.6	**55.6**	**0.27**	21.7

Data: `fondations/journaux/03-entropie/gemv-gptoss20b-secteur.json`, `gemv-gptoss20b-calme.json`.

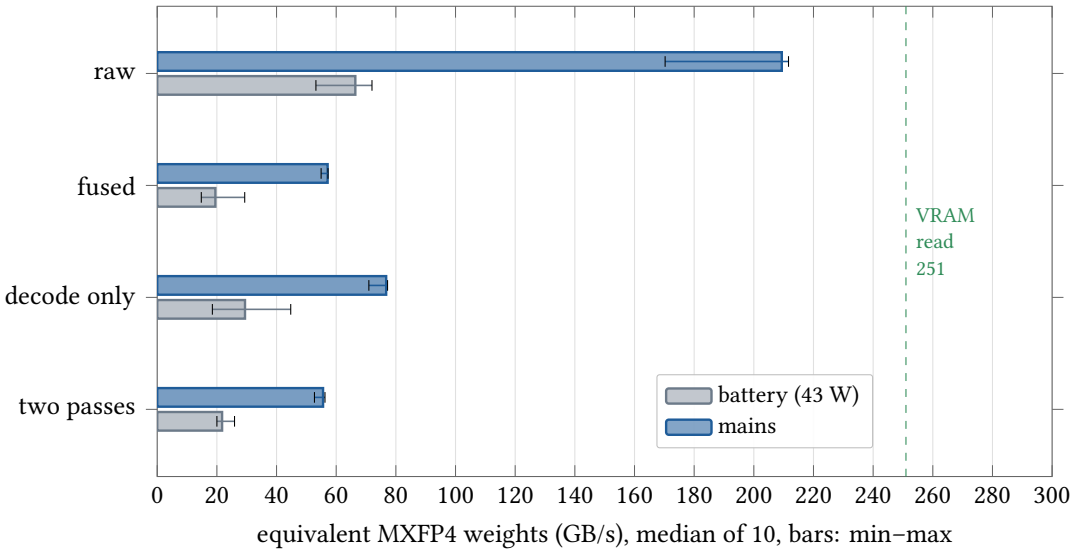


Figure 6.11: Batch-1 matrix–vector product on real MXFP4 weights of gpt-oss-20b, four paths, in GB/s of equivalent MXFP4 weights [meas.]. The raw path reaches 83 % of the VRAM read throughput; fused with the decoder it is 3.7 times slower, and no faster than two passes.

Data: `memoire/figures/donnees/entropie-gemv-{secteur,batterie}.csv` (from `gemv-gptoss20b-{secteur,calme}.json`).

Reading. (1) The raw path does 394 billion weights per second for a ceiling of 472: 83 % of the VRAM, memory-bound, and the fastest 4-bit product measured on the machine (study 01 had 122 GB/s in Q4_0); its recipe fits in one line: a subgroup handles 4 rows, each lane loads the 32 x of a block once for the 4, and the word 0xC8643210 replaces any table. (2) **Fused is 3.7 times slower than raw**; the calculation predicted 3.4. (3) **Fusing gains nothing over two passes**: 57.1 against 55.6 GB/s, within the spread. Times add up: $3.72 + 1.36 = 5.08$ ms for 4.99 (fused) and 5.13 (two passes) measured; fusion only saves the round trip of the codes through VRAM (about 2 ms of traffic), which the compute-bound decoder already hides. (4) On battery, the ratios stay at 0.27–0.33.

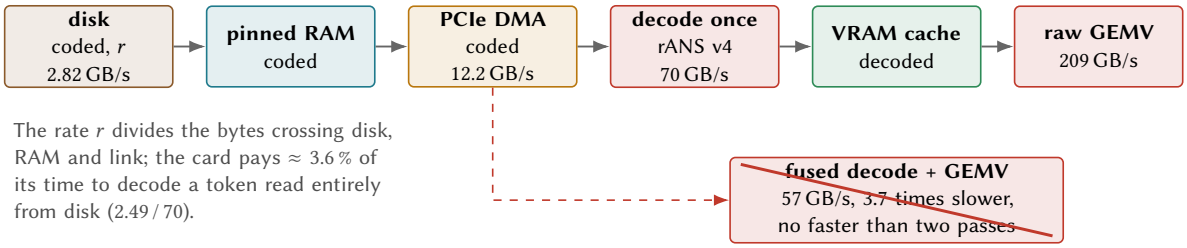


Figure 6.12: The answer to the key question. The coded stream crosses the disk, the RAM and the link; it is decoded once, on arrival in VRAM, before entering the card's cache, where it then serves at the raw product's throughput. The struck-out path, decoding inside the product, is 3.7 times slower [meas.].

Answer to the key question. Decoding *inside* the kernel brings nothing on this card, and decoding *just before* costs as much: in both cases, 3.7 times the raw product's time when the weights reside in VRAM. Entropy decoding pays only where it relieves a stage slower than itself, **the disk and the link** (Figure 6.12). In this scheme, decoding a token that reads everything from disk costs $2.49/70 \approx 3.6$ % of the card's time on mains (9 % on battery) [calc.], where 2.49 GB/s is the volume of codes the disk delivers at 2.8 GB/s of Q4_K blocks ($2.8 \times 256/144/2$); and it gives the whole rate r

back to the disk (corollary 6.2). Reassembling the three class streams, measured separately, writes at 102.4 GB/s against 113.5 for a contiguous copy, exact output and a control that fails [meas.]; decoding then reassembly together make 41.6 GB/s, still 17 times the 2.49 GB/s of codes the disk delivers [calc.].

The only conclusion that could depend on the quality of the fused kernel holds on its own: no writing of the fused kernel can go below the time of the decoder alone, which is already $3.72/1.36 = 2.7$ times that of the raw product; with the CUDA decoder, 15 % faster, the ratio would stay above 2.3 [calc.]. A decoder two to three times faster would be needed to reverse the conclusion, which neither measurement nor literature provides on a card of this class. vram-compression [51], which fuses an rANS into a batch-1 product on an RTX 3050 Laptop, reports a kernel "2 to 3 times slower than bf16 cuBLAS" [lit.]: the same conclusion, by another path.

6.8 Beyond lossless: a quantization tailored to the coder

Why do K-quants leave so little? Because their scale and minimum search clips to fill the 16 levels: almost uniform codes are the optimum of a *fixed-rate* code. Classical theory says what fixed rate costs.

Theorem 6.16 (Gish and Pierce [101], high resolution)**.** *A uniform scalar quantizer of step Δ followed by ideal entropy coding (ECSQ) achieves, as $\Delta \to 0$,*

$$D = \frac{\Delta^2}{12}, \qquad R = H(Q) = h(X) - \log_2 \Delta + o(1), \qquad \textit{hence} \qquad R_{\mathrm{ECSQ}}(D) = h(X) - \tfrac{1}{2}\log_2(12D). \tag{6.5}$$

Proof (high resolution). The rate comes from Proposition 6.5. For the distortion: on a cell where the density is almost constant, the error $X - Q(X)$ is almost uniform on $[-\Delta/2, \Delta/2]$, of variance $\int_{-\Delta/2}^{\Delta/2} u^2 \,\mathrm{d}u/\Delta = \Delta^2/12$; averaging over cells keeps this value. Eliminate Δ: $\log_2 \Delta = \frac{1}{2}\log_2(12D)$. ∎

Corollary 6.17 (The gap of ECSQ to the Shannon bound)**.** *The Shannon lower bound for squared error is $R(D) \geq h(X) - \frac{1}{2}\log_2(2\pi e D)$ [91]. ECSQ lies at*

$$R_{\mathrm{ECSQ}}(D) - R_{\mathrm{SLB}}(D) = \tfrac{1}{2}\log_2 \frac{2\pi e}{12} = \tfrac{1}{2}\log_2 \frac{\pi e}{6} = \tfrac{1}{2}\log_2 1.4233 = 0.2546 \textit{ bit, i.e. } 1.53 \textit{ dB.}$$

What fixed rate costs, for a Gaussian. The best scalar quantizer with 2^R levels achieves at high resolution $D \approx \frac{1}{12}\big(\int p^{1/3}\big)^3 2^{-2R}$ (Panter and Dite [94]). For $\mathcal{N}(0, \sigma^2)$:

$$\int p^{1/3} = (2\pi\sigma^2)^{-1/6} \int e^{-x^2/(6\sigma^2)}\,\mathrm{d}x = (2\pi\sigma^2)^{-1/6}\sqrt{6\pi}\,\sigma,$$
$$\Big(\int p^{1/3}\Big)^3 = (2\pi)^{-1/2}\sigma^{-1}(6\pi)^{3/2}\sigma^3 = 6\sqrt{3}\,\pi\,\sigma^2,$$

hence $D_{\mathrm{PD}} = \frac{\sqrt{3}\,\pi}{2}\,\sigma^2 2^{-2R}$. Compared with the exact bound $D(R) = \sigma^2 2^{-2R}$ (theorem 5.1), the loss in rate is $\frac{1}{2}\log_2(\sqrt{3}\pi/2) = \frac{1}{2}\log_2 2.7207 = 0.722$ bit; ECSQ loses only 0.2546 (corollary 6.17, the Gaussian reaching the Shannon bound). **Difference:** $0.722 - 0.255 = 0.467$ **bit per weight** [calc.], what entropy coding gains in theory, at high rate, over the best fixed rate.

The measurement, on surrogates. No BF16 weight being on the machine, the study took dequantized surrogates (two Q8_0 tensors and one Q6_K of Qwen3.5-4B) and compared, at equal squared error, a Q4_K-style quantization rewritten from scratch (KQ4, 4.5 bits per weight, whose codes carry

3.874 to 3.883 bits against 3.865 for the real Q4_K: same shape) with an ECSQ (step $\Delta = c\,\sigma$ per block of 256, σ in fp16, unbounded integer codes, one table per tensor) [meas.] (`rd-proxy-4B.json`). **At equal error, ECSQ needs 0.41 to 0.46 bit less per weight** (Figure 6.13), in agreement with the theory's 0.467 despite a 4-bit regime that is not asymptotic and non-Gaussian sources. That is twice the lossless ceiling of Q4_K (0.24 bit), and the decoder is v4, with an alphabet of 38 to 96 symbols.

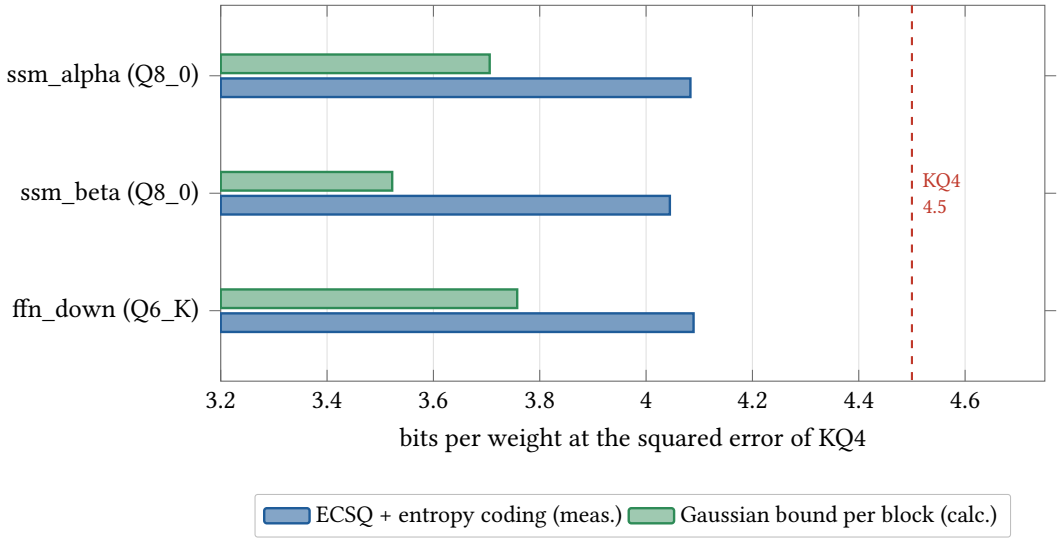


Figure 6.13: Bits per weight of an ECSQ followed by entropy coding, at the squared error of KQ4 (4.5 bits per weight), on three surrogates [meas.], and the per-block Gaussian bound [calc.]. The gain ranges from 0.411 to 0.455 bit per weight; 0.3 to 0.5 bit remains to the bound.

Data: `fondations/journaux/03-entropie/csv/rd-proxy.csv` (from `rd-proxy-4B.json`).

This figure is not a quality: squared error is not perplexity, and an error weighted by weight importance can move the gain. vram-compression publishes, on Qwen2.5-1.5B, a perplexity of a per-column-step ECSQ at 3.25 bits (9.81) almost equal to that of GPTQ at 4.25 bits (9.78) [lit.]: about one bit gained at almost equal quality, twice our measurement at equal squared error; their Hessian-weighted per-column step, which our measurement lacks, may explain the gap. EntQuant [49] (ICML 2026) does the same family of things in Float8 coded by ANS, decoded per block before the product, and WaterSIC [50] sits 0.255 bit from the information-theoretic limit through a per-column rate allocation. What these works still lack, with respect to the mission: running without CUDA, aiming at the regime of streaming from disk, and applying to the formats the machine runs.

6.9 The state of the art, technique by technique

Table 6.7 sums up the sixteen rows of the study. Two structural lessons come out of it. **In the resident regime, only a constant-time decodable code fits inside the multiplication**: ZipServ beats cuBLAS only with a fixed-length 3-bit code on very sparse BF16 exponents; our 4-bit codes are almost uniform and do not have that shape. **In the streaming regime, a variable-length code pays its whole rate**, provided its decoder is on the card and outpaces the disk by a wide factor: here, by 28.

6.10 What is missing in the world

1. **Nobody codes the scales of MX formats.** None of the works read measures or codes the shared E8M0 scales, although they carry the gain: 0.213 bit per weight against 0.134 for the codes on

Table 6.7: Lossless weight coders and their neighbours, with respect to the mission. Publishers' gains and throughputs [lit.], except troncons' [meas.]. "No CUDA": runs natively under Windows on the machine's card without the CUDA toolkit.

technique	mechanism	on our formats	no CUDA	licence
DFloat11 [41]	Huffman of BF16 exponents, cascaded 256-entry tables	n/a (BF16)	no	Apache-2.0
ZipNN [42]	streams of same-rank bytes, Huffman or zstd	1.042 at best per field [calc.]	yes (CPU)	MIT
Huff-LLM [43]	hardware Huffman decoder in front of a systolic array	n/a	n/a	no code
Ecco [102]	k-means patterns and Huffman tables per group, hardware	n/a	n/a	no code
ZipServ [45]	fixed-length 3-bit code on exponents, bit planes, fused	n/a (BF16)	no	Apache-2.0
ISCA 2026 [46]	DietGPU's rANS, product tiles, fused	int4 claimed, contradicted here	no	no code
EntroPack [48]	E8 lattice, conditional model, independent tiles	n/a (diffusion)	unknown	unverified
nvCOMP [56]	chunked ANS, GDeflate, Bitcomp	$\leq$ 1.018 (bytes)	no	proprietary
DirectStorage [5]	GDeflate and zstd on the card (DX12)	$\leq$ 1.018	**yes**	MIT, Apache-2.0
DietGPU [57]	rANS, warp-cooperative renormalization	same, more per field	no	MIT
Recoil [55]	one rANS stream, split points	same	no	GPL-3.0
zstd, lz4 [58]	LZ and entropy, CPU	1.012 and 1.000 [meas.]	yes	BSD
EntQuant [49]	Float8 under an ℓ_1 Lagrangian, ANS via nvCOMP	n/a (Float8)	no	Apache-2.0
vram-compression [51]	ECSQ and GPTQ, rANS fused into the product	ECSQ, not a GGUF	no	none
ECF8 [44]	FP8 with entropy-coded exponents	n/a (FP8)	unknown	no code
troncons rANS v4	subgroup-interleaved rANS, shared stream, scales by row median	**1.0316** (Q4_K); **1.1026** (MXFP4, 120b) [meas.]	**yes** (Vulkan, 70 GB/s)	ours

gpt-oss-20b [calc.]. The gap is quantified by a real rate, decoded exactly: 1.1026 on gpt-oss-120b, where zstd takes 1.0405 on the same tensors and lz4 nothing. NVFP4 (FP8 scale per block of 16) has the same structure; no file in that format was on the machine to measure it.

2. **No tool entropy-codes the fields of K-quants**: the ceiling is 5.3 %, zstd takes 1.2 %, any byte coder over the whole block at most 1.8 %; on the 9B, 180 MB per re-read token would not cross the disk.
3. **No CUDA-free entropy decoder for weights had a published throughput** on a laptop card; this one does 70 to 72 GB/s through Vulkan, 28 times the disk.
4. **Coder-tailored quantization exists in CUDA** (EntQuant, vram-compression), ~~but exists nowhere for consumer LLMs~~ (sentence of the study's first version, struck out at its resumption); none runs without CUDA, aims at streaming from disk, or applies to GGUF formats.
5. **The resident regime has no variable-length code on this class of card**: the decoder is short by a factor of 3.4, found again by measurement (3.7). A constant-time decodable code that keeps part of ECSQ's gain remains to be invented.
6. **Published int4 entropies diverge from measurement by a factor of 4**, and the exact calculation of Section 6.3.1 shows why: reaching them would take a symmetric grid more than 30 standard deviations wide. A public, tooled measurement per quantization scheme is missing.
7. **No published measurement compares two compilation paths of the same decoder** on

the same card: 15 % here between nvcc and Vulkan. A “CUDA or Vulkan” decision taken on a throughput published in another language is wrong by about that factor.

What could be wrong

- **Contention.** Other agents ran llama.cpp and Python scripts during the measurements; the processor throughputs are only orders of magnitude. The card throughputs quoted come from passes judged quiet on the power drawn (idle 5.5 W, declared threshold 12 W), with a sampler every 500 ms that tells afterwards whether the pass stayed alone.
- **The battery.** The first half of the resumption ran on battery (card capped at 43 W, memory clock at 6 001 MHz instead of 8 001). Only mains figures are quoted as throughputs; battery ones remain, labelled.
- **Gains on fp16 metadata are optimistic** (lemma 6.3): the realistic figure is the per-field byte coding (1.052 for Q4_K), not the ideal 1.057.
- **The throughputs come from batches of 126 to 256 MiB**; one launch per expert matrix (4.4 MB in gpt-oss-120b) will lose at start-up and at the end of the wave. Not measured here.
- **The ECSQ section rests on dequantized surrogates**, lying on a grid, which can lower ECSQ’s entropy and inflate the gain; KQ4 has neither the imatrix nor llama.cpp’s exact search; squared error is not quality.
- **The card’s arithmetic budget** (6 to $7.7 \cdot 10^{12}$ operations per second) is an order of magnitude; Proposition 6.14 and the predicted factor of 3.4 depend on it, the measurement (3.7) does not.
- **The subgroup model** requires subgroups of 32 consecutive invocations: true here (checked by exactness), false on AMD (64). The decoder stops cleanly through a counter if the size differs.
- **The CUDA second path is independent by code and compiler, not by algorithm**: it follows the same format. What makes it probative: exactness is judged against the source, not against the other decoder.
- **The calculation of Section 6.3** assumes independent Gaussian or Laplacian weights; it recovers the measurements to the hundredth, but a real law with heavier tails would lower the entropy of a large-group min-max.
- **Literature throughputs** (DietGPU, Recoil, ZipNN) are the publishers’, on other machines; none is reproduced, although CUDA is now on the machine.

Chapter 7

On-demand loading

Where we start, what this chapter establishes, what remains

At the start, one question: when a model does not fit in memory, where are its weights at the moment a computation needs them, how many bytes must move per token, through which stage, and what share of that movement hides behind the computation? **This chapter establishes** the review of everything that runs a model larger than memory, from 2022's "layer by layer" to the 2026 engines that read the experts of an MoE from disk, judged by a single rule (bytes per token divided by the throughput of each stage); the quantity that decides everything for an MoE, the cache hit rate; the hit rate implied by any published measurement; and the judge rerun with the measured disk. **What remains** is a real routing trace on a 2026 MoE, a clean PCIe pass, and the miss policy that assembles the four possible responses. Source: `fondations/04-chargement.md`, `fondations/journaux/04-chargement/`.

7.1 The law of loading

During decoding, each token reads every active weight once. Theorem 3.2 bounds the throughput between two regimes: perfectly overlapped stages, $d \le 1/\max_i(O_i/W_i)$, and stages in series, $d = 1/\sum_i(O_i/W_i)$. A real system falls between the two, and where it falls can be measured:

$$\text{hidden share} = \frac{\sum_i t_i - t_{\text{measured}}}{\sum_i t_i - \max_i t_i}, \qquad t_i = O_i/W_i. \tag{7.1}$$

Every system we read says that out-of-memory decoding is a problem of bytes, not of compute, and their numbers confirm it: FlexGen spends more than 99.5 % of its time transferring weights; HOBBIT measures 80 ms to load one Mixtral layer against 3 ms to compute it on an RTX 4090 [lit.] [8, 16]. The "bytes over throughput" judge (`chargement_juge.py`) bounds every technique; it was validated on five published measurements, none of which exceeds its bound, and, with throughputs divided by 10, it fails on four of the five: the control can fail.

7.2 The mixture of experts in three quantities

For an MoE with L expert layers, E routed experts per layer, k_e experts chosen per token, stored at b bits per weight, we set (Figure 7.1):

$$B_t = \text{always-read bytes (attention, shared experts, routers, norms, head)},$$
$$e = \frac{P_{\text{experts}}}{E\,L} \cdot \frac{b}{8} \quad \text{(bytes of one routed expert)}, \qquad B_e = k_e\,L\,e \quad \text{(routed bytes per token)}. \tag{7.2}$$

A share h_v of the routed bytes comes from VRAM, h_r from RAM, $h_s = 1 - h_v - h_r$ from disk, which must deliver $h_s B_e$ bytes per token. It is almost always the limiting stage, and **the cache hit rate $h = h_v + h_r$ is the quantity that decides everything**.

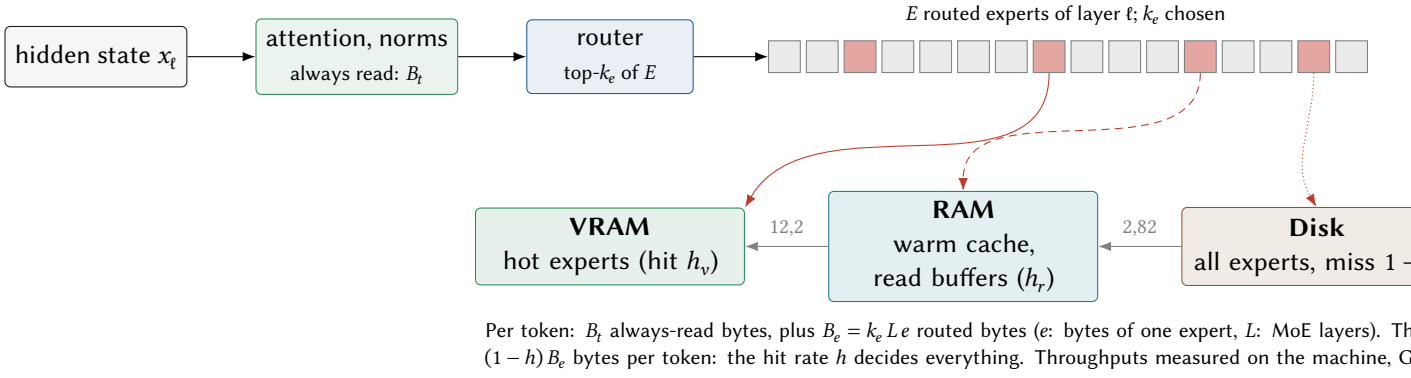


Figure 7.1: A mixture-of-experts layer and the cache hierarchy serving its experts. Always-read tensors reside in VRAM; among the E experts of the layer, the router picks k_e; each is served by VRAM, RAM or disk.

Conversely, a published throughput measurement says something about the locality it does not publish.

Lemma 7.1 (The hit rate implied by a measurement)**.** *If a system reaches d tokens per second with a disk of throughput W_{disk} and B_e routed bytes per token, its cache serves at least a share*

$$h_{\min} = 1 - \frac{W_{\text{disk}}}{d\,B_e} \tag{7.3}$$

of the routed bytes.

Proof. By theorem 3.2 applied to the disk stage, $1/d \geq (1-h)B_e/W_{\text{disk}}$, i.e. $1 - h \leq W_{\text{disk}}/(dB_e)$. ∎

Table 7.1: The cache hit rate implied by three published measurements (lemma 7.1) [calc.] on [lit.].

Published measurement	share of model in fast memory	implied hit rate, at least
Qwen3-235B warm, 0.441 tok/s, disk 2.4 GB/s [103]	about 25–30 %	33 %
GLM-5.2, 2.20 tok/s, disk ≤ 7.877 GB/s [22]	79 GB of 254 (31 %)	53 % (published: 79 %)
moe-stream, Qwen3.6-35B-A3B, 5.05 tok/s, 6 GB [23]	6 of 19.5 GB of experts (31 %)	53 %

The GLM case serves as a control: the bound (53 %) is below the published hit rate (79 %). The moe-stream case shows, without a routing trace, that a cache holding 31 % of the experts serves at least 53 % of the bytes: **locality is worth at least a factor of 1.7 on capacity** for this model.

7.3 Granularity

A stage reaches its throughput only beyond a request size and a queue depth. The machine's disk is the extreme example: 0.22 GB/s at 4 KiB against 2.4 to 2.8 GB/s at 1–16 MiB (Figure 3.1), a factor of 11 to 13. The literature says the same elsewhere: the system page cache delivers 148 MB/s where the same disk gives 1 496 MB/s with explicit expert-sized reads, with 25 000 to 50 000 page faults per

token [23]; in a GGUF file, the three matrices of an expert sit in three different tensors, hence 6 912 pages touched per expert against 192 after regrouping [104] [lit.]. **An expert must be one read, contiguous, aligned on 4 KiB.** On the card, a synchronous transfer costs a fixed latency of about 33 µs (64 KiB move at 1.5 GB/s) [meas.].

7.4 Dense models

Layer by layer (AirLLM [6]). The model is split into one file per layer; at each pass, layer ℓ is read, copied, computed, freed, and a thread prepares $\ell + 1$. This system publishes memory, not speed ("70B on a 4 GB card"). The judge quantifies it: a dense 70B in FP16 reads 140 GB per token; at the measured disk throughput, **49.6 s per token** [calc.]. It is exactly the literal reading of "without RAM or anything", and that is why it is not enough: peak memory is solved, throughput is not.

FlexGen [8] and ZeRO-Inference [7]. FlexGen amortizes the weight read over a large block of batches, with a per-layer cost model where five streams overlap, $T = \max(\text{ctog}, \text{gtoc}, \text{dtoc}, \text{ctod}, \text{comp})$, and an eleven-variable policy chosen by linear programming. It reaches 0.69 tokens/s on OPT-175B, but with an effective batch of 144 and thousands of seconds of latency [lit.]: nothing for interactive use at batch size one.

llama.cpp [9]. Read at commit `843e7dc`: memory mapping of the file (`mmap`, with `PrefetchVirtualMemory` under Windows), the simplest and worst-grained on-demand loading there is; `-ngl N` to put N layers on the card, the rest computed by the processor, without overlap between layers; `-ncmoe N` to leave the experts of N layers on the processor; `--load-mode dio` for a direct read bypassing the system cache; lazy reading of tensors above 4 GiB; and `--moe-cache-mib`, experimental, a VRAM cache of the experts kept in RAM. Reading experts from disk with an explicit cache is an open pull request [22].

LLM in a flash [13] and PowerInfer [11, 12]. The first predicts the active FFN neurons, keeps those of the last tokens in DRAM (windowing), and bundles the row and column of one neuron to read it in one piece: 20 to 25 times better than naive loading on the GPU. The second places "hot" neurons on the card and computes "cold" ones on the processor. Both rely on the activation sparsity of ReLU functions; in modern SwiGLU models it is weak, and Chapter 8 measures it.

7.5 Mixtures of experts

Offloading, prediction, caching. Mixtral-offloading [14] keeps an LRU cache of a few experts per layer in VRAM and applies the router of layer $\ell + 1$ to the hidden state of layer ℓ (the residual changes little) to prefetch: ×1.47 on an RTX 3060. The systems that follow predict better (Table 7.2); two lessons weigh for what follows. HOBBIT [16] replaces a missing expert by its low-precision copy for at most one point of loss. Edge0 [19] measures that prediction removes no bytes but takes latency off the critical path, and that only a full token of lead pays off in a real engine.

Computing the expert where it is. Fiddler [20] computes on the processor an expert absent from the card if that is cheaper than transferring it; KTransformers [21] computes experts in RAM with AVX-512 and AMX kernels and *defers* some experts: their output is added to layer $\ell + 2$ instead of $\ell + 1$, which lets the processor compute during the next attention, for at most two points of loss.

Table 7.2: Predicting and prefetching experts: published accuracy and gain [lit.].

System	Prediction and published result	Machine
Mixtral-offloading [14]	router ℓ+1 on state ℓ; ×1.47 with LRU	RTX 3060
ProMoE [15]	learned predictor, 84.7 %; decoding ×2.07	RTX 4090
HOBBIT [16]	routers ℓ+1..ℓ+3, 96 % at ℓ+1; up to ×9.93	RTX 4090, Orin
AdapMoE [105]	adaptive gating (−25 % of experts); ×1.35	edge
ExpertFlow [17]	whole-path predictor; up to ×10	one card
SeqMoE [18]	multi-step, probabilistic Belady; 96.97 % at 45 % residency	—
Edge0 [19]	pre-router, one token ahead; +80 to +84 % in decoding	Mac mini M4 Pro

On Stone's machine, the study had quantified that an expert of Qwen3-30B (2.86 MB in Q4_K_M) costs 82 µs to compute on the processor (RAM assumed at 35 GB/s) against 33 + 448 µs to transfer at 6.4 GB/s: ~~transferring costs 5.9 times more than computing in place~~. Redone with the throughputs measured in Chapter 3 [calc.]: computing on the processor costs 2.86/15.6 = 183 µs, transferring 33 + 2.86/12.2 = 267 µs, i.e. only **1.5 times** more; computing on the card from VRAM, 11 µs. With a single RAM channel, Fiddler's rule barely holds, and each expert kept in VRAM is worth sixteen kept in RAM.

Reading experts from disk (2026). llama.cpp pull request #25294 [22] manages a cache of slots in card memory, reads misses through a thread pool and reaches 2.20 tokens/s on GLM-5.2 (254 GB) with a 79 % hit rate, against 0.87 with mmap. moe-stream [23] runs under Windows, on the processor alone: Qwen3.6-35B-A3B goes from 0.82 tokens/s with mmap to 5.05 with an explicit 6 GB cache. SSD-LLaMA [24] treats SSD, RAM and VRAM as one hierarchy and **compresses the experts** (disk traffic −33.2 %), on an RTX 5090 with PCIe 5.0. SmallThinker [106] takes the opposite stance: a model designed for local use, whose router sits before attention so that experts are read during attention; locality can be bought at training time, not on an existing model [lit.].

Table 7.3: Expert cache policies: published hit rates [lit.].

Policy	Published result	Source
LRU	65.9 % at 13.4 % of budget (Qwen3-30B, one layer)	[103]
LFU	60.1 % at the same point	[103]
Static pinning	59.2 % at the same point	[103]
Belady (offline optimum)	79.1 % at the same point	[103]
"Least-Stale"	88–92 % at 1 % capacity	[107]
Learned (recency and frequency)	+51 % hit rate over LRU and LFU	[108]
Cache-conditional routing	DeepSeek-V2 misses 28 % → 7 %, perplexity +0.1 to 3 %	[109]

7.6 Expert locality from one token to the next

Locality exists and it is weak (Table 7.4): a token reuses about a quarter to 44 % of the previous token's experts, two to eight times chance; it is stronger in the middle of the network than at its ends; popularity counts as much as recency; and the domain changes the hot set. The optimum against

Table 7.4: Reuse of experts from one token to the next [lit.].

Model (E, k_e)	Adjacent reuse	"At random"
Mixtral-8x7B (8, 2) [110]	first choice repeated 13.6–14.9 % (layer 0), 24.6–28.4 % (layer 15)	12.5 %
Qwen3-30B-A3B (128, 8) [103]	0.444	0.223
Qwen3.6-35B-A3B (256, 8) [19]	"about a quarter" of a layer's set	3.1 % [calc.]
Qwen1.5-MoE, DeepSeek-V2, Mixtral [109]	mean cache residency 26, 19, 5 tokens	—
Qwen3-30B (4 domains) [103]	52.5 % of experts carry 95 % of traffic; code/other overlap 0.11–0.16	—

which every policy is judged is Belady's.

Theorem 7.2 (Belady [3])**.** *For a request sequence known in advance and a cache of C slots, the policy that, at each miss, evicts the element whose next request is the farthest away minimizes the number of misses.*

Proof. By exchange. Let $\mathcal{O}$ be an optimal policy that first differs from Belady's $\mathcal{B}$ at time t: $\mathcal{B}$ evicts x, whose next request is the farthest, and $\mathcal{O}$ evicts $y \neq x$. Build $\mathcal{O}'$ that evicts x at time t then imitates $\mathcal{O}$, with y in place of x, until the next request for y (which precedes that for x). At that request, $\mathcal{O}'$ has y cached when $\mathcal{O}$ does not: $\mathcal{O}$ pays a miss $\mathcal{O}'$ does not, or else $\mathcal{O}$ has evicted x in the meantime and the two caches become equal again. In every case $\mathcal{O}'$ pays no more misses than $\mathcal{O}$ and coincides with $\mathcal{B}$ one step longer; by induction, $\mathcal{B}$ is optimal. ■

The `chargement_localite.py` tool measures, on a routing trace, adjacent reuse, its two chance references, reuse distances and the hit rates of LRU, LFU, static pinning and Belady for every budget. Its synthetic controls hold: uniform routing (reuse 0.0613 for 0.0625 expected, tolerance of 4 standard deviations, and LRU equal to C/E at every budget), frozen routing (hit rate 0.9998), "sticky" routing with parameter $\rho = 0.6$ (0.604 measured), Belady never exceeded (theorem 7.2). With the token order shuffled, the reuse of sticky routing falls to 0.0635 and the control fails: the instrument sees the order. Two errors of the first pass were found by these controls and corrected. Figure 7.2 already shows the shape of the problem on sticky routing: the 9 to 13 point gap between LRU and Belady is the margin a perfect prediction could recover. **The instrument is ready; the real trace is missing.**

7.7 Double-buffered pipelines

With two buffers, reading piece $\ell + 1$ happens during the computation of piece ℓ; the time per piece becomes $\max(L_\ell, C_\ell)$ instead of $L_\ell + C_\ell$, plus an initial fill (lemma 3.3). To hide a latency λ, a prefetch depth $d \geq \lceil \lambda / C_{\text{layer}} \rceil$ layers is needed.

Proposition 7.3 (Double buffering hides latency, never volume)**.** *Whatever the prefetching, the time per token remains at least* $\max_i O_i/W_i$ *(theorem 3.2); a deep enough prefetch only removes the waits due to request latency.*

Proof. Prefetching changes when transfers happen, not their volume: each stage must still move O_i bytes per token, and the argument of theorem 3.2 does not depend on the order of transfers. ■

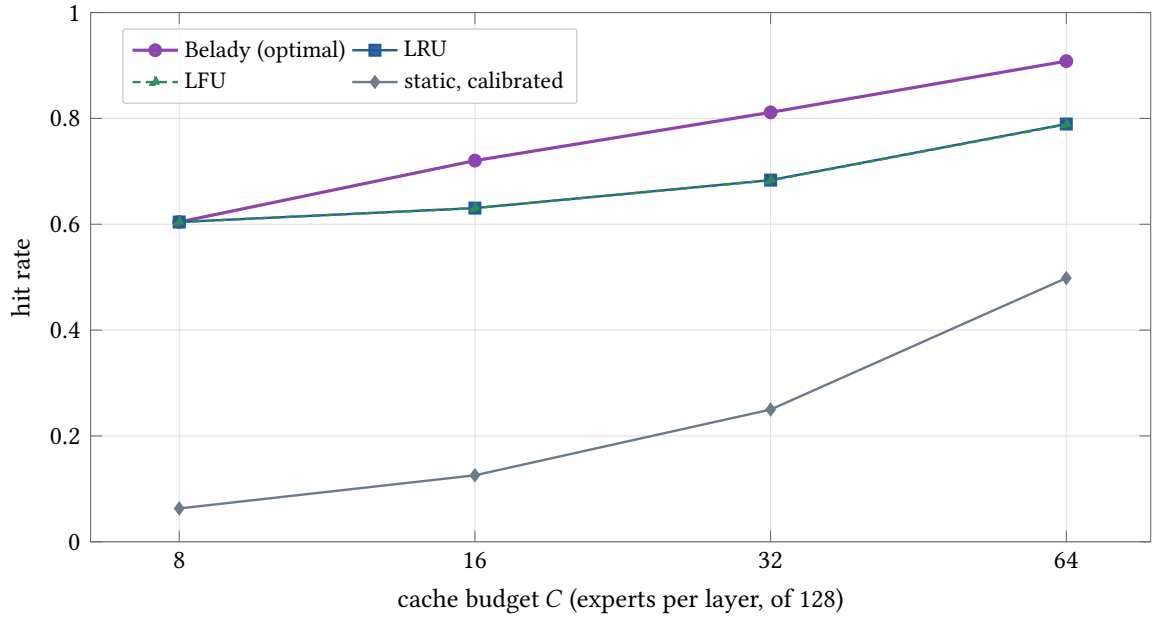


Figure 7.2: Hit rates of cache policies on the synthetic "sticky" routing control ($\rho = 0.6$, $E = 128$, $k_e = 8$) [calc.]. Belady bounds every policy; the LRU–Belady gap is what a perfect prediction would recover.

Data: `fondations/journaux/04-chargement/localite-temoins.json`, control T3.

Example [calc.]: for Qwen3-235B at about 1 token/s, a layer lasts 10.6 ms. The study had quantified reading one expert (10.76 MB) at 1.6 ms assuming a 6.7 GB/s disk; at the measured throughput (2.82 GB/s), it lasts 3.8 ms, and eight missing experts 30.5 ms, three times a layer. One layer of lead hides the latency of one miss; no lead hides the volume of eight.

What the machine gives the pipeline [meas.]. A host-to-device copy overlaps almost perfectly with VRAM traffic on another stream (hidden share 0.98 to 1.01, VRAM slowed by 5.6 %): the copy engines are free while the card works. Pinned memory is worth 21 % more throughput than pageable memory. And the link reaches generation 4 only if the card is in a high performance state (Figure 3.4): **the card must compute while we copy**, which is convenient, since that is the point of overlap.

The Windows paths. Unbuffered reads through IOCP or IoRing (the equivalent of io_uring with `O_DIRECT`); DirectStorage, which reads the NVMe into a Direct3D 12 resource and decompresses on the GPU (GDeflate since version 1.1, Zstandard in preview 1.4 [83]); CUDA, unpacked on the machine, for asynchronous copies and pinned memory. No inference engine we read uses DirectStorage.

7.8 The judge applied to the machine

The study had applied the judge with assumed throughputs (RAM 35 GB/s, disk 6.7 GB/s), then with the first provisional disk reading (2.4 GB/s), crossing out its conclusions at each correction. We rerun it here, same tool, with the throughput measured and retained by Chapter 3 (`--disque-go-s 2.82`); assumed capacities: 6.5 GB of VRAM and 10 GB of RAM for weights.

Reading (Figure 7.3 and Table 7.5):

1. **MoE models with a low B_e (at most 1.5 GB routed per token), three to five times larger than memory, are within reach**: Qwen3-Next-80B (4.6 tokens/s without locality) and Qwen3.8-

Table 7.5: The judge on the machine: bound without locality and required hit rate, disk at 2.4 GB/s (study 04) then 2.82 GB/s (retained measurement) [calc.].

Model	bits/weight	B_e (GB)	bound, 2.4	bound, 2.82	h for 5	h for 10	share that fits
Qwen3-30B-A3B	4.85	1.10	21.08	24.77	0.49	0.74	0.90
Qwen3.6-35B-A3B	4.85	0.61	18.27	21.46	0.08	0.54	0.79
gpt-oss-120b	4.25	1.90	1.70	1.99	0.70	0.85	0.26
GLM-4.5-Air	4.85	3.78	0.80	0.94	0.85	0.93	0.21
Qwen3-Next-80B	4.85	0.92	3.89	4.57	0.38	0.69	0.33
Qwen3.5-122B	4.85	2.20	1.35	1.58	0.74	0.87	0.19
Qwen3.8-Flash-Next	4.85	1.43	2.07	2.43	0.61	0.80	0.19
DeepSeek-V4-Flash	4.50	1.83	1.56	1.84	0.69	0.85	0.16
Qwen3-235B (4.56 b)	4.56	8.09	0.33	0.39	0.93	0.97	0.10
Qwen3.5-397B	4.85	4.58	0.55	0.65	0.88	0.94	0.05
Qwen3-235B (2.7 b)	2.70	4.79	0.61	0.72	0.88	0.94	0.18
GLM-5.1	2.70	7.64	0.33	0.39	0.93	0.96	0.04

Flash-Next (2.4) require 38 to 80 % hit rate for 5 to 10 tokens/s.

2. **Large MoE models with thick experts ($B_e \geq 4.5$ GB) require 88 to 93 % hit rate for 5 tokens/s**, with 4 to 18 % of the model in fast memory. The only comparable published point (LRU 65.9 %, Belady 79.1 % at 13.4 % budget) says that cache policies alone cannot get there: B_e must be reduced (Chapter 5) or the routing changed.
3. **gpt-oss-120b**: 1.99 tokens/s without locality, 70 % hit rate for 5 tokens/s. It is the third independent path that finds the roof of about 2 tokens/s (Section 3.8).

7.9 What is missing in the world

1. **No published engine reads experts from disk under Windows all the way into VRAM.** SSD-LLaMA (io_uring), pull request #25294 (Linux), Edge0 (MLX) and KTransformers (Linux) do not; moe-stream runs under Windows but on the processor alone. The dominant cost of ignoring this: existing engines read through the page cache or in small pieces, and this disk gives 11 to 13 times less with small blocks.
2. **The state of a laptop's PCIe link appears in no publication** (Section 3.5).
3. **No "hit rate versus budget" curve for 2026 MoE models at laptop budgets** (5 to 20 % of the experts); a single exact point is published.
4. **Expert deferral has never been used to absorb a read miss.** KTransformers defers present experts to keep the processor busy; Edge0 shows that latency dominates, not volume. Deferring the missing expert by one layer, instead of waiting, takes that latency off the critical path without prediction and without changing the bytes.
5. **Nobody has assembled the four responses to a miss under a measured quality budget** (Figure 7.5): wait, defer, substitute a low-precision copy, re-route to a present expert.
6. **Compression in the read loop is published only on a machine with PCIe 5.0 and an RTX 5090.** On Stone's machine, where the disk is the bound, every point of expert compression is a point of throughput if decompression fits in the compute time (corollary 3.6).

An assembly proposal, for the arbiter. What the review and the judge suggest, as testable hypotheses, each with its control: attention, shared experts, routers and a hot-expert cache in VRAM; RAM as pinned read buffers and warm cache; the disk holding all experts, rewritten one expert per contiguous aligned block, possibly losslessly coded; reads through unbuffered IoRing at queue depth at least 16, or through DirectStorage into VRAM; one-token-ahead prediction; a miss policy chosen by the routing score under a declared perplexity budget; copies launched while the card computes; measurement against llama.cpp `-ncmoe` and `--moe-cache-mib` on the same machine, in ABBA series. It is not a decision: it is the input of Chapter 10.

What could be wrong

- **The judge assumes perfect overlap** and ignores the KV cache, prefill and per-request latency: it bounds, it does not predict. Published measurements fall at 18 to 45 % of their bound.
- **The judge's RAM capacity (10 GB) and PCIe (6.4 GB/s)** are those of the original study; only the disk value was replaced here.
- **The average bits per weight** apply the same precision to experts and to the rest, which is wrong in detail.
- **Several numbers come from community repositories and discussions**, not peer reviewed; two papers contradict themselves. They are labelled, not discarded.
- **The reuse "0.444 against 0.223"** depends on a definition of chance that the paper does not give.
- **Deferral to absorb a miss was not measured**, and someone may have done it under another name.
- **The effect of the performance state on the link** is measured on this card and this driver.

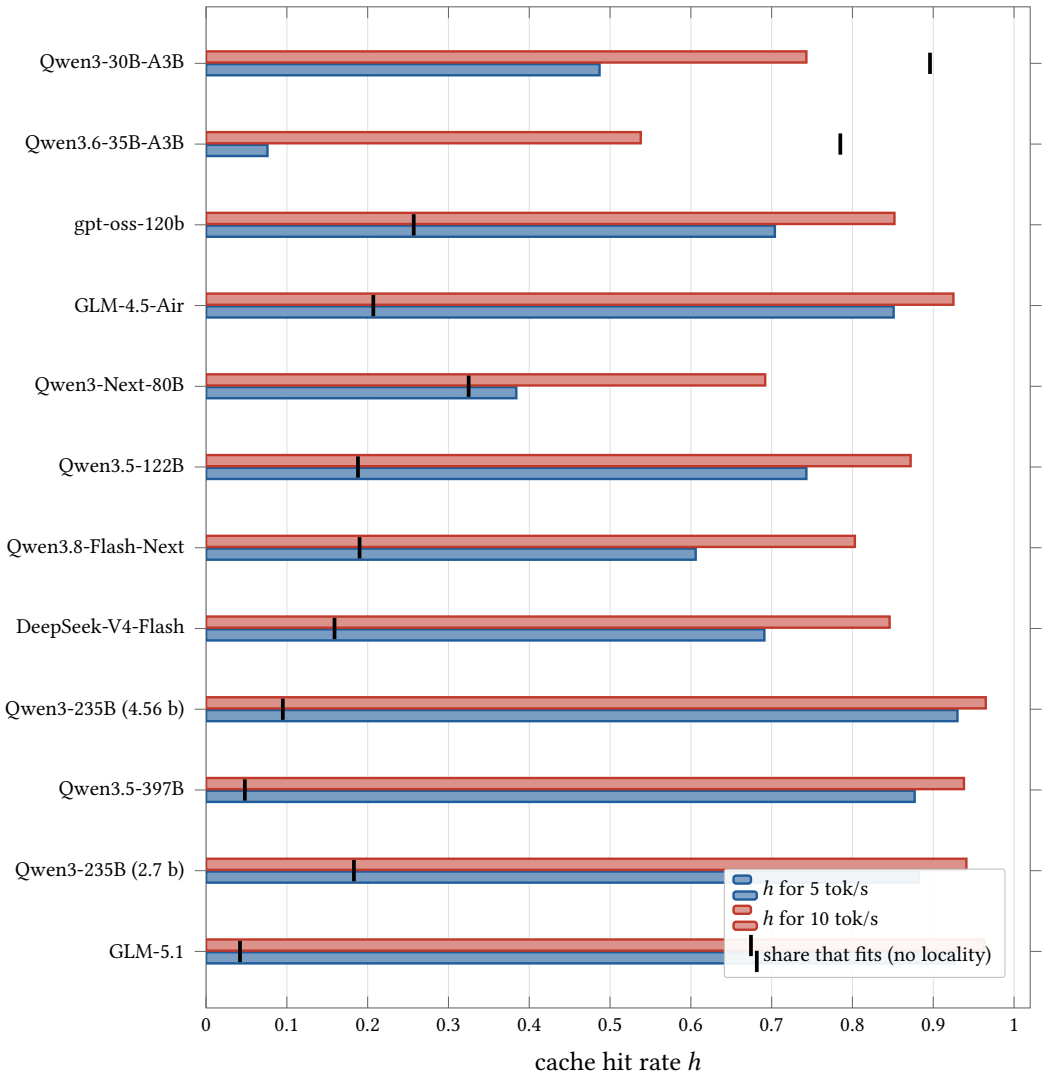


Figure 7.3: Expert-cache hit rate required for 5 and 10 tokens/s, disk measured at 2.82 GB/s, and the share of the model that fits in fast memory (black tick) [calc.]. Wherever the bar exceeds the tick, locality, prediction or fewer bytes are required.

Data: `memoire/figures/donnees/juge-stone-disque-2.82.json` (`outils/chargement_juge.py`) and `fondations/journaux/04-chargement/juge-stone-disque-2.4.json`.

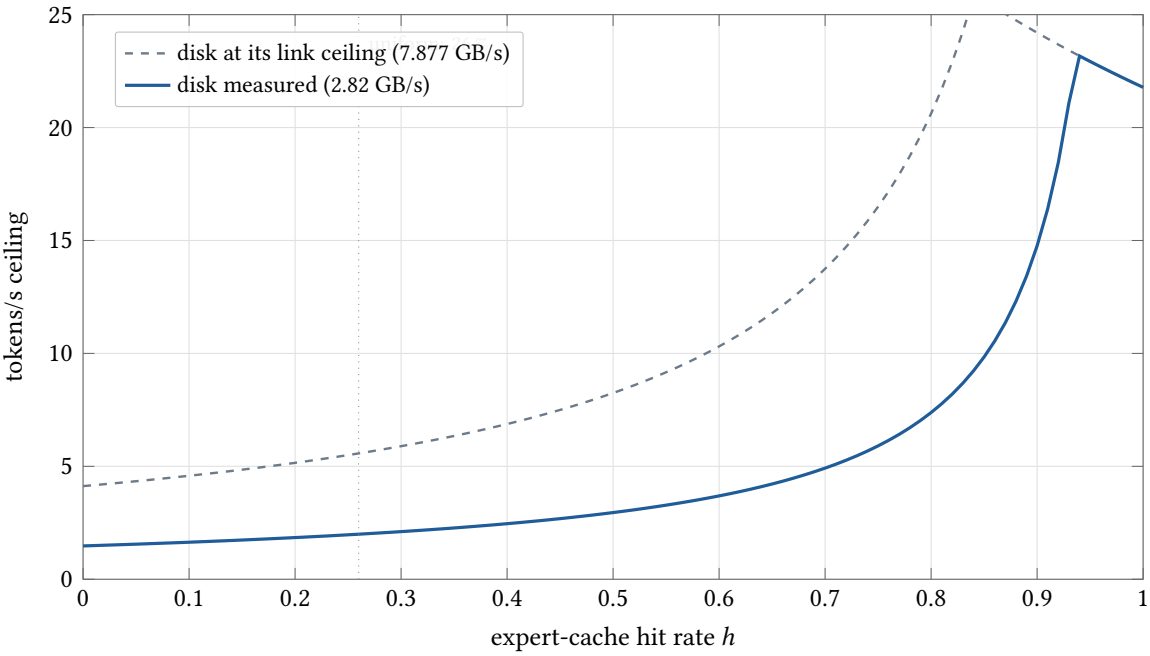
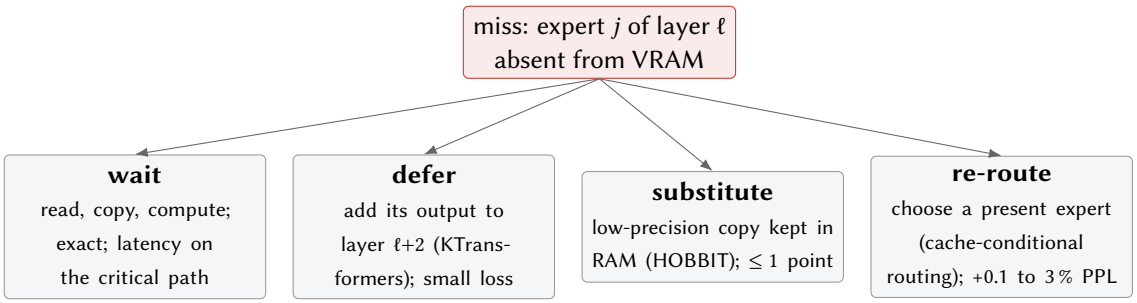


Figure 7.4: Ceiling of gpt-oss-120b versus the expert-cache hit rate h (cost model of the model study, $t = \max(B_t/256,\ hB_e/41.6,\ (1-h)B_e/W_{\text{disk}})$) [calc.]. At the measured disk throughput, going from 26 % (uniform routing) to 80 % hit rate multiplies the ceiling by 3.7.

Data: `memoire/figures/donnees/charge-sensibilite-h.csv`, from `fondations/journaux/01-machine/debits-mesures.json`.



Chosen per miss by the routing score of the missing expert, under a quality budget declared before the measurement: the term nobody has assembled (Section 7.9).

Figure 7.5: The four possible responses to an expert-cache miss, each published separately, never assembled.

Chapter 8

Sparsity and speculative decoding

Where we start, what this chapter establishes, what remains

At the start, two families of techniques promised to read fewer bytes per token: activation sparsity (read only the useful columns) and speculative decoding (read the weights once for several tokens), both published on machines where RAM holds the model. **This chapter establishes** the amortization theorem, $\mathscr{E}(\alpha, k) = (1-\alpha^{k+1})/(1-\alpha)$ tokens per read of the weights, and its corollaries; the exact condition under which speculation pays for a mixture of experts read from disk, which independent routing never satisfies; the measurement, on a SwiGLU model, that sparsity and speculation do not compose; and the acceptance rates measured on the Qwen3.5 family. **What remains** is the real expert-union curve of a 2026 MoE on draft tokens, the agreement of the native MTP module, and tree verification under Vulkan. Source: `fondations/05-sparsite.md`, `fondations/journaux/05-sparsite/`.

8.1 The judge, and the lever with the right slope

A pass of the target over n tokens (a verification batch) costs, with perfectly overlapped stages,

$$T(n) = \max\Big(\frac{O_{\text{disk}}(n)}{W_{\text{disk}}}, \frac{O_{\text{RAM}}(n)}{W_{\text{PCIe}}}, \frac{O_{\text{VRAM}}(n)}{W_{\text{VRAM}}}, \frac{C(n)}{F}\Big), \tag{8.1}$$

and, without overlap, the sum of the same terms (theorem 3.2). $O_x(n)$ is the number of weight bytes the pass reads from stage x, $C(n) \approx 2nN_{\text{active}}$ the number of operations, F the effective compute throughput. For a **dense** model, $O_x(n)$ does not depend on n: that is the whole point. For an **MoE** or a model with **sparse activations**, $O_x(n)$ grows with n, because the pass reads the union of what its n tokens request.

Token throughput is $\mathscr{E}/(T(n) + T_{\text{draft}})$, where $\mathscr{E}$ is the number of tokens accepted per pass. Three levers: reduce $O(n)$ (sparsity, hot neurons, expert prediction); increase $\mathscr{E}$ at almost unchanged $T(n)$ (speculative decoding); hide T_{draft} inside $T(n)$. When weights come from disk, $T(n)$ is counted in seconds and T_{draft} in milliseconds: **the second lever crushes the other two, on one condition, that** $O(n)$ **stays flat in** n. Everything that follows quantifies that condition.

Proposition 8.1 (The compute knee)**.** *For a dense model of N active weights at b bytes per weight, read from a stage of throughput W, verifying n tokens stays bounded by reading as long as*

$$n < n^\star = \frac{F\,b}{2\,W}. \tag{8.2}$$

Proof. Reading costs Nb/W seconds whatever n; computing costs $2nN/F$. The latter stays below the former as long as $2nN/F < Nb/W$, i.e. $n < Fb/(2W)$. ∎

With $F \approx 20$ Tflop/s (order of magnitude of the FP16 tensor cores of an RTX 4060 Laptop, not measured, and lowered by dequantization) and $b = 0.6$ byte per weight (about 4.8 bits) [calc.]: $n^\star \approx 24$ if weights are in VRAM (251.3 GB/s), ≈ 490 if they cross the link (12.2 GB/s; the study, with the link loaded at 6.38 GB/s, gave 940), $\approx 2\,100$ if they come from disk (2.82 GB/s). This table says why SpecExec verifies trees of 1 024 to 2 048 tokens in the offloaded regime, and why EAGLE on a card that holds the model stops at a few dozen: it is not a choice, it is $n^\star$. The measurement of Figure 8.1 confirms it despite its noise: up to 32 tokens, an all-in-VRAM pass costs at most 1.35 times a one-token pass.

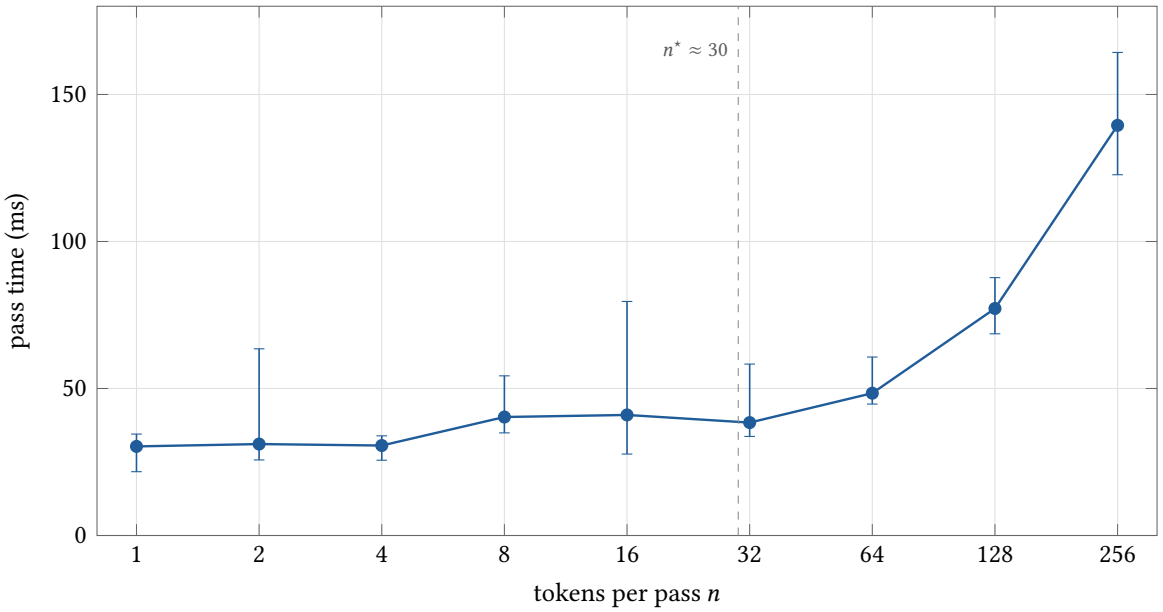


Figure 8.1: Time of a pass of n tokens, Qwen3.5-4B Q4_K_M all in VRAM, median and range of four ABBA series [meas.]. The declared noise threshold (10 %) is reached for no n (shared card): indicative numbers. The knee lies between 32 and 128, where Proposition 8.1 puts it.

Data: `fondations/journaux/05-sparsite/cn/resume-4B-ngl99.txt`.

8.2 Speculative decoding

8.2.1 The verification rule

Leviathan, Kalman and Matias [59] and Chen et al. [60] state the same rule. A draft with law q proposes $x_1, \dots, x_k$; the target p computes in one pass $p(\cdot \mid \text{prefix}, x_1, \dots, x_i)$ for $i = 0, \dots, k$. Each x_i is accepted with probability $\min(1, p(x_i)/q(x_i))$; at the first refusal, a token is drawn from the residual law $r = (p - q)^+ / \|(p - q)^+\|_1$; if everything is accepted, one more token is drawn from p.

Proposition 8.2 (Speculation is exact)**.** *The token emitted at each position follows exactly the target law p, and the acceptance probability of a position is $\alpha = \sum_x \min(p(x), q(x)) = 1 - \frac{1}{2}\|p - q\|_1$.*

Proof. The probability of emitting x is that of proposing and accepting it, plus that of refusing then drawing x from r:

$$q(x)\min\Big(1, \frac{p(x)}{q(x)}\Big) + (1 - \alpha)\,\frac{(p(x) - q(x))^+}{1 - \alpha} = \min(p(x), q(x)) + (p(x) - q(x))^+ = p(x),$$

using $\|(p-q)^+\|_1 = \sum_x (p(x) - \min(p(x), q(x))) = 1-\alpha$. The acceptance probability is $\sum_x q(x) \min(1, p(x)/q(x)) = \sum_x \min(p(x), q(x))$, and $\sum_x \min(p, q) = 1 - \frac{1}{2}\sum_x |p - q|$ since $\min(a, b) = \frac{1}{2}(a + b - |a - b|)$. ■

In greedy decoding, a token is accepted if it is the target's argmax; α is then the argmax agreement rate, measured in Section 8.5.1.

8.2.2 The amortization theorem

Theorem 8.3 (Tokens per read of the weights)**.** *If each of the k drafted positions is accepted with the same probability $\alpha < 1$, independently of the others and as long as none has been refused, the number N of tokens produced by one verification pass (one full read of the active weights) has expectation*

$$\mathscr{E}(\alpha, k) \;=\; \mathbb{E}[N] \;=\; \sum_{i=0}^{k} \alpha^i \;=\; \frac{1 - \alpha^{k+1}}{1 - \alpha}. \tag{8.3}$$

Proof. Let A be the number of drafted tokens accepted before the first refusal, $0 \le A \le k$. The pass always returns one token more than A: the correction drawn from r at the first refusal, or the bonus token drawn from p if everything is accepted. Hence $N = A + 1$. The first i positions are all accepted with probability α^i, so $\mathbb{P}(A \ge i) = \alpha^i$ for $0 \le i \le k$, and $\mathbb{P}(A \ge i) = 0$ beyond. For a non-negative integer variable, $\mathbb{E}[A] = \sum_{i \ge 1} \mathbb{P}(A \ge i)$; thus $\mathbb{E}[N] = 1 + \sum_{i=1}^{k} \alpha^i = \sum_{i=0}^{k} \alpha^i$, a geometric sum equal to $(1 - \alpha^{k+1})/(1 - \alpha)$. ■

Corollary 8.4 (The chain bound)**.** *$\mathscr{E}(\alpha, k) < 1/(1 - \alpha)$ for every k, and the chain reaches the fraction $1 - \alpha^{k+1}$ of that bound. At $\alpha = 0.8$, $k = 8$ already gives 87 % of the bound: beyond, only a better draft or a* tree *raises $\mathscr{E}$.*

Proof. $\mathscr{E}(\alpha, k)(1 - \alpha) = 1 - \alpha^{k+1} < 1$. ■

Corollary 8.5 (The optimal k when the pass is expensive)**.** *With a pass costing T and a draft costing t_b per proposed token, the throughput $\mathscr{E}(\alpha, k)/(T + kt_b)$ is maximal at a $k^\star$ that grows as $\log(T/t_b)/\log(1/\alpha)$; for $T = 8$ s and $t_b = 5$ ms, $k^\star = 25$ at $\alpha = 0.8$ and 48 at $\alpha = 0.9$ [calc.].*

Proof. The marginal gain of the $(k+1)$-th drafted token is α^{k+1} token, its marginal cost t_b for a total cost $T + kt_b$; the optimum roughly satisfies $\alpha^{k^\star} \mathscr{E}^{-1} \approx t_b/(T + k^\star t_b)$, hence $k^\star \approx \log(T/t_b)/\log(1/\alpha)$ up to logarithmic terms. The numerical values come from the exact maximization of $(1 - \alpha^{k+1})/((1 - \alpha)(T + kt_b))$ over the integers. ■

The control. The `amortissement.py` tool simulates 20 000 chains per case and compares against Equation (8.3): gap below 4 standard errors in all 9 cases. With a wrong formula (without the bonus token), 7 cases out of 9 fail [meas.]. Figure 8.2 and Table 8.1 give the values.

8.2.3 Trees

With a tree of K nodes, the pass verifies all branches at once (tree attention mask) and keeps the longest accepted path (Figure 8.3). SpecInfer [68] samples the branches, Sequoia [69] builds the optimal tree by dynamic programming under a budget, SpecExec [70] keeps the K most probable continuations of the draft: it is the variant made for the offloaded regime. Its decisive property: $\mathscr{E}$ is no longer bounded by $1/(1 - \alpha)$ but by the coverage of the tree. SpecExec reports up to 20.6 tokens accepted per pass (budget 2 048) on offloaded Llama 2 70B, and, **on a desktop RTX 4060, 70B in GPTQ: 9.7 tokens per pass, 3.28 tokens/s,** ×4.6 [lit.].

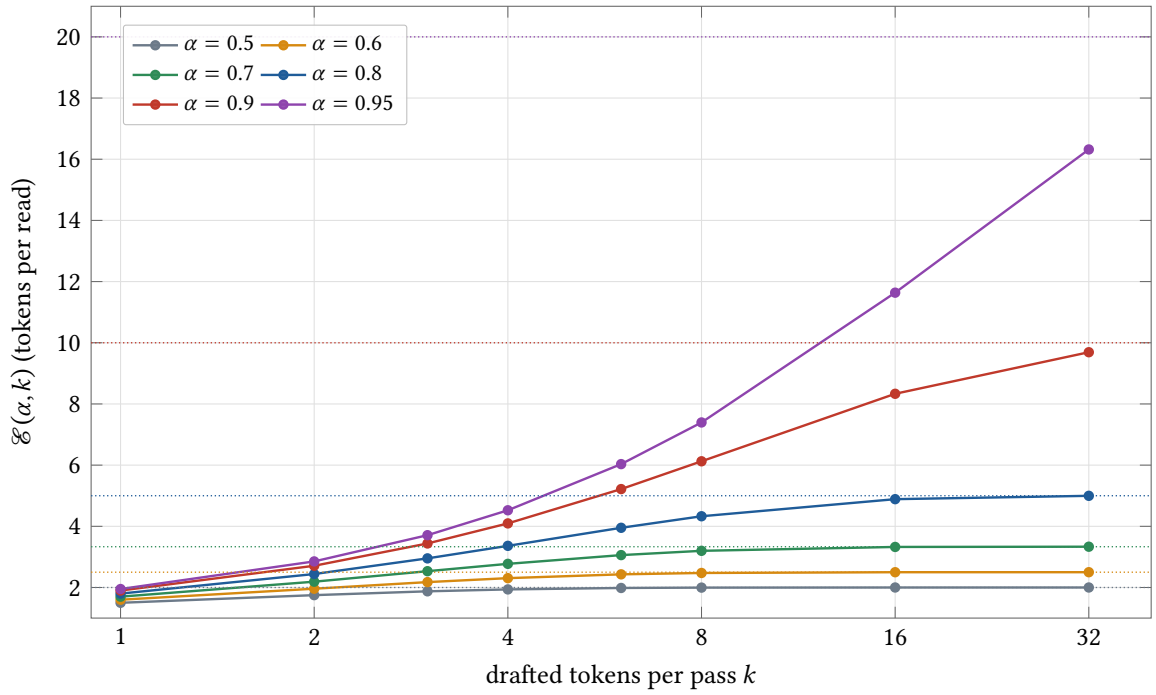


Figure 8.2: Tokens produced per full read of the active weights, $\mathscr{G}(\alpha, k)$, and their asymptotes $1/(1-\alpha)$ dotted (theorem 8.3 and corollary 8.4) [calc.]. The bound arrives fast: past $k \approx 8$, a longer chain barely helps.

Data: `fondations/journaux/05-sparsite/amortissement-tables.json` (`outils/05-sparsite/amortissement.py`).

8.2.4 The variants

The trap of hybrid models. Qwen3.5, 3.6, 3.8 and Qwen3-Next mix full-attention layers with delta-rule linear-attention layers (Gated DeltaNet), whose state is a matrix per head updated at every token. A refused token cannot be removed from a recurrent state: the previous state must have been kept. For Qwen3.5-9B (32 value heads, 128×128 state per head, 24 linear layers out of 32) [calc.]:

state per layer $= 32{\times}128{\times}128{\times}4$ bytes $= 2$ MiB, model state $= 24{\times}2 = 48$ MiB per kept position.

A tree of 256 nodes requires 12 GiB: impossible on 8 GB. STree [113] verifies trees for state-space models by exploiting the structure of their transition matrices; the delta-rule transition, $\alpha_t(I - \beta_t k_t k_t^\top)$, is not diagonal, and no tree verification is published for it.

8.3 Mixtures of experts and speculation

A verification pass of an MoE reads the union of the experts of its n tokens.

Lemma 8.6 (The union of experts under independent routing)**.** *If each token chooses k_e distinct experts out of E, uniformly and independently of the other tokens, the expected number of distinct experts requested by n tokens in a layer is*

$$U(n) = E\left(1 - \left(1 - \frac{k_e}{E}\right)^n\right). \tag{8.4}$$

Proof. A given expert is not chosen by a token with probability $1 - k_e/E$; by independence, it is chosen by none of the n tokens with probability $(1 - k_e/E)^n$. By linearity of expectation, the expected number of experts chosen by at least one token is E times the complementary probability. ∎

Table 8.1: Tokens produced per full read of the active weights, $\mathscr{E}(\alpha, k)$ [calc.].

$\alpha \setminus k$	1	2	3	4	6	8	16	∞
0.5	1.50	1.75	1.88	1.94	1.98	2.00	2.00	2
0.6	1.60	1.96	2.18	2.31	2.43	2.47	2.50	2.5
0.7	1.70	2.19	2.53	2.77	3.06	3.20	3.33	3.33
0.8	1.80	2.44	2.95	3.36	3.95	4.33	4.89	5
0.9	1.90	2.71	3.44	4.10	5.22	6.13	8.33	10
0.95	1.95	2.85	3.71	4.52	6.03	7.40	11.64	20

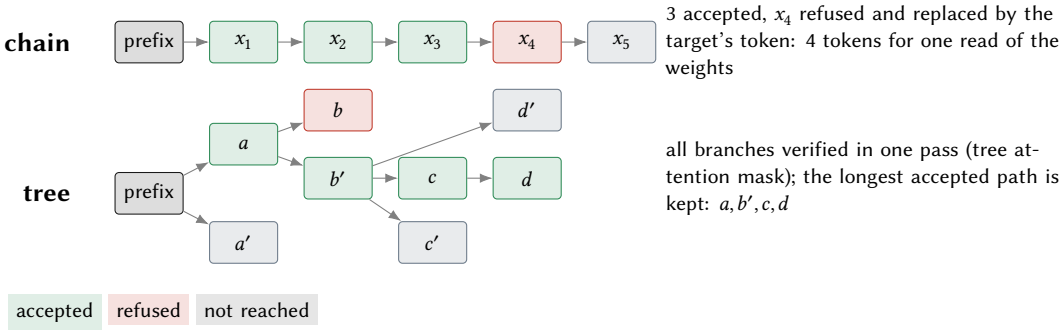


Figure 8.3: A verification pass as a chain and as a tree. The chain stops at the first refusal; the tree verifies all its branches in the same pass and keeps the longest accepted path.

Proposition 8.7 (When speculation pays for an MoE read from disk)**.** *If the bytes coming from disk are dominated by routed experts, verifying k drafted tokens reduces the bytes read per produced token if and only if*

$$\mathscr{E}(\alpha, k) \;>\; \frac{U(k+1)}{k_e}. \tag{8.5}$$

Proof. Without speculation, a token reads k_e experts per layer. The verification pass covers $k+1$ positions (the prefix and the k drafted tokens) and reads $U(k+1)$ experts per layer; it produces $\mathscr{E}(\alpha, k)$ tokens on average (theorem 8.3). Bytes per produced token drop if and only if $U(k+1)/\mathscr{E}(\alpha, k) < k_e$. ∎

Corollary 8.8 (Speculation never pays on independent fine-grained experts)**.** *For $k = 1$, the condition becomes $1+\alpha > 2-k_e/E$, i.e. $\alpha > 1-k_e/E$: $\alpha > 0.977$ for DeepSeek-V4-Flash ($k_e/E = 6/256$), $\alpha > 0.980$ for Qwen3.8-Flash-Next ($10/512$) [calc.]. In the fine-grained limit ($k_e(k+1) \ll E$), $U(k+1)/k_e \to k+1 > \mathscr{E}(\alpha, k)$ for every $\alpha < 1$.*

Proof. By lemma 8.6, $U(2)/k_e = 2-k_e/E$ and $\mathscr{E}(\alpha, 1) = 1+\alpha$. In the limit, $(1-k_e/E)^{k+1} \approx 1-(k+1)k_e/E$, so $U(k+1)/k_e \to k+1$; yet $\mathscr{E}(\alpha, k) = \sum_{i=0}^{k} \alpha^i < k+1$ as soon as $\alpha < 1$. ∎

Stone's regime (batch of one, weights on disk) is exactly the one where Cascade [72] observes that speculation slows MoE models down. Speculation therefore pays, for a fine-grained MoE read from disk, only if the real union is far below independence (neighbouring tokens share their experts), or if the expert set of the verification is bounded: MoE-Spec [73] (fixed budget per layer), AcceptMoE [74] (eligibility restricted to experts already cached: ×2.06, −0.27 accuracy point), EcoSpec [75] and EVICT [114] (lossless), DraftExpert [115] [lit.]. The real union over draft tokens is published for no model the machine can read.

Table 8.2: The variants of speculative decoding: draft, published gain, availability under Windows without the CUDA toolkit [lit.]. llama.cpp [9] (commit `e467e5b`) only verifies chains.

Variant	Draft and published gain	Under Vulkan
Small model [59, 60]	model of the same family, 10 to 50 times smaller; lossless	yes (chain)
Medusa [61]	extra heads on the last state; ×2.2 to ×3.6	no
EAGLE-2, EAGLE-3 [62, 63]	one layer reading the target's states, dynamic tree; up to ×6.5	chain only
Native MTP [64, 65]	multi-token prediction module trained with the model; a free draft if resident in VRAM	yes (`draft-mtp`)
DFlash [66]	block diffusion, a whole block per pass; more than ×6	yes
N-grams, prompt lookup [67]	zero cost; huge on rewriting, nearly nil on free prose	yes
LayerSkip, QSpec [111, 112]	the target itself, skipped layers or 4-bit activations; ×1.6 to ×2.2	no
SubSpec [71]	resident 4-bit substitutes of the offloaded layers; 27 to 35 tokens per pass; Qwen2.5-7B under 8 GB: 2.68 → 27.1 tokens/s	no (PyTorch)
SpecExec, Sequoia [69, 70]	trees of hundreds of nodes in the offloaded regime; 10 to 20 tokens per pass	no

Table 8.3: Speculation and MoE models whose experts come from disk, uniform independent routing (pessimistic) [calc.].

Model	B_t / B_e (GB)	E / k_e	$U(4)/U(1)$	optimal k, $\alpha = 0.9$
DeepSeek-V4-Flash Q2_K_XL	6.69 / 2.10	256 / 6	3.86	**0**
Qwen3.8-Flash-Next Q2_K_XL	3.54 / 0.90	512 / 10	3.88	**0**
gpt-oss-120b MXFP4	1.69 / 1.91	128 / 4	3.82	**0**
Kimi-K2.6 Q2_K_XL	8.52 / 6.90	384 / 8	3.88	**0**

Predicting experts does not reduce bytes. Mixtral-offloading, ProMoE, HOBBIT (Section 7.5) predict in order to prefetch: this hides the loading latency behind computation (Proposition 7.3). In the disk regime, where the disk works all the time, prefetching only yields the step from the sum to the maximum of theorem 3.2. Only caching and lossy substitution reduce the bytes coming from disk.

8.4 Activation sparsity

What it takes for a zero to become an unread byte. In VRAM: column-major weights and a kernel that loads only the live columns (TEAL's kernel requires it: `assert weight.stride(1) > 1`); no such kernel exists in llama.cpp, whose quantized types are row-major. On disk: the granularity is the page, and an FFN neuron of Qwen3.5-9B at 4.5 bits (three vectors of 4 096) weighs about 6.9 KiB [calc.], two pages, far below the size at which this disk delivers its throughput (Figure 3.1).

8.4.1 The measurement: unions, window, hot neurons

The instrument. `llama_numpy.py` is a numpy forward pass of a "llama" model read from the GGUF, written from scratch, with a hook on the input of every projection. Model: SmolLM2-

Table 8.4: Activation sparsity methods [lit.].

Method	What is sparse, published gain	Condition
Deja Vu [10]	heads and neurons per token, predicted one layer ahead; ×2	ReLU (OPT)
PowerInfer [11]	hot neurons on the card, cold on the CPU; up to ×11.69	ReLU
LLM in a flash [13]	sliding window of recent neurons; row–column bundling	ReLU
CATS [116]	threshold on the input of down; +15 % speed	—
TEAL [117]	magnitude threshold on the input of every matrix; ×1.53 at 40 %	column-major weights
R-Sparse [118]	channels and singular components; +43 %	—
TurboSparse [119]	retrained dReLU; ×2 to ×5	retraining

135M-Instruct F16 (30 layers, SwiGLU). Control: the perplexity of the numpy pass must match `llama-perplexity` within 0.5 % (threshold declared before); it gives 22.6768 against 22.6739 over 16 windows (0.013 %, largest gap 0.18 %), and **the control fails when broken** (RoPE rotation by halves instead of adjacent pairs: 232.8 against 10.4) [meas.].

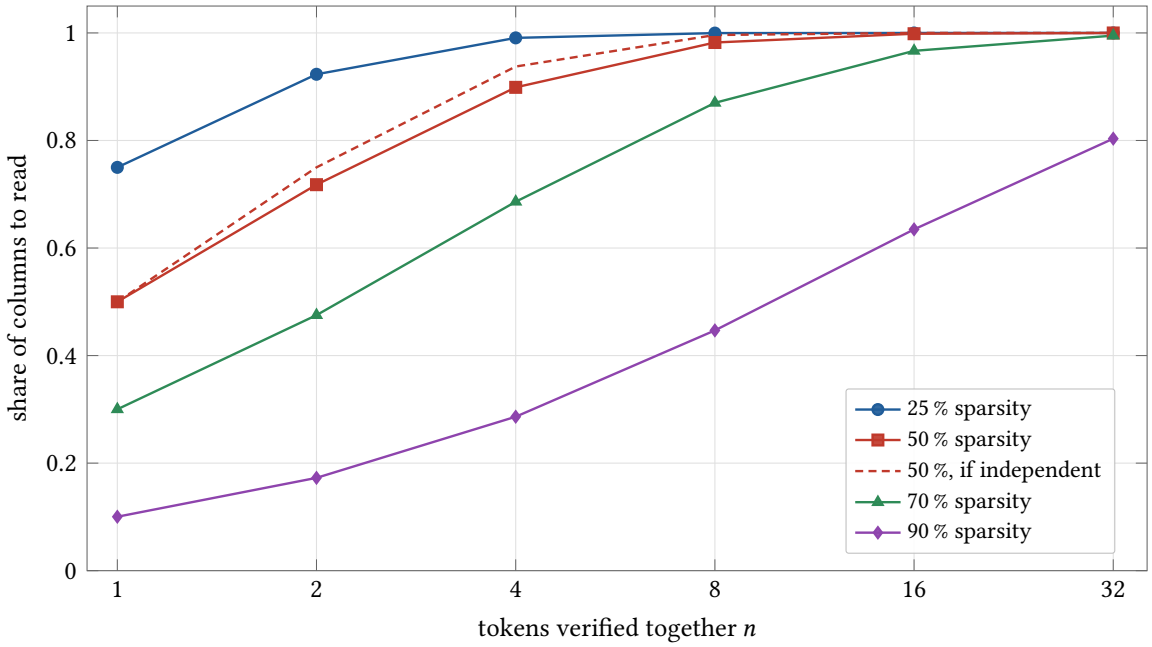


Figure 8.4: Share of the columns of down that a verification of n consecutive tokens must read if each keeps its top-k mask, versus per-token sparsity, and the same union if masks were independent (SmolLM2-135M, 2 048 tokens) [meas.]. At 50 % sparsity, four tokens already read 90 % of the columns.

Data: `fondations/journaux/05-sparsite/stats-smol.json`.

The unions (Figure 8.4) lie below independence: correlation between neighbouring tokens exists, but it is weak. Weighted by the bytes of the four inputs of a block, the union is **0.895 at $n = 4$ and 0.976 at $n = 8$** for 50 % per-token sparsity [calc.]: a verification of four tokens reads only 10.5 % less than a dense pass. **Composed naively with speculation, per-token sparsity vanishes.**

LLM in a flash's sliding window reduces traffic at the price of a RAM holding almost the whole matrix: at equal RAM (72 % of the columns), a dense static cache leaves 28 % of the bytes to the disk, the window $w = 1$ leaves 21.8 %, a gain of only 1.28. And **there is no power law of hot neurons in a SwiGLU model**: covering 80 % of the selections at 50 % sparsity takes 74 % of the channels of down

[meas.]. PowerInfer's factor of 11.69 assumes ReLU sparsity (more than 90 % inactive neurons) and a power law that SwiGLU does not give.

8.4.2 A mask shared by the verified block

The only way to keep the bytes of sparsity under verification is for the n tokens of a block to **share a single mask**. Measured rule: for each block of n consecutive tokens, keep the $(1-s)D$ channels of largest energy $\sum_i v_{ij}^2$, optionally weighted by $\|W_{:,j}\|^2$, the norm of the column we give up reading.

Table 8.5: Perplexity of SmolLM2-135M under different sparsity rules, WikiText-2, 1 020 paired positions [meas.].

Rule	columns read per pass of n tokens	perplexity	gap
none (dense)	100 %	17.86	—
per-token top-k, $s = 25$ %	75 % at $n = 1$; ≈ 99 % at $n = 4$	18.24	+2.1 %
per-token top-k, $s = 50$ %	50 % at $n = 1$; 89.5 % at $n = 4$	27.09	+51.6 %
shared mask, block of 2, weighted, $s = 25$ %	75 % for $n \leq 2$	19.84	+11.0 %
shared mask, block of 4, weighted, $s = 25$ %	75 % for $n \leq 4$	23.96	+34.1 %
shared mask, block of 2, weighted, $s = 50$ %	50 % for $n \leq 2$	62.87	×3.52
shared mask, block of 4, weighted, $s = 50$ %	50 % for $n \leq 4$	301.0	×16.9
shared mask, block of 16, weighted, $s = 50$ %	50 % for $n \leq 16$	4 276	×239

Data: `fondations/journaux/05-sparsite/ppl-smol.json`.

On a dense SwiGLU without retraining, activation sparsity and speculative verification do not compose (Table 8.5): either each token keeps its mask and the pass reads almost everything, or tokens share a mask and quality collapses. The diagnosis confirms it: the share of a token's energy kept at density 0.5 drops from 0.956–0.986 (per-token mask) to 0.805–0.918 (block of 16), and the lost energy, compounded over 30 layers and 4 inputs, blows up perplexity; a random mask, the control, keeps 0.499 to 0.502. Caveat: 135 million parameters; TEAL reports smaller losses at 7–70 billion.

8.5 The computation for Stone's machine

8.5.1 Acceptance rates, measured

The "same top token" of `llama-perplexity --kl-divergence` is the share of positions where the draft's argmax equals the target's, the text serving as prefix: it is the greedy acceptance rate of the first drafted position, rather a lower bound (in real generation, the prefix is the target's own text, usually more predictable).

The control falls at exactly 100 % when the target is the draft: the tool does measure agreement (Table 8.6). A draft of the same family, half the size, returns at most 4.3 tokens per pass as a chain; a quantized copy of the target itself agrees much better (91.9 % at about 4.8 bits), hence SubSpec's acceptance length. **The naive substitute collapses**: the 9B Q2_K requantized without an importance matrix takes perplexity from 8.47 to 105 and agreement to 34 %. A substitute is made with the best available quantization, not the simplest.

Table 8.6: Measured greedy acceptance rates and the tokens per pass they imply (theorem 8.3) [meas.] [calc.].

Draft → target	α	KLD	$\mathscr{E}(\alpha, 8)$	$1/(1-\alpha)$
control: 4B Q4_K_M against itself	**100.000 %**	0	—	—
Qwen3.5-4B Q4_K_M → Qwen3.5-9B Q4_K_M	76.8 ± 0.9 %	0.279	3.91	4.31
4B IQ4_XS (4.76 bits) → 4B Q4_K_M	91.9 ± 0.3 %	0.029	6.57	12.35
4B IQ3_XXS (3.60 bits) → 4B Q4_K_M	82.1 ± 0.4 %	0.148	4.64	5.59
4B IQ2_XXS (2.91 bits) → 4B Q4_K_M	65.8 ± 0.5 %	0.608	2.86	2.92
9B naive Q2_K (3.41 bits, no imatrix) → 9B Q4_K_M	34.0 ± 1.0 %	2.60	1.52	1.52

Data: `fondations/journaux/05-sparsite/alpha-*.log`, `temoin-alpha-4B-contre-lui-meme.log`, study 02 for the quantized copies of the 4B.

8.5.2 The resident substitute, transposed

SubSpec keeps a 4-bit copy of the target in VRAM and streams the 16-bit target over the link from 128 GB of RAM. At Stone's, 16 GB of RAM impose an already quantized target (about 4.8 bits) and 8 GB of VRAM a very low-rate substitute. With the measured agreements, a 6 GB VRAM budget for the substitute and a Q4_K_M target in RAM, the study computes for a 14B target (9.1 GB, 1.42 s pass, link loaded at 6.38 GB/s): 0.70 tokens/s without speculation, **1.75 tokens/s (**×2.5**)** with the best substitute that fits (IQ2_XXS, 5.1 GB, $\alpha = 0.66$, $k^\star = 7$) [calc.]. A 2-bit substitute keeping 85 % agreement would take the 14B to about 3.6 tokens/s. **This is the exact point where quantization and speculation multiply**: extreme quantization no longer only reads less, it manufactures the draft.

8.5.3 Dense models read from disk

Table 8.7: Tokens/s of a dense model partly on disk, without and with speculation (resident draft, 5 ms per token); disk assumed between 3 and 7 GB/s by the study, the measured throughput (2.82 GB/s) lying below the lower bound [calc.].

Case	disk	$T(n)$	no spec.	$\alpha = 0.8$	$\alpha = 0.9$	tree, 10/pass
70B Q4 (42 GB), 24 GB on disk	3 GB/s, serial	9.76 s	0.10	0.50	0.99	1.02
same	7 GB/s, overlapped	3.43 s	0.29	1.40	2.72	2.87
70B ≈2.3 bits (20 GB), 2 GB on disk	3 GB/s, serial	2.43 s	0.41	1.96	3.76	4.04
same	overlapped	1.72 s	0.58	2.72	5.15	5.64

Data: `fondations/journaux/05-sparsite/amortissement-tables.json`.

In the disk cases, speculation multiplies throughput by $\mathscr{E}$ almost exactly (Table 8.7): $k^\star$ is large, the draft negligible (×5 at $\alpha = 0.8$, ×10 at $\alpha = 0.9$). Quantization and speculation multiply: going from 4.8 to 2.3 bits nearly takes the model off the disk, speculation then multiplies by $\mathscr{E}$, and it adds no quality loss.

8.6 What is missing in the world

1. **Speculation with weights read from the NVMe when RAM is smaller than the model.** SpecExec (140–192 GB of RAM), SubSpec (128 GB), Sequoia, TriForce read weights from RAM; on 16 GB and a 2.82 GB/s disk, it is an unmeasured factor of 10.

2. **Trees under Windows without CUDA**: llama.cpp verifies chains, capped at $1/(1-\alpha)$ (corollary 8.4).
3. **The real expert-union curves of 2026 fine-grained MoE models over draft tokens**: all profitability depends on the gap to Equation (8.4).
4. **Splitting 8 GB of VRAM between draft and target**: each GB given to the draft goes back to the link or the disk; nobody publishes the optimum.
5. **Tree verification for delta-rule hybrids** (48 MiB of state per node).

Recommendations, for the arbiter. The cheapest and most accurate draft is the target itself, quantized very low and resident, or its native MTP module; verify trees, not chains; for MoE models, verify only within the experts already in memory and measure the loss; do not invest in per-token activation sparsity for the disk regime on SwiGLU models.

What could be wrong

- **The sparsity model is small** (135 M); large models are sparser, and the correlation between neighbouring tokens may be stronger.
- **The per-token top-k mask is not exactly TEAL** (calibrated thresholds, variable density); it has the same mean density.
- **The constant, independent α assumption** overestimates $\mathscr{E}$ for long chains.
- **Independent routing** is pessimistic; if neighbouring tokens share many experts, "optimal $k = 0$" falls.
- $F = 20$ **Tflop/s** is not measured; $n^\star$ is an order of magnitude.
- **The "same top token" is not exactly α**: forced by a text, first position only, greedy.
- **The substitute agreements come from the 4B**; a 14B quantizes better at equal bits.
- **The machines were shared**: perplexities do not depend on it, timings do.

Chapter 9

The target models

Where we start, what this chapter establishes, what remains

At the start, the mission speaks of "any ultra-heavy model": we had to say which exist, what they read per token, under which licence, and how to judge troncons against a baseline. **This chapter establishes** an inventory of 49 official repositories read from their headers, without downloading a single weight, validated against the publishers' numbers by a control that fails; the bytes read per token of the exact candidate files; their ceilings on the machine, recomputed at the measured disk throughput; the 2026 trends that play for troncons; and a judging protocol in which one measurement has already corrected an assumption. **What remains** is the routing trace of a 2026 model and the first baseline run on gpt-oss, downloaded and verified. Source: `fondations/06-modeles.md`, `fondations/journaux/06-modeles/`.

9.1 What "ultra heavy" means here

The machine offers about 6.5 GB of VRAM and 11 GB of RAM for weights (stated assumptions: 8 188 MiB minus the display, the context and a short KV cache; 15.6 GB minus the system), about 17.5 GB in all, and 404 GB of free disk. A model is "ultra heavy" when its file clearly exceeds these 17.5 GB; it becomes interesting for troncons when it is a multiple of it (three times or more) **and** its bytes read per token stay small. A dense model reads all its weights at every token: at 4 bits, Llama-3.3-70B reads 34.8 GB per token [calc.]; the ground is the mixture of experts.

9.2 The inventory, read from the headers

The method. A safetensors file starts with 8 bytes of length followed by a JSON giving the type and shape of every tensor. Two HTTP range requests per shard suffice: we get the exact shape of every tensor of a 2.8-trillion-parameter model without downloading its weights (`inventaire_modeles.py`). The tool sorts each tensor into a category: routed experts (one of E read per router choice), always read, embedding (one row per token), MTP (read only during speculative decoding), lookup (Engram or n-gram tables), non-text. Active parameters per token = always read + $\sum_{\text{layers}}(k_e/E)\times$ routed experts of the layer. For the GGUF files we would download, the tool reads the header of the exact file (from the public specification [120]); its built-in control, the sum of tensor sizes plus the header equals the file size, holds to the byte for the 8 files read.

The control, and its fault. Second path: the numbers published by the publisher. Thresholds declared in the tool before the first run: total within ±3 % (publishers round to the billion; 3 % lets through neither a factor of 2 nor duplicated files), active within ± max(0.6 G; 10 %).

- **Pass 3 (the good one)**: 37 agreements out of 38; the only failure, Qwen3.8-Flash-Next (6.67 G active against "6B"), is explained by the output head (0.636 G), which we count: without it, 6.03 G. The threshold stays as it was and the failure is written down.
- **Pass 2 (the fault)**: the same control finds 6 failures, and they are 6 real bugs that pass 3 fixes: Mistral counted twice (two copies of the same weights at the root), MiMo V2.6 and Kimi K3 counted at half (FP4 packed in bytes not recognized), Kimi K3 with 50 active experts instead of 16 (the `top_k` key of its configuration is a sampling parameter), Step-3.5-Flash at 196 G active, Qwen3.8 at 57.9 G active (the n-gram table counted as always read).

A predecessor's first pass gave ~~DeepSeek-V4-Flash: 149.2 G in total, 10.6 G active~~ (packed FP4 not recognized, output head missed); corrected: 284.3 G without MTP and 13.27 G active, against 284 G and 13 G in the paper [121].

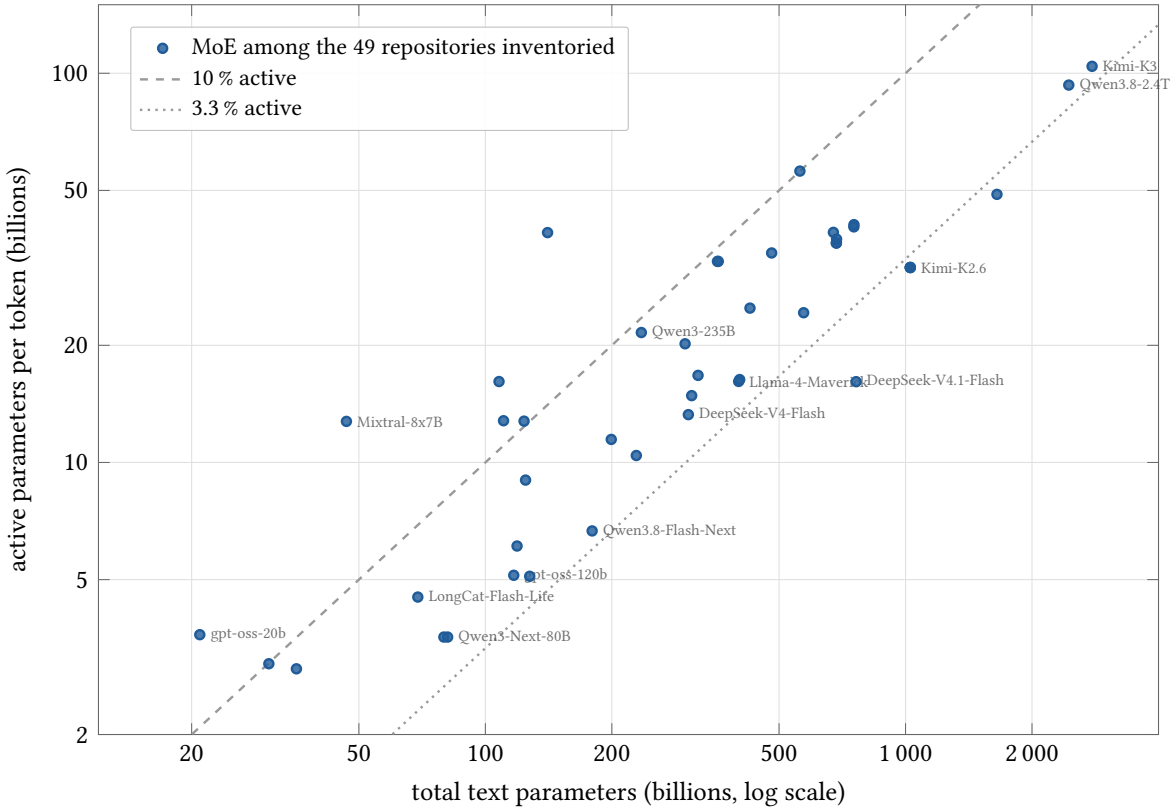


Figure 9.1: Total (text) parameters against active parameters per token of the mixtures of experts inventoried from their headers [meas.]. Most 2025–2026 models activate only 3 to 10 % of their parameters per token: the property that makes an "ultra-heavy" model playable on 8 GB.

Data: `fondations/journaux/06-modeles/octets-par-jeton.json` (`outils/inventaire_modeles.py`, pass 3).

9.3 Bytes read per token

With A the always-read parameters, R the routed expert parameters read per token and b the bits per weight of the experts, two formulas:

$$O_{\text{uniform}} = (A + R)\,\frac{b}{8}, \qquad O_{\text{mixed}} = A\,\frac{8.5}{8} + R\,\frac{b}{8}, \tag{9.1}$$

Table 9.1: A selection of the inventoried mixtures of experts: text total, active per token, architecture, licence [meas.] (literature where stated). L layers, E routed experts, k_e active, P shared experts.

Model	total	active	architecture, published format	licence
gpt-oss-20b [122]	20.9 G	3.61 G	L 24, E 32, k_e 4; MXFP4	Apache-2.0
gpt-oss-120b [122]	116.8 G	5.13 G	L 36, E 128, k_e 4; GQA 64/8, sliding window every other layer; native MXFP4 experts	Apache-2.0
Qwen3-Next-80B-A3B	81.3 G	3.56 G	L 48, E 512, k_e 10 + P; Gated DeltaNet 3:1	Apache-2.0
Qwen3.5-122B-A10B	124.6 G	9.01 G	L 48, E 256, k_e 8 + P, MTP	Apache-2.0
Qwen3.8-Flash-Next	179.6 G	6.67 G	L 48, E 512, k_e 10 + P; 51.2 G n-gram table	qwen-community
DeepSeek-V4-Flash-0731 [121]	284.3 G	13.27 G	L 43, E 256, k_e 6 + 1; 3 hash-routed layers; FP4 experts	MIT
DeepSeek-V4.1-Flash	762.7 G	16.13 G	E 384, k_e 6 + 1; 196.6 G Engram [123]	MIT
Kimi-K2.6 [124]	1 026 G	31.69 G	L 61, E 384, k_e 8 + 1, MLA; native INT4	modified MIT
Kimi-K3	2 780 G	104.19 G	L 93, E 896, k_e 16 + 2; native MXFP4	kimi-k3
GLM-4.5-Air [125]	110.5 G	12.80 G	L 46, E 128, k_e 8 + 1, MTP	MIT
Qwen3-235B-A22B [126]	235.1 G	21.57 G	L 94, E 128, k_e 8	Apache-2.0
LongCat-Flash-Lite	69.1 G	4.51 G	L 14, E 256, k_e 12; 31.4 G n-grams	MIT

the second being the real choice of dynamic quantizations (always-read tensors in Q8_0). Embedding, lookup tables and MTP are excluded. Second path: the headers of the exact files (Table 9.2).

Table 9.2: Bytes read per token, GGUF headers of the exact files [meas.] against the mixed formula [calc.].

File	always read + experts read (exact)	mixed formula	gap
gpt-oss-120b MXFP4	1.69 + 1.91 = 3.60 GB	3.55 GB	−1.4 %
DeepSeek-V4-Flash UD-Q2_K_XL	6.69 + 2.10 = 8.79 GB	8.83 GB	+0.5 %
Qwen3.8-Flash-Next UD-Q2_K_XL	3.54 + 0.90 = 4.44 GB	5.17 GB	+16 %
Kimi-K2.6 UD-Q2_K_XL	8.52 + 6.90 = 15.42 GB	16.49 GB	+7 %

The formula is right where the always-read tensors really are at 8 bits; it overestimates Qwen3.8 (part of its always-read tensors are at 4 to 6 bits) and errs in the other direction on Kimi. **To decide, we take the file header, never the formula.** Two readings of the published sizes: gpt-oss-120b weighs about 63 GB at every quantization level, because its experts stay in MXFP4 and only the attention changes type; and for models born at 4 bits (Kimi-K2.6, DeepSeek-V4-Flash), the “8-bit” level weighs almost as much as the “4-bit”: there is nothing above native 4-bit to keep.

9.4 The physical judge

The study's cost model (`modeles_physique.py`, declared before any use): always-read tensors reside on the card if they fit in 6.5 GB; routed experts share a cache of 11 GB of RAM plus the remaining VRAM, and the disk; routing is assumed uniform and memoryless, so the hit rate is $h = C/E_{\text{bytes}}$, the cache's worst case; experts found in RAM are computed by the processor at 41.6 GB/s, misses are read from disk;

$$t_{\text{opt}} = \max\Big(\frac{A}{256}, \frac{hR}{41.6}, \frac{(1-h)R}{W_{\text{disk}}}\Big), \qquad t_{\text{pess}} = \text{the sum.} \tag{9.2}$$

The study took the link ceiling (7.877 GB/s) for W_{disk}. Chapter 3 measured 2.82 GB/s: we recompute the column that depends on it.

Table 9.3: The candidate files and their tokens/s ceiling, uniform routing, at the link ceiling (study 06) and at the measured disk throughput [calc.].

File	size	always read	experts read	h	ceiling, 7.877	ceiling, 2.82
gpt-oss-20b MXFP4	12.1 GB	—	1.27 GB	100 %	32.7	32.7
gpt-oss-120b MXFP4	63.4 GB	1.69 GB	1.91 GB	26 %	~~5.6~~	**2.0**
Qwen3.8-Flash-Next UD-Q2_K_XL	78.9 GB	3.54 GB	0.90 GB	30 %	~~12.6~~	**4.5**
Qwen3.8-Flash-Next UD-Q4_K_XL	111.3 GB	4.83 GB	1.50 GB	16 %	~~6.3~~	**2.2**
DeepSeek-V4-Flash UD-Q2_K_XL	96.8 GB	6.69 GB	2.10 GB	12 %	~~4.3~~	**1.5**
DeepSeek-V4-Flash UD-Q4_K_XL	155.1 GB	—	3.45 GB	7 %	~~2.5~~	**0.9**
Kimi-K2.6 UD-Q2_K_XL	340.5 GB	8.52 GB	6.90 GB	3 %	~~1.2~~	**0.4**

Data: `fondations/journaux/06-modeles/plafonds-fichiers.md`, recomputed by Equation (9.2) with $W_{\text{disk}} = 2.82$ GB/s.

The crossed-out ceilings assumed the disk at its link ceiling; at the measured throughput, they are 2.8 times lower, because the disk is the bound wherever the cache does not serve everything (Table 9.3). The ranking does not change: **the best useful-to-physical trade-off is Qwen3.8-Flash-Next UD-Q2_K_XL, then gpt-oss-120b**, which has the decisive advantage of a lossless reference (its published weights are already at 4 bits). DeepSeek-V4-Flash overflows the card with its always-read tensors (6.69 GB in UD-Q2_K_XL); Kimi-K2.6 fits on disk (340 GB of 404) but tops out below half a token per second: it is the "one trillion parameters on a laptop" demonstration, not a tool. The dominant lever is the expert-cache hit rate (Figure 7.4).

9.5 What changed in 2026 and serves troncons

1. **Lookup tables, made to live outside the card.** The Engram paper [123] sets the principle: rows addressed by a deterministic hash of token n-grams, hence known before computation and prefetchable; it measures a 100 G table in host DRAM and proposes, without measuring it, an NVMe tier. Three open models carry such a table (26 to 45 % of their parameters): DeepSeek-V4.1-Flash, Qwen3.8-Flash-Next and LongCat-Flash-Lite. A Qwen3.8 token reads 16 rows there, about 1.4 KiB of useful data in a 28.8 GB file [calc.]: less than a thousandth of a token's disk budget. llama.cpp can already leave such a table on disk (`TENSOR_READ_LAZY`, prefetching only the requested rows).
2. **Weights born at 4 bits**: gpt-oss (MXFP4), Kimi K2-Thinking and K2.6 (INT4 through quantization-aware training), Kimi K3 (MXFP4), DeepSeek V4 and MiMo V2.6 (FP4 experts). On these models,

the only lossless compression left is entropy coding of the 4-bit codes and their scales; the E8M0 scales of V4-Flash weigh 9.3 GB for 148.2 GB of FP4 codes (6.3 %) [meas.].

3. **Hash routing**: DeepSeek V4 routes its first 3 layers by a token → experts table; those experts (about 7 % of the expert bytes) are known as soon as the token is sampled, a full pass ahead. Nobody uses it to prefetch.
4. **Built-in multi-token prediction** (DeepSeek, GLM, Qwen, MiMo): a free draft, under the condition of Proposition 8.7.
5. **Hybrid linear attention** (Gated DeltaNet, KDA): the KV cache stops being a constraint at short context on 8 GB; the cost is in the experts.

9.6 The judging protocol

The baseline. llama.cpp b11515 as is, loaded with mmap (the only mode that starts for a file larger than RAM), all layers on the card and the experts in RAM or mmap (`--cpu-moe`, or `--n-cpu-moe` N tuned for the best); both points are published.

A run. `modeles-base.py` runs a completion on a fixed real text (the first 3 000 bytes of wikitext-2 test, SHA-256 recorded), 256 tokens, greedy, and records tokens per second, process memory (peak and last reading, every 100 ms), VRAM, bytes read from disk (system counter read through the PDH API under its English name, integrated, after 10 s of rest to quantify the others' load) and the hash of the produced text. Why not `llama-bench`: its generation starts from meaningless tokens, whereas for an out-of-memory MoE the choice of experts, hence the bytes read, depends on the content.

What the controls have already taught. The starting assumption, ~~the correctness control of a lossless engine is equality of greedy tokens with llama.cpp~~, was refuted by measurement: llama.cpp itself, on the same file, produces a different greedy text depending on the compute engine (divergence near the 25th token between card and processor). Three levels, measured on Qwen3.5-4B:

- **instrument floor** (same engine, the card against its own logits): mean KL divergence zero, maximum $6.8 \cdot 10^{-5}$, same top choice 100 %;
- **gap between two correct engines** (processor alone against card, same weights): mean KL 0.0055, 99.9th percentile 0.187, same top choice 95.96 %: the order of magnitude of one quantization step;
- **the instrument fails when it should**: Qwen3.5-9B against the 4B baseline, mean KL 0.294, same top choice 76.9 %.

Resulting rule: **a lossless engine is judged against a baseline on the same compute stage**, and must fall back within the instrument floor there (KL at most $7 \cdot 10^{-5}$, top choice 100 %). That is why gpt-oss comes first: its MXFP4 is the original weight, so any divergence above the same-stage floor is a troncons error, not a quantization effect.

Quality and decision. Wikitext-2 perplexity (40 windows of 512), KL divergence against reference logits, HellaSwag, Winogrande, MMLU and ARC, identical for the baseline and for troncons; validated on Qwen3.5-4B (perplexity 8.7259 ± 0.2295, deterministic; HellaSwag 72.50 %; Winogrande 67.50 ± 2.34). ABBA series repeated at least twice; a difference is real only if it exceeds twice the largest noise and 3 %. For a lossy variant, no arbitrary threshold: we plot mean KL against bytes read per token for the standard quantizations of the same model, and the variant wins if it falls below that frontier. Physical efficiency (measured tokens/s over computed ceiling) is published with every result.

9.7 The downloads

With Stone's agreement ("au nom de la science tu as mon accord", "in the name of science you have my agreement", 2026-10-15), the orchestrator chose the first two files recommended by the study, and they are on the machine, SHA-256 hashes verified against Hugging Face:

- **gpt-oss-20b MXFP4** [127]: 12 109 566 624 bytes, Apache-2.0; same architecture and format as the 120b, but fits in RAM: the engine is developed on it and the in-memory and streamed versions are compared token for token;
- **gpt-oss-120b MXFP4** [128]: 63 387 346 208 bytes (45 resumptions after server cut-offs), Apache-2.0; 3.6 times the usable memory, 5.1 G active, original weights: **the first ultra-heavy model**.

Next, in the recommended order: Qwen3.8-Flash-Next UD-Q4_K_XL (111 GB, n-gram table) and DeepSeek-V4-Flash UD-Q2_K_XL (97 GB, hash routing, FP4-born experts); Kimi-K2.6 only after a proof on the previous ones.

Licences. Apache-2.0 and MIT restrict nothing for personal research in Canada; the modified licences (Kimi, Qwen3.8, GLM-5.3, Llama) only impose attribution or a separate agreement beyond revenue or user thresholds; none excludes Canada (a non-lawyer's reading, not legal advice).

9.8 What is missing in the world

1. **No published measurement of a lookup table served from a consumer NVMe**, although three open models put 26 to 45 % of their parameters in one.
2. **No token-by-token routing locality data on real text for 2026 models** (512 experts in Qwen3.5 and 3.8, 896 in Kimi K3).
3. **DeepSeek V4's hash routing is exploited by nobody to prefetch.**
4. **DeepSeek V4.1-Flash has no local baseline** (no architecture in llama.cpp as of 2026-10-15).
5. **No standard "out-of-memory" benchmark** that publishes bytes read per token and physical efficiency.
6. **Lossless compression of 4-bit-born models** is the only lossless path left on gpt-oss, Kimi, DeepSeek V4 and MiMo.

What could be wrong

- **The two assumed capacities** (6.5 GB of VRAM, 11 GB of RAM for weights) govern h and the ceilings; they are replaced by measurements at the first trial.
- **Compute is not free**: the processor may not dequantize at 41.6 GB/s, and the measured RAM delivers only 15.6 GB/s (Section 3.4).
- **Uniform routing** is pessimistic by construction; it is not a forecast.
- **The definition of active parameters** includes the output head and excludes the embedding and MTP; other counts give ±0.5 to 1 G.
- **Mirrors** (Llama 4 read on copies, Llama 3.1 405B on Hermes-3) and publishers' cards can be wrong.
- **The judge's validation covers a single model**, a hybrid; on gpt-oss (classical attention), the gap between engines may differ.
- **Licences**: a non-lawyer's summary.

Part III

Towards the system

Chapter 10

Architecture: the candidates, the decision, what comes next

Where we start, what this chapter establishes, what remains

Where we start: the token equation (Chapter 4) and four measured levers. No architecture had been chosen. **This chapter establishes** three candidate architectures, opposed by their angle and computed on the same throughputs (`fondations/07-ce-qui-manque.md`, § 4); the new mechanism of the second one, the exact refinement of a native 4-bit code, with its proven proposition and its measurement on the real weights; the correction of a number that wrongly favoured it, struck out in place; the debate of the two advocates and the arbiter's decision; and the first, still provisional, measurements of the target model. **What remains** are the implementation chapter (milestone J1) and the results chapter, which can only be written with a running program. Sources: `fondations/07-ce-qui-manque.md`, `decisions/partisan-A.md`, `decisions/partisan-B.md`, `decisions/ARCHITECTURE.md` (commit `f0b0fea`), `mesures/` (measurement agent, study not yet delivered when this chapter is written).

10.1 Three angles

The three candidates share the machine, the judge (Section 9.6) and the token equation; all make the GPU decode (corollary 3.6). They differ by their angle:

A reduces the bytes *without losing anything* and relies on the GPU alone;
B trades precision, *only where a byte is expensive*: at a cache miss;
C *reads less often* instead of reading less: one read of the weights amortized over several tokens by tree verification.

Figure 10.1 puts them on one axis, together with the estimated llama.cpp baseline.

10.2 A: everything through the GPU, bytes coded end to end, lossless

The model. gpt-oss-120b MXFP4, already on the machine, checksum verified; development twin: gpt-oss-20b (12.1 GB, also larger than the VRAM). No download.

The byte path (Figure 10.2). The disk delivers experts coded by the rANS coder of the entropy study ($r = 1.1026$), at the size of one expert matrix: in the gpt-oss files the expert is the slowest dimension of each tensor, so every expert matrix is a contiguous block of 4,406,400 bytes, readable in one piece without rewriting the file [meas.] (`fondations/journaux/07-synthese/inventaire-gptoss.json`). Pinned RAM holds the arrival buffers and a warm cache of *coded* experts; DMA

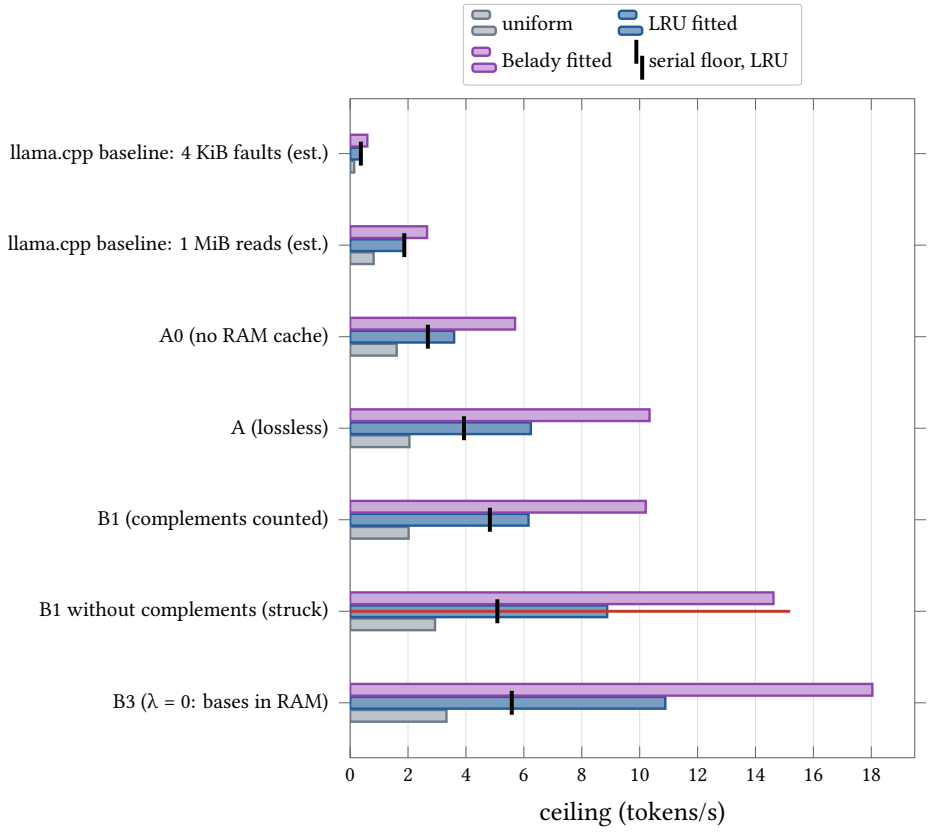


Figure 10.1: Ceilings of the candidate architectures on gpt-oss-120b, three locality scenarios (assumption 4.4); the black tick is the serial floor of the LRU scenario [calc.]. The row struck in red is study 07's number for B1, which did not count the complements (Section 10.3.3). The llama.cpp baseline is estimated, not measured.

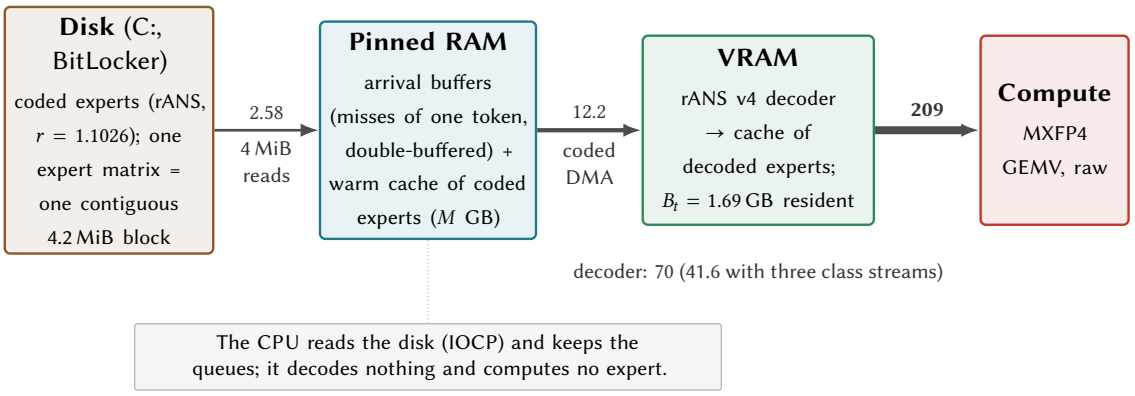
Data: `fondations/journaux/07-synthese/csv/architectures-gptoss120b.csv` and `decisions/journaux/csv/b-complements-gptoss120b.csv`, extracted by `memoire/outils/donnees_figures.py`.

carries coded bytes; the GPU decodes on arrival and keeps a cache of *decoded* experts, because the product fused with the decoder is 3.7 times slower than the raw product (Chapter 4, Section 4.10).

The timeline of a token (Figure 10.3). During layer ℓ, the disk queue receives the misses predicted for $\ell + 1$ by the router of $\ell + 1$ applied to the residual of ℓ (96 % right one layer ahead [lit.], Chapter 7). The disk is busy all the time and the GPU waits for it: about 1 ms of GPU per layer against 4.4 ms of disk on average. The ceiling assumes the queue never runs dry; the floor, that no read is issued before the router of its layer has spoken.

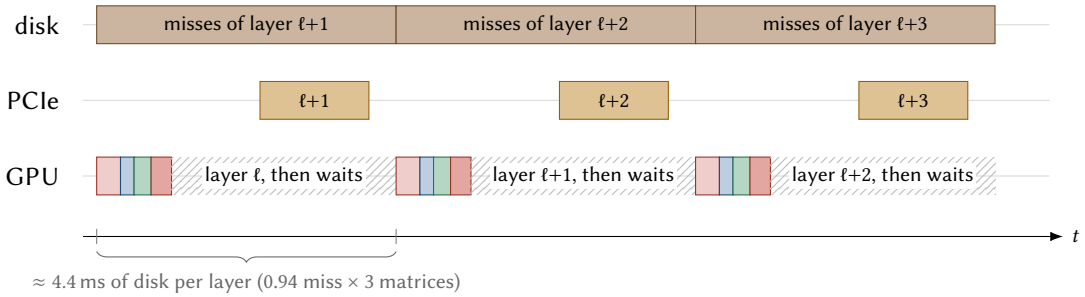
Peak memory. VRAM ≈ 7.5 GB: the always-read tensors (1.69 GB), a 4.8 GB expert cache, the KV cache, activations and decoding buffers (≈ 1 GB). The display goes through the integrated GPU: all 8,188 MiB of the card are available. RAM: from 1.9 to 12.3 GB depending on the warm cache, and 0.64 GB in the minimal window of the advocate of A (Section 4.8).

Quality. Exact by construction, with a double witness: each decoded chunk is compared with its checksum (the rANS decoder is bit-exact on the 108 tensors of gpt-oss-120b [meas.]); and the logits of



Measured throughputs on the machine, GB/s (on mains power). The disk holds the maximum of the token equation (Proposition 4.6).

Figure 10.2: The byte path of candidate architecture A. The CPU only reads and keeps the queues; the GPU decodes and computes. Measured throughputs (Table 4.1).



GPU blocks: attention, router (blue), decoding of the misses (green), expert GEMV. About 1 ms of GPU per layer. The disk queue receives, during layer ℓ, the misses predicted for ℓ+1 by the router of ℓ+1 applied to the residual of ℓ (96 % right one layer ahead [lit.]).

Figure 10.3: Timeline of a token in candidate architecture A: three successive layers. The disk never stops; the GPU computes about a quarter of the time and waits the rest. This is the regime of Proposition 4.6.

A must be *bit-identical* to those of the same engine reading uncoded experts. Against llama.cpp, the comparison is to the measured gap between two correct engines (Section 9.6).

What it gives up. The CPU stays nearly idle (computing warm experts on it would only help the link, which does not hold the maximum: 58 ms against 157); the whole forward pass of gpt-oss has to be written (attention with sinks and a sliding window every other layer, YaRN RoPE, router, clamped SwiGLU); speed remains bounded by locality.

10.3 B: precision by stage

10.3.1 The mechanism

An E2M1 code (the nibble of MXFP4) carries a sign s and a magnitude $m \in \{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$, multiplied by the E8M0 scale of its block of 32 (Chapter 5). We split it into two rANS-coded streams (Figure 10.4):

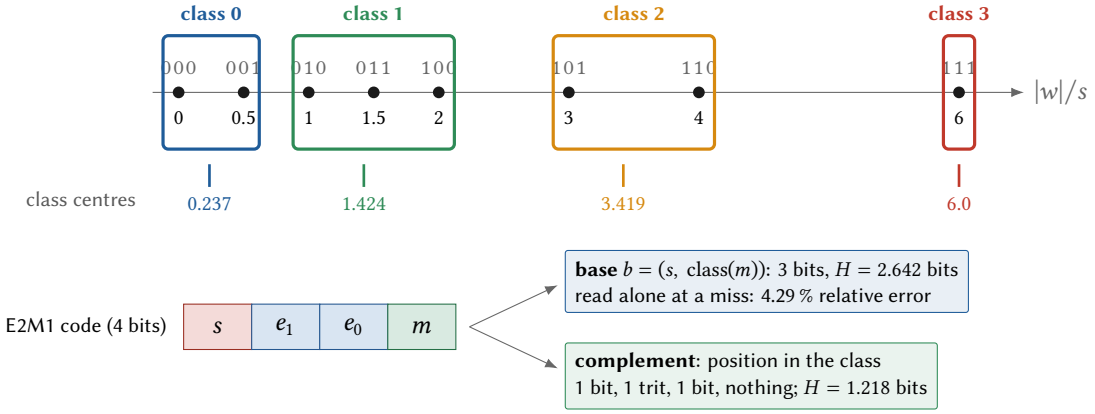
- the **base** $b = (s, \text{class}(m))$, with four magnitude classes;
- the **complement**, the position of m within its class, coded with one table per class.

Table 10.1: Ceilings of candidate architecture A on gpt-oss-120b, disk at 2.58 GB/s, 11 GB warm cache [calc.]. The same path without coding ($r = 1$) gives 1.81 / 5.42 / 8.98.

Scenario	h_v / h_r / h_s	T_D / T_P / T_G (ms)	Ceiling	Serial floor
uniform ($\beta = 1$)	0.079 / 0.199 / 0.723	485 / 131 / 59	2.05	1.47
fitted LRU ($\beta = 0.207$)	0.590 / 0.176 / 0.234	157 / 58 / 36	**6.24**	3.93
fitted Belady ($\beta = 0.117$)	0.744 / 0.118 / 0.139	93 / 36 / 29	10.34	6.17

Data: `fondations/journaux/07-synthese/csv/architectures-gptoss120b.csv`, rows `A_sans_perte` and `A_brut_r1`.

At a cache miss, the disk reads only the base and the GPU serves it, each class replaced by its centre; when the expert enters the cache, reading the complement makes it exact.



Since the base is a function of the code, $H(\text{code}) = H(\text{base}) + H(\text{complement}) = 3.860$ bits (Proposition 10.1): splitting costs nothing in entropy. Centres are weighted by the square of the E8M0 scale, hence optimal in the real domain.

Figure 10.4: Base and complement of an E2M1 code, best four-class partition [meas.]. Top: the eight magnitudes to scale and their 3-bit codes; below them, the class centres (E2M1 units). Bottom: the split of a 4-bit code.

Data: `fondations/journaux/07-synthese/raffinement-mxfp4-gptoss20b.json` (tool `outils/07-synthese/raffinement_mxfp4.py`).

Proposition 10.1 (Exact refinement costs nothing in entropy)**.** *Let X be a discrete random variable (the code) and $B = f(X)$ a deterministic function of X (the base). Then*

$$H(X) \;=\; H(B) + H(X \mid B). \tag{10.1}$$

An ideal entropy coder that codes B, then X given B, therefore writes as many bits as if it coded X in one piece.

Proof. By the chain rule, $H(X, B) = H(X) + H(B \mid X) = H(B) + H(X \mid B)$. Since B is a function of X, knowing X fixes B: $H(B \mid X) = 0$. Hence $H(X) = H(B) + H(X \mid B)$. The optimal code of X given B reaches $H(X \mid B)$ on average, that of B reaches $H(B)$, and the sum is $H(X)$. ∎

This is the trivial case, for a discrete source and a lossless top layer, of the successive refinement of Equitz and Cover [129]; the continuous result is proven by Drop-by-Drop [130] [lit.]. What is new here is not the identity: it is its use on a *native* 4-bit format (the model was trained in it, so the exact

layer is the original itself), tied to the stage, chosen for its distortion rather than by truncation, and its measurement. The neighbours read in full by the advocate of B confirm it: SliceMoE truncates the low-order bits of an integer [131], HOBBIT keeps two separate copies [132] [lit.].

10.3.2 The measurement on the real weights

Table 10.2: Base layers of an E2M1 code, 12 tensors of gpt-oss-20b (layers 0, 8, 16, 23; gate, up, down; 3,185,049,600 weights) [meas.]. H(code) = 3.860 bits; base plus complement equals it within 10^{-9} on every row (witness of Proposition 10.1).

Base	Magnitude classes	H(base)	H(compl.)	Relative error
sign + 2 bits, best partition	{0, 0.5} {1, 1.5, 2} {3, 4} {6}	**2.642**	1.218	**4.29 %**
sign + 2 bits, truncation ("Matryoshka")	{0, 0.5} {1, 1.5} {2, 3} {4, 6}	2.907	0.953	4.47 %
sign + 1 bit	{0 … 1.5} {2 … 6}	1.957	1.903	16.4 %
reference: MXFP4 on a Gaussian				*1.33 %*

Data: `fondations/journaux/07-synthese/raffinement-mxfp4-gptoss20b.json`; break: `raffinement-mxfp4-gptoss20b-casse.json`.

Three readings of Table 10.2. **Truncation loses 0.27 bit per weight at equal error**: the levels of the format are not uniform, and cutting low-order bits groups the magnitudes badly. **The 1-bit base is killed by the number**: 16.4 % error, twelve times the format's own noise. **The class centres** are centroids weighted by the square of the E8M0 scale, hence optimal in the real domain: 0.237, 1.424, 3.419, 6.0; the decoder keeps them in four registers, and the product on the base costs as much as MXFP4. With scales and headers, the base reads at 2.684 bits per weight against 3.855 for lossless: 3.855/2.684 = 1.436 times fewer bytes per miss [calc.].

10.3.3 The number that had to be struck out

Study 07 defined two variants: **B1** (VRAM and RAM keep exact experts; a miss reads the base alone and serves it; the complement is read only if the expert enters the cache) and **B2** (RAM keeps only bases). It put B1 at ~~8.88 tokens/s, i.e. ×1.42 over the ceiling of A in the fitted-LRU scenario~~.

The advocate of B found the error, against his own side: the tool did not count the bytes of the complements. Yet under LRU every miss enters the cache, so every miss eventually reads its complement. Counted, **the ceiling of B1 falls back to that of A**: 6.16 tokens/s against 6.24, because the disk reads base and complement, 3.906 bits against 3.855; the gain of B1 only lives in the serial floor, 3.93 → **4.83** tokens/s (+23 %), because the complement leaves the critical path [calc.] (`decisions/partisan-B.md`, § 2). The arbiter redid it by an independent path, importing no one else's tool: the struck number reproduces (8.879, witness W4), the corrected one too (6.161 and 4.827, witness W5), and the break that zeroes the complements makes W5 alone fall [calc.] (`decisions/journaux/arbitre/verif-plafonds.json`, `verif-plafonds-casse.json`). The ×1.42 is struck out in study 07 itself.

10.3.4 Where B really gains: B3 and the λ knob

The real throughput gain of B is **B3**: RAM keeps bases (1.44 times more warm experts), VRAM an exact hot set, and a share λ of the out-of-VRAM uses demands exact weights (the complement is then read). At $\lambda = 0$, the ceiling is 10.89 tokens/s (×1.74 over A, fitted LRU), at the cost of 48.5 % of

expert uses served by the base; it meets A near $\lambda \approx 0.67$ (Figure 10.5). The knob λ is the miss policy by router weight: serve the base to experts that weigh little, exact weights to the others.

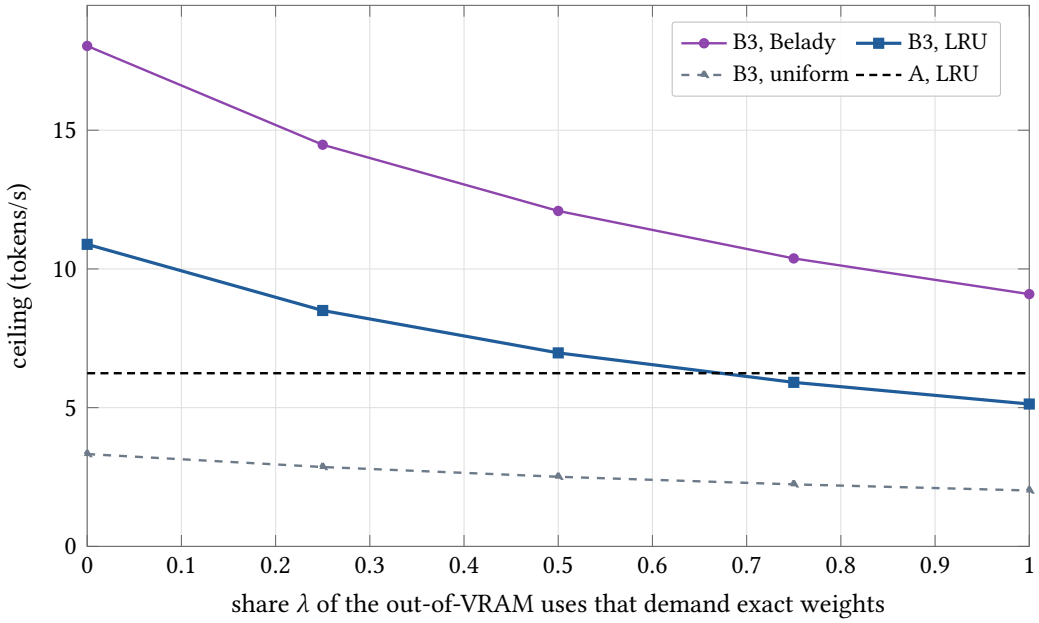


Figure 10.5: Architecture B3 on gpt-oss-120b: ceiling against the share λ of the out-of-VRAM uses that demand exact weights [calc.]. At $\lambda = 0$, ×1.74 over A (fitted LRU); the LRU curve crosses A near $\lambda \approx 0.67$.

Data: `decisions/journaux/csv/b3-lambda-gptoss120b.csv` (tool `outils/decisions/partisan_b.py`).

Quality, the unknown that decides. Through HIGGS linearity (small errors, loss ≈ weighted sum of relative squared errors [lit.]), the advocate of B estimates the KL divergence of B3 without a policy between 0.05 and 0.12, i.e. 9 to 22 times the budget $\kappa = 0.0055$ of Definition 4.5; with a policy that serves the base only to experts carrying a tenth of the routing energy, B1 would land between 0.002 and 0.006 *(assumption)*. These are not measurements. They can be made without an engine: a base "snapped" onto existing E2M1 values (0, 1.5, 3, 6) leaves a valid MXFP4 file that llama.cpp reads as is, at 5.91 % error instead of 4.29 % (bench Q0).

10.4 C: reading less often

A dense model reads all its weights at every pass, but a verification pass of n tokens costs almost the same time as long as $n < n^\star = F\,b/(2\,W_{\text{eff}})$ (Proposition 8.1). For Llama-3.3-70B IQ3_XXS (27,469,499,360 bytes, Llama 3.3 Community licence), with $F \approx 8$ Tflop/s usable (assumption: 60 % of the 13.4 measured in f16), $b = 26.9/70.6 = 0.381$ byte per parameter and $W_{\text{eff}} = 26.9\,\text{GB}/3.79\,\text{s} = 7.10\,\text{GB/s}$:

$$n^\star = \frac{8 \cdot 10^{12} \times 0.381}{2 \times 7.10 \cdot 10^{9}} \approx 215 \quad \text{[calc.]}.$$

A tree of 64 to 128 nodes is free in compute. Without speculation, 0.26 token/s; with SpecExec's tree (9.7 tokens per pass, measured on a desktop RTX 4060 with a 70B GPTQ [lit.]), **2.50** tokens/s (serial floor 1.68) [calc.]. That is below A in the LRU scenario, for 28.3 GB of download, a tree engine to write, and a Llama 3 that quantizes badly (+76 % perplexity at 2.07 bits [lit.], Chapter 5).

10.5 The debate and the decision

The project's decision cycle sets two advocates who do not read each other against each other, then an arbiter who settles with measurements of his own.

The advocate of A (`decisions/partisan-A.md`): A is not an alternative to B, it is its *floor* (B serving all its misses exact *is* A) and its *judge* (B's quality budget is measured against A; without a bit-exact A, an ordinary engine error eats the whole budget). He concedes: an A alone would make a good engineering report, not a research paper.
The advocate of B (`decisions/partisan-B.md`): B is the only mechanism that does not exist yet, and that is what Stone asks for; he corrects B1's number against his own side, and proposes to measure B's quality first, without an engine.

The arbiter's decision (`decisions/ARCHITECTURE.md`; answer by delegation of 2026-10-15, reversible). We build **troncons**, an engine of our own, in Rust with CUDA kernels launched through the driver API.

Table 10.3: The milestones decided by the arbiter.

Milestone	Content	Entry gate
J1	exact A on gpt-oss-120b: `.trc` format v1.0, IOCP reads at expert size, DMA from pinned memory, rANS decoding on the GPU, expert cache in VRAM, full forward pass; modes A0 and A	none
J1+	prefetching (router of $\ell + 1$ on the residual of ℓ), SLRU cache	J1
Q0	quality of B without an engine, by static files judged by llama.cpp, in parallel from day one	the GPU free for 2 to 4 h
J2	B in the engine: two-level format v1.1, B1 (latency), B3 (throughput, policy λ)	J1 holds and Q0 has not killed B
J3	unencrypted disk, speculation if ρ allows, Vulkan port	future decision

His reasons, in numbers: B first means A plus four modules, without a day gained on the first working program, and B1 no longer gains anything at the ceiling; A alone does not meet the demand for a contribution; C caps at 2.50 against 6.24. **CUDA rather than Vulkan for J1**, by the measurements: the rANS decoder is 14.9 % faster under CUDA (80.8 and 82.1 against 70.2 and 71.6 GB/s, ABBA [meas.]); copy–compute overlap (hidden share 0.98 to 1.01) and pinned memory (+21 %) were only measured with CUDA streams; wgpu offers neither host-memory import nor a second queue. The price, stated: the program requires an NVIDIA GPU; the measured WGSL kernels remain the second path.

The arbiter's measurement F1: what the split costs. The advocates disagreed on the format: a single stream in v1 (A), or the base + complement split as the only format (B). The arbiter measured it with the real rANS coder, expert by expert, each stream decoded and compared byte for byte, thresholds declared before the run (Figure 10.6). Results [meas.]: the table chosen *per lane* by the scale class of the block equals the three class streams of the entropy study (3.8431 against 3.8434 bits per weight, threshold S2 holds) with no reassembly; the split conditioned on the scale class costs +0.41 % (threshold S1 of 0.40 %, the header cost of one more stream, missed by 0.01 point); the rANS-coded complement is 0.970 times its raw packing (S3); the base error (4.2920 %) and that of the snapped base

(5.913 %) are recovered by this second path (S4). After the first pass, the arbiter declared a variant and its threshold S1bis (the same 0.40 %) before the second: the complement also conditioned on the class of the neighbouring nibble and, for the high nibble, on the already decoded low complement. It costs +0.31 %: **S1bis holds**. Hence the rule: v1.0 of J1 takes the per-lane table (a single level, the simplest kernel); **the two-level v1.1 becomes the only format of J2**, where A pays 0.31 % more bytes, below the read noise. With the bits measured by F1, the completed B1 rises to 6.22 tokens/s (serial 4.87) against 6.24 for A, and B3 to 11.10 (×1.78) [calc.] (`decisions/journaux/arbitre/verif-plafonds.json`, section `avec_F1`). The arbiter sums it up: the advocate of A was right about the order, the advocate of B about the final format.



Figure 10.6: The arbiter's measurement F1: bits per weight written by each format variant, 12 tensors of gpt-oss-20b, real rANS coder, exact decoding [meas.]. The total is printed at the end of each bar; the base + complement split costs +0.41 %, +0.31 % with the rich context kept for J2.

Data: `decisions/journaux/arbitre/decoupe-gptoss20b.json` (pass 2; tool `outils/arbitre/decoupe/`).

The `.trc` format (Figure 10.7). One self-contained file per model, little-endian, aligned on 4,096 bytes (what unbuffered reads on Windows require); **one chunk per expert**, in one piece, so that a miss costs one request; streams ordered by **refinement level**, any prefix that stops at the end of a level decoding on its own. In J1, one level; the stream kinds of B are reserved now, so that J2 changes neither the reader, nor the cache, nor the scheduler. The name comes from the project: a *tronçon* (chunk) is the part of a model that one reads, decodes, serves and forgets, Stone's "chunks like in minecraft".

The success criteria of J1 , declared before any construction and derived from named measurements: logits of the coded path bit-identical to those of the raw path of the same engine; an engine at least as close to the exact mathematics of the model (double-precision reference) as llama.cpp; peak RAM $\leq$ 1.9 GB in mode A0, equal within 0.5 MB between gpt-oss-20b and gpt-oss-120b (the witness of Definition 1.1); VRAM under 8,188 − 909 MiB; tokens/s at least 0.8 times the serial floor recomputed with the hit rate counted by the engine.

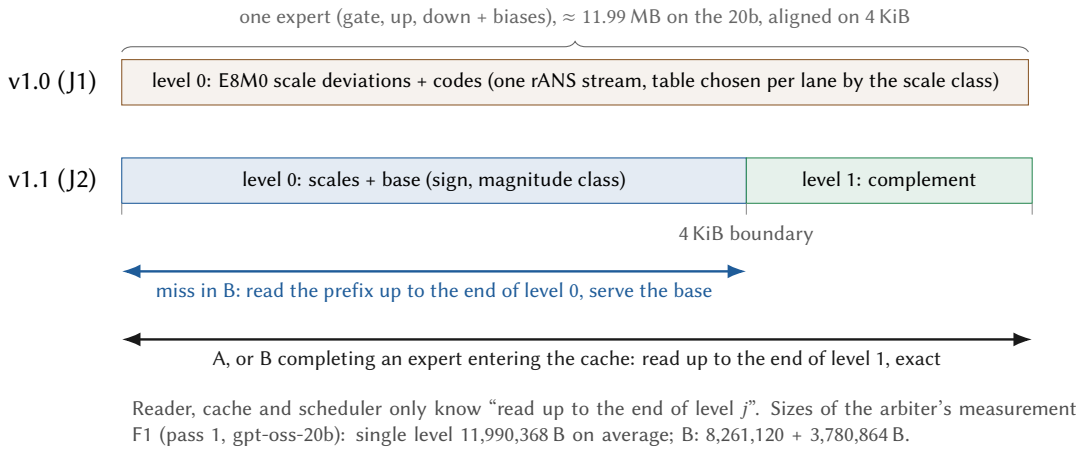


Figure 10.7: A chunk of the `.trc` format: one expert in one piece, its refinement levels aligned on 4 KiB. In v1.0 (J1), a single level; in v1.1 (J2), the base then the complement, and a miss in B reads only the prefix.

Data: `decisions/ARCHITECTURE.md` (§ 4, principles) and `decisions/journaux/arbitre/decoupe-gptoss20b-passe1.json` (sizes).

10.6 The first measurements of the target model

The measurement agent works on the GPU while this chapter is being written; its study is not delivered, but its series and traces are committed with their witnesses. Two figures are drawn from them, to be read as provisional.

The llama.cpp baseline on gpt-oss-20b (Figure 10.8). ABBA series of four passes, on mains power, llama.cpp b11515: experts on the CPU, 11.38 tokens/s in generation; placement fitted by llama.cpp's tool (ten expert layers out of 24 on the GPU), 18.92; expert cache of 4,096 MiB, 10.15, at 84.59 % hits according to llama.cpp's log [meas.]. The gap of the fitted placement over experts on the CPU is real (+66 %, above the noise of 0.30); that of the cache is within the noise (`mesures/base-gpt-oss/20b/resume.csv`). On gpt-oss-120b, a single-pass exploration, not retained by the agent itself, gives 0.49 token/s with experts on the CPU and 295 GB read in 12 minutes 46 seconds (`mesures/REPRISE-gpt-oss.md`): around the low end of the range estimated for the baseline (0.14 to 0.81 in the uniform scenario).

Locality on gpt-oss-20b (Figure 10.9). Real routing traces (four text genres, reading and generation), replayed by the instrument of the loading study; exact witness: a replica of llama.cpp's cache policy replayed on these traces gives back, hit for hit, the numbers of llama.cpp's log. On the generation trace:

- **The shape of assumption 4.4 does not hold, but its order of magnitude does.** At small budgets the measured hit rate is below the assumption (per-layer cache, 15 %: LRU 0.627 against $0.15^{0.2075} = 0.675$; Belady 0.686 against 0.801); at large ones, above (40 %: LRU 0.882 against 0.827; Belady 0.935 against 0.899) [calc.] on [meas.]. The real curve is steeper than a power law fitted at one point.
- **A global LRU cache below $L\,k$ slots returns zero** (96 slots for 24 layers with 4 active experts): each layer evicts the experts of the next before they are used. This cyclic pathology is avoided by Belady and LFU.

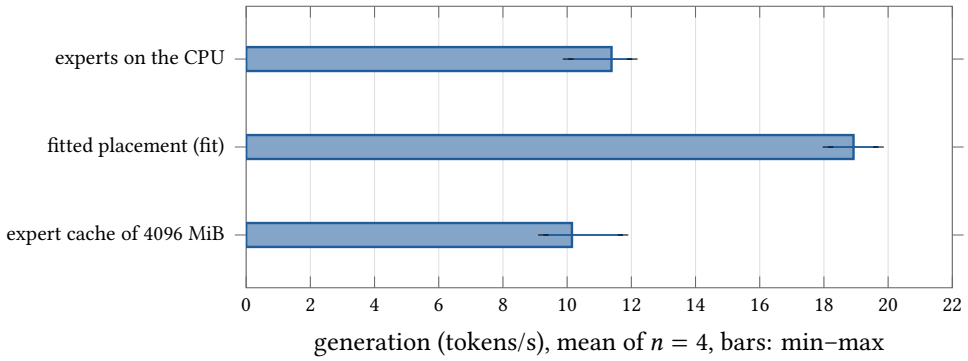


Figure 10.8: llama.cpp b11515 baseline on gpt-oss-20b, generation, three configurations, ABBA series of four passes on mains power [meas.]. Bars: minimum and maximum. Provisional measurement of the measurement agent.

Data: `mesures/base-gpt-oss/20b/resume.csv`, extracted by `memoire/outils/donnees_figures.py`.

- **Token-to-token reuse is** $\rho = 0.577$, against 0.125 under uniform chance and 0.223 under chance with the real popularity (`mesures/localite-gpt-oss/20b/localite-gpt-oss-20b.json`); the instrument measures it as the share of the experts of a token already chosen by the previous token, in the same layer, i.e. the ρ of Proposition 4.8. On gpt-oss-20b, speculation would thus pay from $\alpha > 0.42$, far from the 0.875 that independence imposes [calc.]. It remains to be measured on the 120b.

10.7 What comes next

The next chapters of this part will be written with the program, whose construction started while this one was being written: when this thesis is built, package K has committed the kernels of the per-lane-table decoder and of the MXFP4 GEMV with an independent bench, package E the reading of WDDM hardware scheduling, package R the judge's counters and their witnesses (`git` log, 2026-10-15). **Implementation**: the modules of troncons and their seams, the `.trc` format to the bit, the CUDA kernels and their witnesses, each work package closed by a break that falls. **Results**: tokens/s, peak RAM and VRAM, bytes read (system counter), physical efficiency (measured over ceiling) on gpt-oss-120b, against the llama.cpp baseline measured in ABBA; then, at milestone J2, the quality of B against A. Chapter 12 gives the order.

What could be wrong

- **All the ceilings of this chapter depend on assumption 4.4**, whose shape the first measurement on gpt-oss-20b contradicts; they must be recomputed with the measured curve of the 120b.
- **The decision is an answer by delegation, reversible**: it changes by a written, dated decision in `decisions/ARCHITECTURE.md`, and this chapter will follow.
- **The quality of B is only estimated**, by a linearity valid for small errors; 4.29 % on one expert is not small.
- **The base layer is measured on 12 tensors of gpt-oss-20b**, not on the 120b; the code entropies of the two models differ by 0.001 bit, which makes a gap unlikely, but it is not

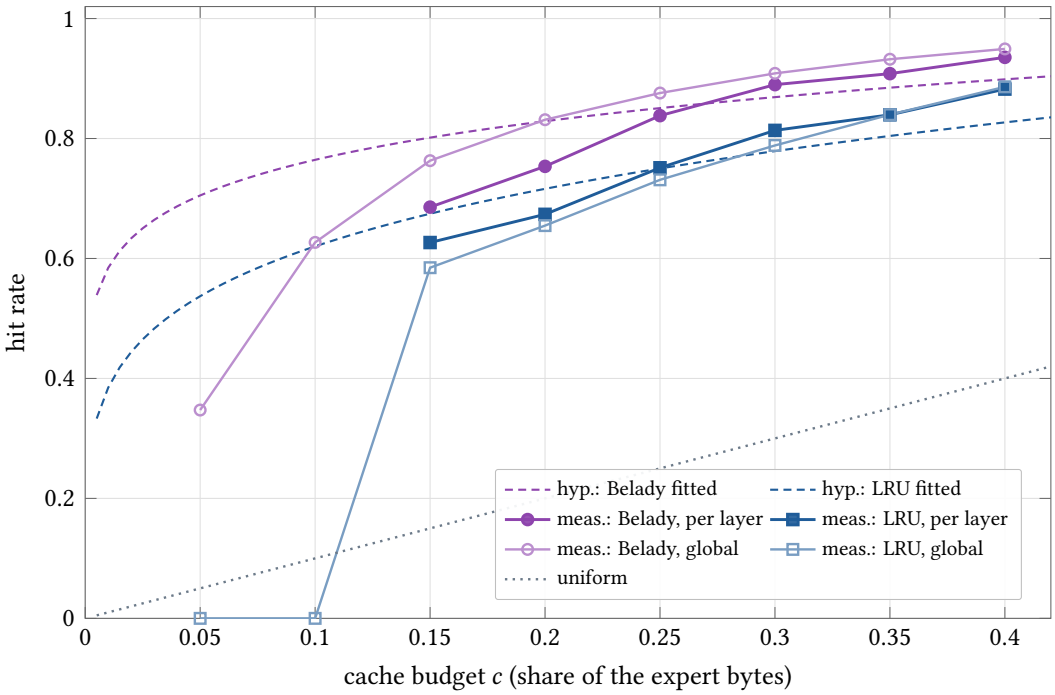


Figure 10.9: Expert-cache hit rate on the real routing traces of gpt-oss-20b (generation trace) [meas.], against the fitted scenarios of Chapter 4 (dashed). The per-layer cache has no point below one expert per layer; global LRU returns zero below $L\,k = 96$ slots. Provisional measurement of the measurement agent, on the 20b and not on the 120b.

Data: `mesures/localite-gpt-oss/20b/courbes-gpt-oss-20b.csv`, extracted by `memoire/outils/donnees_figures.py`.

measured.

- **The neighbours of B** (SliceMoE, TRACE, MatGPTQ, AnyBCQ) are not all read in full; one of them may do, in its body, part of what this chapter calls new.
- **The measurements of Section 10.6 are provisional**: the study that carries them is not delivered, and the machine was shared during the series.

Chapter 11

Discussion and limits

Where we start, what this chapter establishes, what remains

Where we start: nine chapters that each measure, prove and decide on their own. **This chapter establishes** what they say together; what is solid in them and what is fragile; the ideas the numbers killed; the numbers that had to be struck out, gathered in one place; and the limits of the method itself. **What remains** is to confront it with the program: each fragility named here has its planned measurement in Chapter 12.

11.1 What the first part establishes

1. **The judge is a maximum, and on this machine the disk holds it.** For gpt-oss-120b in the retained locality scenario, the disk takes 157 ms per token, the link 58, the GPU 36 (Section 4.7). Any lever that does not touch $h_s B_e/(r_d W_{\mathrm{D}})$ is, here, ornament (corollary 4.7).
2. **The disk delivers 47 % of its rating**, because the volume is encrypted by BitLocker and goes through Intel RST VMD (Chapter 3): the largest lever of the machine is not software.
3. **Every bit removed is worth as much as every byte of throughput gained** (elasticities ±0.98, Proposition 4.6); extreme quantization and lossless compression act on the same term, and the Shannon bound says what is left to take (Chapter 5).
4. **The cache hit rate decides everything for a mixture of experts** (Chapter 7): between uniform routing and a fitted LRU, the ceiling of A changes by a factor of 3.
5. **Speculation cannot be judged without expert reuse**: the condition $\alpha > 1 - \rho$ (Proposition 4.8) replaces the "never" of Chapter 8; on gpt-oss-20b, $\rho = 0.577$ (Section 10.6).
6. **Memory can stop depending on the model**: A0 serves a 63.4 GB model with 1.9 GB of peak RAM, and the minimal window goes down to 0.64 GB (Section 4.8).
7. **A new mechanism exists, narrower than announced**: the exact refinement of a native 4-bit code costs nothing in entropy (Proposition 10.1) and its base loses 0.27 bit less than truncation at equal error; but once the complements are counted, its throughput gain is only real at the price of a tunable share of precision (Section 10.3.3).

11.2 What is solid, what is fragile

Solid. The stage throughputs, each measured with a witness that can fall, in ABBA series, and redone by a second path (system counter for the disk, encryption API for BitLocker); the theorems and their proofs (theorems 3.2 and 8.3, Propositions 4.6 and 10.1); the exactness of the rANS decoder on the 108 tensors of gpt-oss-120b; the inventory of models read from their headers, redone on the local file.

Fragile. Everything that depends on locality: the ceilings of A and B rest on a power law fitted at one point (assumption 4.4) whose shape is already contradicted on gpt-oss-20b; the llama.cpp baseline of gpt-oss-120b, still estimated; the quality of B, only estimated by a linearity valid for small errors; several neighbouring works, read only in abstract.

11.3 The ideas killed by the numbers

The SOCLE asks to kill an idea by numbers, not by opinion. Table 11.1 gathers them.

Table 11.1: The ideas killed by the numbers, and the number that kills them.

Idea	The number that kills it	Where
Decoding inside the product (fused rANS + GEMV)	3.7 times slower than the raw product, no faster than two passes [meas.]	Chapter 6
"Multiplication-free" trellis on the GPU	52 against 121 GB/s: bit counts cost more than the multiplication [meas.]	Chapter 5
Decoding on the CPU	0.075 GB/s per thread [meas.]	study 03, § 4.4
Activation sparsity under verification	the union of 4 tokens touches 89.5 % of the weights; a shared mask (block of 4, 50 %) multiplies perplexity by 16.9 [meas.]	Table 8.5
Speculation on a fine-grained MoE under independence	would need $\alpha > 0.969$ for gpt-oss-120b [calc.]; qualified by ρ	Proposition 4.8
DirectX 12 through wgpu	neither f16 nor subgroups; 32 GB/s in 4 bits against 122 [meas.]	study 01, § 5.3
IoRing instead of IOCP	gap of 0 to 5 %, below the noise [meas.]	Chapter 3
Faster kernels in the disk regime	zero elasticity, under 2 % [calc.]	corollary 4.7
Computing warm experts on the CPU	the link (58 ms) does not hold the maximum (disk 157 ms) [calc.]	Section 10.2
A 1-bit-magnitude base for B	16.4 % error, twelve times the format's own noise [meas.]	Table 10.2
C (a 70B dense read from disk, with a tree)	2.50 tokens/s at best against 6.24 for A, and 28.3 GB to download [calc.]	Section 10.4

11.4 The struck-out numbers

When a measurement contradicted a published number, even ours, the sentence was struck out in place and what changed was stated. This thesis keeps the list (Table 11.2): it is the trace that the method worked.

11.5 The limits of the method

- **One machine.** Everything is measured on Stone's laptop; it is the target, but the conclusions about the limiting stage change with another disk (an unencrypted volume hands the lead to the link, Chapter 12).
- **A shared machine.** Other agents were measuring at the same time; "best" throughputs are lower bounds of the idle machine, and each pass logs the foreign load.

Table 11.2: The numbers struck out during the foundations, and what replaced them.

Struck out	Replaced by, and why	Where
~~Ceiling of gpt-oss-120b without cache: 5.5 tokens/s~~	≈ 2 tokens/s: 5.5 assumed the disk at its link ceiling, it delivers 36 % of it	Section 3.8
~~$r^\star$ is 1.05 bit stored for 4.5 served~~	1.17 bit: Q4_K_M serves 5.08 bits, not 4.5	Section 3.8
~~A resident matrix deserves 2.6 more bits~~	2.18 bits (link) and 3.24 bits (disk), with the measured throughputs	Chapter 5
~~Disk-cache witness: rereading from cache exceeds the link~~	copying from the Windows cache is bound by one thread (3.1 GB/s); replaced by the system counter	Chapter A
~~Correctness of a lossless engine: same tokens as llama.cpp~~	llama.cpp diverges from itself across backends; KL budget 0.0055	Section 9.6
~~Lossless rate on the cache: +14.4 %~~	+13.1 %: the GPU keeps its experts decoded	Section 4.10
~~B1: 8.88 tokens/s, ×1.42 over A~~	6.16 (serial 4.83): the complements were not counted	Section 10.3.3
~~B1 beats A as soon as 4.2 % of misses are served by the base~~	B1 never beats A at the ceiling, only in series	Section 10.3.3
~~Optimal $k = 0$ for a MoE read from disk~~	depends on ρ; $\alpha > 0.42$ would suffice on gpt-oss-20b	Proposition 4.8

- **Two machines in one.** On battery, the GPU, the link and the CPU change regime (Section 3.6.1); all retained measurements are on mains power, and the power state is logged with each pass (Chapter A).
- **A quality judge calibrated on another model.** The budget $\kappa = 0.0055$ comes from Qwen3.5-4B; it must be remeasured on gpt-oss.
- **Literature in abstract.** Several neighbours of B (TRACE, MatGPTQ, AnyBCQ) are read only in abstract: nothing is published before they are read in full.
- **A tooling choice that closes a door.** J1 will be in CUDA (Section 10.5): the program will require an NVIDIA GPU until the Vulkan port.

11.6 Reading “without RAM or anything”

The retained reading (Definition 1.1) was never substituted for Stone’s request: it is his to confirm. The numbers make it concrete. Zero RAM is impossible; a RAM that does not depend on the model is reachable, and its price can be read on Figure 4.3: each GB of RAM given back to the system costs about 0.24 token/s in the fitted-LRU scenario. “Without RAM” becomes a knob, which Stone can turn according to what else he does on his machine.

What could be wrong

- The summary inherits every fragility of the chapters: first locality, then the estimated baseline, finally the quality of B.
- The list of killed ideas holds for this regime (encrypted disk, batch of one); several come back if the disk stops limiting.
- The list of struck-out numbers is complete only for what the studies published; a wrong

> number not yet contradicted is, by definition, not in it.

Chapter 12

Future work

Where we start, what this chapter establishes, what remains

Where we start: the fragilities of Chapter 11 and the milestones of Section 10.5. **This chapter establishes** the order of the coming work, each step with the measurement that judges it; the open research questions, each quantified by the token equation; the only gesture that falls to Stone; and the downloads that could help, none of them needed for the first program. **What remains** is to do it.

12.1 The deciding measurements, first

1. **The curve $h(c)$ and the reuse ρ of gpt-oss-120b**, on its own routing traces. They replace assumption 4.4 in every ceiling; the 20b measurement (Figure 10.9) already says the assumed shape is wrong. Witness: the replica of llama.cpp's cache replayed on the traces, which must give back the numbers of its log.
2. **The llama.cpp baseline of gpt-oss-120b in ABBA series**, on mains power, following the protocol of Section 9.6: this is the first measurement that can kill A (the ×2.5 to ×17 ratio of study 07 rests on an estimated baseline).
3. **The Q0 bench**: the quality of B without an engine, by static MXFP4 files (base snapped onto existing E2M1 values) judged by llama.cpp, and the frontier of uniform requantizations. This is the measurement that decides J2.
4. **The gap between two correct engines on gpt-oss**, which recalibrates the quality budget κ of Definition 4.5.

12.2 Milestone J1: building A

The arbiter splits the work into five work packages with disjoint files, each closed by a witness, its break and a declared threshold (`decisions/ARCHITECTURE.md`, § 6):

F, format and coder : rewriting the original GGUF into a `.trc` v1.0 file; witness: every decoded chunk equals the original, byte for byte.
E, byte path : IOCP reader at expert size into pinned memory, DMA; witness: end-to-end throughput exceeds no stage, and bytes arrive exact.
K, kernels : per-lane-table rANS decoder, MXFP4 product, gpt-oss forward pass; witness: the already measured WGSL kernels, on the same bytes (second path).
R, reference and judge : a double-precision CPU reference, the quality judge, the Q0 bench.
O, cache, scheduler, engine : expert cache in VRAM, miss queue, modes A0 and A; witness: bit-identical logits between the coded path and the raw path.

The milestone is judged on gpt-oss-120b: tokens/s, peak RAM and VRAM, bytes read according to the system counter, and physical efficiency, i.e. measured throughput over the ceiling of Equation (4.1) recomputed with the hit rate counted by the engine. Then J1+ adds one-layer-ahead prefetching and an SLRU cache, and is judged against the best measured llama.cpp baseline.

12.3 The research questions

Table 12.1: The open research questions, and what each is worth according to the token equation [calc.] (`fondations/07-ce-qui-manque.md`, § 3.2, and Section 10.3.3).

	Question	What it is worth, and what is missing
R1	Exact refinement of a native 4-bit format, tied to the stage (B)	in throughput: B3, ×1.74 at the cost of 48.5 % of uses served by the base; in latency: B1, floor +23 %. Missing: quality (Q0)
R2	Joint allocation of bits and stage	with the machine's throughputs, a resident matrix deserves $\frac{1}{2}\log_2(209/2.58) = 3.2$ more bits than a matrix read from disk; no tool applies it
R3	A miss policy under a quality budget	wait, defer, serve the base, re-route, miss by miss; the gain lives between floor and ceiling (A, LRU: 3.93 → 6.24, ×1.59)
R4	Speculation under real reuse	condition $\alpha > 1 - \rho$ (Proposition 4.8); $\rho = 0.577$ on gpt-oss-20b; to be measured on the 120b, and for $k > 1$
R5	A second-generation 2-bit code decodable without CUDA	trellis decoder written and measured at 124 GB/s, encoder missing; B_e divided by 1.9 (4.25 → 2.25 bits); quality unknown on a model trained in 4 bits
R6	Verification trees for delta-rule hybrids	48 MiB of state per kept node, 12 GiB for 256 nodes; blocks C on the Qwen models
R7	A constant-time-decodable code for the resident regime	at best +10 % VRAM capacity, +2 % ceiling; low priority

12.4 The gesture that falls to Stone

The largest lever of the machine is not software (Section 11.1). An unencrypted `D:` volume, created by a script already written that Stone would run himself as administrator (`outils/creer-disque-modeles.ps1`), would raise the ceiling of A from 6.24 to **14.1 tokens/s** in the fitted-LRU scenario if the disk reached its rated 6.0 GB/s there [calc.] (`fondations/journaux/07-synthese/csv/architectures-gptoss120b.csv`, row `A_sans_perte_disque_non_chiffre_6`). This is an upper bound: Intel RST VMD stays on the path, and its share of the 53 % lost is not separated. Beyond that, the link takes over (58 ms against 67 for the disk): keeping the link at Gen 4 and VRAM capacity become levers again.

12.5 Possible downloads

None is needed to build and judge A and B on gpt-oss-120b, already on the machine. Any download over 500 MB requires Stone's agreement, with name, source, size and licence (Table 12.2).

Table 12.2: Downloads that could help, sorted by need [lit.] (sizes read through the Hugging Face or NuGet API, `fondations/07-ce-qui-manque.md`, § 5).

Rank	Name	Size	Licence	For
1	DirectStorage 1.4 (preview)	3.99 MB	Microsoft	comparing the disk → GPU path
2	Qwen3.6-35B-A3B UD-Q4_K_M	22.13 GB	Apache-2.0	second fine-grained MoE, trace and ρ
3	OLMoE-1B-7B Q4_K_M	≈ 4.2 GB	Apache-2.0	quick routing trace
4	Llama-3.3-70B IQ3_XXS + Llama-3.2-1B Q4_K_M	28.28 GB	Llama 3.3 / 3.2	architecture C
5	Qwen3.8-Flash-Next UD-Q2_K_XL	78.87 GB	qwen-community-1.0	second model for B
6	Qwen3.5-9B-MTP Q4_K_M	5.87 GB	Apache-2.0	acceptance of native MTP

12.6 The thesis itself

Two chapters will be added to the third part, before the discussion: **implementation** (the modules of troncons, the `.trc` format to the bit, the kernels and their witnesses) and **results** (each series of milestone J1 as a figure, with error bars, witness and declared threshold). ~~The entropy study, which this thesis quotes without giving it a chapter because it was being completed by another agent, will get its own~~ (it has: Chapter 6), and the gpt-oss measurement, once delivered, will replace the provisional figures of Section 10.6.

What could be wrong

- The proposed order assumes the measured llama.cpp baseline does not exceed the ceiling of A; if it did, A would have to be rethought before being built.
- The gains of Table 12.1 are ceilings computed under assumption 4.4; they will move with the measured curve.
- The gain of the unencrypted volume is an upper bound based on a manufacturer's rating, not on a measurement.

Appendix A

Measurement protocols

This appendix gathers the rules that produced every [meas.] number of the thesis. They were written and committed *before* the measurements they judge (`fondations/journaux/01-machine/00-seuils-declares.md`, committed at 01:35 on 2026-10-15, before the first run of the bench); none was adjusted afterwards.

A.1 The labels

Every number carries its provenance: [meas.] measured here, on Stone's machine, with a committed tool and log; [calc.] computed by a written formula and a committed tool; [lit.] literature (publisher's number, not reproduced). A number taken from a study keeps its label and its study number.

A.2 The witness, and its break

Every measurement has its witness: a case whose answer is known in advance. Every new witness must be able to fail: its condition is forced to false (option `--fausser` or `--casser` of the tool), the run is repeated, the instrument is checked to fall, then restored. Examples from the studies:

- **disk**: every 4 KiB page carries its number; wrong pages must be zero, and all verified pages under `--fausser`;
- **RAM**: the sum read must equal the known sum of the pattern; `--fausser` changes the expected value and must fail;
- **GPU**: host → device → host round trip byte for byte; matrix-vector product compared to a CPU reference (relative gap $\leq 10^{-3}$ on 64 rows);
- **physical ceilings**: an unbuffered read above the NVMe link (7.877 GB/s), a RAM read above one DDR5-5200 module (41.6 GB/s) prove a read served by a cache, and make the instrument fall;
- **token equation**: three numbers published by other studies redone within 1 %; replacing the measured disk throughput by the link ceiling makes exactly the one that depends on it fall (Table 4.3);
- **coders**: every stream decoded and compared byte for byte with the original; a falsified frequency table must make decoding inexact (Section 10.5).

A.3 The threshold declared beforehand

A threshold is not set by eye: it derives from a named measured quantity, and it is declared before the run. Thus the quality budget $\kappa = 0.0055$ is the measured gap between two correct engines (Definition 4.5); the threshold of measurement F1 (0.40 %) is the measured header cost of one more stream; the thermal throttling threshold (95 % of the initial throughput) is more than twice the expected noise of a repeated GEMV.

A.4 ABBA series and noise

A cost is never based on a single pass. Every A/B comparison is measured in series A1 B1 B2 A2, repeated at least twice. The noise of a program against itself is $|A_1 - A_2|/\overline{A}$. A difference is declared real only if

$$\frac{|\overline{A} - \overline{B}|}{\overline{A}} \geq 2\max(\text{noise}_A, \text{noise}_B) \quad \text{and} \quad \geq 3\,\%.$$

This rule declares "within the noise" the gap between IOCP and IoRing (Chapter 3) and between llama.cpp's expert cache and experts on the CPU for gpt-oss-20b (Section 10.6).

A.5 The second path

Every published number we depend on is redone by an independent path. The disk throughput is redone by the cumulative system counter (`Win32_PerfRawData_PerfDisk_PhysicalDisk`, sampled every 0.5 s; unbuffered, the disk must have seen at least 90 % of the tool's bytes); the cost of BitLocker by the Windows cryptography API; the gpt-oss inventory by reading the local file header after the remote one; the architecture ceilings by an arbiter's tool that imports no one else's tool.

A.6 Power and the shared machine

On battery, the machine changes regime (Section 3.6.1). The power state is read at the start of each measurement with `Win32_Battery.BatteryStatus` (2: mains; 1: battery) and logged; only measurements on mains power are retained as reference throughputs, the others are stated as such. Other agents were working on the same machine: each pass logs the foreign load (disk counters, `nvidia-smi`), and "best" throughputs are lower bounds of the idle machine.

A.7 The quality judge

The protocol of the model study (Section 9.6): llama.cpp b11515 as is for the baseline; a fixed real text (the first 3,000 bytes of WikiText-2 test, SHA-256 recorded), 256 tokens, greedy decoding; tokens/s of prefill and generation, peak process memory (every 100 ms), peak VRAM (every 250 ms), bytes read from disk (integrated PDH counter, with 10 s of rest to quantify the foreign load), checksum of the produced text. Quality is judged by the KL divergence against the baseline and the same-top-choice rate: instrument floor (same engine) 0.000055 at the 99.9th percentile; gap between two correct engines 0.0055 on average, 95.96 % same top choice.

A.8 The struck-out sentence

When a measurement contradicts a published number, even ours, the sentence is struck out in place (~~like this~~) and what changed is stated; never a silent removal. Table 11.2 gathers those of the foundations.

Appendix B

Tools

Every [meas.] or [calc.] number of the thesis comes out of a committed tool. This appendix lists them, sorted by study, each with the first sentence of its own description, quoted as is (the tools are written in French, as is the whole repository). The table is generated by `memoire/outils/annexes.py` from ~~`git ls-files`~~ `git ls-tree`: it is not written by hand and ~~follows the repository~~ describes the repository at the publication of this volume, at commit `b8af154`. Corrected on 2026-10-29: the tool read the repository of the day, which kept growing after this volume; it now reads this commit, and the table stays that of the published volume.

B.1 The thesis toolchain

- `memoire/construire.sh` (and `memoire/Makefile` for those who have `make`): regenerates the figure data and the appendix tables, then builds both PDFs with `latexmk` (lualatex, biber); it fails at the first error, at any missing reference or citation, and at any entry biber cannot find.
- `memoire/outils/donnees_figures.py`: extracts from each committed log the CSV of one figure, with no number written by hand (the only constants are the datasheet ceilings, each with its derivation).
- `memoire/outils/annexes.py`: the tables of this appendix and the next.
- `memoire/outils/bib_etudes.py`: the bibliographies of the studies, in biblatex.

B.2 The tools of the studies

Tool	Its own description (first line, quoted)
01 machine	
`outils/banc/Cargo.toml`	Dépendances : toutes déjà dans le cache cargo de la machine (construction –offline). windows-sys : MIT OU Apache-2.0. wgpu : MIT OU Apache-2.0. pollster : MIT OU Apache-2.0. bytemuck …
`outils/banc/aes_xts.py`	Débit du déchiffrement XTS-AES sur le processeur, par l'API CNG de Windows (bcrypt.dll), de 1 à N fils : le coût logiciel de BitLocker, mesuré par un second chemin.
`outils/banc/analyse_carte.py`	Lecture du journal du banc carte : tableaux par opération, taille et interface (Vulkan, DX12), série ABBA vulkan, dx12, dx12, vulkan …
`outils/banc/analyse_disque.py`	Lecture du journal du banc disque : tableaux bloc x profondeur (moyenne des passages A et B, bruit ABBA), comparaison IOCP / IoRing selon la règle déclarée, second chemin (compteur du …

Tool	Its own description (first line, quoted)
`outils/banc/analyse_thermique.py`	Analyse d'une passe thermique (pilote_thermique.py) : débit de la GEMV par fenêtre de 5 s et relevé nvidia-smi par seconde.
`outils/banc/csv_disque.py`	Extrait du journal du balayage disque (disque-balayage.jsonl) un CSV pour les figures du mémoire …
`outils/banc/pilote_carte.py`	Pilote du banc carte : chaque passe de carte.exe encadrée par un relevé nvidia-smi (utilisation, VRAM occupée, horloges, température, puissance) ; témoins –fausser …
`outils/banc/pilote_disque.py`	Pilote du banc disque : témoins, balayage bloc × profondeur de file en ordre ABBA, comparaison IOCP / IoRing, second chemin par le compteur cumulé du système (CIM).
`outils/banc/pilote_llama.py`	Base de comparaison : llama.cpp b11515 (CUDA 12.4) et Qwen3.5-9B Q4_K_M, en VRAM puis avec une partie des couches sur le processeur, en projection de fichier (mmap) ou en lecture directe. …
`outils/banc/pilote_thermique.py`	Thermique sur 5 minutes : GEMV q4b en boucle (carte.exe thermique) et relevé nvidia-smi chaque seconde (température, horloges, puissance, raisons de bridage). …
`outils/banc/src/bin/carte.rs`	Banc de la carte graphique par wgpu (Vulkan ou DirectX 12), sans boîte à outils CUDA.
`outils/banc/src/bin/disque.rs`	Banc du disque : lecture NVMe sans tampon (FILE_FLAG_NO_BUFFERING), séquentielle et aléatoire, par port de complétion (IOCP) ou par IoRing, à profondeur de file fixée.
`outils/banc/src/bin/memoire.rs`	Banc de la RAM : débit de lecture, d'écriture et de copie, de 1 à N fils, et latence d'un accès dépendant (chasse de pointeurs sur un cycle aléatoire).
`outils/banc/src/lib.rs`	Pièces communes du banc : motif de données vérifiable, statistiques, arguments.
`outils/banc/suite_secteur.py`	La suite « sur secteur » : les mesures que la batterie fausse, relancées dès que la machine est branchée.
`outils/banc/toit.py`	Le toit (roofline) de troncons : plafond de jetons par seconde selon l'étage d'où viennent les poids, et facteur de compression qui rend la lecture disque aussi rapide que la PCIe.
02 quantization	
`outils/quant-bib.py`	Sources de l'étude 02 en BibTeX (biblatex) pour le mémoire : memoire/sources/quantification.bib.
`outils/quant-csv-memoire.py`	Données de l'étude 02 (quantification) en CSV, pour les figures pgfplots du mémoire.
`outils/quant-echelle.sh`	Échelle de quantification llama.cpp sur un modèle : imatrix, requantification vers chaque type, bits par poids réels, divergence KL et perplexité contre la base.
`outils/quant-gemv-treillis.cu`	Coût du décodage dans le noyau : produit matrice-vecteur (lot de 1) sur la RTX 4060, poids stockés en FP16, en scalaire 2 bits, en scalaire 4 bits, et en treillis à décalage 2 bits (code …
`outils/quant-gguf-bits.py`	Bits par poids réels d'un fichier GGUF, métadonnées comprises, et octets lus par jeton.
`outils/quant-mse-gaussien.py`	Erreur quadratique de quantificateurs à 1, 2 et 3 bits par poids sur une source gaussienne i.i.d. …

Tool	Its own description (first line, quoted)
outils/quant-mse-iq2xxs.py	Place d'IQ2_XXS (llama.cpp) sur le banc gaussien du § 2 de fondations/02-quantification.md.
outils/quant-mse-xor.py	Treillis à décalage à code linéaire par parités (sans multiplication), sur source N(0,1).
outils/quant-resume-echelle.py	Résumé d'une échelle de quantification produite par quant-echelle.sh.
outils/quant-vitesse.py	Vitesse de décodage (lot de 1) de fichiers GGUF quantifiés et débit effectif des poids.
outils/quant-vulkan-gemv/Cargo.toml	Dépendances (mêmes versions que outils/banc, déjà dans le cache cargo) : wgpu : MIT OU Apache-2.0. pollster : MIT OU Apache-2.0.
outils/quant-vulkan-gemv/src/main.rs	Coût du décodage dans un shader de calcul portable (wgpu : Vulkan ou DirectX 12) : produit matrice-vecteur 16 384 × 16 384, poids en scalaire 2 bits, en treillis à décalage 2 bits (code …
03 entropy	
outils/03-entropie/attendre_calme.sh	Lance une commande seulement quand la machine est calme, trois relevés de suite à 10 s d'écart. mode « processeur » : charge du processeur sous 20 % et aucun processus llama-*. …
outils/03-entropie/champs_k.py	Découpe des blocs quantifiés ggml en leurs champs, écrite de zéro.
outils/03-entropie/cuda/construire.bat	Construit rans_v4.exe (second chemin du decodeur) et dispersion.exe (reassemblage des flux de classe), CUDA, sm_89, hors du depot. …
outils/03-entropie/cuda/dispersion.cu	Coût de la passe de réassemblage des codes MXFP4 répartis en trois flux de classe (§1.6 de 03-entropie.md) : chaque bloc de 16 octets revient à sa place depuis le flux de sa classe. …
outils/03-entropie/cuda/rans_v4.cu	Second chemin pour le débit du décodeur rANS v4 : le même algorithme que decoder_v4 de decodeur.wgsl, écrit en CUDA (intrinsèques de warp …
outils/03-entropie/debits_cpu.py	Débits de décompression sur le processeur : zstd et lz4 contre une recopie mémoire.
outils/03-entropie/flux_octets.py	Ce qu'un codeur d'octets d'ordre 0 (le décodeur v4) peut prendre à chaque type K, si l'on sépare le bloc en flux d'octets de même nature (le « byte grouping » de ZipNN, appliqué aux champs …
outils/03-entropie/gguf_lecture.py	Lecteur GGUF minimal, écrit de zéro pour troncons.
outils/03-entropie/mesure_entropie.py	Entropie réelle des poids quantifiés d'un GGUF, champ par champ.
outils/03-entropie/mxfp4_contexte.py	Entropie des échelles E8M0 des tenseurs MXFP4, avec et sans contexte.
outils/03-entropie/passes_carte.sh	Passes de carte de l'étude 03, chacune dans sa propre fenêtre calme (attendre_calme.sh). 1. banc_gemv sur les codes MXFP4 réels de gpt-oss-20b : brut, fondu, décodage, deux passes ; 2. …
outils/03-entropie/passes_secteur.sh	Passes de carte de l'étude 03, machine sur secteur (relevé à 08:33 le 2026-10-15 : Win32_Battery = 2, horloges 2 250 / 8 101 MHz), chacune dans sa fenêtre calme.
outils/03-entropie/proportions_classes.py	Proportions des trois classes d'échelle (e < médiane de ligne, e = médiane, e > médiane) sur tous les tenseurs MXFP4 d'un GGUF …

Tool	Its own description (first line, quoted)
outils/03-entropie/rans/Cargo.toml	Dépendances, toutes dans le cache cargo de la machine (construction –offline) : wgpu 30.0.1 : MIT OU Apache-2.0. pollster : MIT OU Apache-2.0. bytemuck : Zlib OU Apache-2.0 OU MIT. rayon …
outils/03-entropie/rans/src/bin/banc_gemv.rs	Banc de la question clé : décoder le flux entropique DANS le produit matrice-vecteur, ou JUSTE AVANT, quand les poids résident en VRAM.
outils/03-entropie/rans/src/bin/banc_rans.rs	Banc du rANS entrelacé sur les codes Q4_K d'un GGUF.
outils/03-entropie/rans/src/bin/micro_cpu.rs	Micro-banc du décodeur processeur sur données synthétiques (distribution des codes Q4_K mesurée), pour séparer le coût du code de la contention de la machine.
outils/03-entropie/rans/src/bin/taux_mxfp4.rs	Taux réel d'un codage MXFP4 par champs, sur tous les tenseurs MXFP4 d'un GGUF.
outils/03-entropie/rans/src/bin/vider_flux.rs	Écrit sur le disque un lot codé par le rANS v4 (alphabet octet), pour qu'un second décodeur, écrit dans un autre langage (CUDA, outils/03-entropie/cuda/rans_v4.cu), le décode et soit …
outils/03-entropie/rans/src/decodeur.wgsl	Décodeur rANS entrelacé par sous-groupe (format décrit dans lib.rs). Un sous-groupe de 32 invocations décode une tuile …
outils/03-entropie/rans/src/gemv.wgsl	Produit matrice-vecteur (lot 1) sur des poids MXFP4, avec et sans décodage rANS fondu.
outils/03-entropie/rans/src/lib.rs	rANS entrelacé par sous-groupe pour les codes 4 bits des blocs Q4_K.
outils/03-entropie/rd_proxy.py	Au-delà du sans-perte : à erreur égale, combien de bits gagne une quantification taillée pour un codeur entropique (ECSQ), face à une quantification à débit fixe ?
outils/03-entropie/resume_entropie.py	Résumé des journaux de mesure_entropie.py : par type, par rôle, et pour le modèle.
outils/03-entropie/temoin_champs.py	Témoin : notre découpe des champs reconstruit exactement les poids que la bibliothèque gguf (llama.cpp, licence MIT, second chemin indépendant) déquantifie.
outils/03-entropie/vers_csv.py	Tire des journaux de l'étude 03 les CSV des figures du mémoire.
04 loading	
outils/chargement_juge.py	Le juge physique du chargement : octets lus par jeton, étage par étage, divisés par le débit de chaque étage. Décodage, un jeton à la fois (lot 1), modèle MoE ou dense.
outils/chargement_localite.py	Localité des experts d'un jeton au suivant, et ce qu'un cache en tire.
outils/chargement_pcie.py	Mesure des étages RAM -> PCIe -> VRAM de la machine, sans boîte à outils CUDA.
05 sparsity	
outils/05-sparsite/amortissement.py	Amortissement d'une lecture de poids sur plusieurs jetons : le calcul de l'étude 05, avec son témoin par simulation.
outils/05-sparsite/gguf_f16.py	Lecteur GGUF réduit au nécessaire de l'étude 05 : métadonnées et tenseurs F32 / F16 en tableaux numpy (memmap, sans copie).
outils/05-sparsite/llama_numpy.py	Passe avant d'un modèle d'architecture « llama » (GGUF F16), en numpy pur, avec un crochet sur l'entrée de chaque projection linéaire.

Tool	Its own description (first line, quoted)
outils/05-sparsite/mesures_sparsite.py	Mesures de sparsité des activations sur la passe avant numpy (étude 05).
outils/05-sparsite/resume_cn.py	Résumé des séries ABBA de llama-bench (pp n, -n 0) : temps d'une passe de n jetons, médiane des séries, bruit, et verdict contre le seuil déclaré.
outils/05-sparsite/temoin_passe_avant.py	Témoin de la passe avant numpy : sa perplexité doit retrouver celle de llama-perplexity (b11515) sur les mêmes jetons, fenêtre 512, 16 fenêtres.
outils/sparsite-inventaire-gguf.py	Inventaire d'un fichier GGUF, sans dépendance : métadonnées scalaires et octets par famille de tenseurs.
06 models	
outils/creer-disque-modeles.ps1	creer-disque-modeles.ps1 – cree le disque D: "Modeles", non chiffre, pour les poids des modeles.
outils/inventaire_modeles.py	Inventaire des modèles lourds : paramètres totaux et actifs, lus dans les en-têtes safetensors.
outils/modeles-base.py	Banc de la base de comparaison : llama.cpp tel quel (mmap), sur un texte réel fixé, en glouton.
outils/modeles-fichiers-a-telecharger.py	Liste exacte des fichiers à demander : nom, taille, empreinte SHA-256 (champ LFS de l'API).
outils/modeles-gguf-entete-distant.py	Octets lus par jeton d'un fichier GGUF précis, lus dans son en-tête à distance (requêtes Range).
outils/modeles-gguf-tailles.py	Tailles réelles des quantifications GGUF publiées, lues dans l'arbre des dépôts Hugging Face.
outils/modeles-juge.sh	Juge de qualité léger, identique pour tous les modèles et tous les moteurs. Usage : modeles-juge.sh <modèle.gguf> <étiquette> <dossier-sortie> [base-logits.kld] Sans 4e argument …
outils/modeles-octets-par-jeton.py	Octets lus par jeton et plafonds physiques, pour chaque modèle de l'inventaire.
outils/modeles-plafonds-fichiers.py	Plafonds physiques par jeton pour les fichiers GGUF exacts candidats au téléchargement.
outils/modeles-temoin-disque.py	Témoin du compteur de disque de modeles-base.py (PDH, PhysicalDisk(_Total) / Disk Read Bytes/sec).
outils/modeles-temoin-inventaire.py	Témoin de l'inventaire : nos comptes de paramètres contre ceux que les éditeurs publient.
outils/modeles_physique.py	Plafonds physiques de la machine de Stone et modèle de coût par jeton (calculé, jamais mesuré).
07 synthesis	
outils/07-synthese/inventaire_gptoss.py	Inventaire local des fichiers gpt-oss MXFP4, par lecture de leur en-tête GGUF.
outils/07-synthese/plafonds.py	L'équation du jeton, appliquée aux architectures candidates de troncons.
outils/07-synthese/raffinement_mxfp4.py	Raffinement exact d'un format MXFP4 natif : ce que coûte et ce que vaut une couche de base.
decisions	
outils/arbitre/decoupe/Cargo.toml	Une seule dépendance, déjà dans le cache cargo de la machine (construction –offline) : rayon 1.12.0 : MIT OU Apache-2.0. …

Tool	Its own description (first line, quoted)
`outils/arbitre/decoupe/src/main.rs`	Mesure F1 de l'arbitre (decisions/ARCHITECTURE.md) : ce que coûte, en octets réels, la découpe d'un code MXFP4 en base + complément, contre le flux unique de l'architecture A.
`outils/arbitre/verif_plafonds.py`	Second chemin de l'arbitre : les plafonds de A, A0, B1 et B3 refaits par des formules écrites ici.
`outils/arbitre/vers_csv.py`	Données des figures du mémoire tirées des journaux de l'arbitre (rien n'est recalculé ici).
`outils/decisions/partisan_a.py`	Les chiffres du partisan de l'architecture A (« l'exactitude et l'ingénierie d'abord »).
`outils/decisions/partisan_b.py`	Les chiffres du partisan de B (« la contribution d'abord ») : ce que B gagne vraiment quand on compte le trafic des compléments, et ce que vaut une base lisible par llama.cpp.
gpt-oss measurements	
`outils/gptoss-routage/construire.bat`	Construit traceur.exe contre les DLL officielles de llama.cpp b11515 (binaires CUDA 12.4). en-têtes : sources officielles du même commit (3d65c90d0, licence MIT), dossier SRC …
`outils/gptoss-routage/faire_def.py`	Tire un fichier .def (bibliothèque d'importation) de la sortie de « dumpbin /exports ».
`outils/gptoss-routage/tracer.sh`	Relève les traces de routage de gpt-oss (20b ou 120b) sur le corpus en quatre genres, puis sur l'invite fixée de la base (étude 06) avec la même génération gloutonne que la base, pour le …
`outils/gptoss-routage/traceur.cpp`	traceur.cpp : relève, jeton par jeton et couche par couche, les experts choisis par le routeur d'un modèle MoE (écrit pour gpt-oss) en faisant tourner llama.cpp b11515 tel quel.
`outils/gptoss_base_resume.py`	Résumé d'une série de la base (passes.csv de gptoss_base_serie.py) : moyenne, bruit, décision.
`outils/gptoss_base_serie.py`	Séries alternées de la base llama.cpp b11515 sur gpt-oss (protocole de fondations/06-modeles.md § 7).
`outils/gptoss_comparer_traces.py`	Compare deux traces de routage du même texte : part des choix (jeton, couche) identiques.
`outils/gptoss_corpus.py`	Corpus de texte réel pour relever le routage des experts de gpt-oss : quatre genres.
`outils/gptoss_figures.py`	Données et figures pgfplots du mémoire pour les mesures sur gpt-oss (localité et base).
`outils/gptoss_localite.py`	Localité des experts de gpt-oss sur traces réelles, et plafond de jetons/s qui en découle.
`outils/gptoss_octets.py`	Octets par expert et octets toujours lus de gpt-oss, lus dans l'entête des fichiers GGUF locaux.

Appendix C

Logs and reproduction of the figures

What produces a published number is committed: tools and logs. This appendix says where the logs are and which data each graph reads. Both tables are generated by `memoire/outils/annexes.py`.

C.1 Rebuilding the thesis

From `memoire/`, in Git Bash, with MiKTeX (lualatex, biber, latexmk) and Python 3.13 with no dependency:

```
./construire.sh          # figure data, appendices, then both PDFs
./construire.sh en       # one language only
./construire.sh propre   # removes intermediate files
```

Each pgfplots graph reads a committed CSV; each TikZ diagram is a file in `memoire/figures/schemas/`; the figure commands are shared by both languages, which only differ by their texts (`\EF{English}{French}`). Each thesis reads in parallel with the other: same chapters, same labels, same figures.

C.2 The logs

Log folder	Files	Size (MB)
`decisions/journaux/`	2	0.05
`decisions/journaux/arbitre/`	9	0.05
`decisions/journaux/csv/`	2	0.00
`decisions/journaux/partisan-a/`	5	0.03
`fondations/journaux/01-machine/`	31	0.67
`fondations/journaux/02-quantification/`	106	0.89
`fondations/journaux/03-entropie/`	81	1.94
`fondations/journaux/04-chargement/`	11	0.08
`fondations/journaux/05-sparsite/`	26	0.19
`fondations/journaux/06-modeles/`	36	1.02
`fondations/journaux/07-synthese/`	10	0.11
`mesures/`	2	0.04
`mesures/base-gpt-oss/`	48	0.19
`mesures/localite-gpt-oss/`	32	3.60
`mesures/moteur/`	118	0.88

C.3 The data of each graph

Graph (command)	Data files it reads
\GrapheArchitectures	memoire/figures/donnees/archi-plafonds.csv
\GrapheBtroisLambda	decisions/journaux/csv/b3-lambda-gptoss120b.csv
\GrapheFun	memoire/figures/donnees/archi-f1.csv
\GrapheLocaliteMesuree	memoire/figures/donnees/form-localite.csv memoire/figures/donnees/archi-localite-20b.csv
\GrapheBaseVingt	memoire/figures/donnees/archi-base-20b.csv
\GrapheJuge	memoire/figures/donnees/charge-juge.csv
\GrapheSensibiliteH	memoire/figures/donnees/charge-sensibilite-h.csv
\GrapheLocalite	memoire/figures/donnees/charge-localite-collant.csv
\GrapheHistogrammeCodes	../../fondations/journaux/03-entropie/csv/fig-histogramme-codes.csv
\GrapheEntropieGroupes	memoire/figures/donnees/entropie-groupes.csv memoire/figures/donnees/entropie-groupes-mesures.csv
\GrapheEntropiePlage	memoire/figures/donnees/entropie-plage.csv
\GrapheEchelles	memoire/figures/donnees/entropie-echelles-20b.csv memoire/figures/donnees/entropie-echelles-120b.csv
\GrapheTauxTenseurs	memoire/figures/donnees/entropie-taux-20b-down.csv memoire/figures/donnees/entropie-taux-20b-gate.csv memoire/figures/donnees/entropie-taux-20b-up.csv memoire/figures/donnees/entropie-taux-120b-down.csv memoire/figures/donnees/entropie-taux-120b-gate.csv memoire/figures/donnees/entropie-taux-120b-up.csv
\GrapheDebitsDecodeur	memoire/figures/donnees/entropie-debits-batterie.csv memoire/figures/donnees/entropie-debits-secteur.csv memoire/figures/donnees/entropie-debits-mxfp4.csv memoire/figures/donnees/entropie-debits-cuda.csv

Graph (command)	Data files it reads
\GrapheGemvEntropie	memoire/figures/donnees/entropie-gemv-secteur.csv memoire/figures/donnees/entropie-gemv-batterie.csv
\GrapheRdProxy	../../fondations/journaux/03-entropie/csv/rd-proxy.csv
\GrapheLocaliteHypotheses	memoire/figures/donnees/form-localite.csv memoire/figures/donnees/form-localite-calage.csv
\GraphePlafondHs	fondations/journaux/07-synthese/csv/plafond-gptoss120b-contre-hs.csv
\GraphePlafondRam	fondations/journaux/07-synthese/csv/plafond-gptoss120b-contre-ram.csv
\GrapheDisque	memoire/figures/donnees/machine-disque-alea-qd1.csv memoire/figures/donnees/machine-disque-alea-qd4.csv memoire/figures/donnees/machine-disque-alea-qd16.csv memoire/figures/donnees/machine-disque-alea-qd64.csv
\GrapheEtages	memoire/figures/donnees/machine-etages.csv
\GrapheAES	memoire/figures/donnees/machine-aes-128.csv memoire/figures/donnees/machine-aes-256.csv
\GrapheLatence	memoire/figures/donnees/machine-ram-latence.csv
\GrapheThermique	memoire/figures/donnees/machine-thermique-secteur.csv memoire/figures/donnees/machine-thermique-batterie.csv
\GraphePCIe	memoire/figures/donnees/machine-pcie-generation.csv
\GrapheGEMV	memoire/figures/donnees/machine-gemv.csv
\GrapheToitR	fondations/journaux/01-machine/toit-9b-compression.csv
\GrapheToitPlacements	memoire/figures/donnees/machine-toit-placements.csv
\GrapheMoE	memoire/figures/donnees/modeles-moe.csv
\GrapheMSE	fondations/journaux/02-quantification/csv/fig-q1-lloyd.csv fondations/journaux/02-quantification/csv/fig-q1-vq8.csv fondations/journaux/02-quantification/csv/fig-q1-iq2xxs.csv fondations/journaux/02-quantification/csv/fig-q1-treillis.csv

Graph (command)	Data files it reads
\GrapheGEMVQuant	memoire/figures/donnees/quant-gemv-vulkan.csv memoire/figures/donnees/quant-gemv-dx12.csv
\GrapheEchelle	fondations/journaux/02-quantification/csv/fig-q3-4B.csv fondations/journaux/02-quantification/csv/fig-q3-9B.csv
\GrapheVitesseQuant	fondations/journaux/02-quantification/csv/fig-q6-vitesse-4B.csv fondations/journaux/02-quantification/csv/fig-q6-vitesse-9B.csv
\GrapheIsoOctets	fondations/journaux/02-quantification/csv/fig-q4-llama2-7B.csv fondations/journaux/02-quantification/csv/fig-q4-llama2-13B.csv fondations/journaux/02-quantification/csv/fig-q4-llama2-70B.csv
\GrapheMMLU	fondations/journaux/02-quantification/csv/fig-q4-qwen3-mmlu-tetra.csv fondations/journaux/02-quantification/csv/fig-q4-qwen3-mmlu-awq.csv fondations/journaux/02-quantification/csv/fig-q4-qwen3-mmlu-fp16.csv
\GrapheAmortissement	memoire/figures/donnees/sparse-E-50.csv memoire/figures/donnees/sparse-E-60.csv memoire/figures/donnees/sparse-E-70.csv memoire/figures/donnees/sparse-E-80.csv memoire/figures/donnees/sparse-E-90.csv memoire/figures/donnees/sparse-E-95.csv
\GrapheUnions	memoire/figures/donnees/sparse-union-s25.csv memoire/figures/donnees/sparse-union-s50.csv memoire/figures/donnees/sparse-union-s70.csv memoire/figures/donnees/sparse-union-s90.csv
\GrapheCn	memoire/figures/donnees/sparse-cn.csv

Bibliography

[1] S. Williams, A. Waterman, and D. Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Communications of the ACM* 52.4 (2009-04). Origine du « toit » ; ici étendu à une chaîne d’étages de stockage (proposition 1)., pp. 65–76. DOI: 10.1145/1498765.1498785. (Visited on 2026-10-15).

[2] P. J. Denning. “The working set model for program behavior”. In: *Communications of the ACM* 11.5 (1968-05). Métadonnées relues par l’API de Crossref le 2026-10-15, pp. 323–333. DOI: 10.1145/363095.363141. (Visited on 2026-10-15).

[3] L. A. Belady. “A Study of Replacement Algorithms for a Virtual-Storage Computer”. In: *IBM Systems Journal* 5.2 (1966), pp. 78–101. DOI: 10.1147/sj.52.0078. (Visited on 2026-10-15).

[4] S. Barrett. *Sparse Virtual Textures*. Exposé à la Game Developers Conference 2008 ; la page ne signe que du prénom « Sean » ; le nom complet est celui du titulaire du site, connu par ailleurs, et n’est pas relu sur la page. 2008. URL: https://silverspaceship.com/src/svt/ (visited on 2026-10-15).

[5] Microsoft DirectX team. *DirectStorage 1.4 Release Adds Support for Zstandard*. Dépôt github.com/microsoft/DirectStorage : MIT, GDeflate Apache-2.0, commit c53f149 du 2026-08-04. 2026-03-11. URL: https://devblogs.microsoft.com/directx/directstorage-1-4-release-adds-support-for-zstandard/ (visited on 2026-10-15).

[6] lyogavin. *AirLLM*. Apache-2.0, commit 0f679cc. 2026-10-14. URL: https://github.com/lyogavin/airllm (visited on 2026-10-15).

[7] DeepSpeed Team. *ZeRO-Inference: Democratizing Massive Model Inference*. 2022-09-09. URL: https://www.deepspeed.ai/2022/09/09/zero-inference.html (visited on 2026-10-15).

[8] Y. Sheng et al. *FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU*. ICML 2023 ; dernière version 2023-06-12. 2023-03-13. arXiv: 2303.06865. URL: https://arxiv.org/abs/2303.06865 (visited on 2026-10-15).

[9] ggml-org. *llama.cpp*. Version commit 843e7dc ; exécutables b11515 (2026-10-09). Licence MIT. 2026-10-15. URL: https://github.com/ggml-org/llama.cpp (visited on 2026-10-15).

[10] Z. Liu et al. *Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time*. ICML 2023. 2023-10-26. arXiv: 2310.17157. URL: https://arxiv.org/abs/2310.17157 (visited on 2026-10-15).

[11] Y. Song et al. *PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU*. SOSP 2024 ; dernière version 2024-12-12. 2023-12-16. arXiv: 2312.12456. URL: https://arxiv.org/abs/2312.12456 (visited on 2026-10-15).

[12] Z. Xue et al. *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. dernière version 2024-12-12. 2024-06-10. arXiv: 2406.06282. URL: https://arxiv.org/abs/2406.06282 (visited on 2026-10-15).

[13] K. Alizadeh et al. *LLM in a Flash: Efficient Large Language Model Inference with Limited Memory*. ACL 2024 ; dernière version 2024-07-30. 2023-12-12. arXiv: 2312.11514. URL: https://arxiv.org/abs/2312.11514 (visited on 2026-10-15).

[14] A. Eliseev and D. Mazur. *Fast Inference of Mixture-of-Experts Language Models with Offloading*. 2023-12-28. arXiv: 2312.17238. URL: https://arxiv.org/abs/2312.17238 (visited on 2026-10-15).

[15] X. Song et al. *ProMoE: Fast MoE-based LLM Serving using Proactive Caching*. dernière version 2025-09-01. 2024-10-29. arXiv: 2410.22134. URL: https://arxiv.org/abs/2410.22134 (visited on 2026-10-15).

[16] P. Tang et al. *HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference*. dernière version 2024-11-06. 2024-11-03. arXiv: 2411.01433. URL: https://arxiv.org/abs/2411.01433 (visited on 2026-10-15).

[17] X. He et al. *ExpertFlow: Optimized Expert Activation and Token Allocation for Efficient Mixture-of-Experts Inference*. DAC 2026 ; dernière version 2026-04-02. 2024-10-23. arXiv: 2410.17954. URL: https://arxiv.org/abs/2410.17954 (visited on 2026-10-15).

[18] Wang et al. *SeqMoE: Sequence-level Expert Prefetching for Offloaded MoE Inference*. 2026-09-11. arXiv: 2609.12978. URL: https://arxiv.org/abs/2609.12978 (visited on 2026-10-15).

[19] Lin et al. *The Other Half of the Memory Wall (Edge0)*. 2026-09-17. arXiv: 2609.18063. URL: https://arxiv.org/abs/2609.18063 (visited on 2026-10-15).

[20] K. Kamahori et al. *Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models*. ICLR 2025 ; dernière version 2025-05-01. 2024-02-10. arXiv: 2402.07033. URL: https://arxiv.org/abs/2402.07033 (visited on 2026-10-15).

[21] H. Chen et al. *KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models*. SOSP 2025, DOI 10.1145/3731569.3764843. 2025-10. URL: https://madsys.cs.tsinghua.edu.cn/publication/ktransformers-unleashing-the-full-potential-of-cpu/gpu-hybrid-inference-for-moe-models/SOSP25-chen.pdf (visited on 2026-10-15).

[22] ggml-org. *llama.cpp pull request 25294: experts read from disk with an explicit cache*. ouverte au 2026-10-15. 2026-07-04. URL: https://github.com/ggml-org/llama.cpp/pull/25294 (visited on 2026-10-15).

[23] Wok3b. *moe-stream*. licence MIT d'après la page du dépôt. 2026. URL: https://github.com/Wok3b/moe-stream (visited on 2026-10-15).

[24] Liang et al. *SSD-LLaMA: SSD, RAM and VRAM as One Hierarchy for Trillion-Parameter MoE Inference*. 2026-09-16. arXiv: 2609.18110. URL: https://arxiv.org/abs/2609.18110 (visited on 2026-10-15).

[25] M. Chen, W. Shao, P. Xu, et al. *EfficientQAT: Efficient Quantization-Aware Training for Large Language Models*. Première version 2024-07-10, dernière 2025-05-19 (2407.11062v3). ACL 2025 Main, camera ready version. 2024-07-10. arXiv: 2407.11062. URL: https://arxiv.org/abs/2407.11062 (visited on 2026-10-15).

[26] V. Malinovskii, A. Panferov, I. Ilin, H. Guo, P. Richtárik, and D. Alistarh. *Pushing the Limits of Large Language Model Quantization via the Linearity Theorem*. Première version 2024-11-26, dernière 2024-11-26 (2411.17525v1). 2024-11-26. arXiv: 2411.17525. URL: https://arxiv.org/abs/2411.17525 (visited on 2026-10-15).

[27] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. D. Sa. *QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks*. Première version 2024-02-06, dernière 2024-06-04 (2402.04396v2). ICML 2024. 2024-02-06. arXiv: 2402.04396. URL: https://arxiv.org/abs/2402.04396 (visited on 2026-10-15).

[28] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh. *Extreme Compression of Large Language Models via Additive Quantization*. Première version 2024-01-11, dernière 2024-09-11 (2401.06118v4). ICML, 2024. 2024-01-11. arXiv: 2401.06118. URL: https://arxiv.org/abs/2401.06118 (visited on 2026-10-15).

[29] V. Malinovskii, D. Mazur, I. Ilin, et al. *PV-Tuning: Beyond Straight-Through Estimation for Extreme LLM Compression*. Première version 2024-05-23, dernière 2024-05-30 (2405.14852v2). Preprint. 2024-05-23. arXiv: 2405.14852. URL: https://arxiv.org/abs/2405.14852 (visited on 2026-10-15).

[30] Y. Liu, J. Wen, Y. Wang, et al. *VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models*. Première version 2024-09-25, dernière 2024-10-22 (2409.17066v2). EMNLP 2024, Main, Poster. 2024-09-25. arXiv: 2409.17066. URL: https://arxiv.org/abs/2409.17066 (visited on 2026-10-15).

[31] A. Tseng, Q. Sun, D. Hou, and C. D. Sa. *QTIP: Quantization with Trellises and Incoherence Processing*. Première version 2024-06-17, dernière 2025-06-18 (2406.11235v4). NeurIPS 2024 Spotlight. 2024-06-17. arXiv: 2406.11235. URL: https://arxiv.org/abs/2406.11235 (visited on 2026-10-15).

[32] G. Ding, Z. Wang, R. Li, and J. Liu. *CurveTQ: Rotation-Free Trellis Quantization of LLM Weights via Curvature-Weighted Search*. Première version 2026-10-06, dernière 2026-10-06 (2610.09212v1). 2026-10-06. arXiv: 2610.09212. URL: https://arxiv.org/abs/2610.09212 (visited on 2026-10-15).

[33] X. Que, N. Elkayam, S. Pyakurel, S. Pan, and D. Gope. *XOR-Trellis: Ultra-Low-Complexity Dequantization and Curvature-Aware Hadamard-Free LLM Quantization*. Première version 2026-09-30, dernière 2026-09-30 (2610.00432v1). 2026-09-30. arXiv: 2610.00432. URL: https://arxiv.org/abs/2610.00432 (visited on 2026-10-15).

[34] P.-J. Malandrino. *Tetra: Serving Leech-Lattice Quantized LLMs at 2.7 Bits per Parameter*. Première version 2026-09-28, dernière 2026-09-28 (2609.35465v1). 15 pages, 4 figures, 8 tables. Code, measurement logs and preregistrations: https://github.com/pjmalandrino/llvq. 2026-09-28. arXiv: 2609.35465. URL: https://arxiv.org/abs/2609.35465 (visited on 2026-10-15).

[35] S. Ma, H. Wang, L. Ma, et al. *The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits*. Première version 2024-02-27, dernière 2024-02-27 (2402.17764v1). Work in progress. 2024-02-27. arXiv: 2402.17764. URL: https://arxiv.org/abs/2402.17764 (visited on 2026-10-15).

[36] Z. Liu, C. Zhao, H. Huang, et al. *ParetoQ: Improving Scaling Laws in Extremely Low-bit LLM Quantization*. Première version 2025-02-04, dernière 2025-10-13 (2502.02631v2). NeurIPS 2025. Model weights are available at https://huggingface.co/collections/facebook/mobilellm-6722be18cb86c20ebe113e95. 2025-02-04. arXiv: 2502.02631. URL: https://arxiv.org/abs/2502.02631 (visited on 2026-10-15).

[37] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Définit MXFP4 : éléments E2M1, échelle partagée E8M0 par bloc de 32. 2023. URL: https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf (visited on 2026-10-15).

[38] NVIDIA, F. Abecassis, A. Agrusa, et al. *Pretraining Large Language Models with NVFP4*. Première version 2025-09-29, dernière 2026-03-04 (2509.25149v2). Update includes: (1) fixing a typo in eq. 2 (2) updating author list, and (3) adding a related work. 2025-09-29. arXiv: 2509.25149. URL: https://arxiv.org/abs/2509.25149 (visited on 2026-10-15).

[39] S. Ashkboos, A. Mohtashami, M. L. Croci, et al. *QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs*. Première version 2024-03-30, dernière 2024-10-29 (2404.00456v2). 21 pages, 7 figures. 2024-03-30. arXiv: 2404.00456. URL: https://arxiv.org/abs/2404.00456 (visited on 2026-10-15).

[40] Z. Liu, C. Zhao, I. Fedorov, et al. *SpinQuant: LLM quantization with learned rotations*. Première version 2024-05-26, dernière 2025-02-20 (2405.16406v4). ICLR 2025. 2024-05-26. arXiv: 2405.16406. URL: https://arxiv.org/abs/2405.16406 (visited on 2026-10-15).

[41] T. Zhang, M. Hariri, S. Zhong, et al. *70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float (DFloat11)*. NeurIPS 2025 ; v3 du 2026-01-01. Code : github.com/LeanModels/DFloat11, Apache-2.0, commit 4577338 du 2025-11-24. 2025-04-15. arXiv: 2504.11651. URL: https://arxiv.org/abs/2504.11651 (visited on 2026-10-15).

[42] M. Hershcovitch, A. Wood, L. Choshen, et al. *ZipNN: Lossless Compression for AI Models*. v2 du 2025-06-04. Code : github.com/zipnn/zipnn, MIT, commit 6009704 du 2026-04-11. 2024. arXiv: 2411.05239. URL: https://arxiv.org/abs/2411.05239 (visited on 2026-10-15).

[43] P. Yubeaton, T. Mahmoud, S. Naga, et al. *Huff-LLM: End-to-End Lossless Compression for Efficient LLM Inference*. Aucun code publié trouvé. 2025-02-02. arXiv: 2502.00922. URL: https://arxiv.org/abs/2502.00922 (visited on 2026-10-15).

[44] Z. Yang, T. Zhang, J. Xie, C. Li, Z. Xu, and A. Shrivastava. *To Compress or Not? Pushing the Frontier of Lossless GenAI Model Weights Compression with Exponent Concentration*. ECF8 ; aucun code lié sur la page. 2025-10-03. arXiv: 2510.02676. URL: https://arxiv.org/abs/2510.02676 (visited on 2026-10-15).

[45] R. Fan, X. Yu, X. Pan, et al. "ZipServ: Fast and Memory-Efficient LLM Inference with Hardware-Aware Lossless Compression". In: *Proceedings of ASPLOS 2026*. Code : github.com/HPMLL/ZipServ_ASPLOS26, Apache-2.0, commit 6f8a209 du 2025-12-19. 2026-03-18. arXiv: 2603.17435. URL: https://arxiv.org/abs/2603.17435 (visited on 2026-10-15).

[46] H. Tan, Y. Chen, G. Alonso, W.-F. Wong, and B. He. "Approaching Shannon Bound with Lossless LLM Weight Compression". In: *Proceedings of ISCA 2026*. Code non trouvé. 2026-06-14. arXiv: 2606.15789. URL: https://arxiv.org/abs/2606.15789 (visited on 2026-10-15).

[47] A. Sanyal, G. Datta, P. Mukherjee, S. P. Chinchali, and M. Orshansky. *EntroLLM: Entropy Encoded Weight Compression for Efficient Large Language Model Inference on Edge Devices*. v4 du 2026-01-26. 2025-05-05. arXiv: 2505.02380. URL: https://arxiv.org/abs/2505.02380 (visited on 2026-10-15).

[48] H. Zhang, Z. Duan, and Y. Chen. *EntroPack: Fast and Accurate Entropy-Coded Weight Compression at Arbitrary Bitrates*. Code : github.com/modelscope/entropack, non cloné, licence non vérifiée. 2026-09-28. arXiv: 2609.34185. URL: https://arxiv.org/abs/2609.34185 (visited on 2026-10-15).

[49] P. Putzky, M. Genzel, M. Mollenhauer, S. Schulze, T. Wollmann, and S. Dietzel. "Float8@2bits: Entropy Coding Enables Data-Free Model Compression". In: *Proceedings of ICML 2026*. EntQuant ; v2 du 2026-05-29. Code : github.com/merantix-momentum/entquant (licence non vérifiée). 2026-01-30. arXiv: 2601.22787. URL: https://arxiv.org/abs/2601.22787 (visited on 2026-10-15).

[50] E. Lifar, S. Savkin, O. Ordentlich, and Y. Polyanskiy. *WaterSIC: Information-Theoretically (Near) Optimal Linear Layer Quantization*. v2 du 2026-06-01. Code : github.com/egorlifar/watersic (licence non vérifiée). 2026-03-05. arXiv: 2603.04956. URL: https://arxiv.org/abs/2603.04956 (visited on 2026-10-15).

[51] S. Nuri. *vram-compression: Entropy-coded LLM weight quantization with a fused rANS decode GEMV kernel*. Un seul commit (1af9a93, 2026-09-23). Aucun fichier de licence : tous droits réservés par défaut, lu seulement. 2026-09-23. URL: https://github.com/snuri00/vram-compression (visited on 2026-10-15).

[52] J. Duda. *Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding*. 2013. arXiv: 1311.2540. URL: https://arxiv.org/abs/1311.2540 (visited on 2026-10-15).

[53] F. Giesen. *Interleaved entropy coders*. 2014. arXiv: 1402.3392. URL: https://arxiv.org/abs/1402.3392 (visited on 2026-10-15).

[54] A. Weissenberger and B. Schmidt. "Massively Parallel ANS Decoding on GPUs". In: *Proceedings of the 48th International Conference on Parallel Processing (ICPP)*. Code : github.com/weissenberger/multians, LGPL-3.0, commit 55515d8 du 2019-07-26. 2019. DOI: 10.1145/3337821.3337888. (Visited on 2026-10-15).

[55] F. Lin, K. Arunruangsirilert, H. Sun, and J. Katto. "Recoil: Parallel rANS Decoding with Decoder-Adaptive Scalability". In: *Proceedings of the 52nd International Conference on Parallel Processing (ICPP)*. Code : github.com/lin-toto/recoil, GPL-3.0, commit c36e25d du 2026-07-12. 2023. DOI: 10.1145/3605573.3605588. arXiv: 2306.12141. (Visited on 2026-10-15).

[56] NVIDIA. *nvCOMP Release Notes and Algorithm Selection*. Version 6.0.0, non datée ; binaire propriétaire. 2026. URL: https://docs.nvidia.com/cuda/nvcomp/release_notes.html (visited on 2026-10-15).

[57] Meta (facebookresearch). *DietGPU: GPU-based lossless compression for numerical data*. MIT, commit a4d70a1 du 2026-03-18. URL: https://github.com/facebookresearch/dietgpu (visited on 2026-10-15).

[58] Meta (facebook). *Zstandard*. README, zstd 1.5.7 et lz4 1.10.0 mesurés par lzbench sur Silesia, i7-9700K. URL: https://github.com/facebook/zstd (visited on 2026-10-15).

[59] Y. Leviathan, M. Kalman, and Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023 ; dernière version 2023-05-18. 2022-11-30. arXiv: 2211.17192. URL: https://arxiv.org/abs/2211.17192 (visited on 2026-10-15).

[60] C. Chen et al. *Accelerating Large Language Model Decoding with Speculative Sampling*. 2023-02-02. arXiv: 2302.01318. URL: https://arxiv.org/abs/2302.01318 (visited on 2026-10-15).

[61] T. Cai et al. *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. dernière version 2024-06-14. 2024-01-19. arXiv: 2401.10774. URL: https://arxiv.org/abs/2401.10774 (visited on 2026-10-15).

[62] Y. Li et al. *EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees*. dernière version 2024-06-30. 2024-06-24. arXiv: 2406.16858. URL: https://arxiv.org/abs/2406.16858 (visited on 2026-10-15).

[63] Y. Li et al. *EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test*. dernière version 2025-04-23. 2025-03-03. arXiv: 2503.01840. URL: https://arxiv.org/abs/2503.01840 (visited on 2026-10-15).

[64] F. Gloeckle et al. *Better and Faster Large Language Models via Multi-token Prediction*. 2024-04-30. arXiv: 2404.19737. URL: https://arxiv.org/abs/2404.19737 (visited on 2026-10-15).

[65] DeepSeek-AI. *DeepSeek-V3 Technical Report*. dernière version 2025-02-18. 2024-12-27. arXiv: 2412.19437. URL: https://arxiv.org/abs/2412.19437 (visited on 2026-10-15).

[66] Chen, Liang, and Liu. *DFlash: Block Diffusion Drafting for Speculative Decoding*. ICML 2026 ; dernière version 2026-05-28. 2026-02-05. arXiv: 2602.06036. URL: https://arxiv.org/abs/2602.06036 (visited on 2026-10-15).

[67] Y. Fu et al. *Break the Sequential Dependency of LLM Inference Using Lookahead Decoding*. 2024-02-03. arXiv: 2402.02057. URL: https://arxiv.org/abs/2402.02057 (visited on 2026-10-15).

[68] X. Miao et al. *SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification*. 2023-05-16. arXiv: 2305.09781. URL: https://arxiv.org/abs/2305.09781 (visited on 2026-10-15).

[69] Z. Chen et al. *Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding*. dernière version 2025-07-05. 2024-02-19. arXiv: 2402.12374. URL: https://arxiv.org/abs/2402.12374 (visited on 2026-10-15).

[70] R. Svirschevski et al. *SpecExec: Massively Parallel Speculative Decoding for Interactive LLM Inference on Consumer Devices*. dernière version 2024-11-30. 2024-06-04. arXiv: 2406.02532. URL: https://arxiv.org/abs/2406.02532 (visited on 2026-10-15).

[71] Wang et al. *Substitute Speculative Decoding for Offloaded LLMs (SubSpec)*. NeurIPS 2025 ; dernière version 2025-10-08. 2025-09-22. arXiv: 2509.18344. URL: https://arxiv.org/abs/2509.18344 (visited on 2026-10-15).

[72] Saxena et al. *Utility-Driven Speculative Decoding for Mixture-of-Experts (Cascade)*. 2025-06-17. arXiv: 2506.20675. URL: https://arxiv.org/abs/2506.20675 (visited on 2026-10-15).

[73] McDanel et al. *MoE-Spec: Expert Budgeting for Speculative Decoding of Mixture-of-Experts*. 2026-02-17. arXiv: 2602.16052. URL: https://arxiv.org/abs/2602.16052 (visited on 2026-10-15).

[74] Liang et al. *AcceptMoE: Acceptance-aware Expert Budgets for Offloaded Speculative Decoding*. 2026-08-04. arXiv: 2608.02989. URL: https://arxiv.org/abs/2608.02989 (visited on 2026-10-15).

[75] Xie et al. *Less Experts, Faster Decoding (EcoSpec)*. 2026-07-14. arXiv: 2607.12696. URL: https://arxiv.org/abs/2607.12696 (visited on 2026-10-15).

[76] Samsung Electronics. *Samsung Electronics Unveils High-Performance PC SSD That Raises Everyday Computing and Gaming to a New Level*. PM9C1a : 6 000 Mo/s en lecture séquentielle, 900 k IOPS en lecture aléatoire (éditeur, non reproduit). Date : janvier 2023, d'après les reprises datées du 2023-01-13 ; la page primaire n'a pas répondu à la consultation. L'identification du disque MZVMA1T0HCLD au PM9C1a est probable, non prouvée. 2023-01. URL: https://news.samsungsemiconductor.com/global/samsung-electronics-unveils-high-performance-pc-ssd-that-raises-everyday-computing-and-gaming-to-a-new-level/ (visited on 2026-10-15).

[77] StorageReview. *Samsung PM9C1a SSD*. Reprise des chiffres de l'éditeur. URL: https://storagereview.com/?p=119560 (visited on 2026-10-15).

[78] Microsoft. *BypassIO for Filter Drivers*. Cas « Volume stack bypass is disabled (fvevol.sys) » quand BitLocker est actif ; chemin complet réservé à StorNVMe. 2025-11-05. URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/ifs/bypassio (visited on 2026-10-15).

[79] Microsoft. *CNG Algorithm Identifiers (Bcrypt.h)*. BCRYPT_XTS_AES_ALGORITHM, L"XTS-AES", SP 800-38E et IEEE 1619-2007, depuis Windows 10. 2026-05-30. URL: https://learn.microsoft.com/en-us/windows/win32/seccng/cng-algorithm-identifiers (visited on 2026-10-15).

[80] *IEEE Standard for Cryptographic Protection of Data on Block-Oriented Storage Devices*. Annexe B, vecteur 1 : témoin de aes_xts.py. IEEE, 2008. DOI: 10.1109/IEEESTD.2008.4493450. (Visited on 2026-10-15).

[81] Microsoft. *ioringapi.h header*. IoRing ; version 4 relevée par QueryIoRingCapabilities sur la machine. URL: https://learn.microsoft.com/en-us/windows/win32/api/ioringapi/ (visited on 2026-10-15).

[82] Microsoft. *Microsoft.Direct3D.DirectStorage (paquet NuGet), index des versions*. 1.3.0 (2025-06-27) : 2 816 824 octets ; 1.4.0-preview2-2606.904 (2026-06-16) : 3 986 072 octets (requêtes HEAD, rien téléchargé). URL: https://api.nuget.org/v3/registration5-gz-semver2/microsoft.direct3d.directstorage/index.json (visited on 2026-10-15).

[83] Microsoft DirectX Team. *DirectStorage 1.4 Release Adds Support for Zstandard*. 2026-03-11. URL: https://devblogs.microsoft.com/directx/directstorage-1-4-release-adds-support-for-zstandard/ (visited on 2026-10-15).

[84] gfx-rs. *wgpu*. Version 30.0.1. Licence MIT ou Apache-2.0. Dépendance du banc carte (Vulkan, DirectX 12). URL: https://github.com/gfx-rs/wgpu (visited on 2026-10-15).

[85] ggml-org. *llama.cpp*. Version b11515 (3d65c90d0). Licence MIT. Base de comparaison (llama-bench, moteur CUDA 12.4). URL: https://github.com/ggml-org/llama.cpp (visited on 2026-10-15).

[86] *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. IEEE. 2019. DOI: 10.1109/IEEESTD.2019.8766229. (Visited on 2026-10-15).

[87] B. D. Rouhani, R. Zhao, A. More, et al. *Microscaling Data Formats for Deep Learning*. Première version 2023-10-16, dernière 2023-10-19 (2310.10537v3). 2023-10-16. arXiv: 2310.10537. URL: https://arxiv.org/abs/2310.10537 (visited on 2026-10-15).

[88] P. Micikevicius et al. *FP8 Formats for Deep Learning*. Formats E4M3 et E5M2. 2022-09-12. arXiv: 2209.05433. URL: https://arxiv.org/abs/2209.05433 (visited on 2026-10-15).

[89] D. Kalamkar et al. *A Study of BFLOAT16 for Deep Learning Training*. 2019-05-29. arXiv: 1905.12322. URL: https://arxiv.org/abs/1905.12322 (visited on 2026-10-15).

[90] C. E. Shannon. "Coding Theorems for a Discrete Source with a Fidelity Criterion". In: *IRE National Convention Record*. Vol. 7. 4. Fonction distorsion-débit ; borne de la source gaussienne. 1959, pp. 142–163.

[91] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. 2nd ed. Chapitre 10 : théorème 10.3.2 (source gaussienne) et théorème du débit-distorsion. Hoboken, NJ: Wiley-Interscience, 2006. ISBN: 978-0-471-24195-9. DOI: 10.1002/047174882X. (Visited on 2026-10-15).

[92] S. P. Lloyd. "Least Squares Quantization in PCM". In: *IEEE Transactions on Information Theory* 28.2 (1982). Rapport interne des Bell Labs de 1957, publié en 1982., pp. 129–137. DOI: 10.1109/TIT.1982.1056489. (Visited on 2026-10-15).

[93] J. Max. "Quantizing for Minimum Distortion". In: *IRE Transactions on Information Theory* 6.1 (1960), pp. 7–12. DOI: 10.1109/TIT.1960.1057548. (Visited on 2026-10-15).

[94] P. F. Panter and W. Dite. "Quantization Distortion in Pulse-Count Modulation with Nonuniform Spacing of Levels". In: *Proceedings of the IRE* 39.1 (1951), pp. 44–48. DOI: 10.1109/JRPROC.1951.230419.

[95] M. W. Marcellin and T. R. Fischer. "Trellis Coded Quantization of Memoryless and Gauss-Markov Sources". In: *IEEE Transactions on Communications* 38.1 (1990), pp. 82–93. DOI: 10.1109/26.46532. (Visited on 2026-10-15).

[96] A. J. Viterbi. "Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm". In: *IEEE Transactions on Information Theory* 13.2 (1967), pp. 260–269. DOI: 10.1109/TIT.1967.1054010. (Visited on 2026-10-15).

[97] D. Lee and H. O. Song. *Q-Palette: Fractional-Bit Quantizers Toward Optimal Bit Allocation for Efficient LLM Deployment*. Première version 2025-09-24, dernière 2025-10-22 (2509.20214v2). NeurIPS 2025. 2025-09-24. arXiv: 2509.20214. URL: https://arxiv.org/abs/2509.20214 (visited on 2026-10-15).

[98] A. Fristrup, S. Pandey, and A. Kariryaa. *ApexQuant: Data-Free Elastic Quantization by Residual Re-Isotropization*. Première version 2026-10-06, dernière 2026-10-06 (2610.07904v1). 25 pages, 6 figures. 2026-10-06. arXiv: 2610.07904. URL: https://arxiv.org/abs/2610.07904 (visited on 2026-10-15).

[99] A. Dadgarnia, S. Tabesh, M. Nikdan, et al. *GSQ: Highly-Accurate Low-Precision Scalar Quantization for LLMs via Gumbel-Softmax Sampling*. Première version 2026-04-20, dernière 2026-05-15 (2604.18556v2). 2026-04-20. arXiv: 2604.18556. URL: https://arxiv.org/abs/2604.18556 (visited on 2026-10-15).

[100] *ml-explore/mlx issue 3043*. Fermée le 2026-01-26 par son auteur ; méthode non décrite. 2026-01-22. URL: https://github.com/ml-explore/mlx/issues/3043 (visited on 2026-10-15).

[101] H. Gish and J. N. Pierce. "Asymptotically efficient quantizing". In: *IEEE Transactions on Information Theory* 14.5 (1968), pp. 676–683. DOI: 10.1109/TIT.1968.1054193.

[102] F. Cheng, C. Guo, C. Wei, et al. "Ecco: Improving Memory Bandwidth and Capacity for LLMs via Entropy-aware Cache Compression". In: *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*. 2025-05-11, pp. 793–807. arXiv: 2505.06901. URL: https://arxiv.org/abs/2505.06901 (visited on 2026-10-15).

[103] Suram. *Cacheable by Design? Training Routers for Expert Locality*. un seul auteur ; incohérences internes relevées par l'étude 04. 2026-08-18. arXiv: 2608.18261. URL: https://arxiv.org/abs/2608.18261 (visited on 2026-10-15).

[104] ggml-org. *llama.cpp discussion 27149: reading only the active experts*. communautaire. 2026-08-15. URL: https://github.com/ggml-org/llama.cpp/discussions/27149 (visited on 2026-10-15).

[105] S. Zhong et al. *AdapMoE: Adaptive Sensitivity-based Expert Gating and Management for Efficient MoE Inference*. 2024-08-19. arXiv: 2408.10284. URL: https://arxiv.org/abs/2408.10284 (visited on 2026-10-15).

[106] SmallThinker Team. *SmallThinker: A Family of Efficient Large Language Models Natively Trained for Local Deployment*. 2025-07. arXiv: 2507.20984. URL: https://arxiv.org/abs/2507.20984 (visited on 2026-10-15).

[107] Hoang et al. *SpecMD: Expert Caching for Mixture-of-Experts Decoding*. 2026-02-03. arXiv: 2602.03921. URL: https://arxiv.org/abs/2602.03921 (visited on 2026-10-15).

[108] Kim et al. *FlashMoE: Learned Expert Cache Replacement*. dernière version 2026-10-05. 2026-01-22. arXiv: 2601.17063. URL: https://arxiv.org/abs/2601.17063 (visited on 2026-10-15).

[109] A. Skliar et al. *Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference*. TMLR ; dernière version 2025-06-24. 2024-11-27. arXiv: 2412.00099. URL: https://arxiv.org/abs/2412.00099 (visited on 2026-10-15).

[110] A. Q. Jiang et al. *Mixtral of Experts*. 2024-01-08. arXiv: 2401.04088. URL: https://arxiv.org/abs/2401.04088 (visited on 2026-10-15).

[111] M. Elhoushi et al. *LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding*. ACL 2024 ; dernière version 2024-10-18. 2024-04-25. arXiv: 2404.16710. URL: https://arxiv.org/abs/2404.16710 (visited on 2026-10-15).

[112] J. Zhao et al. *QSpec: Speculative Decoding with Complementary Quantization Schemes*. EMNLP 2025 ; dernière version 2025-10-02. 2024-10-15. arXiv: 2410.11305. URL: https://arxiv.org/abs/2410.11305 (visited on 2026-10-15).

[113] Wu et al. *STree: Speculative Tree Decoding for Hybrid State-Space Models*. NeurIPS 2025 ; dernière version 2025-10-27. 2025-05-20. arXiv: 2505.14969. URL: https://arxiv.org/abs/2505.14969 (visited on 2026-10-15).

[114] Pan et al. *Making Every Verified Token Count (EVICT)*. dernière version 2026-09-05. 2026-05-01. arXiv: 2605.00342. URL: https://arxiv.org/abs/2605.00342 (visited on 2026-10-15).

[115] Han. *DraftExpert: Distilled Draft Experts for On-Demand MoE Inference*. 2026-07-27. arXiv: 2607.24434. URL: https://arxiv.org/abs/2607.24434 (visited on 2026-10-15).

[116] J.-Y. Lee et al. *CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models*. dernière version 2024-11-03. 2024-04-12. arXiv: 2404.08763. URL: https://arxiv.org/abs/2404.08763 (visited on 2026-10-15).

[117] J. Liu et al. *Training-Free Activation Sparsity in Large Language Models (TEAL)*. ICLR 2025 ; dernière version 2025-02-25. 2024-08-26. arXiv: 2408.14690. URL: https://arxiv.org/abs/2408.14690 (visited on 2026-10-15).

[118] Zhang et al. *R-Sparse: Rank-Aware Activation Sparsity for Efficient LLM Inference*. ICLR 2025. 2025-04-28. arXiv: 2504.19449. URL: https://arxiv.org/abs/2504.19449 (visited on 2026-10-15).

[119] Y. Song et al. *Turbo Sparse: Achieving LLM SOTA Performance with Minimal Activated Parameters*. dernière version 2024-06-11. 2024-06-10. arXiv: 2406.05955. URL: https://arxiv.org/abs/2406.05955 (visited on 2026-10-15).

[120] ggml-org. *GGUF file format specification (docs/gguf.md)*. Spécification publique lue par l'étude 06. URL: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md (visited on 2026-10-15).

[121] DeepSeek-AI. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. 2026-04-26. arXiv: 2606.19348. URL: https://arxiv.org/abs/2606.19348 (visited on 2026-10-15).

[122] OpenAI. *gpt-oss-120b and gpt-oss-20b Model Card*. 2025-08-08. arXiv: 2508.10925. URL: https://arxiv.org/abs/2508.10925 (visited on 2026-10-15).

[123] DeepSeek-AI. *Conditional Memory via Scalable Lookup (Engram)*. dernière version 2026-07-12. 2026-01-12. arXiv: 2601.07372. URL: https://arxiv.org/abs/2601.07372 (visited on 2026-10-15).

[124] Kimi Team. *Kimi K2: Open Agentic Intelligence*. dernière version 2026-02-03. 2025-07-28. arXiv: 2507.20534. URL: https://arxiv.org/abs/2507.20534 (visited on 2026-10-15).

[125] GLM Team. *GLM-4.5: Agentic, Reasoning, and Coding Foundation Models*. 2025-08-08. arXiv: 2508.06471. URL: https://arxiv.org/abs/2508.06471 (visited on 2026-10-15).

[126] Qwen Team. *Qwen3 Technical Report*. 2025-05-14. arXiv: 2505.09388. URL: https://arxiv.org/abs/2505.09388 (visited on 2026-10-15).

[127] ggml-org. *gpt-oss-20b-GGUF (gpt-oss-20b-MXFP4.gguf, 12 109 566 624 octets)*. Apache-2.0 ; empreinte SHA-256 vérifiée à la réception (REPRISE.md). 2025-08. URL: https://huggingface.co/ggml-org/gpt-oss-20b-GGUF (visited on 2026-10-15).

[128] ggml-org. *gpt-oss-120b-GGUF (gpt-oss-120b-MXFP4.gguf, 63 387 346 208 octets)*. Apache-2.0 ; empreinte SHA-256 vérifiée à la réception (REPRISE.md). 2025-08. URL: https://huggingface.co/ggml-org/gpt-oss-120b-GGUF (visited on 2026-10-15).

[129] W. H. R. Equitz and T. M. Cover. "Successive Refinement of Information". In: *IEEE Transactions on Information Theory* 37.2 (1991), pp. 269–275.

[130] L. Babaoglu, S. Chen, and A. Khisti. *Multi-Bitwidth Quantization for LLMs Using Additive Codebooks*. v2 du 2026-09-26. Résumé seul lu : la source gaussienne est raffinable sans pénalité de débit sous une erreur quadratique pondérée ; un seul modèle sert plusieurs largeurs de bits. 2026-06-11. arXiv: 2606.12876. (Visited on 2026-10-15).

[131] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo. *SliceMoE: Bit-Sliced Expert Caching under Miss-Rate Constraints for Efficient MoE Inference*. v5 du 2026-09-08. Corps lu (HTML) : INT asymétrique G32, tranche haute par décalage (AMAT, troncature), experts critiques selon les scores du routeur, tranches basses évincées en premier, aucun codage entropique, flash UFS 3.1 et LPDDR4, accélérateur simulé ; DeepSeek-V2-Lite et Qwen1.5-MoE (éditeur, non reproduit). 2025-12-15. arXiv: 2512.12990. URL: https://arxiv.org/html/2512.12990 (visited on 2026-10-15).

[132] P. Tang, J. Liu, X. Hou, et al. *HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference*. v2 du 2024-11-06. Corps lu (HTML) : deux copies par expert (FP16 et Int4, ou Int8 et Int2), choix selon la somme des poids de routage des experts mieux classés (seuils 0,6 et 0,9 sur Mixtral), 67/30/3 % ; RTX 4090 et Jetson AGX Orin ; perte au plus 1 point (éditeur, non reproduit). 2024-11-03. arXiv: 2411.01433. URL: https://arxiv.org/html/2411.01433v2 (visited on 2026-10-15).